

Database Security

(CT069-3-3-DBS)

Group Assignment

Lecturer Name : Dr. Kulothunkan A/L Palasundram

Hand out Date : 29 June 2026

Hand in Date : 29 August 2026

Intake Code : APU3F2601CS(DA) / APD3F2601CS(DA)

Group No : 31

Group Members

Student ID	Student Name
TP068983	Chan Chun Ming
TP068580	Eu Jun Hong
TP068495	Lim Beng Rhui (*Group Leader*)
TP067678	Ng Xuan Jack

Table of Contents

Workload Matrix ····· 4
Overview ····· 5
Security Weaknesses in Initial Database Design ····· 6
Proposed Security Architecture ····· 7
Data Dictionary and Data Classification ····· 8
Database: SmartBankProd ····· 8
`dbo.Account` ····· 8
`dbo.Staff` ····· 9
`dbo.Customer` ····· 10
`dbo.TransactionRecord` ····· 11
`Security.UserMapping` ····· 12
`Security.RejectedAttemptLog` ····· 13
Database: SmartBankAnalytics ····· 14
`Reporting.DimCustomer` ····· 14
`Reporting.FactAccount` ····· 14
`Reporting.FactTransaction` ····· 15
`Reporting.ETLRunLog` ····· 15
Database Initialisation ····· 16
Access Control ····· 22
Authentication and User Management ····· 22
Role-Based Access Control ····· 28
Row-Level Security ····· 39
Views ····· 46
Integrity Controls ····· 51
Constraints ····· 51
Stored Procedures ····· 56
Triggers ····· 65
Data Protection Techniques ····· 68
Encryption ····· 68
Encryption at Rest ····· 68
Encryption in Transit (TLS) ····· 77
Data Masking ····· 80

Static Data Masking ····· 80
Dynamic Data Masking ····· 82
Hashing and Anonymisation ····· 84
Backup ····· 86
Backup Strategy ····· 86
Production Database: `SmartBankProd` ····· 86
Analytics Database: `SmartBankAnalytics` ····· 87
Implementation ····· 88
Scheduled Backup ····· 88
Restore and Recovery ····· 95
Audit ····· 101
Audit Strategy ····· 101
Audit Matrix ····· 102
Implementation ····· 107
Initialisation of Audit Objects ····· 107
Login and Logout Audit ····· 108
Data Manipulation Language (DML) Audit ····· 109
Data Definition Language (DDL) Audit ····· 113
Permission Change Audit ····· 115
Audit, Backup and Restore Logging ····· 117
Enabling the Audit Specifications ····· 118
Audit Verification ····· 119
Login and Logout Audit ····· 119
Data Manipulation Language (DML) Audit ····· 122
Data Definition Language (DDL) Audit ····· 125
Permission Change Audit ····· 127
Audit, Backup and Restore Logging ····· 129
Procedure-Level Logging ····· 131
Test Case ····· 132
Conclusion ····· 174
References ····· 175
Appendix A. Initial Database Design ····· 180

Workload Matrix

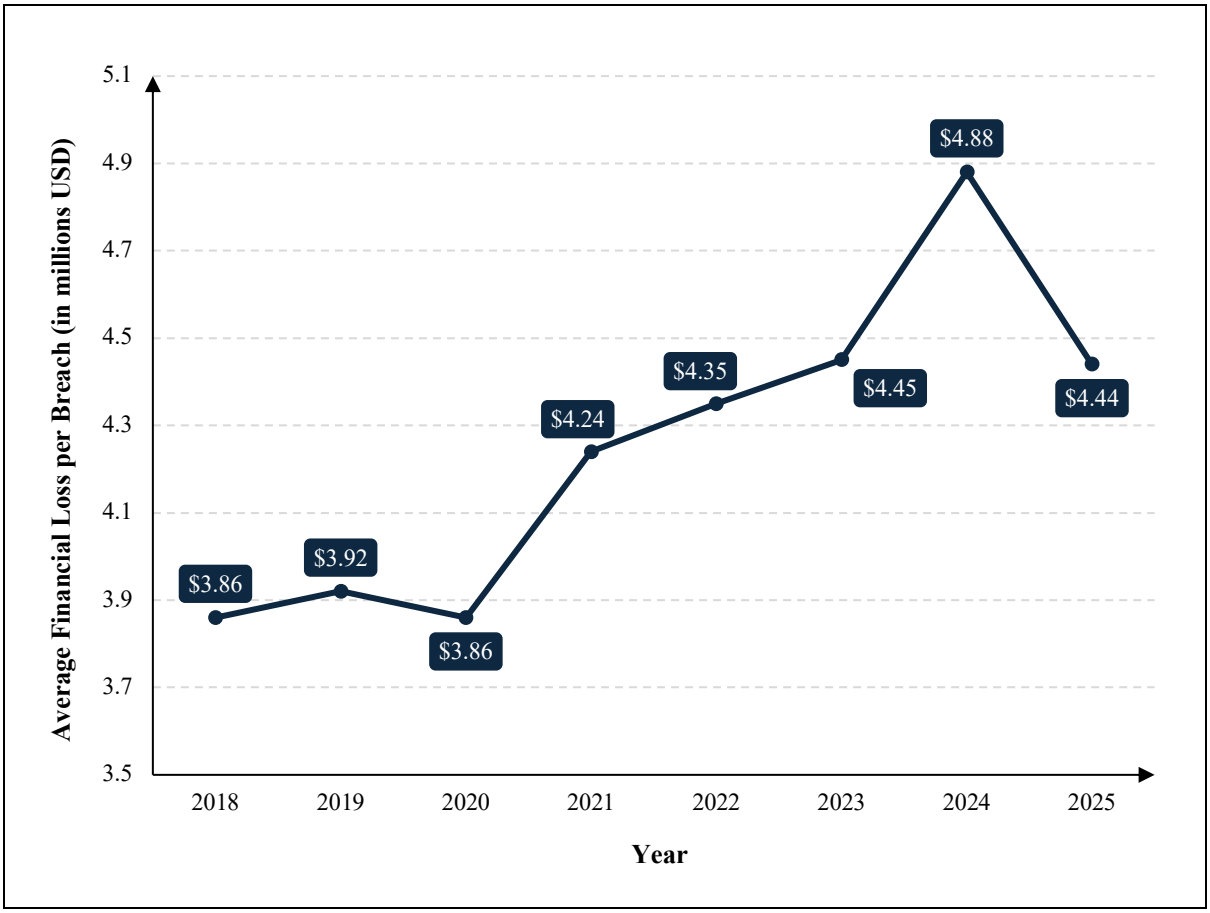
Task		Contribution Percentage			
		Chan Chun Ming (TP068983)	**Eu Jun Hong (TP068580)**	**Lim Beng Rhui (TP068495)**	**Ng Xuan Jack (TP067678)**
Overview		-	-	100%	-
Data Dictionary and Data Classification		-	-	100%	-
Database Initialisation		-	-	100%	-
Access Control	Authentication and User Management	-	-	-	100%
	Role-Based Access Control	-	-	-	100%
	Row-Level Security	-	100%	-	-
	Views	-	100%	-	-
Integrity Controls		100%	-	-	-
Data Protection Techniques	Encryption	100%	-	-	-
	Data Masking	-	-	100%	-
	Hashing and Anonymisation	-	-	100%	-
Backup		-	-	100%	-
Audit		-	-	100%	-
Test Case		-	-	-	100%
Conclusion		100%	-	-	-
Overall Contribution		**25%**	**25%**	**25%**	**25%**

Overview

With the advancement of technology in the current era, companies from different industries are slowly transitioning into the digitalisation phase, where large amount of data are collected and stored to facilitate their day-to-day business operations. Especially as technology like big data and artificial intelligence (AI) evolves, banking firms like SmartBank Malaysia often automate their operational and analytical processes by implementing database systems into their platforms. While database systems are able to allow companies to capture data accurately, without proper security design and configuration, these systems will become the best attack target for malicious users to perform fraudulent activities like cyberattacks and privilege abuse for their own advantages (Almaiah et al., 2024), not only leading to the breach of user data privacy but also incurring millions of losses among companies and negatively affecting their financial and reputation status, as shown in Figure 1.

Figure 1

Worldwide Average Financial Loss per Data Breach



Note. Adapted from *Data Breach Statistics (2026) – Trends, Costs & Impact*, by S. Singh, (2026a). (https://www.demandsage.com/data-breach-statistics/).

Security Weaknesses in Initial Database Design

As a digital bank that operates in the finance industry, SmartBank Malaysia relies on a centralised database system to handle its operations while fulfilling different user requirements, ranging from customers to bank managers. Due to the different types of sensitive information involved in the database, it is crucial for SmartBank to prioritise its database security, not only to comply with the Risk Management in Technology (RMiT) framework by the Bank Negara Malaysia (BNM), but also to ensure that its database remains secure when unexpected incidents like data theft or cyberattacks occur (Ridge Security, 2025). Hence, after examining the initial database design of SmartBank in Appendix A, it is identified that the current design suffers from several critical security weaknesses, with the findings summarised in Table 1.

Table 1

Security Weaknesses Found in Initial Database Design

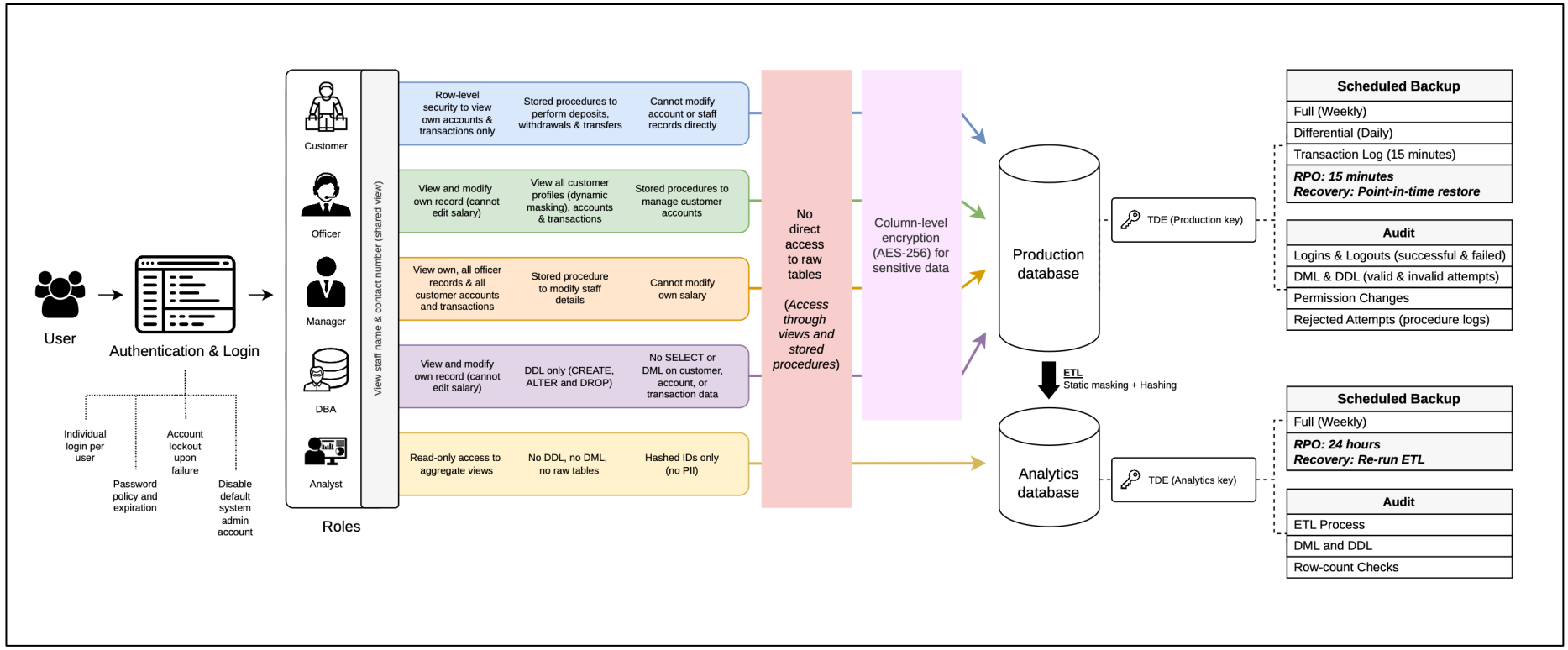
Weakness	CIA	Technical Impact
Lack of architectural separation	Availability	The production and analytics database are not separated, which causes performance bottlenecks and degrading availability when processing real-time transactions and heavy analytical tasks simultaneously.
Lack of authentication and authorisation	Confidentiality, Integrity	Currently, anyone who connects to the SQL server have unrestricted access to all tables in the database, which allows any unauthorised reads and modifications to take place (such as a staff member updating his own salary), hence violating the principle of least privilege.
Missing atomicity and integrity checking	Integrity	Constraints like foreign key and custom rules are not enforced, and data atomicity check is not implemented. This allows invalid or incomplete data to enter the database, breaking data integrity and consistency.
Lack of protection for data at rest	Confidentiality	Offline database and backup files are unencrypted. Malicious users who gain access to these files via physical access will be able to bypass database security checks and extract sensitive information without authorisation.
Sensitive data are stored as plain text	Confidentiality	Due to the lack of mechanisms like column-level encryption (CLE), data masking and hashing, users with access to tables with sensitive data can view confidential information in plain text and breach privacy.
Missing backup and recovery strategy	Integrity, Availability	Data loss that occurred in the database cannot be restored without any backup or recovery plans, potentially causing the entire database to become inoperative and disrupting critical business operations.
Missing audit trail	Confidentiality, Integrity	All actions performed in the database are not recorded in real-time, imposing difficulties for SmartBank to perform backtracking when fraudulent activity or data breach occurs.

Proposed Security Architecture

Based on the security weaknesses in Table 1, the architecture in Figure 2 is designed to identify the actions and restrictions to be implemented at different database security layers while satisfying the three components of the CIA triad. Aside from aligning with the requirements for production, it is proposed to have an additional analytics database to allow analysts perform queries without consuming excessive resources and interrupting the core transactional activities requiring real-time processing, thereby tackling potential system availability issues (Wongaphai & Ongtang, 2025).

Figure 2

Proposed Security Architecture for SmartBank Malaysia



Data Dictionary and Data Classification

Based on the security architecture diagram in Figure 2, the data dictionary for both the production and analytics database are designed, where the attributes involved in each database table are clearly defined and classified to facilitate the implementation of database security mechanisms while ensuring consistent interpretation among different user roles of the database (Panja et al., 2022).

Database: `SmartBankProd`

`dbo.Account`

Purpose: System-versioned temporal table that stores the details of customer accounts registered at SmartBank				
Column Name	***Type***	***Default***	***Note***	***Classification***
`AccountID`	`VARCHAR(10)`	-	• Primary key	Confidential
`CustomerID`	`VARCHAR(6)`	-	• Not null • Foreign key to `CustomerID` in `Customer` Table • Predicate column used for RLS	Confidential
`AccountType`	`VARCHAR(20)`	-	• Not null • Only allow `Savings` and `Current`	Confidential
`Balance`	`DECIMAL(12,2)`	0	• Not null • Value must be not less than 0	Confidential
`SysStartTime`	`DATETIME2`	`SYSUTCDATETIME()`	• Not null • `GENERATE ALWAYS AS ROW START`	Confidential
`SysEndTime`	`DATETIME2`	`9999-12-31 23:59:59.9999999`	• Not null • `GENERATE ALWAYS AS ROW END`	Confidential

`dbo.Staff`

Purpose: System-versioned temporal table that stores the employee details of SmartBank

Column Name	*Type*	*Default*	*Note*	*Classification*
`StaffID`	`VARCHAR(6)`	-	• Primary key • Predicate column used for RLS	Internal
`StaffName`	`VARCHAR(100)`	-	• Not null	Internal
`Position`	`VARCHAR(20)`	-	• Not null • Only allow `DBA`, `Manager` or `Officer`	Confidential
`Branch`	`VARCHAR(50)`	-	• Not null	Internal
`Phone`	`VARCHAR(20)`	-	• Not null	Internal
`Salary`	`DECIMAL(10,2)`	-	• Not null • Value must be not less than 0 • Can only be updated by managers • Enforced by trigger to prevent self-update	Confidential
`SysStartTime`	`DATETIME2`	`SYSUTCDATETIME()`	• Not null • `GENERATE ALWAYS AS ROW START`	Confidential
`SysEndTime`	`DATETIME2`	`9999-12-31 23:59:59.9999999`	• Not null • `GENERATE ALWAYS AS ROW END`	Confidential

`dbo.Customer`

Purpose: System-versioned temporal table that stores the personal details of registered customers of SmartBank

Column Name	*Type*	*Default*	*Note*	*Classification*
`CustomerID`	`VARCHAR(6)`	-	• Primary key • Predicate column used for RLS	Confidential
`CustomerName`	`VARCHAR(100)`	-	• Not null	Confidential
`Phone`	`VARCHAR(20)`	-	• Nullable • Requires dynamic data masking (e.g. `012XXXXXX89`) • `UNMASK` is granted to managers only	Confidential
`Address`	`VARCHAR(200)`	-	• Nullable • Requires dynamic data masking (e.g. `XXXX`) • `UNMASK` is granted to managers only	Confidential
`ICNumber_Enc`	`VARBINARY(256)`	-	• Not null • Require AES-256 cell-level encryption • Replaces the original `ICNumber` (`VARCHAR(20)`) column • Format validation (`XXXXXX-XX-XXXX`) is required before encryption • Decryption is allowed for managers only	Confidential
`SysStartTime`	`DATETIME2`	`SYSUTCDATETIME()`	• Not null • `GENERATE ALWAYS AS ROW START`	Confidential
`SysEndTime`	`DATETIME2`	`9999-12-31 23:59:59.9999999`	• Not null • `GENERATE ALWAYS AS ROW END`	Confidential

`dbo.TransactionRecord`

Purpose: To store the details of each transaction (deposit, withdrawal, transfer) taking place in SmartBank

Column Name	*Type*	*Default*	*Note*	*Classification*
`TransID`	`INT IDENTITY(1,1)`	-	• Primary key • System-generated identifier	Confidential
`AccountID`	`VARCHAR(10)`	-	• Not null • Foreign key to `AccountID` in `Account` table • Predicate column used for RLS	Confidential
`TransDate`	`DATETIME2(0)`	`SYSDATETIME()`	• Not null	Confidential
`Amount`	`DECIMAL(12,2)`	-	• Not null • Value must be greater than 0	Confidential
`TransactionType`	`VARCHAR(20)`	-	• Not null • Only allow `Deposit`, `Withdrawal`, or `Transfer`	Confidential
`ToAccountID`	`VARCHAR(10)`	-	• Nullable • Foreign key to `AccountID` in `Account` table • Not null for `Transfer` type • Must be null for `Withdrawal` and `Deposit` type • Cannot equal `AccountID` • Predicate column used for RLS	Confidential
`PerformedBy`	`SYSNAME`	`ORIGINAL_LOGIN()`	• Not null	Confidential

`Security.UserMapping`

Purpose: System-versioned temporal table that maps each database user to the specific customer or staff registered in the database

Column Name	*Type*	*Default*	*Note*	*Classification*
`DatabaseUser`	`SYSNAME`	-	• Primary key • Predicate column used for RLS	Private
`LoginName`	`SYSNAME`	-	• Not null • Must be unique	Private
`StaffID`	`VARCHAR(6)`	-	• Nullable • Must be null for customers and service accounts • Must be unique for staff accounts	Private
`CustomerID`	`VARCHAR(6)`	-	• Nullable • Must be null for staff and service accounts • Must be unique for customer accounts • Must verify that `StaffID` and `CustomerID` are mutually exclusive	Private
`SysStartTime`	`DATETIME2`	`SYSUTCDATETIME()`	• Not null • `GENERATE ALWAYS AS ROW START`	Private
`SysEndTime`	`DATETIME2`	`9999-12-31 23:59:59.9999999`	• Not null • `GENERATE ALWAYS AS ROW END`	Private

`Security.RejectedAttemptLog`

Purpose: System-versioned temporal table that records the write attempts that are rejected by stored procedures and triggers				
Column Name	***Type***	***Default***	***Note***	***Classification***
`AttemptID`	`INT IDENTITY(1,1)`	-	• Primary key • System-generated identifier	Confidential
`AttemptedOn`	`DATETIME2(0)`	`SYSDATETIME()`	• Not null	Confidential
`LoginName`	`SYSNAME`	-	• Not null	Confidential
`ProcedureName`	`SYSNAME`	-	• Not null	Confidential
`Reason`	`VARCHAR(300)`	-	• Not null	Confidential
`Detail`	`VARCHAR(500)`	-	• Nullable	Confidential
`SysStartTime`	`DATETIME2`	`SYSUTCDATETIME()`	• Not null • `GENERATE ALWAYS AS ROW START`	Confidential
`SysEndTime`	`DATETIME2`	`9999-12-31 23:59:59.9999999`	• Not null • `GENERATE ALWAYS AS ROW END`	Confidential

Database: `SmartBankAnalytics`

`Reporting.DimCustomer`

Purpose: To load anonymised customer processed by the ETL pipeline for analytical purposes

Column Name	*Type*	*Default*	*Note*	*Classification*
`CustomerHash`	`CHAR(64)`	-	• Primary key • Hashed and salted from `CustomerID`	Internal
`MaskedName`	`VARCHAR(100)`	-	• Not null • Statically masked from `CustomerName`	Internal
`LoadedOn`	`DATETIME2(0)`	`SYSDATETIME()`	• Not null	Internal

`Reporting.FactAccount`

Purpose: To store anonymised customer account details for analytical purposes

Column Name	*Type*	*Default*	*Note*	*Classification*
`AccountHash`	`CHAR(64)`	-	• Primary key • Hashed and salted from `AccountID`	Internal
`CustomerHash`	`CHAR(64)`	-	• Not null • Foreign key to `CustomerHash` of `DimCustomer` table	Internal
`AccountType`	`VARCHAR(20)`	-	• Not null	Internal
`Balance`	`DECIMAL(12,2)`	-	• Not null	Internal
`LoadedOn`	`DATETIME2(0)`	`SYSDATETIME()`	• Not null	Internal

`Reporting.FactTransaction`

Purpose: To display anonymised transaction details for analytical purposes				
Column Name	***Type***	***Default***	***Note***	***Classification***
`TransHash`	`CHAR(64)`	-	• Primary key • Hashed and salted from `TransID`	Internal
`AccountHash`	`CHAR(64)`	-	• Not null • Foreign key of `AccountHash` from `Reporting.FactAccount`	Internal
`TransDate`	`DATETIME2(0)`	-	• Not null	Internal
`Amount`	`DECIMAL(12,2)`	-	• Not null	Internal
`TransactionType`	`VARCHAR(20)`	-	• Not null	Internal
`LoadedOn`	`DATETIME2(0)`	`SYSDATETIME()`	• Not null	Internal

`Reporting.ETLRunLog`

Purpose: To record the execution details of the ETL pipeline to detect incomplete and failed loads				
Column Name	***Type***	***Default***	***Note***	***Classification***
`RunID`	`INT IDENTITY(1,1)`	-	• Primary key • System-generated identifier	Internal
`StartedOn`	`DATETIME2(0)`	-	• Not null	Internal
`CompletedOn`	`DATETIME2(0)`	-	• Null until ETL pipeline ends on both success and failure	Internal
`CustomerRows`	`INT`	-	• Null until ETL pipeline completes	Internal
`AccountRows`	`INT`	-	• Null until ETL pipeline completes	Internal
`TransactionRows`	`INT`	-	• Null until ETL pipeline completes	Internal
`RunStatus`	`VARCHAR(20)`	-	• Not null • Only allow `Running`, `Success` or `Failed`	Internal
`Message`	`VARCHAR(500)`	-	• Nullable • Must display error text when `RunStatus` is `Failed`	Internal

Database Initialisation

Using the SQL Server Management Studio 22 platform, the production and analytics databases are initialised on a new SQL server named `SmartBankServer`. As shown in Figure 3, to ensure that all the necessary backup, audit and encryption files can be located and stored centrally yet securely, the script first creates several directories at the `C:\SmartBankServer` of the local machine. This facilitates the configuration and execution of subsequent operations related to monitoring and recovery, thereby contributing to the availability of both databases.

Figure 3

SQL Script to Initialise Directories for Backup, Audit and Encryption Files

```
USE master;
GO

-- Create SmartBankServer directory structure --
EXEC master.dbo.xp_create_subdir 'C:\SmartBankServer';
EXEC master.dbo.xp_create_subdir 'C:\SmartBankServer\Backup';
EXEC master.dbo.xp_create_subdir 'C:\SmartBankServer\Backup\Production';
EXEC master.dbo.xp_create_subdir 'C:\SmartBankServer\Backup\Analytics';
EXEC master.dbo.xp_create_subdir 'C:\SmartBankServer\Audit';
EXEC master.dbo.xp_create_subdir 'C:\SmartBankServer\Encryption';
```

After initialising all the necessary directories, two databases are created for production and analytics purposes. Aside preventing heavy analytical tasks from negatively impacting database responsiveness when performing operational transactions, separating both databases also allows different recovery models to be configured based on the criticality of their data. As shown in Figure 4, the full recovery model is implemented for the production database to maximise data restoration, while the lower-criticality analytics database uses the simple recovery model to reduce storage requirements. This helps to maximise system performance and availability while minimising any additional potential resource overhead (Robidoux, 2025).

Figure 4

SQL Script to Create Production and Analytics Database

```
-- Create the production DB --
CREATE DATABASE SmartBankProd;
GO
ALTER DATABASE SmartBankProd SET RECOVERY FULL;
GO

-- Create the analytics DB --
CREATE DATABASE SmartBankAnalytics;
GO
ALTER DATABASE SmartBankAnalytics SET RECOVERY SIMPLE;
GO
```

To ensure data integrity, the tables are then initialised based on the specifications in the data dictionary, where Figure 5 shows the script to create the fact and dimension tables for the analytics database, while the SQL script in Figure 6 creates the relational tables for production. These tables are also assigned into different schemas, where the `dbo` schema handles business data like account and transaction details, the `Reporting` schema groups tables that are used for analytical workflows, while the `Security` schema governs objects that are related to the enforcement of database security mechanisms. By having a clear logical separation between tables, aside from improving the overall database structure, this also facilitates subsequent access control operations as permissions can be assigned at schema level without requiring administrators to handle each table individually, thereby encouraging the enforcement of the principle of least privilege to promote data confidentiality (Gundapaneni, 2025).

Figure 5

SQL Script to Create `Reporting` *Schema and Initialise Tables for Analytics Database*

```
-- Create "Reporting" schema for analytics DB --
USE SmartBankAnalytics;
GO
CREATE SCHEMA Reporting AUTHORIZATION dbo;
GO

-- Create tables for analytics DB --
CREATE TABLE Reporting.DimCustomer (
    CustomerHash    CHAR(64)          NOT NULL    CONSTRAINT PK_DimCustomer PRIMARY KEY,
    MaskedName      VARCHAR(100)      NOT NULL,
    LoadedOn        DATETIME2(0)      NOT NULL    CONSTRAINT DF_DimCustomer_Loaded DEFAULT (SYSDATETIME())
);
GO

CREATE TABLE Reporting.FactAccount (
    AccountHash     CHAR(64)          NOT NULL    CONSTRAINT PK_FactAccount PRIMARY KEY,
    CustomerHash    CHAR(64)          NOT NULL,
    AccountType     VARCHAR(20)       NOT NULL,
    Balance         DECIMAL(12,2)     NOT NULL,
    LoadedOn        DATETIME2(0)      NOT NULL    CONSTRAINT DF_FactAccount_Loaded DEFAULT (SYSDATETIME())
);
GO

CREATE TABLE Reporting.FactTransaction (
    TransHash       CHAR(64)          NOT NULL    CONSTRAINT PK_FactTransaction PRIMARY KEY,
    AccountHash     CHAR(64)          NOT NULL,
    TransDate       DATETIME2(0)      NOT NULL,
    Amount          DECIMAL(12,2)     NOT NULL,
    TransactionType VARCHAR(20)       NOT NULL,
    LoadedOn        DATETIME2(0)      NOT NULL    CONSTRAINT DF_FactTransaction_Loaded DEFAULT (SYSDATETIME())
);
GO

CREATE TABLE Reporting.ETLRunLog (
    RunID           INT IDENTITY(1,1) NOT NULL    CONSTRAINT PK_ETLRunLog PRIMARY KEY,
    StartedOn       DATETIME2(0)      NOT NULL,
    CompletedOn     DATETIME2(0)      NULL,
    CustomerRows    INT               NULL,
    AccountRows     INT               NULL,
    TransactionRows INT               NULL,
    RunStatus       VARCHAR(20)       NOT NULL,
    [Message]       VARCHAR(500)      NULL
);
GO
```

Figure 6

SQL Script to Create `Security` *Schema and Initialise Tables for Production Database*

```
-- Create "Security" schema for production DB --
USE SmartBankProd;
GO
CREATE SCHEMA [Security] AUTHORIZATION dbo;
GO

-- Create tables for production DB --
CREATE TABLE dbo.Staff (
    StaffID         VARCHAR(6)          NOT NULL    CONSTRAINT PK_Staff PRIMARY KEY,
    StaffName       VARCHAR(100)        NOT NULL,
    Position        VARCHAR(20)         NOT NULL,
    Branch          VARCHAR(50)         NOT NULL,
    Phone           VARCHAR(20)         NOT NULL,
    Salary          DECIMAL(10,2)       NOT NULL
);
GO

CREATE TABLE dbo.Customer (
    CustomerID      VARCHAR(6)          NOT NULL    CONSTRAINT PK_Customer PRIMARY KEY,
    CustomerName    VARCHAR(100)        NOT NULL,
    ICNumber        VARCHAR(20)         NOT NULL,
    Phone           VARCHAR(20)         NULL,
    [Address]       VARCHAR(200)        NULL
);
GO

CREATE TABLE dbo.Account (
    AccountID       VARCHAR(10)         NOT NULL    CONSTRAINT PK_Account PRIMARY KEY,
    CustomerID      VARCHAR(6)          NOT NULL,
    AccountType     VARCHAR(20)         NOT NULL,
    Balance         DECIMAL(12,2)       NOT NULL    CONSTRAINT DF_Account_Balance DEFAULT (0)
);
GO

CREATE TABLE dbo.TransactionRecord (
    TransID         INT IDENTITY(1,1)   NOT NULL    CONSTRAINT PK_TransactionRecord PRIMARY KEY,
    AccountID       VARCHAR(10)         NOT NULL,
    TransDate       DATETIME2(0)        NOT NULL    CONSTRAINT DF_Trans_Date DEFAULT (SYSDATETIME()),
    Amount          DECIMAL(12,2)       NOT NULL,
    TransactionType VARCHAR(20)         NOT NULL,
    ToAccountID     VARCHAR(10)         NULL,
    PerformedBy     SYSNAME             NOT NULL    CONSTRAINT DF_Trans_By DEFAULT (ORIGINAL_LOGIN())
);
GO

CREATE TABLE [Security].UserMapping (
    DatabaseUser     SYSNAME            NOT NULL    CONSTRAINT PK_UserMapping PRIMARY KEY,
    LoginName        SYSNAME            NOT NULL,
    StaffID          VARCHAR(6)         NULL,
    CustomerID       VARCHAR(6)         NULL
);
GO

CREATE TABLE [Security].RejectedAttemptLog (
    AttemptID       INT IDENTITY(1,1)   NOT NULL    CONSTRAINT PK_RejectedAttemptLog PRIMARY KEY,
    AttemptedOn     DATETIME2(0)        NOT NULL    CONSTRAINT DF_Rejected_On DEFAULT (SYSDATETIME()),
    LoginName       SYSNAME             NOT NULL,
    ProcedureName   SYSNAME             NOT NULL,
    Reason          VARCHAR(300)        NOT NULL,
    Detail          VARCHAR(500)        NULL
);
GO
```

As the last step of initialisation, the tables under the `dbo` schema are populated with initial data, as shown by the script in Figure 7. Tables under the `Security` and `Reporting` schemas will have their data populated dynamically as different database security mechanisms are implemented and demonstrated.

Figure 7

SQL Script to Populate Data into Tables under `dbo` *Schema*

```
-- Populate data for tables under "dbo" schema --
USE SmartBankProd;
GO

INSERT INTO dbo.Staff (StaffID, StaffName, Position, Branch, Phone, Salary) VALUES
    ('S00001',  'Ahmad bin Hassan',     'DBA',      'HQ Kuala Lumpur',  '012-3450001',  9500.00),
    ('S00002',  'Aisyah binti Rahman',  'Manager',  'Petaling Jaya',    '012-3450002',  12000.00),
    ('S00003',  'Ravi Subramaniam',     'Manager',  'Shah Alam',        '012-3450003',  11500.00),
    ('S00004',  'Lim Wei Sheng',        'Officer',  'Petaling Jaya',    '012-3450004',  5200.00),
    ('S00005',  'Nurul Izzah',          'Officer',  'Shah Alam',        '012-3450005',  4900.00);
GO

INSERT INTO dbo.Customer (CustomerID, CustomerName, ICNumber, Phone, [Address]) VALUES
    ('C00001',  'Tan Mei Ling',     '900314-14-5521',   '013-7770001',  '12 Jalan Bukit Bintang, 55100 Kuala Lumpur'),
    ('C00002',  'Siti Nurhaliza',   '880725-10-4432',   '013-7770002',  '88 Jalan SS2/24, 47300 Petaling Jaya'),
    ('C00003',  'Kumar Rajendran',  '950612-08-3310',   '013-7770003',  '5 Persiaran Kayangan, 40100 Shah Alam'),
    ('C00004',  'Wong Kah Meng',    '011128-14-2201',   '013-7770004',  '30 Jalan Ampang, 50450 Kuala Lumpur');
GO

INSERT INTO dbo.Account (AccountID, CustomerID, AccountType, Balance) VALUES
    ('ACC1000001',  'C00001',   'Savings',  15000.00),
    ('ACC1000002',  'C00001',   'Current',  4500.00),
    ('ACC1000003',  'C00002',   'Savings',  28000.00),
    ('ACC1000004',  'C00003',   'Savings',  9200.00),
    ('ACC1000005',  'C00004',   'Current',  51000.00);
GO

INSERT INTO dbo.TransactionRecord (AccountID, TransDate, Amount, TransactionType, ToAccountID, PerformedBy) VALUES
    ('ACC1000001',  '2026-07-01 09:15:00',  2000.00,    'Deposit',      NULL,           'seed'),
    ('ACC1000001',  '2026-07-05 14:02:00',  350.00,     'Withdrawal',   NULL,           'seed'),
    ('ACC1000003',  '2026-07-06 11:40:00',  5000.00,    'Deposit',      NULL,           'seed'),
    ('ACC1000001',  '2026-07-08 09:30:00',  500.00,     'Transfer',     'ACC1000002',   'seed'),
    ('ACC1000004',  '2026-07-10 16:25:00',  120.00,     'Withdrawal',   NULL,           'seed'),
    ('ACC1000003',  '2026-07-11 13:50:00',  1000.00,    'Transfer',     'ACC1000005',   'seed'),
    ('ACC1000005',  '2026-07-12 10:05:00',  8000.00,    'Deposit',      NULL,           'seed');
GO
```

To validate the outputs, the SQL snippet in Figure 8 is used to display the structure and contents of all the tables in the production and analytics database. Figure 9 shows that all the tables are initialised and grouped under the correct schema, Figure 10 displays the data that are populated under each table in the `dbo` schema, while Figure 11 shows that the file directories to store backup, audit and encryption files are successful created in the local `C:\` drive.

Figure 8

SQL Snippet for Displaying the Structure and Contents of Database Tables

```
-- Display results (structure) --
SELECT  'SmartBankProd'                AS DatabaseName,
        s.name                         AS SchemaName,
        t.name                         AS TableName,
        t.create_date                  AS CreatedOn
FROM    SmartBankProd.sys.tables AS t
JOIN    SmartBankProd.sys.schemas AS s ON s.schema_id = t.schema_id
UNION ALL
SELECT  'SmartBankAnalytics'           AS DatabaseName,
        s.name                         AS SchemaName,
        t.name                         AS TableName,
        t.create_date                  AS CreatedOn
FROM    SmartBankAnalytics.sys.tables AS t
JOIN    SmartBankAnalytics.sys.schemas AS s ON s.schema_id = t.schema_id
ORDER BY DatabaseName, SchemaName, TableName;
GO

-- Display results (data) --
USE SmartBankProd;
GO

SELECT * FROM dbo.Staff;
SELECT * FROM dbo.Customer;
SELECT * FROM dbo.Account;
SELECT * FROM dbo.TransactionRecord;
GO
```

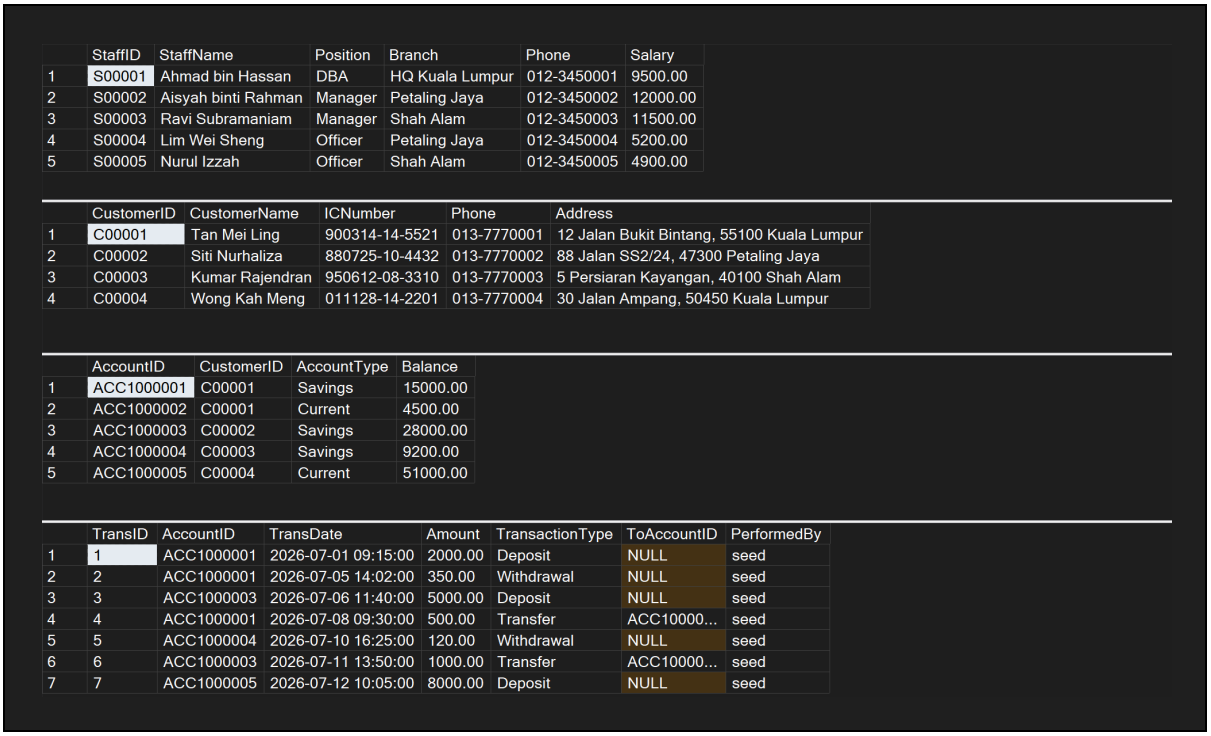
Figure 9

Schema and Details of Existing Tables in Production and Analytics Database

	DatabaseName	SchemaName	TableName	CreatedOn
1	SmartBankAnalytics	Reporting	DimCustomer	2026-08-19 17:10:38.497
2	SmartBankAnalytics	Reporting	ETLRunLog	2026-08-19 17:10:38.503
3	SmartBankAnalytics	Reporting	FactAccount	2026-08-19 17:10:38.500
4	SmartBankAnalytics	Reporting	FactTransaction	2026-08-19 17:10:38.500
5	SmartBankProd	dbo	Account	2026-08-19 17:11:04.347
6	SmartBankProd	dbo	Customer	2026-08-19 17:11:04.343
7	SmartBankProd	dbo	Staff	2026-08-19 17:11:04.340
8	SmartBankProd	dbo	TransactionRecord	2026-08-19 17:11:04.347
9	SmartBankProd	Security	RejectedAttemptLog	2026-08-19 17:11:04.350
10	SmartBankProd	Security	UserMapping	2026-08-19 17:11:04.350

Figure 10

Contents in the Existing Table under `dbo` *Schema*



	StaffID	StaffName	Position	Branch	Phone	Salary
1	S00001	Ahmad bin Hassan	DBA	HQ Kuala Lumpur	012-3450001	9500.00
2	S00002	Aisyah binti Rahman	Manager	Petaling Jaya	012-3450002	12000.00
3	S00003	Ravi Subramaniam	Manager	Shah Alam	012-3450003	11500.00
4	S00004	Lim Wei Sheng	Officer	Petaling Jaya	012-3450004	5200.00
5	S00005	Nurul Izzah	Officer	Shah Alam	012-3450005	4900.00

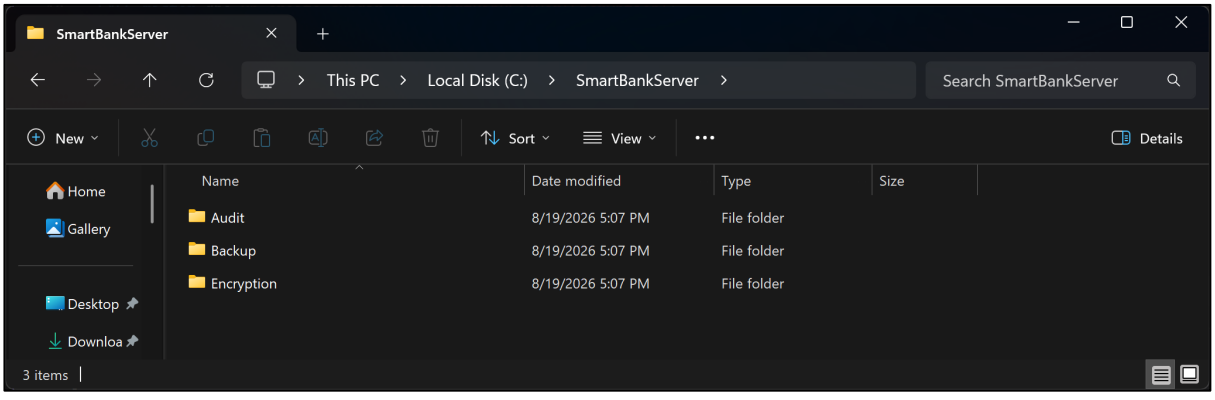
	CustomerID	CustomerName	ICNumber	Phone	Address
1	C00001	Tan Mei Ling	900314-14-5521	013-7770001	12 Jalan Bukit Bintang, 55100 Kuala Lumpur
2	C00002	Siti Nurhaliza	880725-10-4432	013-7770002	88 Jalan SS2/24, 47300 Petaling Jaya
3	C00003	Kumar Rajendran	950612-08-3310	013-7770003	5 Persiaran Kayangan, 40100 Shah Alam
4	C00004	Wong Kah Meng	011128-14-2201	013-7770004	30 Jalan Ampang, 50450 Kuala Lumpur

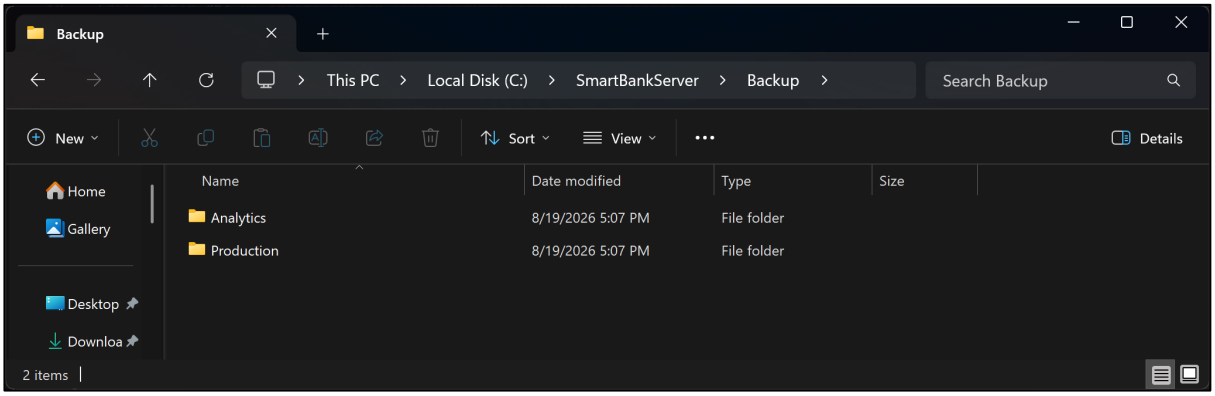
	AccountID	CustomerID	AccountType	Balance
1	ACC1000001	C00001	Savings	15000.00
2	ACC1000002	C00001	Current	4500.00
3	ACC1000003	C00002	Savings	28000.00
4	ACC1000004	C00003	Savings	9200.00
5	ACC1000005	C00004	Current	51000.00

	TransID	AccountID	TransDate	Amount	TransactionType	ToAccountID	PerformedBy
1	1	ACC1000001	2026-07-01 09:15:00	2000.00	Deposit	NULL	seed
2	2	ACC1000001	2026-07-05 14:02:00	350.00	Withdrawal	NULL	seed
3	3	ACC1000003	2026-07-06 11:40:00	5000.00	Deposit	NULL	seed
4	4	ACC1000001	2026-07-08 09:30:00	500.00	Transfer	ACC10000...	seed
5	5	ACC1000004	2026-07-10 16:25:00	120.00	Withdrawal	NULL	seed
6	6	ACC1000003	2026-07-11 13:50:00	1000.00	Transfer	ACC10000...	seed
7	7	ACC1000005	2026-07-12 10:05:00	8000.00	Deposit	NULL	seed

Figure 11

File Directories Created to Store Backup, Audit and Encryption Files





Access Control

As one of the key measures to enforce data confidentiality in database systems, access control measures are implemented to ensure that users are authenticated and authorised before obtaining valid permissions to perform read and write operations on SmartBank's database.

Authentication and User Management

As banking institutions like SmartBank often store and handle financial and personal data that are highly sensitive in nature, integrating user authentication controls into SmartBank is crucial to uniquely identify the person who login to the system, so that all database activities can be recorded and traced back to support data integrity and promote accountability (Mohamed et al., 2023). Without authentication, users can access SmartBank's database via any unrestricted or shared credentials, thereby creating a significant security weakness as such a scenario not only leads to the exposure of banking information to unauthorised users, but also causes invalid modifications to take place, which affects the accuracy and reliability of the data recorded in the database. Therefore, authentication controls are implemented in SmartBank to validate the identity of each user when they login to the `SmartBankServer` server.

In SQL server, two authentication modes are provided to enforce user authentication, namely Windows Authentication mode that allows users to login using their Windows account credentials, as well as Mixed mode, which supports both Windows Authentication and SQL Server Authentication and provides the flexibility to configure logins and roles independently. Although Windows Authentication mode is often recommended as it supports centralised account management via Active Directory with better security (Douglas, 2023), the current project requires the simulation of different business users who do not correspond to real Windows account but must be configured using SQL Server Authentication. Hence, SmartBank will be using Mixed mode for its authentication, as configured in Figure 12. After restarting the SQL server to activate the new configurations, multiple login accounts are created for each user at the SQL server instance level using the code in Figure 13, with several login options configured. The `MUST_CHANGE` option is specified to require users to change their password immediately during their first connection, `CHECK_POLICY = ON` ensures that the password set by users are aligned with the Windows account policies shown in Figure 14 and 15, while `CHECK_EXPIRATION = ON` enforces password expiration mechanism that requires users to change their password after the maximum password age expires. However, `svc_ETL` is exempted from `CHECK_EXPIRATION` to prevent any job failures when running ETL pipeline.

Figure 12

Server Properties Interface to Configure Authentication Mode

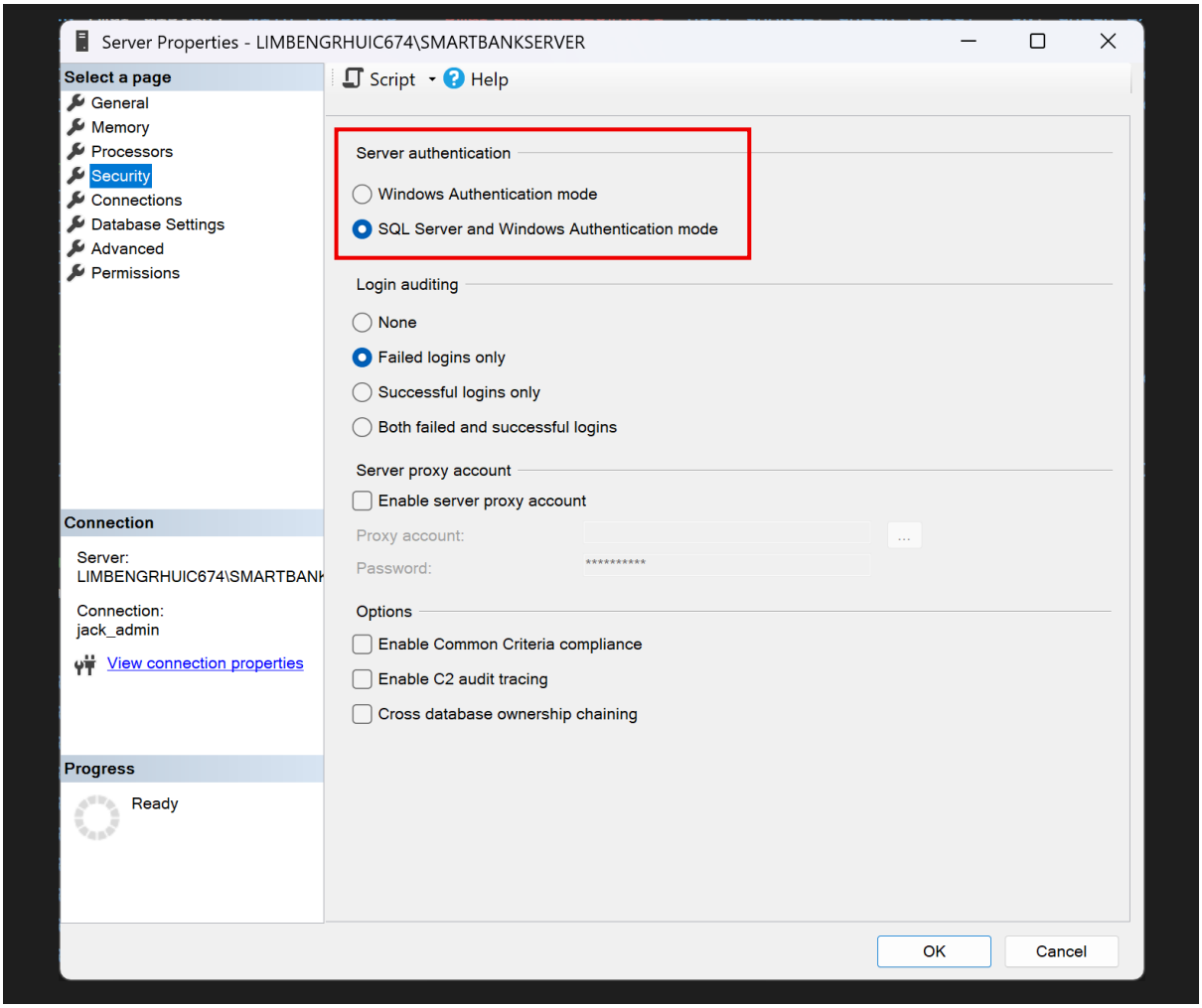


Figure 13

SQL Snippet to Create Logins

```
USE master;
GO

-- Staff logins
CREATE LOGIN [dba_ahmad]   WITH PASSWORD = 'SmartBank@2026#Dba'  MUST_CHANGE, CHECK_POLICY = ON, CHECK_EXPIRATION = ON;
CREATE LOGIN [mgr_aisyah]  WITH PASSWORD = 'SmartBank@2026#Mgr1' MUST_CHANGE, CHECK_POLICY = ON, CHECK_EXPIRATION = ON;
CREATE LOGIN [mgr_ravi]    WITH PASSWORD = 'SmartBank@2026#Mgr2' MUST_CHANGE, CHECK_POLICY = ON, CHECK_EXPIRATION = ON;
CREATE LOGIN [ofc_lim]     WITH PASSWORD = 'SmartBank@2026#Ofc1' MUST_CHANGE, CHECK_POLICY = ON, CHECK_EXPIRATION = ON;
CREATE LOGIN [ofc_nurul]   WITH PASSWORD = 'SmartBank@2026#Ofc2' MUST_CHANGE, CHECK_POLICY = ON, CHECK_EXPIRATION = ON;

-- Customer logins
CREATE LOGIN [cust_tan]    WITH PASSWORD = 'SmartBank@2026#Cus1' MUST_CHANGE, CHECK_POLICY = ON, CHECK_EXPIRATION = ON;
CREATE LOGIN [cust_siti]   WITH PASSWORD = 'SmartBank@2026#Cus2' MUST_CHANGE, CHECK_POLICY = ON, CHECK_EXPIRATION = ON;
CREATE LOGIN [cust_kumar]  WITH PASSWORD = 'SmartBank@2026#Cus3' MUST_CHANGE, CHECK_POLICY = ON, CHECK_EXPIRATION = ON;
CREATE LOGIN [cust_wong]   WITH PASSWORD = 'SmartBank@2026#Cus4' MUST_CHANGE, CHECK_POLICY = ON, CHECK_EXPIRATION = ON;

-- Analytics side
CREATE LOGIN [analyst_lee] WITH PASSWORD = 'SmartBank@2026#Ana1' MUST_CHANGE, CHECK_POLICY = ON, CHECK_EXPIRATION = ON;

-- Special: svc_ETL is a service account, so does not require expiration
CREATE LOGIN [svc_ETL]     WITH PASSWORD = 'SmartBank@2026#Etl0', CHECK_POLICY = ON, CHECK_EXPIRATION = OFF;
GO
```

Figure 14

Windows Password Policy Settings

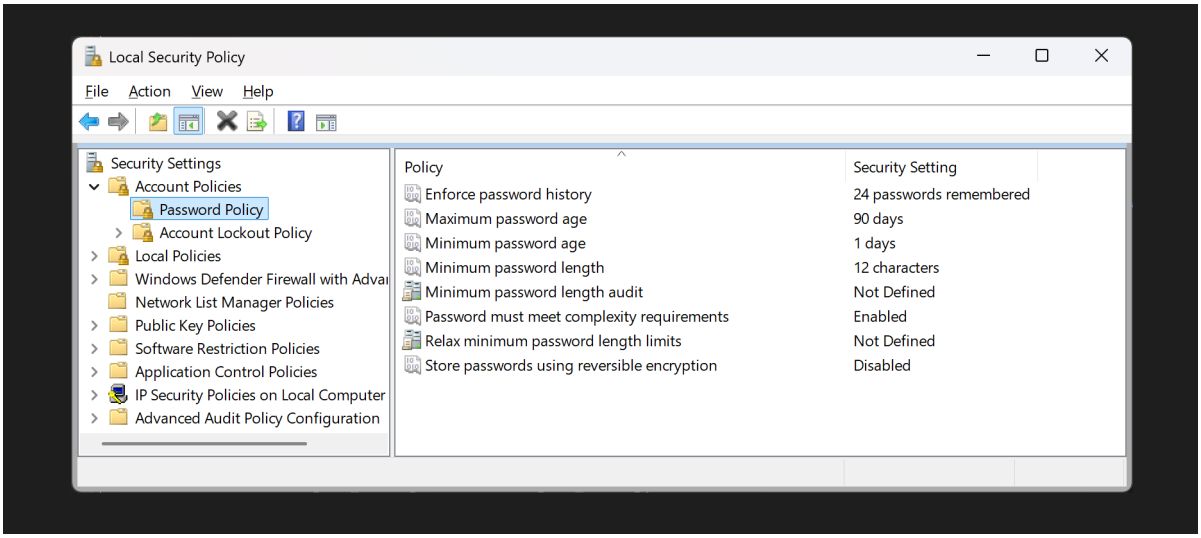
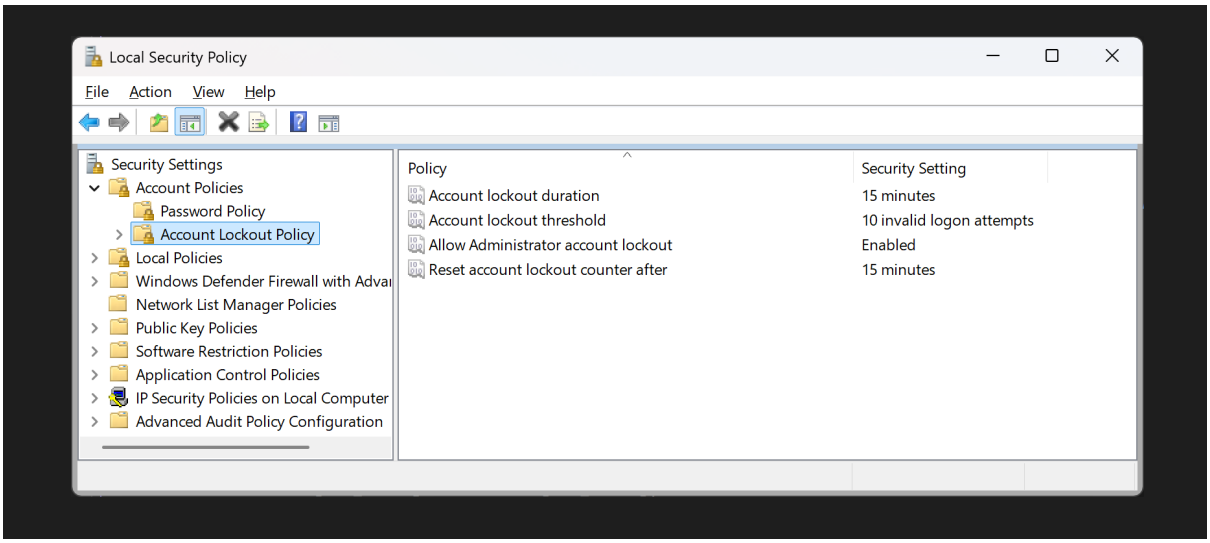


Figure 15

Windows Account Lockout Policy Settings



Once logins are established at the SQL server, the corresponding user identities are created for each of the login so that database-level permissions can be configured for individual roles subsequently. Different users, covering roles like database administrators (DBA), bank managers, officers, customers, and analysts, are created using the `CREATE USER` command, as shown in Figure 16. An additional user named `svc_ETL` that corresponds to the `svc_ETL` login is also created to act as a service account that supports the ETL pipeline by loading data from the production database into the analytics database on a daily basis.

Figure 16

SQL Snippet to Create Users for the Production and Analytics Database

```
-- Create users for production DB (database-level) --
USE SmartBankProd;
GO

CREATE USER [dba_ahmad]  FOR LOGIN [dba_ahmad];
CREATE USER [mgr_aisyah] FOR LOGIN [mgr_aisyah];
CREATE USER [mgr_ravi]   FOR LOGIN [mgr_ravi];
CREATE USER [ofc_lim]    FOR LOGIN [ofc_lim];
CREATE USER [ofc_nurul]  FOR LOGIN [ofc_nurul];
CREATE USER [cust_tan]   FOR LOGIN [cust_tan];
CREATE USER [cust_siti]  FOR LOGIN [cust_siti];
CREATE USER [cust_kumar] FOR LOGIN [cust_kumar];
CREATE USER [cust_wong]  FOR LOGIN [cust_wong];
CREATE USER [svc_ETL]    FOR LOGIN [svc_ETL];
GO

-- Create users for analytics DB (database-level) --
USE SmartBankAnalytics;
GO

CREATE USER [svc_ETL]     FOR LOGIN [svc_ETL];
CREATE USER [analyst_lee] FOR LOGIN [analyst_lee];
GO
```

After creating all the database users, the `Security.UserMapping` table is then populated with the corresponding IDs of each staff member and customer, as shown in Figure 17. The purpose of the table is to simplify the process of mapping each database user to their corresponding IDs, so that SQL queries are able to link the current connected login to the specific staff or customer record. This facilitates the subsequent configuration of database security mechanisms, such as auditing and row-level security (RLS), that requires users who login to SmartBank's server to be accurately identified before access control can be enforced.

Figure 17

SQL Snippet to Initialise `Security.UserMapping` *Table*

```
USE SmartBankProd;
GO
INSERT INTO Security.UserMapping (DatabaseUser, LoginName, StaffID, CustomerID) VALUES
    ('dba_ahmad',  'dba_ahmad',  'S00001', NULL),
    ('mgr_aisyah', 'mgr_aisyah', 'S00002', NULL),
    ('mgr_ravi',   'mgr_ravi',   'S00003', NULL),
    ('ofc_lim',    'ofc_lim',    'S00004', NULL),
    ('ofc_nurul',  'ofc_nurul',  'S00005', NULL),
    ('cust_tan',   'cust_tan',   NULL, 'C00001'),
    ('cust_siti',  'cust_siti',  NULL, 'C00002'),
    ('cust_kumar', 'cust_kumar', NULL, 'C00003'),
    ('cust_wong',  'cust_wong',  NULL, 'C00004'),
    ('svc_ETL',    'svc_ETL',    NULL, NULL);
GO
```

`UNIQUE INDEX` is also added in Figure 18 to prevent duplicate `LoginName`, `StaffID`, and `CustomerID` values, ensuring that users can be mapped to their records accurately.

Figure 18

SQL Snippet to Create `UNIQUE INDEX` *for* `Security.UserMapping`

```
CREATE UNIQUE INDEX UQ_UserMapping_LoginName ON Security.UserMapping (LoginName);
CREATE UNIQUE INDEX UQ_UserMapping_StaffID   ON Security.UserMapping (StaffID) WHERE StaffID IS NOT NULL;
CREATE UNIQUE INDEX UQ_UserMapping_CustomerID ON Security.UserMapping (CustomerID) WHERE CustomerID IS NOT NULL;
GO
```

Privileged access is also handled in SmartBank by creating a new system administrator account named `smartbank_admin` to replace the existing built-in `sa` login, as well as the default Windows Authentication account with `sysadmin` privilege. As shown in Figure 19, both accounts are disabled as the default `sa` account is the common attack target for cybercriminals to obtain full privileged access and perform malicious actions on the database (Tenable, 2024), while the Windows Authentication account is authenticated independently of the current SQL server's password policies, which undermines future access control mechanisms in SmartBank. Although the `smartbank_admin` account becomes the new `sysadmin` account that will be used for subsequent configurations, its usage shall be highly restricted and consistently monitored to prevent any third parties from obtaining access to perform unauthorised reads and modifications towards both databases in SmartBank.

Figure 19

SQL Snippet to Create a New System Admin Account to Replace Existing Ones

```
-- Create a new system admin account --
CREATE LOGIN [smartbank_admin] WITH PASSWORD = 'SmartBank@2026#Adm1' MUST_CHANGE,
                                                                     CHECK_POLICY = ON,
                                                                     CHECK_EXPIRATION = ON;
GO
ALTER SERVER ROLE [sysadmin] ADD MEMBER [smartbank_admin];
GO

-- Check if smartbank_admin is successfully created as 'sa' --
SELECT IS_SRVROLEMEMBER('sysadmin', 'smartbank_admin') AS IsSysadmin;
GO

-- Disable default 'sa' account --
ALTER LOGIN [sa] DISABLE;
GO

-- Disable current Windows Auth account --
ALTER LOGIN [LIMBENGRHUIC674\limbengrhui] DISABLE;
GO
ALTER SERVER ROLE sysadmin DROP MEMBER [LIMBENGRHUIC674\limbengrhui];
GO
```

	IsSysadmin
1	1

After switching to the `smartbank_admin` account, the `SELECT` queries in Figure 20 are executed to review the authentication settings of each SQL login, as well as the customer and staff IDs that correspond to each database user. Based on the results, it is evident that all SQL logins are configured with the correct password policy status and expiration settings, and each database user, except `svc_ETL`, is assigned to the correct IDs. This shows that authentication is configured appropriately for SmartBank's database, thereby aligning with the confidentiality component in the CIA triad as this acts as a first line of defence that prevents unauthorised users from having unrestricted access towards the database.

Figure 20

SQL Snippet to Verify the Configuration for Authentication Control

```
-- Show the details of the users who are allowed to login --
SELECT  name                  AS LoginName,
        is_policy_checked     AS PasswordPolicyEnforced,
        is_expiration_checked AS ExpirationEnforced,
        is_disabled           AS IsDisabled,
        create_date
FROM    sys.sql_logins
WHERE   (name LIKE 'dba[_]%' OR name LIKE 'mgr[_]%' OR name LIKE 'ofc[_]%'
     OR name LIKE 'cust[_]%' OR name LIKE 'analyst[_]%' OR name = 'svc_ETL'
     OR name = 'smartbank_admin' OR name = 'sa')
ORDER BY is_disabled, name;
GO

-- Show the table that maps each login to the respective staff ID or customer ID --
USE SmartBankProd;
GO
SELECT  dp.name         AS DatabaseUser,
        sp.name         AS ServerLogin,
        m.StaffID,
        m.CustomerID
FROM    sys.database_principals AS dp
LEFT JOIN sys.server_principals AS sp ON sp.sid = dp.sid
LEFT JOIN Security.UserMapping  AS m  ON m.DatabaseUser = dp.name
WHERE   dp.type = 'S' AND dp.principal_id > 4
ORDER BY dp.name;
GO
```

	LoginName	PasswordPolicyEnforced	ExpirationEnforced	IsDisabled	create_date
1	analyst_lee	1	1	0	2026-08-19 17:15:18.717
2	cust_kumar	1	1	0	2026-08-19 17:15:18.527
3	cust_siti	1	1	0	2026-08-19 17:15:18.433
4	cust_tan	1	1	0	2026-08-19 17:15:18.337
5	cust_wong	1	1	0	2026-08-19 17:15:18.623
6	dba_ahmad	1	1	0	2026-08-19 17:15:17.860
7	mgr_aisyah	1	1	0	2026-08-19 17:15:17.960
8	mgr_ravi	1	1	0	2026-08-19 17:15:18.050
9	ofc_lim	1	1	0	2026-08-19 17:15:18.147
10	ofc_nurul	1	1	0	2026-08-19 17:15:18.243
11	smartbank_admin	1	1	0	2026-08-19 17:17:32.440
12	svc_ETL	1	0	0	2026-08-19 17:15:18.813
13	sa	1	0	1	2003-04-08 09:10:35.460

	DatabaseUser	ServerLogin	StaffID	CustomerID
1	cust_kumar	cust_kumar	NULL	C00003
2	cust_siti	cust_siti	NULL	C00002
3	cust_tan	cust_tan	NULL	C00001
4	cust_wong	cust_wong	NULL	C00004
5	dba_ahmad	dba_ahmad	S00001	NULL
6	mgr_aisyah	mgr_aisyah	S00002	NULL
7	mgr_ravi	mgr_ravi	S00003	NULL
8	ofc_lim	ofc_lim	S00004	NULL
9	ofc_nurul	ofc_nurul	S00005	NULL
10	svc_ETL	svc_ETL	NULL	NULL

Role-Based Access Control

While authentication establishes the identity for each database user, without mechanisms to define and enforce the permissions for each user, any information in the database is still vulnerable to threats like external attacks or insider misuse (Fernandez et al., 2023). Therefore, role-based access control (RBAC) is implemented in SmartBank to complement existing authentication controls by assigning permissions to users via roles, which not only centralises and simplifies the process of permission management (Butt et al., 2023), but also aligns with the principle of least privilege (PoLP) by ensuring that only necessary access is granted, thereby conforming to the confidentiality component of the CIA triad (Azad et al., 2024).

Before executing any SQL scripts to configure RBAC in SmartBank, an authorisation matrix is first designed to provide an overview of the different types of access granted to each database role in SmartBank. Tables 2 and 3 respectively define the permissions for each role in the production and analytics database, while Table 4 maps each database user to their roles.

Table 2

Authorisation Table for Production Database

Role	Permission	Object Type	Object	Privilege(s)

`role_DBA`	Grant	Database	`SmartBankProd`	`CREATE TABLE`
				`CREATE VIEW`
				`CREATE PROCEDURE`
				`CREATE FUNCTION`

`role_DBA`	Grant	Schema	`dbo`	`ALTER`
`role_DBA`	Deny	Database	`SmartBankProd`	`ALTER ANY USER` `ALTER ANY ROLE`

Role	Permission	Object Type	Object	Privilege(s)
role_DBA role_Manager role_Officer role_Customer	Deny	Table	dbo.Staff dbo.Customer dbo.Account dbo.TransactionRecord	SELECT INSERT UPDATE DELETE
role_DBA role_Manager role_Officer role_Customer	Grant	Table	Security.UserMapping	SELECT
role_DBA role_Manager role_Officer role_Customer role_ETL_Reader	Deny	Table	Security.RejectedAttemptLog dbo.StaffHistory dbo.CustomerHistory dbo.AccountHistory Security.UserMappingHistory Security.RejectedAttemptLogHistory	SELECT INSERT UPDATE DELETE
role_DBA role_Manager role_Officer role_Customer	Grant	View	dbo.vw_StaffDirectory	SELECT
role_DBA role_Manager role_Officer	Grant	View	dbo.vw_MyStaffRecord	SELECT
role_Manager	Grant	View	dbo.vw_AllStaff	SELECT
role_DBA role_Officer role_Customer	Deny	View	dbo.vw_AllStaff	SELECT

Role	Permission	Object Type	Object	Privilege(s)
role_Manager role_Officer	Grant	View	dbo.vw_CustomerProfile	SELECT
role_DBA role_Customer	Deny	View	dbo.vw_CustomerProfile	SELECT
role_Manager role_Officer role_Customer	Grant	View	dbo.vw_CustomerAccounts dbo.vw_CustomerTransactions	SELECT
role_DBA	Deny	View	dbo.vw_CustomerAccounts dbo.vw_CustomerTransactions	SELECT
role_Manager role_Officer	Grant	View	dbo.vw_CustomerIC	SELECT
role_DBA role_Customer	Deny	View	dbo.vw_CustomerIC	SELECT
role_Customer	Grant	Stored Procedure	dbo.usp_Deposit dbo.usp_Withdraw dbo.usp_Transfer	EXECUTE
role_DBA role_Manager role_Officer	Deny	Stored Procedure	dbo.usp_Deposit dbo.usp_Withdraw dbo.usp_Transfer	EXECUTE
role_DBA role_Manager role_Officer	Grant	Stored Procedure	dbo.usp_UpdateOwnStaffRecord	EXECUTE

Role	Permission	Object Type	Object	Privilege(s)
role_Customer	Deny	Stored Procedure	dbo.usp_UpdateOwnStaffRecord	EXECUTE
role_Manager	Grant	Stored Procedure	dbo.usp_UpdateStaffDetails	EXECUTE
role_DBA role_Officer role_Customer	Deny	Stored Procedure	dbo.usp_UpdateStaffDetails	EXECUTE
role_Officer	Grant	Stored Procedure	dbo.usp_CreateCustomerAccount dbo.usp_UpdateCustomerAccount dbo.usp_RegisterCustomer	EXECUTE
role_DBA role_Manager role_Customer	Deny	Stored Procedure	dbo.usp_CreateCustomerAccount dbo.usp_UpdateCustomerAccount dbo.usp_RegisterCustomer	EXECUTE
role_Manager	Grant	Stored Procedure	dbo.usp_GetCustomerIC	EXECUTE
role_DBA role_Officer role_Customer	Deny	Stored Procedure	dbo.usp_GetCustomerIC	EXECUTE
role_Manager	Grant	Symmetric Key	SK_CustomerPII	VIEW DEFINITION
role_DBA role_Officer role_Customer	Deny	Symmetric Key	SK_CustomerPII	VIEW DEFINITION
role_Manager	Grant	Certificate	Cert_CustomerPII	CONTROL

Role	Permission	Object Type	Object	Privilege(s)
role_DBA role_Officer role_Customer	Deny	Certificate	Cert_CustomerPII	CONTROL
role_Manager	Grant	Database	SmartBankProd	UNMASK
role_DBA role_Officer role_Customer	Deny	Database	SmartBankProd	UNMASK
role_ETL_Reader	Grant	Table	dbo.Customer dbo.Account dbo.TransactionRecord	SELECT
role_ETL_Reader	Deny	Table	dbo.Customer dbo.Account dbo.TransactionRecord	INSERT UPDATE DELETE
role_ETL_Reader	Deny	Table	dbo.Staff	SELECT INSERT UPDATE DELETE
[no role]	-	Function	dbo.fn_StaffDirectory Security.fn_StaffAccessPredicate Security.fn_CustomerAccessPredicate Security.fn_AccountAccessPredicate Security.fn_TransactionAccessPredicate Security.fn_UserMappingPredicate	[no grant]
[no role]	-	Stored Procedure	Security.usp_LogRejection	[no grant]

Table 3

Authorisation Table for Analytics Database

Role	Permission	Object Type	Object	Privilege(s)

`role_ETL_Service`	Grant	Schema	`Reporting`	`SELECT`
				`INSERT`
				`UPDATE`
				`DELETE`

`role_ETL_Service`	Grant	Stored Procedure	`Reporting.usp_RunETL`	`EXECUTE`

`role_ETL_Service`	Deny	View	`Reporting.vw_AccountSummary`
			`Reporting.vw_TransactionTrend`
			`Reporting.vw_CustomerActivity`
			`Reporting.vw_ETLHealth`
				`SELECT`

`role_Analyst_ReadOnly`	Deny	Schema	`Reporting`	`INSERT`
				`UPDATE`
				`DELETE`

`role_Analyst_ReadOnly`	Deny	Table	`Reporting.DimCustomer`
			`Reporting.FactAccount`
			`Reporting.FactTransaction`
			`Reporting.ETLRunLog`
				`SELECT`

`role_Analyst_ReadOnly`	Grant	View	`Reporting.vw_AccountSummary`
			`Reporting.vw_TransactionTrend`
			`Reporting.vw_CustomerActivity`
			`Reporting.vw_ETLHealth`
				`SELECT`

`role_Analyst_ReadOnly`	Deny	Stored Procedure	`Reporting.usp_RunETL`	`EXECUTE`

Table 4

Role-User Mapping Table

Roles	Users
role_DBA	dba_ahmad (S00001)
role_Manager	mgr_aisyah (S00002) mgr_ravi (S00003)
role_Officer	ofc_lim (S00004) ofc_nurul (S00005)
role_Customer	cust_tan (C00001) cust_siti (C00002) cust_kumar (C00003) cust_wong (C00004)
role_ETL_Reader	svc_ETL
role_ETL_Service	svc_ETL
role_Analyst_ReadOnly	analyst_lee
sysadmin	smartbank_admin

With the authorisation matrix defined, role-based access control (RBAC) is implemented by first creating database roles, then assigning each database user to their corresponding roles, as shown in Figure 21. An exception is the `smartbank_admin` account, which is assigned to the `sysadmin` server role in Figure 19 rather than database roles, as it requires full server-level privilege to support subsequent configurations. Therefore, after all security mechanisms are configured and implemented in SmartBank, the `smartbank_admin` account shall be restricted to perform emergency operations to minimise its usage and reduce the risk of exposing its credentials.

Figure 21

SQL Snippet to Create Roles for the Production and Analytics Database

```
-- Create roles for production DB --
USE SmartBankProd;
GO
CREATE ROLE role_DBA;
CREATE ROLE role_Manager;
CREATE ROLE role_Officer;
CREATE ROLE role_Customer;
CREATE ROLE role_ETL_Reader;
GO

-- Add users to respective roles --
ALTER ROLE role_DBA         ADD MEMBER [dba_ahmad];
ALTER ROLE role_Manager     ADD MEMBER [mgr_aisyah];
ALTER ROLE role_Manager     ADD MEMBER [mgr_ravi];
ALTER ROLE role_Officer     ADD MEMBER [ofc_lim];
ALTER ROLE role_Officer     ADD MEMBER [ofc_nurul];
ALTER ROLE role_Customer    ADD MEMBER [cust_tan];
ALTER ROLE role_Customer    ADD MEMBER [cust_siti];
ALTER ROLE role_Customer    ADD MEMBER [cust_kumar];
ALTER ROLE role_Customer    ADD MEMBER [cust_wong];
ALTER ROLE role_ETL_Reader  ADD MEMBER [svc_ETL];
GO

-- Create roles for analytics DB --
USE SmartBankAnalytics;
GO
CREATE ROLE role_ETL_Service;
CREATE ROLE role_Analyst_ReadOnly;
GO

-- Add users to respective roles --
ALTER ROLE role_ETL_Service      ADD MEMBER [svc_ETL];
ALTER ROLE role_Analyst_ReadOnly ADD MEMBER [analyst_lee];
GO
```

After establishing the roles for the production and analytics database, each role is granted and denied with their corresponding permissions following the authorisation matrix, as shown in Figure 22. Direct access to all database tables is denied for all business users to reduce the risk of them bypassing access control mechanisms to perform unauthorised operations directly on these tables. Instead, users are only allowed to perform operations through views and stored procedures to align with the need-to-know principle.

Figure 22

SQL Snippet to Grant and Deny Permission for Roles

```
-- Configure user access for production DB --
USE SmartBankProd;
GO

-- Prevent direct access to tables in production --
DENY SELECT, INSERT, UPDATE, DELETE ON dbo.Staff             TO role_DBA, role_Manager, role_Officer, role_Customer;
DENY SELECT, INSERT, UPDATE, DELETE ON dbo.Customer          TO role_DBA, role_Manager, role_Officer, role_Customer;
DENY SELECT, INSERT, UPDATE, DELETE ON dbo.Account           TO role_DBA, role_Manager, role_Officer, role_Customer;
DENY SELECT, INSERT, UPDATE, DELETE ON dbo.TransactionRecord TO role_DBA, role_Manager, role_Officer, role_Customer;

-- DBA can perform DDL tasks but not config user account --
GRANT CREATE TABLE          TO role_DBA;
GRANT CREATE VIEW           TO role_DBA;
GRANT CREATE PROCEDURE      TO role_DBA;
GRANT CREATE FUNCTION       TO role_DBA;
GRANT ALTER ON SCHEMA::dbo  TO role_DBA;
DENY ALTER ANY ROLE         TO role_DBA;
DENY ALTER ANY USER         TO role_DBA;

-- Only read access for Security.UserMapping table (for RLS) --
GRANT SELECT ON [Security].UserMapping TO role_DBA, role_Manager, role_Officer, role_Customer;
DENY SELECT, INSERT, UPDATE, DELETE ON [Security].RejectedAttemptLog
    TO role_DBA, role_Manager, role_Officer, role_Customer, role_ETL_Reader;

-- Service account can only read from production DB --
GRANT SELECT ON dbo.Customer                        TO role_ETL_Reader;
GRANT SELECT ON dbo.Account                         TO role_ETL_Reader;
GRANT SELECT ON dbo.TransactionRecord               TO role_ETL_Reader;
DENY INSERT, UPDATE, DELETE ON dbo.Customer         TO role_ETL_Reader;
DENY INSERT, UPDATE, DELETE ON dbo.Account          TO role_ETL_Reader;
DENY INSERT, UPDATE, DELETE ON dbo.TransactionRecord TO role_ETL_Reader;
DENY SELECT, INSERT, UPDATE, DELETE ON dbo.Staff    TO role_ETL_Reader;
GO

-- Configure user access for analytics DB --
USE SmartBankAnalytics;
GO

-- Only ETL service can write to Reporting --
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::Reporting TO role_ETL_Service;

-- Analysts have no direct access to raw tables --
DENY INSERT, UPDATE, DELETE ON SCHEMA::Reporting TO role_Analyst_ReadOnly;
DENY SELECT ON Reporting.DimCustomer             TO role_Analyst_ReadOnly;
DENY SELECT ON Reporting.FactAccount             TO role_Analyst_ReadOnly;
DENY SELECT ON Reporting.FactTransaction         TO role_Analyst_ReadOnly;
DENY SELECT ON Reporting.ETLRunLog               TO role_Analyst_ReadOnly;
GO
```

To verify the configuration of RBAC, the queries in Figures 23 and 24 are executed to review the existing role memberships and database permissions assigned to each role. Based on the results, it is shown that all users under the production and analytics database are assigned to the correct roles, aligning with the role-user table in Table 4. Furthermore, permissions are also assigned accurately for each of the role, where `DENY` is applied to all raw database tables for all roles, while schemas are granted correctly to database roles so that they have the appropriate access for schema-level objects like views, stored procedures, certificates, and encryption keys, which will be configured in subsequent sections. Therefore, it is demonstrated that RBAC is enforced for the production and analytics database of SmartBank, which not only aligns with data confidentiality as sensitive information is restricted to authorised users, but also enforces integrity as modifications can only be performed via specific roles (Farhadighalati et al., 2025).

Figure 23

SQL Snippet to Verify the Configuration of RBAC for Production Database

```sql
USE SmartBankProd;
GO

-- Verify the roles and members involved in the production DB --
SELECT  r.name  AS RoleName,
        m.name  AS MemberName
FROM    sys.database_role_members rm
JOIN    sys.database_principals r ON r.principal_id = rm.role_principal_id
JOIN    sys.database_principals m ON m.principal_id = rm.member_principal_id
WHERE   r.name LIKE 'role[_]%'
ORDER BY r.name, m.name;
GO

-- Verify if objects in production DB are granted to correct role and access --
SELECT  p.class_desc,
        CASE p.class_desc
            WHEN 'OBJECT_OR_COLUMN' THEN OBJECT_NAME(p.major_id)
            WHEN 'SCHEMA'           THEN SCHEMA_NAME(p.major_id)
            WHEN 'DATABASE'         THEN DB_NAME()
        END                         AS ObjectName,
        dp.name                     AS RoleName,
        p.permission_name,
        p.state_desc                AS GrantOrDeny
FROM    sys.database_permissions p
JOIN    sys.database_principals dp ON dp.principal_id = p.grantee_principal_id
WHERE   dp.name LIKE 'role[_]%'
ORDER BY dp.name, ObjectName, p.permission_name;
GO
```

	RoleName	MemberName
1	role_Customer	cust_kumar
2	role_Customer	cust_siti
3	role_Customer	cust_tan
4	role_Customer	cust_wong
5	role_DBA	dba_ahmad
6	role_ETL_Reader	svc_ETL
7	role_Manager	mgr_aisyah
8	role_Manager	mgr_ravi
9	role_Officer	ofc_lim
10	role_Officer	ofc_nurul

	class_desc	ObjectName	RoleName	permission_name	GrantOrDeny
1	OBJECT_OR_COLUMN	Account	role_Customer	DELETE	DENY
2	OBJECT_OR_COLUMN	Account	role_Customer	INSERT	DENY
3	OBJECT_OR_COLUMN	Account	role_Customer	SELECT	DENY
4	OBJECT_OR_COLUMN	Account	role_Customer	UPDATE	DENY
5	OBJECT_OR_COLUMN	Customer	role_Customer	DELETE	DENY
6	OBJECT_OR_COLUMN	Customer	role_Customer	INSERT	DENY
7	OBJECT_OR_COLUMN	Customer	role_Customer	SELECT	DENY
8	OBJECT_OR_COLUMN	Customer	role_Customer	UPDATE	DENY
9	OBJECT_OR_COLUMN	RejectedAttemptLog	role_Customer	DELETE	DENY
10	OBJECT_OR_COLUMN	RejectedAttemptLog	role_Customer	INSERT	DENY
11	OBJECT_OR_COLUMN	RejectedAttemptLog	role_Customer	SELECT	DENY
12	OBJECT_OR_COLUMN	RejectedAttemptLog	role_Customer	UPDATE	DENY
13	OBJECT_OR_COLUMN	Staff	role_Customer	DELETE	DENY
14	OBJECT_OR_COLUMN	Staff	role_Customer	INSERT	DENY
15	OBJECT_OR_COLUMN	Staff	role_Customer	SELECT	DENY
16	OBJECT_OR_COLUMN	Staff	role_Customer	UPDATE	DENY
17	OBJECT_OR_COLUMN	TransactionRecord	role_Customer	DELETE	DENY
18	OBJECT_OR_COLUMN	TransactionRecord	role_Customer	INSERT	DENY
19	OBJECT_OR_COLUMN	TransactionRecord	role_Customer	SELECT	DENY
20	OBJECT_OR_COLUMN	TransactionRecord	role_Customer	UPDATE	DENY
21	OBJECT_OR_COLUMN	UserMapping	role_Customer	SELECT	GRANT
22	OBJECT_OR_COLUMN	Account	role_DBA	DELETE	DENY
23	OBJECT_OR_COLUMN	Account	role_DBA	INSERT	DENY
24	OBJECT_OR_COLUMN	Account	role_DBA	SELECT	DENY
25	OBJECT_OR_COLUMN	Account	role_DBA	UPDATE	DENY
26	OBJECT_OR_COLUMN	Customer	role_DBA	DELETE	DENY
27	OBJECT_OR_COLUMN	Customer	role_DBA	INSERT	DENY
28	OBJECT_OR_COLUMN	Customer	role_DBA	SELECT	DENY
29	OBJECT_OR_COLUMN	Customer	role_DBA	UPDATE	DENY
30	SCHEMA	dbo	role_DBA	ALTER	GRANT

Figure 24

SQL Snippet to Verify the Configuration of RBAC for Analytics Database

```sql
USE SmartBankAnalytics;
GO

-- Check the users assigned under each role in analytics DB --
SELECT  r.name  AS RoleName,
        m.name  AS MemberName
FROM    sys.database_role_members rm
JOIN    sys.database_principals r ON r.principal_id = rm.role_principal_id
JOIN    sys.database_principals m ON m.principal_id = rm.member_principal_id
WHERE   r.name LIKE 'role[_]%'
ORDER BY r.name, m.name;
GO

-- Verify if the objects in analytics DB are allocated to the correct role and access --
SELECT  p.class_desc,
        CASE p.class_desc
            WHEN 'OBJECT_OR_COLUMN' THEN OBJECT_NAME(p.major_id)
            WHEN 'SCHEMA'           THEN SCHEMA_NAME(p.major_id)
            WHEN 'DATABASE'         THEN DB_NAME()
        END                         AS ObjectName,
        dp.name                     AS RoleName,
        p.permission_name,
        p.state_desc                AS GrantOrDeny
FROM    sys.database_permissions p
JOIN    sys.database_principals dp ON dp.principal_id = p.grantee_principal_id
WHERE   dp.name LIKE 'role[_]%'
ORDER BY dp.name, ObjectName, p.permission_name;
GO
```

	RoleName	MemberName
1	role_Analyst_ReadOnly	analyst_lee
2	role_ETL_Service	svc_ETL

	class_desc	ObjectName	RoleName	permission_name	GrantOrDeny
1	OBJECT_OR_COLUMN	DimCustomer	role_Analyst_ReadOnly	SELECT	DENY
2	OBJECT_OR_COLUMN	ETLRunLog	role_Analyst_ReadOnly	SELECT	DENY
3	OBJECT_OR_COLUMN	FactAccount	role_Analyst_ReadOnly	SELECT	DENY
4	OBJECT_OR_COLUMN	FactTransaction	role_Analyst_ReadOnly	SELECT	DENY
5	SCHEMA	Reporting	role_Analyst_ReadOnly	DELETE	DENY
6	SCHEMA	Reporting	role_Analyst_ReadOnly	INSERT	DENY
7	SCHEMA	Reporting	role_Analyst_ReadOnly	UPDATE	DENY
8	SCHEMA	Reporting	role_ETL_Service	DELETE	GRANT
9	SCHEMA	Reporting	role_ETL_Service	INSERT	GRANT
10	SCHEMA	Reporting	role_ETL_Service	SELECT	GRANT
11	SCHEMA	Reporting	role_ETL_Service	UPDATE	GRANT

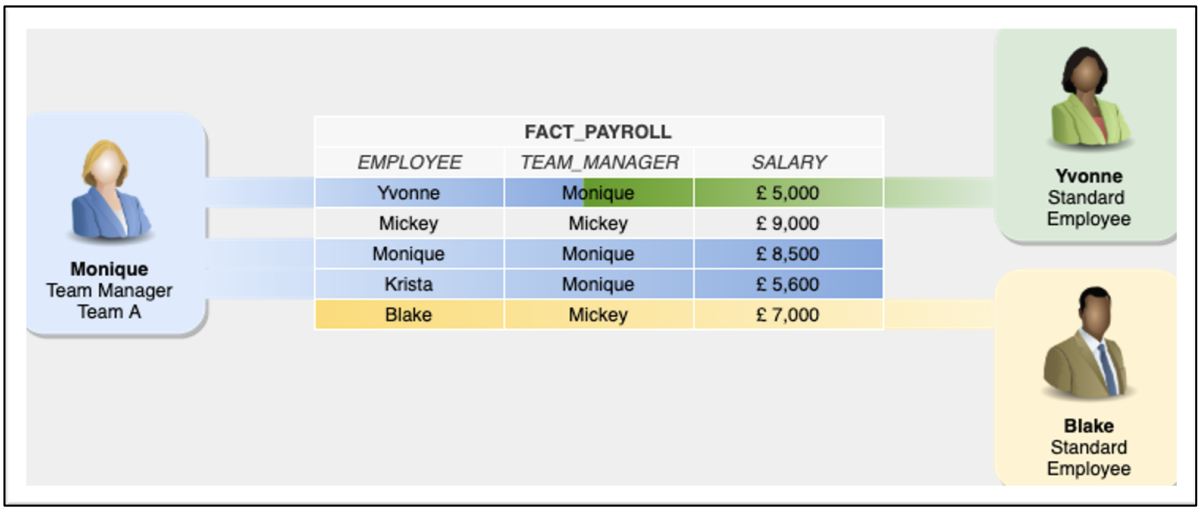
Row-Level Security

Although role-based access control (RBAC) is able to limit the database objects that a user can access, it is insufficient alone to fully protect the database, as authorised users are still able to view others' sensitive information despite these records are irrelevant to them (Singh, 2026b). Taking SmartBank as an example, when customers view the `dbo.TransactionRecord` table, current configuration allows them to view all the records in the table, including transactions that they are not involved in, hence exposing other customers' information and misaligning with the need-to-know principle. Therefore, to enforce data confidentiality, row-level security (RLS) is implemented to complement the existing access control mechanisms.

Row-level security (RLS) prevents users from reading, updating, or deleting rows that are irrelevant to them. Taking Figure 25 as an example, when RLS is implemented on the `Fact_Payroll` table, Yvonne will be able to only view her record at the first row (*highlighted in green*), while Monique will be able to view multiple records that contain her name (*highlighted in blue*). In SQL server, RLS is enforced automatically once the corresponding predicate functions and security policies are defined, which helps to reduce data exposure even if access control mechanisms are misconfigured (Karthik, 2025). There are two types of security predicates to support RLS in SQL server, namely the `FILTER PREDICATE` that hides irrelevant rows from users, as well as the `BLOCK PREDICATE` that prevents users from performing modification towards any rows that they do not have access to (Johnston, 2023).

Figure 25

An Example of Row-Level Security Applied



Note. From *How to configure Row Level Security - A Basic Example*, by M. Beek, 2021 (https://community.thoughtspot.com/customers/s/article/Row-Level-Security-A-Basic-Example).

In SmartBank, row-level security (RLS) is applied to tables in the production database:

- **`dbo.Customer`**

  As shown in Figure 26, RLS is applied to `dbo.Customer` to ensure that customers are only able to view their own records but not the records of other customers to preserve data confidentiality. However, for SmartBank staff like bank managers and officers, they are required to view the full `dbo.Customer` table to perform business operations like loan approvals and customer support. Therefore, when the `dbo.Customer` table is accessed, the predicate function first validates the identity of users by checking their role membership using the `IS_MEMBER()` function. If the users are neither system administrators, bank managers, nor bank officers, the predicate function will then filter the rows in the table by only returning the records that matches the `CustomerID` of the user. This ensures that customers are able to view their own records, while other unauthorised users will be unable to view others' records. The predicate function is then attached to the `dbo.Customer` table via a security policy named `CustomerSecurityPolicy`. Aside from implementing the `FILTER PREDICATE` to hide irrelevant rows from users, the policy also implements a `BLOCK PREDICATE` to prevent any `UPDATE` operations from being applied by users to circumvent the enforcement of RLS and gain unauthorised access to other customers' data.

Figure 26

Row-Level Security Implemented on `dbo.Customer` *Table*

```
CREATE FUNCTION [Security].fn_CustomerAccessPredicate (@CustomerID AS VARCHAR(6))
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN
    SELECT 1 AS fn_AccessResult
    WHERE
           USER_NAME() = 'dbo'
        OR IS_MEMBER('db_owner') = 1
        OR IS_MEMBER('role_ETL_Reader') = 1
        OR IS_MEMBER('role_Manager') = 1
        OR IS_MEMBER('role_Officer') = 1
        OR EXISTS (
              SELECT 1
              FROM   Security.UserMapping AS m
              WHERE  m.DatabaseUser = USER_NAME()
                AND  m.CustomerID   = @CustomerID
          );
GO

CREATE SECURITY POLICY [Security].CustomerSecurityPolicy
    ADD FILTER PREDICATE [Security].fn_CustomerAccessPredicate(CustomerID) ON dbo.Customer,
    ADD BLOCK  PREDICATE [Security].fn_CustomerAccessPredicate(CustomerID) ON dbo.Customer AFTER UPDATE
WITH (STATE = ON);
GO
```

- **`dbo.Account`**

  Similar to the `dbo.Customer` table, `dbo.Account` follows the same configuration for RLS, where staff members like managers and officers are able to view the full table to perform their duties, while customers are only able to view the accounts associated with their `CustomerID`, as shown in Figure 27. The `AccountSecurityPolicy` policy also filters irrelevant rows and blocks invalid updates to enforce data confidentiality.

Figure 27

Row-Level Security Implemented on `dbo.Account` *Table*

```
-- Predicate function for accounts --
CREATE FUNCTION [Security].fn_AccountAccessPredicate (@CustomerID AS VARCHAR(6))
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN
    SELECT 1 AS fn_AccessResult
    WHERE
           USER_NAME() = 'dbo'
        OR IS_MEMBER('db_owner') = 1
        OR IS_MEMBER('role_ETL_Reader') = 1
        OR IS_MEMBER('role_Manager') = 1
        OR IS_MEMBER('role_Officer') = 1
        OR EXISTS (
               SELECT 1
               FROM   Security.UserMapping AS m
               WHERE  m.DatabaseUser = USER_NAME()
                 AND  m.CustomerID   = @CustomerID
           );
GO

-- Security policy for accounts --
CREATE SECURITY POLICY [Security].AccountSecurityPolicy
    ADD FILTER PREDICATE [Security].fn_AccountAccessPredicate(CustomerID) ON dbo.Account,
    ADD BLOCK  PREDICATE [Security].fn_AccountAccessPredicate(CustomerID) ON dbo.Account AFTER UPDATE
WITH (STATE = ON);
GO
```

- **`dbo.TransactionRecord`**

  As shown in Figure 28, the RLS implemented for the `dbo.TransactionRecord` table is similar to the ones implemented for `dbo.Customer` and `dbo.Account` tables, where staff members like managers and officers are able to view the full transaction records, while customers are only allowed to view the transactions that are related to their accounts. Both the `FILTER PREDICATE` and `BLOCK PREDICATE` predicates are also implemented similarly as the other tables. However, the `dbo.TransactionRecord` table implements an additional `AFTER INSERT` predicate to prevent users from inserting any transaction records that do not belong to their customer IDs. This is to ensure the confidentiality and integrity of the `dbo.TransactionRecord` table, where authorised users are only able to view relevant records but not insert any transactional data that does not belong to them, hence ensuring the accuracy and reliability of financial data in SmartBank.

Figure 28

Row-Level Security Implemented on `dbo.TransactionRecord` *Table*

```
-- Predicate function for transaction records --
CREATE FUNCTION [Security].fn_TransactionAccessPredicate
(
    @AccountID            AS VARCHAR(10),
    @ToAccountID AS VARCHAR(10)
)
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN
    SELECT 1 AS fn_AccessResult
    WHERE
           USER_NAME() = 'dbo'
        OR IS_MEMBER('db_owner') = 1
        OR IS_MEMBER('role_ETL_Reader') = 1
        OR IS_MEMBER('role_Manager') = 1
        OR IS_MEMBER('role_Officer') = 1
        OR EXISTS (
               SELECT 1
               FROM   Security.UserMapping AS m
               INNER JOIN dbo.Account AS a
                       ON a.CustomerID = m.CustomerID
               WHERE  m.DatabaseUser = USER_NAME()
                 AND  a.AccountID IN (@AccountID, @ToAccountID)
           );
GO

-- Security policy for transaction records --
CREATE SECURITY POLICY [Security].TransactionSecurityPolicy
    ADD FILTER PREDICATE [Security].fn_TransactionAccessPredicate(AccountID, ToAccountID)
    ON dbo.TransactionRecord,
    ADD BLOCK PREDICATE [Security].fn_TransactionAccessPredicate(AccountID, ToAccountID)
    ON dbo.TransactionRecord AFTER INSERT,
    ADD BLOCK PREDICATE [Security].fn_TransactionAccessPredicate(AccountID, ToAccountID)
    ON dbo.TransactionRecord AFTER UPDATE
WITH (STATE = ON);
GO
```

- **`dbo.Staff`**

  Unlike customer-related tables like `dbo.Customer` and `dbo.Account`, the `dbo.Staff` table requires different permissions, where only privileged roles like bank managers are able to view the full table. On the other hand, staff members like database administrators and bank officers can only view their own records. Therefore, as shown in Figure 29, the `StaffID` and `USER_NAME()` are compared with details from `Security.UserMapping` to filter out irrelevant records before displaying them to staff members like bank officers and database administrators. Once the predicate function is declared, it is then attached to the `dbo.Staff` table via the `StaffSecurityPolicy` policy, with `FILTER PREDICATE` and `BLOCK PREDICATE ... AFTER UPDATE` implemented to prevent staff members from performing any update operations to replace their `StaffID` with the identifier of another employee to bypass RLS and view other employees' information in an unauthorised manner.

Figure 29

Row-Level Security Implemented on `dbo.Staff` *Table*

```
-- Predicate function for staff --
CREATE FUNCTION [Security].fn_StaffAccessPredicate (@StaffID AS VARCHAR(6))
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN
    SELECT 1 AS fn_AccessResult
    WHERE
           USER_NAME() = 'dbo'
        OR IS_MEMBER('db_owner') = 1
        OR IS_MEMBER('role_Manager') = 1
        OR EXISTS (
              SELECT 1
              FROM   Security.UserMapping AS m
              WHERE  m.DatabaseUser = USER_NAME()
                AND  m.StaffID      = @StaffID
          );
GO

-- Security policy for staff --
CREATE SECURITY POLICY [Security].StaffSecurityPolicy
    ADD FILTER PREDICATE [Security].fn_StaffAccessPredicate(StaffID) ON dbo.Staff,
    ADD BLOCK  PREDICATE [Security].fn_StaffAccessPredicate(StaffID) ON dbo.Staff AFTER UPDATE
WITH (STATE = ON);
GO
```

- **`Security.UserMapping`**

  RLS is also applied to the `Security.UserMapping` table to ensure that the table does not expose the login and identity information of other database users, even if unrestricted access is granted to users. As shown in Figure 30, `fn_UserMappingPredicate` ensures that only system administrators are able to view all mappings, while other business users like staff members and customers can only view their own rows from the table. The security policy also applies a `FILTER PREDICATE` to enforce the RLS restriction on the table.

Figure 30

Row-Level Security Implemented on `Security.UserMapping` *Table*

```
-- Predicate function for User Mapping table --
CREATE FUNCTION [Security].fn_UserMappingPredicate (@DatabaseUser AS SYSNAME)
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN
    SELECT 1 AS fn_AccessResult
    WHERE
           USER_NAME() = 'dbo'
        OR IS_MEMBER('db_owner')    = 1
        OR @DatabaseUser = USER_NAME();
GO

-- Security policy for User Mapping table --
CREATE SECURITY POLICY [Security].UserMappingSecurityPolicy
    ADD FILTER PREDICATE [Security].fn_UserMappingPredicate(DatabaseUser) ON [Security].UserMapping
WITH (STATE = ON);
GO
```

After row-level security (RLS) is applied to the tables in the production database, the code in Figure 31 is executed to inspect the configurations of RLS. Several tests are also performed to ensure that RLS is working as intended in SmartBank. As shown in Figure 32, the role `role_Customer` is temporarily granted the `SELECT` permission on the `dbo.Account` table so that `cust_tan` and `cust_siti` can be used to observe the output of RLS-enforced tables. Meanwhile, the `svc_ETL` account is also tested to make sure that it can view all the rows in the production database to facilitate the ETL pipeline.

Figure 31

SQL Snippet to Verify the Configuration of Row-Level Security

```
USE SmartBankProd;
GO

-- Display the security policies that are being enforced --
SELECT  sp.name                             AS PolicyName,
        sp.is_enabled,
        OBJECT_NAME(spr.target_object_id)   AS ProtectedTable,
        spr.predicate_type_desc,
        spr.predicate_definition
FROM    sys.security_policies AS sp
JOIN    sys.security_predicates AS spr
     ON  spr.object_id = sp.object_id
ORDER BY PolicyName, ProtectedTable;
GO
```

Figure 32

SQL Snippet to Demonstrate that Row-Level Security Performs as Intended

```
-- A simple demo for RLS --
GRANT SELECT ON dbo.Account TO role_Customer;
GO

EXECUTE AS USER = 'cust_tan';
    SELECT USER_NAME() AS RunningAs, AccountID, CustomerID, Balance FROM dbo.Account;
REVERT;
GO

EXECUTE AS USER = 'cust_siti';
    SELECT USER_NAME() AS RunningAs, AccountID, CustomerID, Balance FROM dbo.Account;
REVERT;
GO

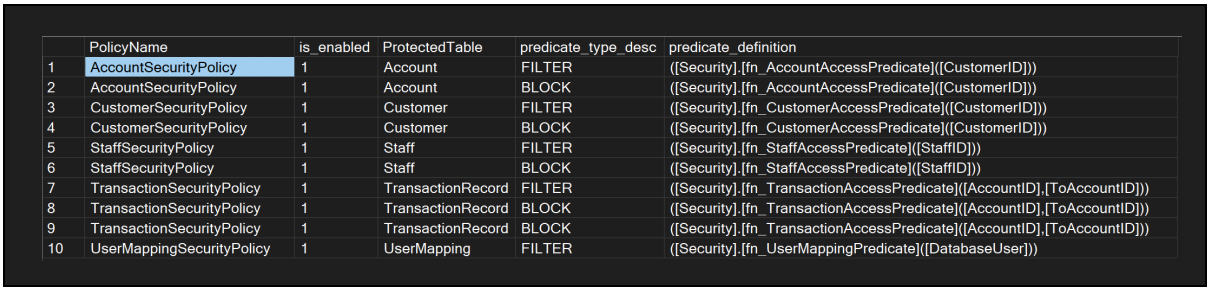
DENY SELECT ON dbo.Account TO role_Customer;
GO

-- Ensure that the svc_ETL account is able to access all data to support ETL pipeline --
EXECUTE AS USER = 'svc_ETL';
    SELECT  USER_NAME()                                     AS RunningAs,
            (SELECT COUNT(*) FROM dbo.Customer)             AS CustomersVisible,
            (SELECT COUNT(*) FROM dbo.Account)              AS AccountsVisible,
            (SELECT COUNT(*) FROM dbo.TransactionRecord)    AS TransactionsVisible;
REVERT;
GO
```

Based on the results in Figure 33, it is evident that all RLS policies are enabled, with filter and block predicates correctly assigned to each table. The results in Figure 34 further proves that customers are only able to view their own account details, while `svc_ETL` is able to view all rows. These results demonstrate that RLS is enforced correctly in SmartBank, contributing to the data confidentiality and integrity in SmartBank.

Figure 33

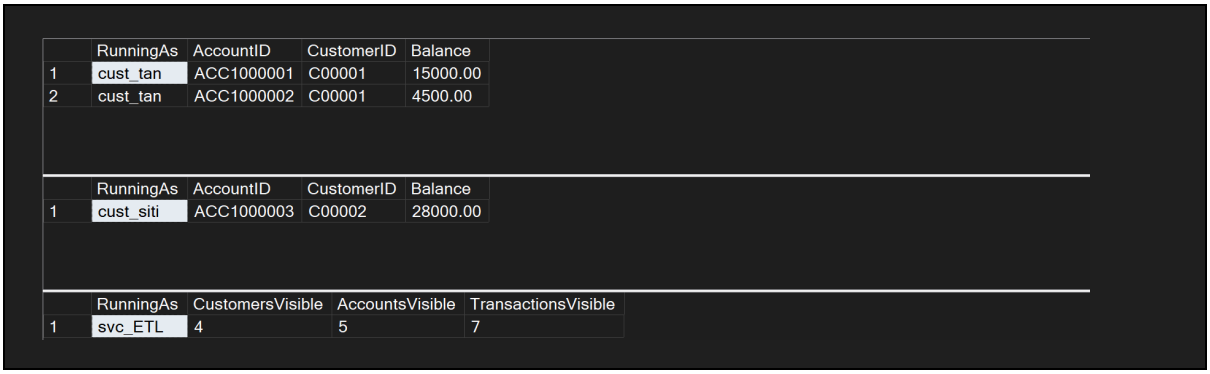
Result Table for the Configuration of Row-Level Security



	PolicyName	is_enabled	ProtectedTable	predicate_type_desc	predicate_definition
1	AccountSecurityPolicy	1	Account	FILTER	([Security].[fn_AccountAccessPredicate]([CustomerID]))
2	AccountSecurityPolicy	1	Account	BLOCK	([Security].[fn_AccountAccessPredicate]([CustomerID]))
3	CustomerSecurityPolicy	1	Customer	FILTER	([Security].[fn_CustomerAccessPredicate]([CustomerID]))
4	CustomerSecurityPolicy	1	Customer	BLOCK	([Security].[fn_CustomerAccessPredicate]([CustomerID]))
5	StaffSecurityPolicy	1	Staff	FILTER	([Security].[fn_StaffAccessPredicate]([StaffID]))
6	StaffSecurityPolicy	1	Staff	BLOCK	([Security].[fn_StaffAccessPredicate]([StaffID]))
7	TransactionSecurityPolicy	1	TransactionRecord	FILTER	([Security].[fn_TransactionAccessPredicate]([AccountID],[ToAccountID]))
8	TransactionSecurityPolicy	1	TransactionRecord	BLOCK	([Security].[fn_TransactionAccessPredicate]([AccountID],[ToAccountID]))
9	TransactionSecurityPolicy	1	TransactionRecord	BLOCK	([Security].[fn_TransactionAccessPredicate]([AccountID],[ToAccountID]))
10	UserMappingSecurityPolicy	1	UserMapping	FILTER	([Security].[fn_UserMappingPredicate]([DatabaseUser]))

Figure 34

Result Table to Verify that Row-Level Security Performs as Intended



	RunningAs	AccountID	CustomerID	Balance
1	cust_tan	ACC1000001	C00001	15000.00
2	cust_tan	ACC1000002	C00001	4500.00

	RunningAs	AccountID	CustomerID	Balance
1	cust_siti	ACC1000003	C00002	28000.00

	RunningAs	CustomersVisible	AccountsVisible	TransactionsVisible
1	svc_ETL	4	5	7

Views

While row-level security (RLS) controls the rows that a user can access, views are equally important as they determine the attributes that a user can view from a table. Taking SmartBank as an example, when customers view the details of a staff member, they shall be able to view non-sensitive information like `Name` and `Branch`, but not financial details like `Salary` which are confidential. Therefore, to protect personal and financial information from being disclosed, views play a crucial role in aligning with the principle of need-to-know by limiting users to only view the information required for their duties, not only complementing RLS by providing a more comprehensive security protection for the database tables in SmartBank, but also contributing to the confidentiality element of the CIA triad (McCallister et al., 2010).

Based on the authorisation matrix and business requirements, several views are implemented for SmartBank, including:

- **`dbo.vw_StaffDirectory`**

  As shown in Figure 35, the `dbo.vw_StaffDirectory` view allows authorised users to view the information of staff members in SmartBank, including `StaffID`, `StaffName`, `Phone` and `Branch` without sensitive columns like `Salary`. However, as row-level security (RLS) is enforced on the `dbo.Staff` table, it is necessary to create a helper function that allows `EXECUTE AS OWNER` to be specified, so that all staff records can be passed to the view to allow authorised users view all records without any RLS restrictions.

Figure 35

SQL Code for Function `dbo.fn_StaffDirectory` *and View* `dbo.vw_StaffDirectory`

```
CREATE FUNCTION dbo.fn_StaffDirectory()
RETURNS @Directory TABLE (
    StaffID   VARCHAR(6),
    StaffName VARCHAR(100),
    Phone     VARCHAR(20),
    Branch    VARCHAR(50)
)
WITH EXECUTE AS OWNER
AS BEGIN
    INSERT INTO @Directory (StaffID, StaffName, Phone, Branch)
    SELECT StaffID, StaffName, Phone, Branch
    FROM   dbo.Staff;
    RETURN;
END;
GO

CREATE VIEW dbo.vw_StaffDirectory
AS
    SELECT StaffID, StaffName, Phone, Branch
    FROM   dbo.fn_StaffDirectory();
GO
```

- **`dbo.vw_MyStaffRecord`**

  The `dbo.vw_MyStaffRecord` view allows staff members of SmartBank to view their own staff record, including personal and financial details like `StaffName`, `Position`, and `Salary`, as shown by the code in Figure 36. By performing `INNER JOIN` with the `Security.UserMapping` table, regardless of the user's role, only their own record will be displayed. This ensures that database confidentiality is enforced, even when the view is queried by privileged users who have the permission to view all staff records.

  Figure 36

  SQL Code for View `dbo.vw_MyStaffRecord`

```
CREATE VIEW dbo.vw_MyStaffRecord AS
    SELECT s.StaffID, s.StaffName, s.Position, s.Branch, s.Phone, s.Salary
    FROM   dbo.Staff AS s
    INNER JOIN Security.UserMapping AS m ON s.StaffID = m.StaffID
    WHERE  m.DatabaseUser = USER_NAME();
GO
```

- **`dbo.vw_AllStaff`**

  The `dbo.vw_AllStaff` view in Figure 37 is similar to `dbo.vw_MyStaffRecord`, except that it is intended to be queried by bank managers to retrieve all staff records with complete details. Therefore, the permission to query this view will be assigned to the `role_Manager` role only to align with the role-based access control (RBAC) mechanism.

  Figure 37

  SQL Code for View `dbo.vw_AllStaff`

```
CREATE VIEW dbo.vw_AllStaff AS
    SELECT StaffID, StaffName, Position, Branch, Phone, Salary
    FROM   dbo.Staff;
GO
```

- **`dbo.vw_CustomerProfile`**

  The `dbo.vw_CustomerProfile` view is created to allow officers and managers to view customer information like `CustomerName`, `Phone` and `Address` to support their daily operations, as shown in Figure 38. The IC number is explicitly excluded from the view as it is classified as confidential information that can only be decrypted by managers, which is the only role that holds the permission to open the encryption key. Hence, managers will retrieve customers' IC number using another view named `dbo.vw_CustomerIC`, which is defined later under the section of column-level encryption (CLE).

Figure 38

SQL Code for View `dbo.vw_CustomerProfile`

```
CREATE VIEW dbo.vw_CustomerProfile
AS
    SELECT CustomerID, CustomerName, Phone, [Address]
    FROM   dbo.Customer;
GO
```

- **`dbo.vw_CustomerAccounts`**

  The `dbo.vw_CustomerAccounts` view in Figure 39 inherits the row-level security (RLS) enforced on the `dbo.Account` table, which allows managers and officers to view the full records of customer accounts, while customers are only able to view their own records.

Figure 39

SQL Code for View `dbo.vw_CustomerAccounts`

```
CREATE VIEW dbo.vw_CustomerAccounts
AS
    SELECT  a.AccountID, a.CustomerID, a.AccountType, a.Balance
    FROM    dbo.Account AS a;
GO
```

- **`dbo.vw_CustomerTransactions`**

  Similar with `dbo.vw_CustomerAccounts`, the `dbo.vw_CustomerTransactions` view in Figure 40 also inherits row-level security (RLS) from the `dbo.TransactionRecord` table, allowing bank managers and officers to view the full transaction records, while customers are only able to view the records that they are involved in. The `PerformedBy` column is excluded from the view as it is mainly used for internal audit tracking but not required for business operations, hence aligning with the need-to-know principle.

Figure 40

SQL Code for View `dbo.vw_CustomerTransactions`

```
CREATE VIEW dbo.vw_CustomerTransactions
AS
    SELECT  t.TransID,
            t.AccountID,
            t.TransDate,
            t.Amount,
            t.TransactionType,
            t.ToAccountID
    FROM    dbo.TransactionRecord AS t;
GO
```

After views are created, to ensure that authorised users gain the permissions to access these views, the code in Figure 41 is executed to perform `GRANT` and `DENY` operations according to the authorisation matrix. This is to ensure that database users are still able to access data despite not having direct access to the database tables, hence complying with the role-based access control (RBAC) mechanisms while ensuring database confidentiality and availability.

Figure 41

SQL Snippet to Grant and Deny Permissions for Views

```
GRANT SELECT ON dbo.vw_StaffDirectory       TO role_DBA, role_Manager, role_Officer, role_Customer;
GRANT SELECT ON dbo.vw_MyStaffRecord        TO role_DBA, role_Manager, role_Officer;
GRANT SELECT ON dbo.vw_AllStaff             TO role_Manager;
DENY  SELECT ON dbo.vw_AllStaff             TO role_DBA, role_Officer, role_Customer;
GRANT SELECT ON dbo.vw_CustomerProfile      TO role_Manager, role_Officer;
DENY  SELECT ON dbo.vw_CustomerProfile      TO role_DBA, role_Customer;
GRANT SELECT ON dbo.vw_CustomerAccounts     TO role_Manager, role_Officer, role_Customer;
DENY  SELECT ON dbo.vw_CustomerAccounts     TO role_DBA;
GRANT SELECT ON dbo.vw_CustomerTransactions TO role_Manager, role_Officer, role_Customer;
DENY  SELECT ON dbo.vw_CustomerTransactions TO role_DBA;
```

The configuration for views is then reviewed using the code in Figure 42. Based on the result, it is shown that each of the view is configured with the correct permission, aligning with the authorisation matrix.

Figure 42

SQL Snippet to Verify the Configuration of Views in SmartBank

```
SELECT  v.name  AS ViewName,
        dp.name AS RoleName,
        p.permission_name,
        p.state_desc AS GrantOrDeny
FROM    sys.database_permissions p
JOIN    sys.views v                 ON v.object_id = p.major_id
JOIN    sys.database_principals dp ON dp.principal_id = p.grantee_principal_id
ORDER BY v.name, dp.name;
GO
```

	ViewName	RoleName	permission_name	GrantOrDeny
1	vw_AllStaff	role_Customer	SELECT	DENY
2	vw_AllStaff	role_DBA	SELECT	DENY
3	vw_AllStaff	role_Manager	SELECT	GRANT
4	vw_AllStaff	role_Officer	SELECT	DENY
5	vw_CustomerAccounts	role_Customer	SELECT	GRANT
6	vw_CustomerAccounts	role_DBA	SELECT	DENY
7	vw_CustomerAccounts	role_Manager	SELECT	GRANT
8	vw_CustomerAccounts	role_Officer	SELECT	GRANT
9	vw_CustomerProfile	role_Customer	SELECT	DENY
10	vw_CustomerProfile	role_DBA	SELECT	DENY
11	vw_CustomerProfile	role_Manager	SELECT	GRANT
12	vw_CustomerProfile	role_Officer	SELECT	GRANT
13	vw_CustomerTransactions	role_Customer	SELECT	GRANT
14	vw_CustomerTransactions	role_DBA	SELECT	DENY
15	vw_CustomerTransactions	role_Manager	SELECT	GRANT
16	vw_CustomerTransactions	role_Officer	SELECT	GRANT
17	vw_MyStaffRecord	role_DBA	SELECT	GRANT
18	vw_MyStaffRecord	role_Manager	SELECT	GRANT
19	vw_MyStaffRecord	role_Officer	SELECT	GRANT
20	vw_StaffDirectory	role_Customer	SELECT	GRANT
21	vw_StaffDirectory	role_DBA	SELECT	GRANT
22	vw_StaffDirectory	role_Manager	SELECT	GRANT
23	vw_StaffDirectory	role_Officer	SELECT	GRANT

Several tests are also performed to validate if users are able to access the created views. As shown in Figure 43, when `ofc_lim` attempts to query the `dbo.vw_StaffDirectory` view, all the five staff directory entries are extracted with attributes like `StaffName`, `Phone` and `Branch` displayed without the `Salary` column, indicating that `dbo.vw_StaffDirectory` is able to display all staff records without being affected by RLS. Also, when `cust_tan` and `cust_siti` perform query on the `dbo.vw_CustomerTransactions` view, each customer is only able to view their own records, indicating that the view inherits the RLS mechanism enforced on the `dbo.TransactionRecord` table. As the results align with the expected output, this indicates that views are properly configured in SmartBank, not only aligning with the principle of least privilege (PoLP) as the access towards database information is restricted to authorised users only, but also enforcing the confidentiality component of the CIA triad since sensitive data is hidden from unauthorised users.

Figure 43

SQL Snippet to Demonstrate that Views Perform as Intended in SmartBank

```
EXECUTE AS USER = 'ofc_lim';
    SELECT USER_NAME() AS RunningAs, * FROM dbo.vw_StaffDirectory;
REVERT;
GO

EXECUTE AS USER = 'cust_tan';
    SELECT USER_NAME() AS RunningAs, * FROM dbo.vw_CustomerTransactions;
REVERT;
GO

EXECUTE AS USER = 'cust_siti';
    SELECT USER_NAME() AS RunningAs, * FROM dbo.vw_CustomerTransactions;
REVERT;
GO
```

	RunningAs	StaffID	StaffName	Phone	Branch
1	ofc_lim	S00001	Ahmad bin Hassan	012-3450001	HQ Kuala Lumpur
2	ofc_lim	S00002	Aisyah binti Rahman	012-3450002	Petaling Jaya
3	ofc_lim	S00003	Ravi Subramaniam	012-3450003	Shah Alam
4	ofc_lim	S00004	Lim Wei Sheng	012-3450004	Petaling Jaya
5	ofc_lim	S00005	Nurul Izzah	012-3450005	Shah Alam

	RunningAs	TransID	AccountID	TransDate	Amount	TransactionType	ToAccountID
1	cust_tan	1	ACC1000001	2026-07-01 09:15:00	2000.00	Deposit	NULL
2	cust_tan	2	ACC1000001	2026-07-05 14:02:00	350.00	Withdrawal	NULL
3	cust_tan	4	ACC1000001	2026-07-08 09:30:00	500.00	Transfer	ACC1000002

	RunningAs	TransID	AccountID	TransDate	Amount	TransactionType	ToAccountID
1	cust_siti	3	ACC1000003	2026-07-06 11:40:00	5000.00	Deposit	NULL
2	cust_siti	6	ACC1000003	2026-07-11 13:50:00	1000.00	Transfer	ACC1000005

Integrity Controls

Although implementing access control mechanisms restricts the privilege of each role towards objects inside the production and analytics database, this does not completely ensure data integrity as authorised users may still make human errors when handling data and tables, such as inputting erroneous data into tables, creating opportunity for breaches like financial data corruption to take place (Amoresano & Yankson, 2023). Therefore, integrity controls are also applied in SmartBank's database to ensure the accuracy and reliability of the database.

Constraints

To maintain data consistency and integrity, constraints are implemented onto database tables to enforce validation rules and prevent users from entering invalid data (Dharani, 2025). At the data initialisation phase, several core constraints, including `PRIMARY KEY` and `NOT NULL`, are defined to ensure that records can be uniquely identified without containing any missing information, thereby maintaining the entity and domain integrity of the database. In addition to that, several advanced constraints are also defined for the tables in the production and analytics database to improve data reliability, including:

- **Foreign Key Constraints**

  To maintain referential integrity in a relational database, foreign key constraints are widely implemented to ensure that values of a column can correspond to an existing record in another table (Slawinska, 2021). From the perspective of database security, having such a control is crucial to prevent having orphaned records, such as having transaction records associated with invalid customer accounts, thereby allowing SmartBank to align with the integrity component of the CIA triad to ensure data accuracy and trustworthiness. As shown in Figure 44 and 45, foreign key constraints are implemented in both the production and analytics database of SmartBank, not only to maintain consistency across different data environments, but also to ensure that invalid values will not be inserted into the columns that are protected by foreign keys. As an example, when a new record is inserted into the `dbo.TransactionRecord` table, both its `AccountID` and `ToAccountID` column must consist of existing account IDs in the `dbo.Account` table. Any attempts to remove records in `dbo.Account` will also be checked by the SQL server automatically to ensure that account IDs that are still referenced by the transaction records will not be deleted, adhering to the foreign key dependency between tables (Microsoft, 2026a). This acts as an additional layer of protection to maintain the integrity of relationships between tables.

Figure 44

Foreign Key Constraints Added to the Production Database

```
USE SmartBankProd;
GO

ALTER TABLE dbo.Account
    ADD CONSTRAINT FK_Account_Customer
    FOREIGN KEY (CustomerID) REFERENCES dbo.Customer (CustomerID);

ALTER TABLE dbo.TransactionRecord
    ADD CONSTRAINT FK_Transaction_Account
    FOREIGN KEY (AccountID) REFERENCES dbo.Account (AccountID);

ALTER TABLE dbo.TransactionRecord
    ADD CONSTRAINT FK_Transaction_ToAccount
    FOREIGN KEY (ToAccountID) REFERENCES dbo.Account (AccountID);
GO
```

Figure 45

Foreign Key Constraints Added to the Analytics Database

```
USE SmartBankAnalytics;
GO

ALTER TABLE Reporting.FactAccount
    ADD CONSTRAINT FK_FactAccount_DimCustomer
    FOREIGN KEY (CustomerHash) REFERENCES Reporting.DimCustomer (CustomerHash);

ALTER TABLE Reporting.FactTransaction
    ADD CONSTRAINT FK_FactTransaction_FactAccount
    FOREIGN KEY (AccountHash) REFERENCES Reporting.FactAccount (AccountHash);
GO
```

- **Check Constraints**

  To enforce domain integrity in SmartBank, check constraints are also implemented to ensure that values are validated by predefined conditional rules, such as restricting values into an appropriate range of numbers, before the values are inserted into the database tables (AxialSQL, 2021). Such a validation process is crucial in terms of database security as it helps to maintain data consistency by preventing unexpected data values from being stored into the database. Similar to the foreign key constraints, check constraints are also widely implemented in both production and analytics database of SmartBank, as shown in Figure 46. For instance, the `Salary` column in the `dbo.Staff` table is checked to ensure no negative values are inserted, while columns like `TransactionType` and `AccountType` in both the `dbo.TransactionRecord` and `dbo.Account` tables are restricted to a fixed set of values to align with business requirements. By having these predefined rules, check constraints help to ensure the accuracy of input data to enforce data integrity in SmartBank.

Figure 46

Check Constraints Added to SmartBank

```
-- Check constraints (production DB) --
USE SmartBankProd;
GO

ALTER TABLE dbo.Account
    ADD CONSTRAINT CK_Account_BalanceNonNegative
    CHECK (Balance >= 0);

ALTER TABLE dbo.Account
    ADD CONSTRAINT CK_Account_Type
    CHECK (AccountType IN ('Savings', 'Current'));

ALTER TABLE dbo.Staff
    ADD CONSTRAINT CK_Staff_Position
    CHECK (Position IN ('DBA', 'Manager', 'Officer'));

ALTER TABLE dbo.Staff
    ADD CONSTRAINT CK_Staff_SalaryNonNegative
    CHECK (Salary >= 0);

ALTER TABLE dbo.TransactionRecord
    ADD CONSTRAINT CK_Transaction_Type
    CHECK (TransactionType IN ('Deposit', 'Withdrawal', 'Transfer'));

ALTER TABLE dbo.TransactionRecord
    ADD CONSTRAINT CK_Transaction_AmountPositive
    CHECK (Amount > 0);

ALTER TABLE dbo.TransactionRecord
    ADD CONSTRAINT CK_Transaction_ToAccountNotSelf
    CHECK (ToAccountID IS NULL OR ToAccountID <> AccountID);

ALTER TABLE dbo.TransactionRecord
    ADD CONSTRAINT CK_Transaction_ToAccountLogic
    CHECK ((TransactionType = 'Transfer' AND ToAccountID IS NOT NULL)
           OR (TransactionType <> 'Transfer' AND ToAccountID IS NULL)
    );

ALTER TABLE Security.UserMapping
    ADD CONSTRAINT CK_UserMapping_OneIdentity
    CHECK ((StaffID IS NOT NULL AND CustomerID IS NULL)
           OR (StaffID IS NULL AND CustomerID IS NOT NULL)
           OR (StaffID IS NULL AND CustomerID IS NULL)         -- This is for service accounts
    );
GO

-- Check constraints (analytics DB) --
USE SmartBankAnalytics;
GO

ALTER TABLE Reporting.ETLRunLog
    ADD CONSTRAINT CK_ETLRunLog_RunStatus
    CHECK (RunStatus IN ('Running', 'Success', 'Failed'));
GO
```

To ensure that foreign keys and check constraints are configured correctly, the code in Figure 47 is used to verify the existing configurations by using a `SELECT` query to check the status of all existing constraints from the `sys.objects` table. Based on the results, it is shown that each of the foreign key constraint and check constraints are applied to their corresponding database tables, hence proving that these constraints are configured as expected.

Figure 47

SQL Code to Verify that Foreign Keys and Check Constraints are Configured Correctly

```
USE SmartBankProd;              -- Verification for production DB --
GO

SELECT  OBJECT_SCHEMA_NAME(parent_object_id) AS SchemaName,
        OBJECT_NAME(parent_object_id)        AS TableName,
        name                                 AS ConstraintName,
        type_desc                            AS ConstraintType
FROM    sys.objects
WHERE   type IN ('F', 'C')
ORDER BY TableName, ConstraintType, ConstraintName;
GO

USE SmartBankAnalytics;         -- Verification for analytics DB --
GO

SELECT  OBJECT_SCHEMA_NAME(parent_object_id) AS SchemaName,
        OBJECT_NAME(parent_object_id)        AS TableName,
        name                                 AS ConstraintName,
        type_desc                            AS ConstraintType
FROM    sys.objects
WHERE   type IN ('F', 'C')
ORDER BY TableName, ConstraintType, ConstraintName;
GO
```

	SchemaName	TableName	ConstraintName	ConstraintType
1	dbo	Account	CK_Account_BalanceNonNegative	CHECK_CONSTRAINT
2	dbo	Account	CK_Account_Type	CHECK_CONSTRAINT
3	dbo	Account	FK_Account_Customer	FOREIGN_KEY_CONSTRAINT
4	dbo	Staff	CK_Staff_Position	CHECK_CONSTRAINT
5	dbo	Staff	CK_Staff_SalaryNonNegative	CHECK_CONSTRAINT
6	dbo	TransactionRecord	CK_Transaction_AmountPositive	CHECK_CONSTRAINT
7	dbo	TransactionRecord	CK_Transaction_ToAccountLogic	CHECK_CONSTRAINT
8	dbo	TransactionRecord	CK_Transaction_ToAccountNotSelf	CHECK_CONSTRAINT
9	dbo	TransactionRecord	CK_Transaction_Type	CHECK_CONSTRAINT
10	dbo	TransactionRecord	FK_Transaction_Account	FOREIGN_KEY_CONSTRAINT
11	dbo	TransactionRecord	FK_Transaction_ToAccount	FOREIGN_KEY_CONSTRAINT
12	Security	UserMapping	CK_UserMapping_OneIdentity	CHECK_CONSTRAINT

	SchemaName	TableName	ConstraintName	ConstraintType
1	Reporting	ETLRunLog	CK_ETLRunLog_RunStatus	CHECK_CONSTRAINT
2	Reporting	FactAccount	FK_FactAccount_DimCustomer	FOREIGN_KEY_CONSTRAINT
3	Reporting	FactTrans...	FK_FactTransaction_FactAc...	FOREIGN_KEY_CONSTRAINT

Several tests are also conducted to verify that both foreign keys and check constraints are able to function as intended, as shown in Figure 48. To validate foreign key constraints, an attempt to insert a new account to the `dbo.Account` table is performed using an invalid `CustomerID` that does not exist in the `dbo.Customer` table. With the result displaying that the insertion operation is rejected due to the violation of `FK_Account_Customer`, this successfully demonstrates that foreign keys are able to enforce referential integrity in the database. On the other hand, check constraints are tested by inserting a record into `dbo.TransactionRecord` with an invalid `TransactionType` value, `Random type`. The operation is also rejected by the constraint named `CK_Transaction_Type`, indicating that the `TransactionType` column only accepts three values, namely `Deposit`, `Withdrawal`, and `Transfer`. Hence, these tests have proven that the constraints implemented in SmartBank are working as intended, helping to prevent invalid data from being inserted into the database.

Figure 48

SQL Code to Verify that Foreign Keys and Check Constraints can Function as Intended

```
USE SmartBankProd;
GO

BEGIN TRAN;
BEGIN TRY
    INSERT INTO dbo.Account (AccountID, CustomerID, AccountType, Balance)
    VALUES ('ACC9999999', 'C99999', 'Savings', 100.00);
    SELECT 'FAILED - orphan account was accepted' AS Result;
END TRY
BEGIN CATCH
    SELECT 'PASSED - FK_Account_Customer blocked the orphan' AS Result,
           ERROR_MESSAGE() AS ErrorMessage;
END CATCH
ROLLBACK TRAN;
GO

BEGIN TRAN;
BEGIN TRY
    INSERT INTO dbo.TransactionRecord (AccountID, Amount, TransactionType)
    VALUES ('ACC1000001', 100.00, 'Random type');
    SELECT 'FAILED - invalid transaction type was accepted' AS Result;
END TRY
BEGIN CATCH
    SELECT 'PASSED - CK_Transaction_Type rejected the value' AS Result,
           ERROR_MESSAGE() AS ErrorMessage;
END CATCH
ROLLBACK TRAN;
GO
```

	Result	ErrorMessage
1	PASSED - FK_Account_Customer blocked the orphan	The INSERT statement conflicted with the FOREIGN KEY constraint "FK_Account_Customer". ...

	Result	ErrorMessage
1	PASSED - CK_Transaction_Type rejected the value	The INSERT statement conflicted with the CHECK constraint "CK_Transaction_Type". ...

Stored Procedures

Although constraints are effective mechanisms that help to enforce data validity in the database, it is insufficient to purely rely on constraints when handling complex business requirements as constraints can only validate simple and predefined rules but not conditional logic or multi-step business rules. As an example, performing fund transfer operation requires multiple steps of verifications, including checking the existence of the source and destination accounts, as well as verifying that funds are sufficient for the operation, which are unable to be handled by constraints alone. Hence, to overcome this limitation, stored procedures, which support the execution of a predefined sequence of SQL commands, are implemented for the databases of SmartBank, not only to support integrity controls by centralising business operation logic, but also to provide the database with a better control over user interactions, thereby aligning with the principle of least privilege (PoLP).

Different stored procedures are created for the production and analytics database of SmartBank to facilitate its business operations, including:

- **`Security.usp_LogRejection`**

  To ensure that every invalid action that occur internally within any user-defined stored procedures is accurately captured, a procedure named `Security.usp_LogRejection` is created to support procedural-level logging by recording all rejected attempts into the `Security.RejectedAttemptLog` table. As shown in Figure 49, the procedure captures several important details, such as login name, name of the procedure where rejection takes place, reason of rejection, and other relevant details, to support audit traceability. After its creation, this procedure will be invoked within other user-defined procedures to ensure that the audit trail of rejected actions can be captured consistently and accurately.

Figure 49

SQL Code for Stored Procedure named `Security.usp_LogRejection`

```
CREATE PROCEDURE [Security].usp_LogRejection
    @ProcedureName SYSNAME,
    @Reason        VARCHAR(300),
    @Detail        VARCHAR(500) = NULL
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO Security.RejectedAttemptLog (LoginName, ProcedureName, Reason, Detail)
    VALUES (ORIGINAL_LOGIN(), @ProcedureName, @Reason, @Detail);
END;
GO
```

- **`dbo.usp_Deposit`**

  The `dbo.usp_Deposit` stored procedure is created to allow customers to deposit money into their bank accounts. As shown in Figure 50, the procedure first verifies whether if the database users are customers performing deposit on their own account, which acts as an additional check layer that avoids unauthorised users from performing deposit even if the `EXEC` permission is mistakenly granted, thereby aligning with the principle of defence in depth (DiD). Then, the procedure validates if the inputted deposit amount is valid before updating the account balance of the customer and logging the transaction record onto the `TransactionRecord` table. If any layer of validation fails, the error message, with details like `AccountID`, will be logged by `Security.usp_LogRejection` to facilitate audit.

Figure 50

SQL Code for Stored Procedure named `dbo.usp_Deposit`

```
CREATE PROCEDURE dbo.usp_Deposit
    @AccountID VARCHAR(10),
    @Amount    DECIMAL(12,2)
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;
    DECLARE @Caller SYSNAME = ORIGINAL_LOGIN();
    DECLARE @CallerCustomerID VARCHAR(6);
    BEGIN TRY
        IF IS_ROLEMEMBER('role_Customer', @Caller) <> 1
        BEGIN EXEC [Security].usp_LogRejection 'usp_Deposit',
                                               'Caller is not a member of role_Customer',
                                               @AccountID;
              THROW 51001, 'Only customers may perform transactions.', 1;
        END
        SELECT @CallerCustomerID = m.CustomerID
        FROM   [Security].UserMapping AS m
        WHERE  m.LoginName = @Caller;
        IF NOT EXISTS (SELECT 1 FROM dbo.Account WHERE AccountID = @AccountID
                                                   AND CustomerID = @CallerCustomerID)
        BEGIN EXEC Security.usp_LogRejection 'usp_Deposit',
                                             'Account does not belong to caller',
                                             @AccountID;
              THROW 51002, 'You may only transact on your own account.', 1;
        END
        IF @Amount <= 0
        BEGIN EXEC Security.usp_LogRejection 'usp_Deposit', 'Non-positive amount', @AccountID;
              THROW 51003, 'Deposit amount must be greater than zero.', 1;
        END
        BEGIN TRANSACTION;
            UPDATE dbo.Account SET Balance = Balance + @Amount WHERE AccountID = @AccountID;
            INSERT INTO dbo.TransactionRecord (AccountID, Amount, TransactionType, PerformedBy)
            VALUES (@AccountID, @Amount, 'Deposit', @Caller);
        COMMIT TRANSACTION;

        PRINT 'Deposit successful.';
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO
```

- **`dbo.usp_Withdraw`**

  Another procedure named `dbo.usp_Withdraw` is created to allow customers withdraw money from their own accounts. Similar with `dbo.usp_Deposit`, a verification step is performed to determine if the database users are customer withdrawing money from their own account. As shown in Figure 51, an additional check is also carried out by the procedure to determine if the account has sufficient balance for the operation, which avoids customers from withdrawing money exceeding their balance.

Figure 51

SQL Code for Stored Procedure named `dbo.usp_Withdraw`

```
CREATE PROCEDURE dbo.usp_Withdraw
    @AccountID VARCHAR(10),
    @Amount    DECIMAL(12,2)
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;
    DECLARE @Caller SYSNAME = ORIGINAL_LOGIN();
    DECLARE @CallerCustomerID VARCHAR(6);
    DECLARE @Balance DECIMAL(12,2);
    BEGIN TRY
        IF IS_ROLEMEMBER('role_Customer', @Caller) <> 1
        BEGIN EXEC Security.usp_LogRejection 'usp_Withdraw',
                                             'Caller is not a member of role_Customer',
                                             @AccountID;
              THROW 51001, 'Only customers may perform transactions.', 1;
        END
        SELECT @CallerCustomerID = m.CustomerID
        FROM   Security.UserMapping AS m
        WHERE  m.LoginName = @Caller;
        IF NOT EXISTS (SELECT 1 FROM dbo.Account WHERE AccountID = @AccountID
                                                   AND CustomerID = @CallerCustomerID)
        BEGIN EXEC Security.usp_LogRejection 'usp_Withdraw',
                                             'Account does not belong to caller',
                                             @AccountID;
              THROW 51002, 'You may only transact on your own account.', 1;
        END
        IF @Amount <= 0
        BEGIN EXEC Security.usp_LogRejection 'usp_Withdraw', 'Non-positive amount', @AccountID;
              THROW 51003, 'Withdrawal amount must be greater than zero.', 1;
        END
        SELECT @Balance = Balance FROM dbo.Account WHERE AccountID = @AccountID;
        IF @Balance < @Amount
        BEGIN EXEC Security.usp_LogRejection 'usp_Withdraw', 'Insufficient funds', @AccountID;
              THROW 51004, 'Insufficient funds for this withdrawal.', 1;
        END
        BEGIN TRANSACTION;
            UPDATE dbo.Account
            SET    Balance = Balance - @Amount
            WHERE  AccountID = @AccountID;
            INSERT INTO dbo.TransactionRecord (AccountID, Amount, TransactionType, PerformedBy)
            VALUES (@AccountID, @Amount, 'Withdrawal', @Caller);
        COMMIT TRANSACTION;
        PRINT 'Withdrawal successful.';
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO
```

- `dbo.usp_Transfer`

  As this procedure is intended to allow customers perform money transfer from their account to other bank accounts, aside from running the same validation process as the withdrawal procedure, an additional validation is performed in Figure 52 to ensure that the source and destination accounts are valid but not the same, hence ensuring the validity of all transfers.

Figure 52

SQL Code for Stored Procedure named `dbo.usp_Transfer`

```
CREATE PROCEDURE dbo.usp_Transfer
    @FromAccountID VARCHAR(10),
    @ToAccountID   VARCHAR(10),
    @Amount        DECIMAL(12,2)
WITH EXECUTE AS OWNER
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;
    DECLARE @Caller SYSNAME = ORIGINAL_LOGIN();
    DECLARE @CallerCustomerID VARCHAR(6);
    DECLARE @Balance DECIMAL(12,2);
    BEGIN TRY
        IF IS_ROLEMEMBER('role_Customer', @Caller) <> 1
        BEGIN EXEC Security.usp_LogRejection 'usp_Transfer',
                                             'Caller is not a member of role_Customer',
                                             @FromAccountID;
              THROW 51001, 'Only customers may perform transactions.', 1;
        END
        SELECT @CallerCustomerID = m.CustomerID
        FROM   Security.UserMapping AS m
        WHERE  m.LoginName = @Caller;
        IF NOT EXISTS (SELECT 1 FROM dbo.Account WHERE AccountID = @FromAccountID
                                                   AND CustomerID = @CallerCustomerID)
        BEGIN EXEC Security.usp_LogRejection 'usp_Transfer',
                                             'Source account does not belong to caller',
                                             @FromAccountID;
              THROW 51002, 'You may only transfer from your own account.', 1;
        END
        IF @FromAccountID = @ToAccountID
        BEGIN EXEC Security.usp_LogRejection 'usp_Transfer',
                                             'Source and destination are the same',
                                             @FromAccountID;
            THROW 51005, 'Source and destination accounts must differ.', 1;
        END
        IF NOT EXISTS (SELECT 1 FROM dbo.Account WHERE AccountID = @ToAccountID)
        BEGIN EXEC Security.usp_LogRejection 'usp_Transfer',
                                             'Destination account does not exist',
                                             @ToAccountID;
            THROW 51006, 'Destination account does not exist.', 1;
        END
        IF @Amount <= 0
        BEGIN EXEC Security.usp_LogRejection 'usp_Transfer', 'Non-positive amount', @FromAccountID;
              THROW 51003, 'Transfer amount must be greater than zero.', 1;
        END
        SELECT @Balance = Balance FROM dbo.Account WHERE AccountID = @FromAccountID;
        IF @Balance < @Amount
        BEGIN EXEC Security.usp_LogRejection 'usp_Transfer', 'Insufficient funds', @FromAccountID;
              THROW 51004, 'Insufficient funds for this transfer.', 1;
        END
        BEGIN TRANSACTION;
            UPDATE dbo.Account SET Balance = Balance - @Amount WHERE AccountID = @FromAccountID;
            UPDATE dbo.Account SET Balance = Balance + @Amount WHERE AccountID = @ToAccountID;
            INSERT INTO dbo.TransactionRecord
                   (AccountID, Amount, TransactionType, ToAccountID, PerformedBy)
            VALUES (@FromAccountID, @Amount, 'Transfer', @ToAccountID, @Caller);
        COMMIT TRANSACTION;
        PRINT 'Transfer successful.';
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO
```

- **`dbo.usp_UpdateOwnStaffRecord`**

  The procedure named `dbo.usp_UpdateOwnStaffRecord` allows the staff in SmartBank to update their own records. However, they are only permitted to change their phone number and branch, as these information are considered low-risk and will not lead to privilege escalation issues although changes are made towards these details. This helps to prevent data tampering cases from happening, such as staff modifying their salary or promoting themselves to a higher position without permission, hence contributing to the accuracy and integrity of the database. As shown in Figure 53, the procedure not only performs authorisation checks like validating staff identity before executing any `UPDATE` operations, but also ensures that users are only able to modify their own details but not the staff record of others, which demonstrates defence in depth (DiD) as any unauthorised modifications procedure will be rejected by the procedure itself even when incorrect permissions are granted, hence protecting data confidentiality and integrity.

Figure 53

SQL Code for Stored Procedure named `dbo.usp_UpdateOwnStaffRecord`

```
CREATE PROCEDURE dbo.usp_UpdateOwnStaffRecord
    @Phone  VARCHAR(20),
    @Branch VARCHAR(50) = NULL
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @MyStaffID VARCHAR(6);

    IF IS_MEMBER('role_DBA')         <> 1
       AND IS_MEMBER('role_Manager') <> 1
       AND IS_MEMBER('role_Officer') <> 1
    BEGIN
        EXEC Security.usp_LogRejection 'usp_UpdateOwnStaffRecord',
                                       'Caller is not a member of any staff role',
                                       NULL;
        THROW 51009, 'Only staff members may update their own record.', 1;
    END

    SELECT @MyStaffID = m.StaffID
    FROM   Security.UserMapping AS m
    WHERE  m.DatabaseUser = USER_NAME();

    IF @MyStaffID IS NULL
    BEGIN
        EXEC Security.usp_LogRejection 'usp_UpdateOwnStaffRecord',
                                       'No staff record mapped to caller',
                                       NULL;
        THROW 51010, 'No staff record is mapped to your login.', 1;
    END

    UPDATE dbo.Staff
    SET    Phone  = @Phone,
           Branch = ISNULL(@Branch, Branch)
    WHERE  StaffID = @MyStaffID;

    PRINT 'Your staff record has been updated.';
END;
GO
```

- `dbo.usp_UpdateStaffDetails`

  In addition to updating own staff details, managers also have the privilege to update the details of other staff members using the `dbo.usp_UpdateStaffDetails` procedure. However, similar to the `dbo.usp_UpdateOwnStaffRecord` procedure, it is also necessary to prevent manager from abusing their privileges and modifying the sensitive details of their own like `Salary`, so that any actions that could breach the integrity and reliability of the database can be prevented. Hence, as shown in Figure 54, the `UPDATE` operation of the procedure is separated into two parts, where the general information of staff like `Branch` and `Phone` is updated first, followed by the individual update on the `Salary` column. This is to ensure that the `dbo.trg_Staff_SalaryGuard` trigger can rollback any unauthorised salary modifications without affecting other legitimate changes even if the identity check is removed from the stored procedure.

Figure 54

SQL Code for Stored Procedure named `dbo.usp_UpdateStaffDetails`

```
CREATE PROCEDURE dbo.usp_UpdateStaffDetails
    @StaffID   VARCHAR(6),
    @StaffName VARCHAR(100) = NULL,
    @Position  VARCHAR(20)  = NULL,
    @Branch    VARCHAR(50)  = NULL,
    @Phone     VARCHAR(20)  = NULL,
    @Salary    DECIMAL(10,2) = NULL
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @MyStaffID VARCHAR(6);

    IF IS_MEMBER('role_Manager') <> 1
    BEGIN EXEC Security.usp_LogRejection 'usp_UpdateStaffDetails', 'Caller is not a Bank Manager', @StaffID;
          THROW 51011, 'Only Bank Managers may manage staff details.', 1;
    END

    IF NOT EXISTS (SELECT 1 FROM dbo.Staff WHERE StaffID = @StaffID)
    BEGIN EXEC Security.usp_LogRejection 'usp_UpdateStaffDetails', 'Staff record not found', @StaffID;
          THROW 51012, 'Staff record not found.', 1;
    END

    SELECT @MyStaffID = m.StaffID FROM Security.UserMapping AS m WHERE m.DatabaseUser = USER_NAME();

    IF @MyStaffID = @StaffID
    BEGIN EXEC Security.usp_LogRejection 'usp_UpdateStaffDetails',
                                         'Manager attempted to update own staff record',
                                         @StaffID;
          THROW 51013, 'Managers may not update their own staff record.
                        Use usp_UpdateOwnStaffRecord instead.', 1;
    END

    UPDATE dbo.Staff
    SET    StaffName = ISNULL(@StaffName, StaffName),
           Position  = ISNULL(@Position,  Position),
           Branch    = ISNULL(@Branch,    Branch),
           Phone     = ISNULL(@Phone,     Phone)
    WHERE  StaffID = @StaffID;

    IF @Salary IS NOT NULL
    BEGIN UPDATE dbo.Staff SET Salary = @Salary WHERE StaffID = @StaffID;
    END

    PRINT 'Staff record updated.';
END;
GO
```

- **`dbo.usp_CreateCustomerAccount`**

  The procedure `dbo.usp_CreateCustomerAccount` is created to allow bank officers create new accounts for customers in SmartBank. Similar to other procedures, the database user is first verified to ensure bank officers are the ones executing the stored procedure, as shown in Figure 55. This is followed by checking the validity of the input `CustomerID` to ensure that the new account belongs to a valid customer, as well as validating the input `OpeningBalance` to avoid invalid negative balances from being inserted into the database. Additionally, considering that customers are allowed to have multiple bank accounts under real-world scenario, the SmartBank system does not perform any checks to verify if the customer holds any existing accounts in the bank to avoid imposing any incorrect one-account-per-customer restrictions towards SmartBank's banking operations. By doing so, the production database of SmartBank is able to mimic real-world banking practices while ensuring data integrity and maintaining appropriate access control mechanisms.

Figure 55

SQL Code for Stored Procedure named `dbo.usp_CreateCustomerAccount`

```
CREATE PROCEDURE dbo.usp_CreateCustomerAccount
    @AccountID   VARCHAR(10),
    @CustomerID  VARCHAR(6),
    @AccountType VARCHAR(20),
    @OpeningBalance DECIMAL(12,2) = 0
AS BEGIN

    SET NOCOUNT ON;

    IF IS_MEMBER('role_Officer') <> 1
    BEGIN EXEC Security.usp_LogRejection 'usp_CreateCustomerAccount',
                                         'Caller is not a Bank Officer',
                                         @AccountID;
          THROW 51013, 'Only Bank Officers may manage customer accounts.', 1;
    END

    IF NOT EXISTS (SELECT 1 FROM dbo.Customer WHERE CustomerID = @CustomerID)
    BEGIN EXEC Security.usp_LogRejection 'usp_CreateCustomerAccount',
                                         'Customer does not exist',
                                         @CustomerID;
          THROW 51014, 'Customer does not exist.', 1;
    END

    IF @OpeningBalance < 0
    BEGIN EXEC Security.usp_LogRejection 'usp_CreateCustomerAccount',
                                         'Negative opening balance',
                                         @AccountID;
          THROW 51015, 'Opening balance cannot be negative.', 1;
    END

    INSERT INTO dbo.Account (AccountID, CustomerID, AccountType, Balance)
    VALUES (@AccountID, @CustomerID, @AccountType, @OpeningBalance);

    PRINT 'Customer account created.';

END;
GO
```

- **`dbo.usp_UpdateCustomerAccount`**

  The procedure is created for authorised bank officers to assist customers in modifying their account type. To ensure data integrity, the procedure only allows bank officers to switch `AccountType` to either `Savings` or `Current` without involving additional attributes like `Balance`. To align with the concept of defence in depth (DiD), the procedure first verifies the role of the database user before performing any `UPDATE` operation, as shown in Figure 56. Validation is also performed to track the number of row changes after executing `UPDATE` to determine if the operation is successful, where unsuccessful attempts indicate that the input `AccountID` does not correspond to a valid account.

Figure 56

SQL Code for Stored Procedure named `dbo.usp_UpdateCustomerAccount`

```
CREATE PROCEDURE dbo.usp_UpdateCustomerAccount
    @AccountID   VARCHAR(10),
    @AccountType VARCHAR(20)
AS
BEGIN
    SET NOCOUNT ON;

    -- Reject if user is not an officer --
    IF IS_MEMBER('role_Officer') <> 1
    BEGIN
        EXEC Security.usp_LogRejection 'usp_UpdateCustomerAccount',
                                       'Caller is not a Bank Officer',
                                       @AccountID;
        THROW 51013, 'Only Bank Officers may manage customer accounts.', 1;
    END

    -- Update account type based on account ID --
    UPDATE dbo.Account
    SET    AccountType = @AccountType
    WHERE  AccountID = @AccountID;

    -- If nothing changed, this means the account ID is invalid --
    IF @@ROWCOUNT = 0
    BEGIN
        EXEC Security.usp_LogRejection 'usp_UpdateCustomerAccount',
                                       'Account not found',
                                       @AccountID;
        THROW 51016, 'Account not found.', 1;
    END

    -- Show output message once done --
    PRINT 'Customer account updated.';
END;
GO
```

After all the stored procedures are implemented, their `EXEC` access shall then be granted to the appropriate users and roles according to the authorisation matrix, as shown in Figure 57. This is to ensure that business users are only able to perform database operations via stored procedures level to align with the principle of least privilege (PoLP).

Figure 57

SQL Code to Handle Permission for Stored Procedures

```
GRANT EXECUTE ON dbo.usp_Deposit  TO role_Customer;
GRANT EXECUTE ON dbo.usp_Withdraw TO role_Customer;
GRANT EXECUTE ON dbo.usp_Transfer TO role_Customer;
DENY  EXECUTE ON dbo.usp_Deposit  TO role_DBA, role_Manager, role_Officer;
DENY  EXECUTE ON dbo.usp_Withdraw TO role_DBA, role_Manager, role_Officer;
DENY  EXECUTE ON dbo.usp_Transfer TO role_DBA, role_Manager, role_Officer;

GRANT EXECUTE ON dbo.usp_UpdateOwnStaffRecord TO role_DBA, role_Manager, role_Officer;
DENY  EXECUTE ON dbo.usp_UpdateOwnStaffRecord TO role_Customer;
GRANT EXECUTE ON dbo.usp_UpdateStaffDetails   TO role_Manager;
DENY  EXECUTE ON dbo.usp_UpdateStaffDetails   TO role_DBA, role_Officer, role_Customer;

GRANT EXECUTE ON dbo.usp_CreateCustomerAccount TO role_Officer;
GRANT EXECUTE ON dbo.usp_UpdateCustomerAccount TO role_Officer;
DENY  EXECUTE ON dbo.usp_CreateCustomerAccount TO role_DBA, role_Manager, role_Customer;
DENY  EXECUTE ON dbo.usp_UpdateCustomerAccount TO role_DBA, role_Manager, role_Customer;
```

The configuration for the stored procedures is then reviewed using the `SELECT` query in Figure 58 to ensure that each procedure is successfully created and configured with the appropriate access and permissions. Based on the result table, it is evident that each database role is granted with the correct permissions, aligning with the authorisation matrix, thereby supporting both the confidentiality and integrity of the database.

Figure 58

SQL Query to Verify the Permission Configuration for Stored Procedures

```
SELECT  OBJECT_SCHEMA_NAME(p.major_id) AS SchemaName,
        OBJECT_NAME(p.major_id)        AS ProcedureName,
        dp.name                        AS RoleName,
        p.permission_name,
        p.state_desc                   AS GrantOrDeny
FROM    sys.database_permissions AS p
JOIN    sys.procedures AS pr        ON pr.object_id = p.major_id
JOIN    sys.database_principals AS dp ON dp.principal_id = p.grantee_principal_id
ORDER BY ProcedureName, RoleName;
```

	SchemaName	ProcedureName	RoleName	permission_name	GrantOrDeny
1	dbo	usp_CreateCustomerAccount	role_Customer	EXECUTE	DENY
2	dbo	usp_CreateCustomerAccount	role_DBA	EXECUTE	DENY
3	dbo	usp_CreateCustomerAccount	role_Manager	EXECUTE	DENY
4	dbo	usp_CreateCustomerAccount	role_Officer	EXECUTE	GRANT
5	dbo	usp_Deposit	role_Customer	EXECUTE	GRANT
6	dbo	usp_Deposit	role_DBA	EXECUTE	DENY
7	dbo	usp_Deposit	role_Manager	EXECUTE	DENY
8	dbo	usp_Deposit	role_Officer	EXECUTE	DENY
9	dbo	usp_Transfer	role_Customer	EXECUTE	GRANT
10	dbo	usp_Transfer	role_DBA	EXECUTE	DENY
11	dbo	usp_Transfer	role_Manager	EXECUTE	DENY
12	dbo	usp_Transfer	role_Officer	EXECUTE	DENY
13	dbo	usp_UpdateCustomerAccount	role_Customer	EXECUTE	DENY
14	dbo	usp_UpdateCustomerAccount	role_DBA	EXECUTE	DENY
15	dbo	usp_UpdateCustomerAccount	role_Manager	EXECUTE	DENY
16	dbo	usp_UpdateCustomerAccount	role_Officer	EXECUTE	GRANT
17	dbo	usp_UpdateOwnStaffRecord	role_Customer	EXECUTE	DENY
18	dbo	usp_UpdateOwnStaffRecord	role_DBA	EXECUTE	GRANT
19	dbo	usp_UpdateOwnStaffRecord	role_Manager	EXECUTE	GRANT
20	dbo	usp_UpdateOwnStaffRecord	role_Officer	EXECUTE	GRANT
21	dbo	usp_UpdateStaffDetails	role_Customer	EXECUTE	DENY
22	dbo	usp_UpdateStaffDetails	role_DBA	EXECUTE	DENY
23	dbo	usp_UpdateStaffDetails	role_Manager	EXECUTE	GRANT
24	dbo	usp_UpdateStaffDetails	role_Officer	EXECUTE	DENY
25	dbo	usp_Withdraw	role_Customer	EXECUTE	GRANT
26	dbo	usp_Withdraw	role_DBA	EXECUTE	DENY
27	dbo	usp_Withdraw	role_Manager	EXECUTE	DENY
28	dbo	usp_Withdraw	role_Officer	EXECUTE	DENY

Triggers

Relying purely on stored procedures to control the execution of database operations is often insufficient. If procedures are accidentally granted with the incorrect permission, unauthorised users can gain access to execute sensitive operations, hence breaching the confidentiality and integrity of the database. Therefore, as an additional protection layer, triggers are also applied in SmartBank to ensure that specific database events, such as `INSERT`, `UPDATE`, or `DELETE` that takes place on tables or views, are validated in the background before the actual operation takes place (Richman, 2026). Some of the triggers that are defined in SmartBank include:

- **`dbo.trg_Staff_SalaryGuard`**

  The `dbo.trg_Staff_SalaryGuard` trigger is implemented to prevent staff, including managers, from modifying their own salary. As shown in Figure 59, this is achieved by checking if the `StaffID` of the modified record matches the `StaffID` of the database user. If both IDs match, the trigger will automatically roll back the modification operation and log the rejection to record the case where a staff is trying to modify their own salary. Aside from preventing managers from abusing their privilege to perform unauthorised actions, this also aligns with the defence in depth (DiD) principle as sensitive financial information like `Salary` is protected by numerous security mechanisms.

Figure 59

SQL Code for Trigger named `dbo.trg_Staff_SalaryGuard`

```
CREATE TRIGGER dbo.trg_Staff_SalaryGuard ON dbo.Staff
AFTER UPDATE
AS BEGIN
    SET NOCOUNT ON;
    IF NOT UPDATE(Salary) RETURN;
    DECLARE @Caller SYSNAME = ORIGINAL_LOGIN();
    DECLARE @Detail VARCHAR(500) = (SELECT TOP (1) 'StaffID: ' + i.StaffID FROM inserted AS i);
    IF EXISTS (SELECT 1 FROM inserted AS i
               JOIN [Security].UserMapping AS m ON m.StaffID = i.StaffID
               WHERE  m.LoginName = @Caller)
    BEGIN ROLLBACK TRANSACTION;
          EXEC [Security].usp_LogRejection 'trg_Staff_SalaryGuard',
                                           'Attempted to modify own salary',
                                           @Detail;
          THROW 51020, 'Refused: you cannot modify your own salary.', 1;
    END
    IF NOT EXISTS (SELECT 1 FROM sys.database_role_members
                   WHERE role_principal_id   = DATABASE_PRINCIPAL_ID('role_Manager')
                   AND member_principal_id = DATABASE_PRINCIPAL_ID(@Caller))
    BEGIN ROLLBACK TRANSACTION;
          EXEC [Security].usp_LogRejection 'trg_Staff_SalaryGuard',
                                           'Non-manager attempted to modify a salary',
                                           @Detail;
          THROW 51021, 'Refused: only Bank Managers may modify salary.', 1;
    END
END;
GO
```

- `dbo.trg_Transaction_NoDelete`

  To maintain accuracy in financial records and audit trail, it is also crucial to ensure that transaction records cannot be deleted once created. Therefore, a trigger with the name of `dbo.trg_Transaction_NoDelete` is implemented in Figure 60 to reject any `DELETE` attempts taking place on `dbo.TransactionRecord` by using the `INSTEAD OF DELETE` clause. This ensures that transaction records are immutable, thereby enhancing data integrity and accountability of the production database in SmartBank.

Figure 60

SQL Code for Trigger named `dbo.trg_Transaction_NoDelete`

```
CREATE TRIGGER dbo.trg_Transaction_NoDelete
ON dbo.TransactionRecord
INSTEAD OF DELETE
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @Detail VARCHAR(500) = (SELECT TOP (1) 'TransID: ' + CONVERT(VARCHAR(12), d.TransID)
                                    FROM deleted AS d);
    ROLLBACK TRANSACTION;
    EXEC [Security].usp_LogRejection 'trg_Transaction_NoDelete',
                                     'Attempted deletion of a transaction record',
                                     @Detail;
    THROW 51030, 'Transaction records are append-only and cannot be deleted.', 1;
END;
GO
```

The code in Figure 61 is then executed to examine the configuration for both triggers, while the code in Figure 62 is used to determine if triggers are functioning as intended in the database. Based on both result tables, it is shown that all triggers are enabled, and the `DELETE` operation performed on the `dbo.TransactionRecord` table is denied. This indicates that triggers are configured correctly, hence contributing to the accountability and integrity of the database.

Figure 61

SQL Query to Verify the Configuration for Triggers

```
SELECT  t.name  AS TableName,
        tr.name AS TriggerName,
        tr.is_disabled,
        tr.is_instead_of_trigger
FROM    sys.triggers AS tr
JOIN    sys.tables   AS t ON t.object_id = tr.parent_id
ORDER BY t.name, tr.name;
GO
```

	TableName	TriggerName	is_disabled	is_instead_of_trigger
1	Staff	trg_Staff_SalaryGuard	0	0
2	TransactionRecord	trg_Transaction_NoDelete	0	1

Figure 62

SQL Query to Demonstrate that Trigger is Fired on `dbo.TransactionRecord` *Table*

```
BEGIN TRY
    DELETE FROM dbo.TransactionRecord WHERE TransID = 1;
    SELECT 'FAILED - a transaction row was deleted' AS Result;
END TRY
BEGIN CATCH
    SELECT 'PASSED - ledger is append-only' AS Result, ERROR_MESSAGE() AS ErrorMessage;
END CATCH
GO
```

	Result	ErrorMessage
1	PASSED - ledger is append-only	Transaction records are append-only and cannot be deleted.

Data Protection Techniques

Currently, the contents of the data in both the production and analytics database of SmartBank is unprotected, indicating that unauthorised users will be able to view sensitive data from existing database tables without any restriction if they can successfully bypass the access control mechanisms implemented (Mollashaik, 2023), hence compromising the confidentiality component of the CIA triad. To tackle such an issue, different data protection techniques are employed in the SmartBank database, as outlined in this section.

Encryption

Encryption is a common security technique often implemented to protect sensitive information by transforming data from plain text into an unreadable form called cipher text (PentaSecurity, 2020). Encryption is reversible in nature, where the original data can always be retrieved as long as users have the corresponding key to perform decryption. This provides an additional layer of protection as unauthorised users are unable to access encrypted data without gaining access to the encryption keys, thereby safeguarding data confidentiality in SmartBank.

Encryption at Rest

Encryption at rest is often applied when data is stored in files or tables (Erik, 2026). Several encryption techniques are implemented to protect data at rest in SmartBank, including:

- **Transparent Data Encryption (TDE)**

  Transparent data encryption (TDE) protects data by encrypting physical database files stored on disk, including primary and secondary database files like `.mdf` and `.ndf`, as well as transactional log files like `.ldf` (Madyatmadja et al., 2021). Without protection by TDE, unauthorised users can easily steal the physical database files and restore all data on another SQL server without having to perform data decryption or undergo existing access control mechanisms, hence breaching data confidentiality in SmartBank. As shown in Figure 63, TDE is implemented by first creating a database master key (DML) that is protected by a strong password, followed by the creation of two separate TDE certificates for each database in SmartBank, namely `TDECert_Production` for the production database and `TDECert_Analytics` for the analytics database. This is to reduce the risk of having both databases compromised simultaneously, as attackers will have to obtain two certificates separately to decrypt the data in both databases, hence enhancing database security during unexpected incidents like cybersecurity attacks.

Figure 63

SQL Code to Setup Database Master Key and Certificates

```
USE master;
GO

-- Create the database master key for master DB (TDE) --
IF NOT EXISTS (SELECT 1 FROM sys.symmetric_keys WHERE name = '##MS_DatabaseMasterKey##')
    CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'M@sterKey#SmartBank2026';
GO

-- If certificate is not there, create a new certificate for production DB (TDE) --
IF NOT EXISTS (SELECT 1 FROM sys.certificates WHERE name = 'TDECert_Production')
    CREATE CERTIFICATE TDECert_Production WITH SUBJECT = 'TDE Certificate - SmartBankProd Production';
GO

-- If certificate is not there, create a new certificate for analytics DB (TDE) --
IF NOT EXISTS (SELECT 1 FROM sys.certificates WHERE name = 'TDECert_Analytics')
    CREATE CERTIFICATE TDECert_Analytics WITH SUBJECT = 'TDE Certificate - SmartBankAnalytics';
GO
```

Before continuing with the subsequent steps of TDE encryption, it is necessary to back up the TDE certificates first to prevent the scenario where, despite being an authorised user, decryption and database recovery could not be performed due to the loss of TDE certificates. Hence, as shown in Figure 64, backups are carried out on the TDE certificates (`.cer`) and their private keys (`.pvk`), where both files are additionally encrypted by a password before they are stored under the directory `C:\SmartBankServer\Encryption` .

Figure 64

SQL Code to Backup Certificates

```
-- Backup certificates: Check if there is existing cert files for production DB --
DECLARE @FileExists INT;
CREATE TABLE #FileCheck (FileExists INT, IsDirectory INT, ParentExists INT);
INSERT INTO #FileCheck EXEC master.dbo.xp_fileexist 'C:\SmartBankServer\Encryption\TDECert_Production.cer';
SELECT @FileExists = FileExists FROM #FileCheck;

-- Backup the production certificate if file does not exist --
IF @FileExists = 0
    BACKUP CERTIFICATE TDECert_Production
        TO FILE = 'C:\SmartBankServer\Encryption\TDECert_Production.cer'
        WITH PRIVATE KEY (
            FILE = 'C:\SmartBankServer\Encryption\TDECert_Production.pvk',
            ENCRYPTION BY PASSWORD = 'C3rtBackup#Prod2026'
        );

-- Backup certificate: Check if there is existing cert files for analytics DB --
TRUNCATE TABLE #FileCheck;
INSERT INTO #FileCheck EXEC master.dbo.xp_fileexist 'C:\SmartBankServer\Encryption\TDECert_Analytics.cer';
SELECT @FileExists = FileExists FROM #FileCheck;

-- Backup the analytics certificate if file does not exist --
IF @FileExists = 0
    BACKUP CERTIFICATE TDECert_Analytics
        TO FILE = 'C:\SmartBankServer\Encryption\TDECert_Analytics.cer'
        WITH PRIVATE KEY (
            FILE = 'C:\SmartBankServer\Encryption\TDECert_Analytics.pvk',
            ENCRYPTION BY PASSWORD = 'C3rtBackup#Ana2026'
        );

-- Cleanup: The FileCheck table is temporarily used for checking only --
DROP TABLE #FileCheck;
GO
```

After backing up the TDE certificates, the database encryption keys (DEK) are created and enabled, as shown in Figure 65. Protected by the certificates, the database encryption keys will be used to encrypt and decrypt the physical files of the production and analytics database. The AES-256 algorithm is selected as the encryption algorithm as it is a well-established standard that uses 256 bits for its key space, making it computationally infeasible for attackers to guess the encryption key using exhaustive brute-force methods, thereby providing a high level of data security. Moreover, compared to other encryption algorithms like RSA and ECC, the AES-256 algorithm consumes lesser computational resources when encrypting large database files, making it the most appropriate algorithm to be applied for SmartBank as it offers both security and performance (Gomes et al., 2021).

Figure 65

SQL Code to Create Database Encryption Keys

```
-- Create and enable TDE on production DB --
USE SmartBankProd;
GO

IF NOT EXISTS (SELECT 1 FROM sys.dm_database_encryption_keys WHERE database_id = DB_ID())
    CREATE DATABASE ENCRYPTION KEY
        WITH ALGORITHM = AES_256
        ENCRYPTION BY SERVER CERTIFICATE TDECert_Production;
GO

ALTER DATABASE SmartBankProd SET ENCRYPTION ON;
GO

-- Create and enable TDE on analytics DB --
USE SmartBankAnalytics;
GO

IF NOT EXISTS (SELECT 1 FROM sys.dm_database_encryption_keys WHERE database_id = DB_ID())
    CREATE DATABASE ENCRYPTION KEY
        WITH ALGORITHM = AES_256
        ENCRYPTION BY SERVER CERTIFICATE TDECert_Analytics;
GO

ALTER DATABASE SmartBankAnalytics SET ENCRYPTION ON;
GO
```

Once TDE is configured, the SQL query in Figure 66 is executed to validate the encryption status of both the production and analytics database in SmartBank. Based on the status retrieved from the `sys.dm_database_encryption_keys` table, it is shown that both databases are successfully encrypted by TDE with the AES-256 algorithm, and the TDE certificates are also correctly assigned to protect each database individually. The `tempdb` database, which is used internally by the SQL server to perform operations like sorting and hashing, is also automatically encrypted to ensure that all data, including its temporary copies or intermediate representations, are written to its physical files in ciphertext format.

Figure 66

SQL Code to Check the Encryption Status for Each Database

```
SELECT    DB_NAME(dek.database_id) AS DatabaseName, dek.encryption_state,
          CASE dek.encryption_state WHEN 1 THEN 'Unencrypted'
                                    WHEN 2 THEN 'Encryption in progress'
                                    WHEN 3 THEN 'Encrypted'
                                    WHEN 4 THEN 'Key change in progress'
                                    WHEN 5 THEN 'Decryption in progress'
          END AS StateDescription,
          dek.key_algorithm, dek.key_length, c.name AS CertificateUsed
FROM      sys.dm_database_encryption_keys AS dek
LEFT JOIN sys.certificates AS c ON c.thumbprint = dek.encryptor_thumbprint;
```

	DatabaseName	encryption_state	StateDescription	key_algorithm	key_length	CertificateUsed
1	tempdb	3	Encrypted	AES	256	NULL
2	SmartBankProd	3	Encrypted	AES	256	TDECert_Production
3	SmartBankAnalytics	3	Encrypted	AES	256	TDECert_Analytics

An additional demonstration is also performed to display the difference between TDE-encrypted and unencrypted database files. Before TDE is implemented, a command is executed in PowerShell to search for records containing `Aisyah` from the physical `.mdf` file, as shown in Figure 67, where the result shows that database information can be extracted using the search command. However, when the same PowerShell command is executed after TDE is configured, Figure 68 shows that no values are returned, indicating that the same technique cannot be used to extract database information anymore.

Figure 67

PowerShell Command to Search for Keywords in Database Files before TDE

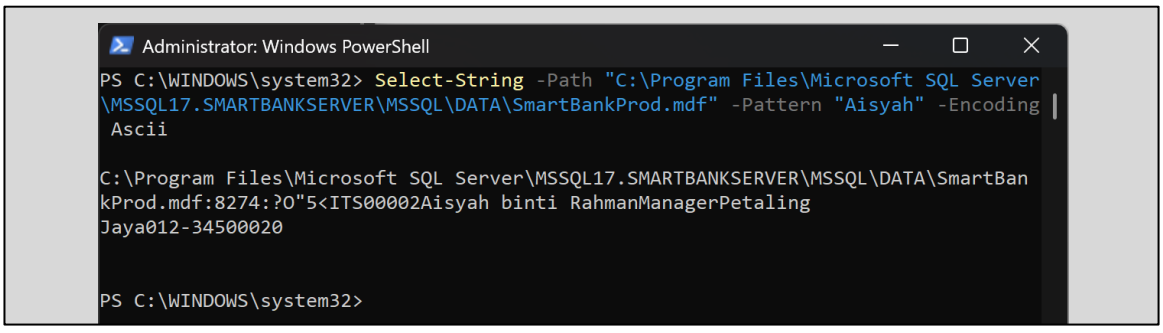
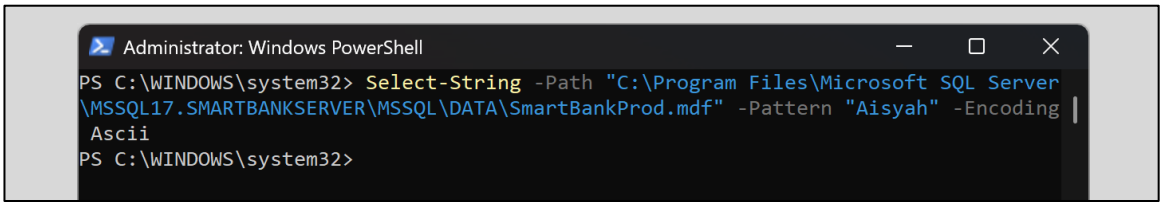


Figure 68

PowerShell Command to Search for Keywords in Database Files after TDE



- **Column-Level Encryption (CLE)**

  Column-level encryption (CLE) is also implemented to encrypt sensitive columns in a table (Banerjee, 2023). Similar to TDE, only database users with the correct key are able to view the original values of the column, hence enforcing data confidentiality while aligning with the principle of least privilege (PoLP) as data is restricted to authorised users only. As shown in Figure 69, enforcing CLE follows a similar process as TDE, where a database master key (DMK) is first created in the production database, followed by the creation and backup of certificates before initialising the encryption key with the AES-256 algorithm.

Figure 69

SQL Code to Perform Column-Level Encryption on Production Database

```
USE SmartBankProd;
GO

-- Create the database master key for production DB (CLE) --
IF NOT EXISTS (SELECT 1 FROM sys.symmetric_keys WHERE name = '##MS_DatabaseMasterKey##')
    CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'SmartBankProd#DMK@2026';
GO

-- If certificate is unavailable, create it --
IF NOT EXISTS (SELECT 1 FROM sys.certificates WHERE name = 'Cert_CustomerPII')
    CREATE CERTIFICATE Cert_CustomerPII WITH SUBJECT = 'Certificate protecting the customer PII symmetric key';
GO

-- Backup certificate if there is no existing ones --
DECLARE @FileExists INT;
CREATE TABLE #FileCheck (FileExists INT, IsDirectory INT, ParentExists INT);
INSERT INTO #FileCheck EXEC master.dbo.xp_fileexist 'C:\SmartBankServer\Encryption\Cert_CustomerPII.cer';
SELECT @FileExists = FileExists FROM #FileCheck;

IF @FileExists = 0
    BACKUP CERTIFICATE Cert_CustomerPII
        TO FILE = 'C:\SmartBankServer\Encryption\Cert_CustomerPII.cer'
        WITH PRIVATE KEY (FILE = 'C:\SmartBankServer\Encryption\Cert_CustomerPII.pvk',
                          ENCRYPTION BY PASSWORD = 'C3rtBackup#PII2026');

DROP TABLE #FileCheck;
GO

-- Create encryption key for CLE --
IF NOT EXISTS (SELECT 1 FROM sys.symmetric_keys WHERE name = 'SK_CustomerPII')
    CREATE SYMMETRIC KEY SK_CustomerPII
        WITH ALGORITHM = AES_256
        ENCRYPTION BY CERTIFICATE Cert_CustomerPII;
GO
```

Since IC number is classified as a highly confidential information, the `ICNumber` column in the `dbo.Customer` table is applied with column-level encryption to ensure that only managers are allowed to view the original IC number of customers to support banking operations. Therefore, as shown in Figure 70, after using `Cert_CustomerPII` to open the symmetric key `SK_CustomerPII`, the original `ICNumber` column is migrated to a new encrypted column named `ICNumber_Enc` with `VARBINARY(256)` as its data type. A query is also executed to ensure that the values of `ICNumber_Enc` matches with the `ICNumber` column before dropping the old `ICNumber` column.

Figure 70

SQL Code to Validate IC Number before Performing Column-Level Encryption

```
-- Add the ICNumber_Enc column (make it nullable first coz it's empty initially) --
ALTER TABLE dbo.Customer ADD ICNumber_Enc VARBINARY(256) NULL;
GO

-- Migrate the values from ICNumber to ICNumber_Enc --
OPEN SYMMETRIC KEY SK_CustomerPII DECRYPTION BY CERTIFICATE Cert_CustomerPII;

UPDATE dbo.Customer
SET    ICNumber_Enc = ENCRYPTBYKEY(KEY_GUID('SK_CustomerPII'), ICNumber);

CLOSE SYMMETRIC KEY SK_CustomerPII;
GO

-- Validate that the IC numbers are transferred correctly before dropping old column --
OPEN SYMMETRIC KEY SK_CustomerPII DECRYPTION BY CERTIFICATE Cert_CustomerPII;

SELECT  CustomerID,
        ICNumber                                        AS Plaintext_BeforeDrop,
        ICNumber_Enc                                    AS Ciphertext,
        CONVERT(VARCHAR(20), DECRYPTBYKEY(ICNumber_Enc))  AS DecryptedBack
FROM    dbo.Customer;

CLOSE SYMMETRIC KEY SK_CustomerPII;
GO

-- Drop old IC column if everything looks good --
ALTER TABLE dbo.Customer DROP COLUMN ICNumber;
GO

-- Now convert the ICNumber_Enc to not null since all data is loaded --
ALTER TABLE dbo.Customer ALTER COLUMN ICNumber_Enc VARBINARY(256) NOT NULL;
GO
```

	CustomerID	Plaintext_BeforeDrop	Ciphertext	DecryptedBack
1	C00001	900314-14-5521	0x008F6DD18DD923449E0BFB5A240AD4F10200000025B151769973081 46A0B7F6E...	900314-14-5521
2	C00002	880725-10-4432	0x008F6DD18DD923449E0BFB5A240AD4F102000000D63C3A93DF618996BAF8DB...	880725-10-4432
3	C00003	950612-08-3310	0x008F6DD18DD923449E0BFB5A240AD4F102000000C9F061F0A7D324777EA92A7...	950612-08-3310
4	C00004	011128-14-2201	0x008F6DD18DD923449E0BFB5A240AD4F102000000C9E2CC8B0884532067311D1...	011128-14-2201

Once the configuration for CLE encryption is completed, the access to the certificate and symmetric key shall be granted to authorised users. Based on the authorisation matrix, since managers are the only ones who have the permission to view the complete IC number of customers, both the certificate and key are granted to managers but denied for other users like officers and DBA, as shown in Figure 71.

Figure 71

SQL Code to Grant Access of CLE Certificate and Key to Users

```
GRANT VIEW DEFINITION ON SYMMETRIC KEY::SK_CustomerPII TO role_Manager;
GRANT CONTROL         ON CERTIFICATE::Cert_CustomerPII TO role_Manager;

DENY VIEW DEFINITION  ON SYMMETRIC KEY::SK_CustomerPII TO role_Officer, role_Customer, role_DBA;
DENY CONTROL          ON CERTIFICATE::Cert_CustomerPII TO role_Officer, role_Customer, role_DBA;
GO
```

Since users do not have direct access to raw database tables, including the newly updated `dbo.Customer` table, additional views and stored procedures are created to facilitate business processes. Figure 72 shows the `vw_CustomerIC` view that allows officers and managers to view customer details, while Figure 73 displays the `usp_GetCustomerIC` procedure that allows manager to view the decrypted `ICNumber` of customers. The `usp_RegisterCustomer` procedure is also defined in Figure 74 to allow officers to register new customers into SmartBank before allowing them to create new bank accounts.

Figure 72

SQL Code for the View named `dbo.vw_CustomerIC`

```
CREATE VIEW dbo.vw_CustomerIC
AS
    SELECT  CustomerID,
            CustomerName,
            CONVERT(VARCHAR(20), DECRYPTBYKEY(ICNumber_Enc)) AS ICNumber
    FROM    dbo.Customer;
GO

GRANT SELECT ON dbo.vw_CustomerIC TO role_Manager, role_Officer;
DENY  SELECT ON dbo.vw_CustomerIC TO role_DBA, role_Customer;
GO
```

Figure 73

SQL Code for the Stored Procedure named `dbo.usp_GetCustomerIC`

```
CREATE PROCEDURE dbo.usp_GetCustomerIC
    @CustomerID VARCHAR(6)
AS
BEGIN
    SET NOCOUNT ON;

    -- Make sure that the user is a manager --
    IF IS_MEMBER('role_Manager') <> 1
    BEGIN EXEC Security.usp_LogRejection 'usp_GetCustomerIC',
                                         'Caller is not a Bank Manager',
                                         @CustomerID;
          THROW 51040, 'Only Bank Managers may retrieve a customer IC number.', 1;
    END

    -- Allow managers to retrieve customer information --
    OPEN SYMMETRIC KEY SK_CustomerPII DECRYPTION BY CERTIFICATE Cert_CustomerPII;

    SELECT  CustomerID,
            CustomerName,
            CONVERT(VARCHAR(20), DECRYPTBYKEY(ICNumber_Enc)) AS ICNumber
    FROM    dbo.Customer
    WHERE   CustomerID = @CustomerID;

    CLOSE SYMMETRIC KEY SK_CustomerPII;
END;
GO

GRANT EXECUTE ON dbo.usp_GetCustomerIC TO role_Manager;
DENY  EXECUTE ON dbo.usp_GetCustomerIC TO role_Officer, role_Customer, role_DBA;
GO
```

Figure 74

SQL Code for the Stored Procedure named `dbo.usp_RegisterCustomer`

```
-- Stored procedure to register customer --
CREATE PROCEDURE dbo.usp_RegisterCustomer
    @CustomerID   VARCHAR(6),
    @CustomerName VARCHAR(100),
    @ICNumber     VARCHAR(20),
    @Phone        VARCHAR(20)  = NULL,
    @Address      VARCHAR(200) = NULL
WITH EXECUTE AS OWNER
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    -- Retrieve the user who calls the SP --
    DECLARE @Caller SYSNAME = ORIGINAL_LOGIN();

    -- Reject if the user is not an officer --
    IF IS_ROLEMEMBER('role_Officer', @Caller) <> 1
    BEGIN
        EXEC Security.usp_LogRejection 'usp_RegisterCustomer',
                                       'Caller is not a Bank Officer',
                                       @CustomerID;
        THROW 51013, 'Only Bank Officers may manage customer accounts.', 1;
    END

    -- Validate if customer has already exist in the system --
    IF EXISTS (SELECT 1 FROM dbo.Customer WHERE CustomerID = @CustomerID)
    BEGIN
        EXEC Security.usp_LogRejection 'usp_RegisterCustomer',
                                       'Customer already exists',
                                       @CustomerID;
        THROW 51041, 'That CustomerID is already registered.', 1;
    END

    -- Perform validation on IC number --
    IF @ICNumber NOT LIKE '[0-9][0-9][0-9][0-9][0-9][0-9]-[0-9][0-9]-[0-9][0-9][0-9][0-9]'
    BEGIN
        EXEC Security.usp_LogRejection 'usp_RegisterCustomer',
                                       'IC number failed format validation',
                                       @CustomerID;
        THROW 51042, 'IC number must be in the format YYMMDD-PB-###G.', 1;
    END

    -- If everything is okay, update the Customer table --
    OPEN SYMMETRIC KEY SK_CustomerPII DECRYPTION BY CERTIFICATE Cert_CustomerPII;

    INSERT INTO dbo.Customer (CustomerID, CustomerName, ICNumber_Enc, Phone, [Address])
    VALUES (@CustomerID,
            @CustomerName,
            ENCRYPTBYKEY(KEY_GUID('SK_CustomerPII'), @ICNumber),
            @Phone,
            @Address);

    CLOSE SYMMETRIC KEY SK_CustomerPII;

    -- Output results once done --
    PRINT 'Customer registered. IC number stored encrypted.';
END;
GO

-- Only officers can register customers --
GRANT EXECUTE ON dbo.usp_RegisterCustomer TO role_Officer;
DENY  EXECUTE ON dbo.usp_RegisterCustomer TO role_DBA, role_Manager, role_Customer;
GO
```

Lastly, to ensure that CLE is able to function as intended in the production database, two `SELECT` queries are executed by impersonating two different users, namely `ofc_lim` who is a bank officer, as well as `mgr_aisyah` who is a bank manager. Since managers are the only ones who have access to both the certificate and symmetric key, the decrypted `ICNumber` column shall only be visible to managers but not officers. As shown in Figure 75, the results align with the expected outputs, indicating that CLE performs as intended in SmartBank.

Figure 75

SQL Code to Verify that Column-Level Encryption is Functioning as Intended

```
EXECUTE AS USER = 'ofc_lim';
    SELECT USER_NAME() AS TestedAs, * FROM dbo.vw_CustomerIC;
REVERT;
GO

EXECUTE AS USER = 'mgr_aisyah';
    OPEN SYMMETRIC KEY SK_CustomerPII DECRYPTION BY CERTIFICATE Cert_CustomerPII;
    SELECT USER_NAME() AS TestedAs, * FROM dbo.vw_CustomerIC;
    CLOSE SYMMETRIC KEY SK_CustomerPII;
REVERT;
GO
```

	TestedAs	CustomerID	CustomerName	ICNumber
1	ofc_lim	C00001	Tan Mei Ling	NULL
2	ofc_lim	C00002	Siti Nurhaliza	NULL
3	ofc_lim	C00003	Kumar Rajendran	NULL
4	ofc_lim	C00004	Wong Kah Meng	NULL

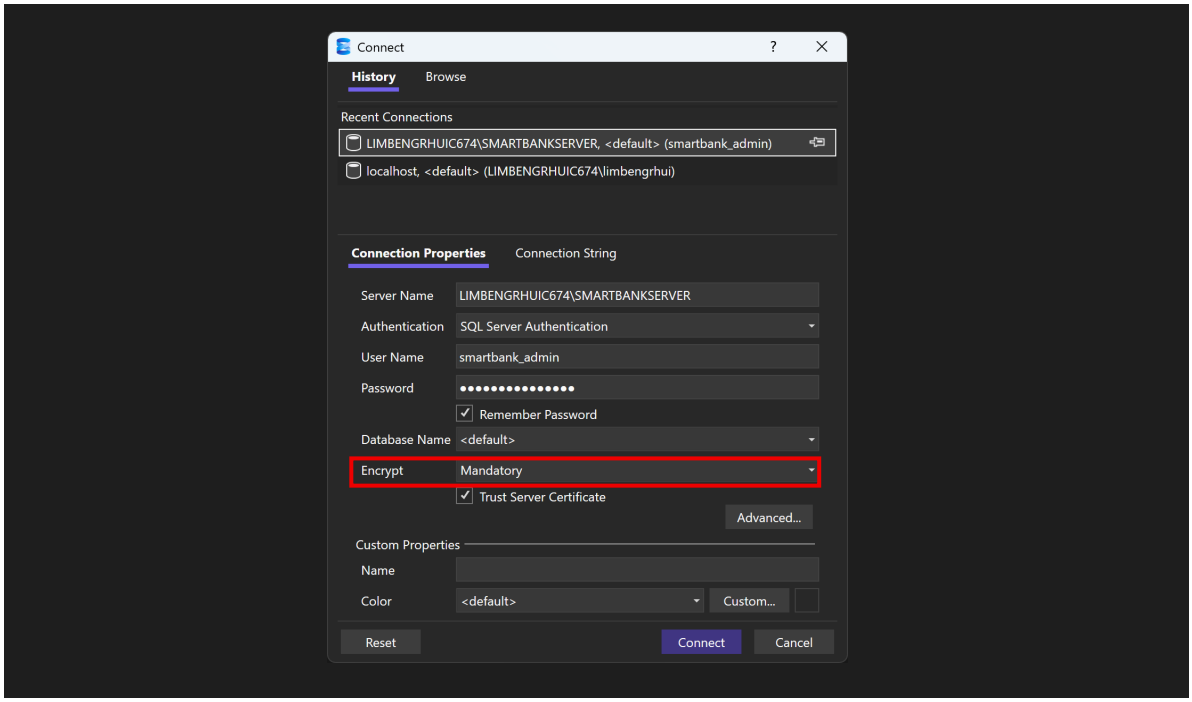
	TestedAs	CustomerID	CustomerName	ICNumber
1	mgr_aisyah	C00001	Tan Mei Ling	900314-14-5521
2	mgr_aisyah	C00002	Siti Nurhaliza	880725-10-4432
3	mgr_aisyah	C00003	Kumar Rajendran	950612-08-3310
4	mgr_aisyah	C00004	Wong Kah Meng	011128-14-2201

Encryption in Transit (TLS)

While encryption-at-rest techniques are implemented to protect information when it is stored, the encryption-in-transit mechanism protects data by converting information into cipher text when it is transmitted through a network, so that unauthorised users are unable to extract sensitive information even though the data is intercepted (Rajeshkumarin, 2026). In SQL Server, when a client communicates with a database server, the data will first be converted into a Tabular Data Stream (TDS) format before the Transport Layer Security (TLS) encrypts the data to secure the communication channel. However, unlike other security mechanisms, encryption in transit is applied by default in SQL Server Management Studio 22 (Microsoft, 2026b), where the connection property for `Encrypt` is set to `Mandatory` in default, as shown in Figure 76. Therefore, this section details the steps taken to demonstrate that all network connections involving SSMS are encrypted by TLS.

Figure 76

Connect Dialog Window Showing that Encrypt is Mandatory by Default



Since the SQL server `SmartBankServer` and the SSMS client are running on the same local machine, the client-server connection is currently using the shared memory protocol, as shown in Figure 77. In other words, both the server and client currently carry out communication by writing to a shared region of memory on the local machine without passing through any network routes. Therefore, the current setup is unsuitable to demonstrate encryption in transit.

Figure 77

SQL Code to Check Current Client-Server Connection

```
SELECT  c.session_id,
        s.login_name,
        c.encrypt_option,
        c.auth_scheme,
        c.net_transport
FROM    sys.dm_exec_connections AS c
JOIN    sys.dm_exec_sessions    AS s ON s.session_id = c.session_id
WHERE   s.is_user_process = 1 AND auth_scheme = 'SQL';
GO
```

	session_id	login_name	encrypt_option	auth_scheme	net_transport
1	51	smartbank_admin	TRUE	SQL	Shared memory
2	52	smartbank_admin	TRUE	SQL	Shared memory
3	53	smartbank_admin	TRUE	SQL	Shared memory
4	54	smartbank_admin	TRUE	SQL	Shared memory
5	56	smartbank_admin	TRUE	SQL	Shared memory
6	69	smartbank_admin	TRUE	SQL	Shared memory

To switch the current connection protocol to a network protocol, a PowerShell command is executed to identify the TCP configurations and ports currently used by `SmartBankServer`. As shown in Figure 78, the SQL server currently uses the TCP port of `51433`.

Figure 78

PowerShell Command to Check TCP Port

```
Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\system32> $instancePath = "HKLM:\SOFTWARE\Microsoft\Microsoft SQL Server\MSSQL17.SMARTBANKSERVER\MSSQLServ
er\SuperSocketNetLib\Tcp\IPAll"
PS C:\WINDOWS\system32> Get-ItemProperty -Path $instancePath -Name TcpPort, TcpDynamicPorts


TcpPort          : 51433
TcpDynamicPorts  :
PSPath           : Microsoft.PowerShell.Core\Registry::HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Microsoft SQL
                   Server\MSSQL17.SMARTBANKSERVER\MSSQLServer\SuperSocketNetLib\Tcp\IPAll
PSParentPath     : Microsoft.PowerShell.Core\Registry::HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Microsoft SQL
                   Server\MSSQL17.SMARTBANKSERVER\MSSQLServer\SuperSocketNetLib\Tcp
PSChildName      : IPAll
PSDrive          : HKLM
PSProvider       : Microsoft.PowerShell.Core\Registry


PS C:\WINDOWS\system32>
```

As shown in Figure 79, a new SSMS connection is established using the identified port number, `tcp:LIMBENGRHUIC674,51433`. The client-server connection is then re-validated using the code in Figure 80, where the result table shows that all active connections are using the TCP protocol, with `encrypt_option` returning `TRUE`. This indicates that encryption in transit is enabled for the current client-server connection to `SmartBankServer`, hence ensuring that any sensitive information in the database is protected during transmission.

Figure 79

Connect Dialog Window to Establish TCP Protocol for Client-Server Communication

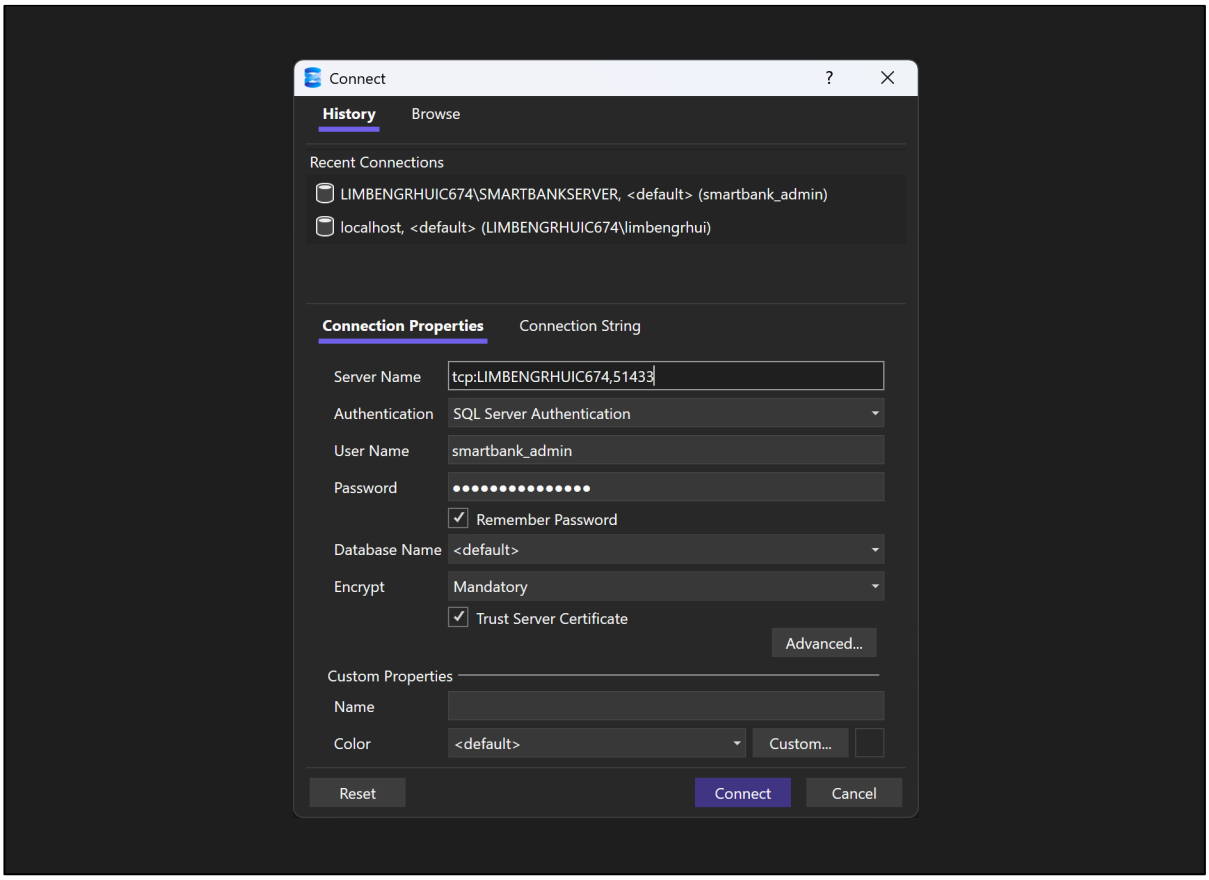


Figure 80

SQL Code to Re-Verify Current Client-Server Connection

```
SELECT  c.session_id,
        s.login_name,
        c.encrypt_option,
        c.auth_scheme,
        c.net_transport
FROM    sys.dm_exec_connections AS c
JOIN    sys.dm_exec_sessions    AS s ON s.session_id = c.session_id
WHERE   s.is_user_process = 1 AND auth_scheme = 'SQL';
GO
```

	session_id	login_name	encrypt_option	auth_scheme	net_transport
1	51	smartbank_admin	TRUE	SQL	TCP
2	52	smartbank_admin	TRUE	SQL	TCP
3	53	smartbank_admin	TRUE	SQL	TCP
4	54	smartbank_admin	TRUE	SQL	TCP
5	56	smartbank_admin	TRUE	SQL	TCP
6	69	smartbank_admin	TRUE	SQL	TCP

Data Masking

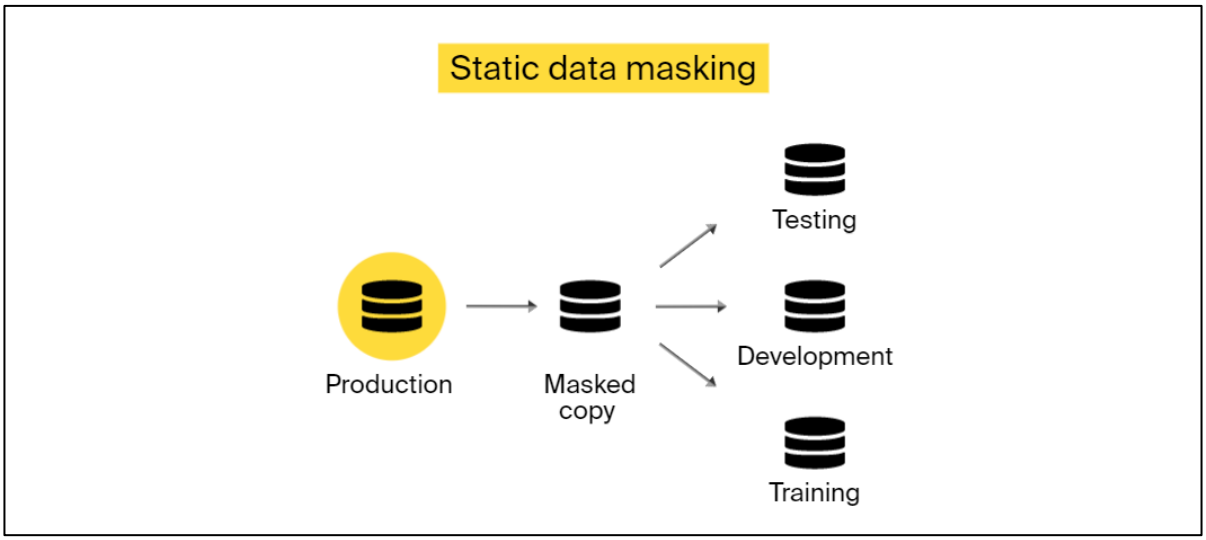
While encryption techniques like column-level encryption (CLE) provide complete restriction when protecting sensitive data, these methods not only are often complex to be configured, but also lead to reduced flexibility as SQL queries are unable to perform operations like searching and indexing on encrypted data (Sun et al., 2024). Therefore, in order to maintain the usability of data for business operations while reducing the risk of exposure of sensitive information, data masking techniques, which partially hide or conceal the data without compromising its structure, are often implemented to preserve and enhance data confidentiality (AWS, 2026).

Static Data Masking

As shown in Figure 81, after creating a copy of existing data from the database, static data masking permanently replaces the copied data with fictitious values by using techniques like substitution or redaction. This helps to ensure that confidential information, such as personally identifiable information (PII), is completely obfuscated from unauthorised users, which helps mitigating privacy risks like re-identification to improve confidentiality. However, as the process is irreversible, this causes static data masking to be unsuitable for the production database as actual data values are often required to perform business operations and calculations, making it more appropriate for environments like development and analytics that do not require the unmasking of real transactional data (Bowman, 2024).

Figure 81

Concept of Static Data Masking



Note. From *What is data masking, and how to implement it the right way?*, by N. Alkhaldi, 2023 (https://itrexgroup.com/blog/what-is-data-masking-and-how-to-implement-it-the-right-way/).

In terms of SmartBank, static data masking technique is applied on the `CustomerName` attribute when performing extract-transform-load (ETL) operations from production to the analytics database to ensure that anyone who is granted with illegal access to the analytics database will still be unable to access customers' personal data. As shown in Figure 82, under the procedure `Reporting.usp_RunETL`, the `CustomerName` attribute is partially redacted with the asterisk (`*`) symbol immediately when the record is extracted from the `Customer` table, while its original value is not stored or written elsewhere in the procedure. As a result, only the first letter of `CustomerName` remains in the `Reporting.DimCustomer` table after the ETL pipeline runs, as shown in Figure 83, with the discarded characters unrecoverable from the masked output.

Figure 82

Static Data Masking Applied on `CustomerName` *in* `Reporting.usp_RunETL`

```
-- Extracted from stored procedure Reporting.usp_RunETL --
INSERT INTO Reporting.DimCustomer (CustomerHash, MaskedName)
SELECT
    CONVERT(CHAR(64), HASHBYTES('SHA2_256', @Salt + c.CustomerID), 2),
    LEFT(c.CustomerName, 1) + REPLICATE('*', 7)
FROM SmartBankProd.dbo.Customer AS c;
```

Figure 83

SQL Query and Result after Executing `Reporting.usp_RunETL`

```
-- Demo: Execute the entire ETL pipeline once --
EXEC Reporting.usp_RunETL;
GO

-- Show that customer name is statically masked --
SELECT CustomerHash, MaskedName, LoadedOn FROM Reporting.DimCustomer;
GO
```

	CustomerHash	MaskedName	LoadedOn
1	BAC6B559F1B1149CC49C1B409EE7912700F4B9741E78FD3A4...	S*******	2026-08-19 22:58:28
2	C5B2DFFA45E38BC0EF05C9938CB774A7ACE562541F548F74...	K*******	2026-08-19 22:58:28
3	DA9D4554F8D0F1FDE36EC4478F5F4B9A0568E98696F36E8A0...	W*******	2026-08-19 22:58:28
4	F186E19390EE7CEBA10CB16AD7F068A7E87BF559F8AA0E0E...	T*******	2026-08-19 22:58:28

Dynamic Data Masking

Unlike static dynamic masking, dynamic data masking does not alter the existing values of data as the technique only applies its masking rules towards its result set during query execution. Although requiring additional performance overhead for execution, dynamic data masking is often preferred in production as not only it promotes data confidentiality by allowing administrators to explicitly control the permission of `GRANT UNMASK` for different database roles, but also supports data integrity since no additional copying or modification is required before processing production data (Imperva, 2024). Hence, as shown in Figure 84, dynamic data masking is applied on SmartBank's production database to ensure that only bank managers are able to view complete customer information like `Phone` and `Address` to support day-to-day business operations without compromising their data privacy.

Figure 84

Dynamic Data Masking Applied on `Phone` *and* `Address` *in* `dbo.Customer`

```
USE SmartBankProd;
GO

-- DDM on phone number --
ALTER TABLE dbo.Customer
    ALTER COLUMN Phone ADD MASKED
    WITH (FUNCTION = 'partial(3,"XXXXXX",2)');
GO

-- DDM on address --
ALTER TABLE dbo.Customer
    ALTER COLUMN [Address] ADD MASKED
    WITH (FUNCTION = 'default()');
GO

-- Granting UNMASK only to managers, others should not have the privilege to view --
GRANT UNMASK TO role_Manager;
DENY  UNMASK TO role_Officer;
DENY  UNMASK TO role_Customer;
DENY  UNMASK TO role_DBA;
GO
```

To verify that dynamic data masking is successfully implemented in the database, the code in Figure 85 is executed to display database configuration, such as the details of the masked columns and the database roles who are granted with the `UNMASK` permission. The query result shows that dynamic data masking is configured correctly following requirements. Figure 86 further validates the reliability of dynamic data mask on different user roles, as it shows that Phone and Address are masked for officers like `ofc_lim`, while managers like `mgr_aisyah` are able to view complete customer details without any content redaction.

Figure 85

SQL Query and Result to Verify the Configuration of Dynamic Data Masking

```sql
-- Verify that DDM is applied on the correct column --
SELECT  OBJECT_SCHEMA_NAME(c.object_id) AS SchemaName,
        OBJECT_NAME(c.object_id)        AS TableName,
        c.name                          AS ColumnName,
        c.masking_function              AS MaskFunction
FROM    sys.masked_columns AS c
ORDER BY TableName, ColumnName;
GO

-- Verify that unmask is granted to the correct user --
SELECT  dp.name AS RoleName, p.permission_name, p.state_desc AS GrantOrDeny
FROM    sys.database_permissions AS p
JOIN    sys.database_principals AS dp ON dp.principal_id = p.grantee_principal_id
WHERE   p.permission_name = 'UNMASK'
ORDER BY dp.name;
GO
```

	SchemaName	TableName	ColumnName	MaskFunction
1	dbo	Customer	Address	default()
2	dbo	Customer	Phone	partial(3, "XXXXXX", 2)

	RoleName	permission_name	GrantOrDeny
1	role_Customer	UNMASK	DENY
2	role_DBA	UNMASK	DENY
3	role_Manager	UNMASK	GRANT
4	role_Officer	UNMASK	DENY

Figure 86

SQL Query and Result to Validate the Reliability of Dynamic Data Masking on Different Users

```sql
-- Validate that DDM works on different roles --
EXECUTE AS USER = 'ofc_lim';
    SELECT USER_NAME() AS RunningAs, CustomerID, CustomerName, Phone, [Address]
    FROM   dbo.vw_CustomerProfile;
REVERT;
GO

EXECUTE AS USER = 'mgr_aisyah';
    SELECT USER_NAME() AS RunningAs, CustomerID, CustomerName, Phone, [Address]
    FROM   dbo.vw_CustomerProfile;
REVERT;
GO
```

	RunningAs	CustomerID	CustomerName	Phone	Address
1	ofc_lim	C00001	Tan Mei Ling	013XXXXXX01	xxxx
2	ofc_lim	C00002	Siti Nurhaliza	013XXXXXX02	xxxx
3	ofc_lim	C00003	Kumar Rajendran	013XXXXXX03	xxxx
4	ofc_lim	C00004	Wong Kah Meng	013XXXXXX04	xxxx

	RunningAs	CustomerID	CustomerName	Phone	Address
1	mgr_aisyah	C00001	Tan Mei Ling	013-7770001	12 Jalan Bukit Bintang, 55100 Kuala Lumpur
2	mgr_aisyah	C00002	Siti Nurhaliza	013-7770002	88 Jalan SS2/24, 47300 Petaling Jaya
3	mgr_aisyah	C00003	Kumar Rajendran	013-7770003	5 Persiaran Kayangan, 40100 Shah Alam
4	mgr_aisyah	C00004	Wong Kah Meng	013-7770004	30 Jalan Ampang, 50450 Kuala Lumpur

Hashing and Anonymisation

While data encryption and data masking techniques are able to restrict user access to sensitive data values and protect data confidentiality, both approaches have their own limitations when analytics tasks are involved. The reversible nature of encryption allows analysts to recover the original values of confidential information if they gain unauthorised access to encryption keys, meanwhile data masking techniques that replace actual data with fictitious values introduces non-uniqueness across different records, thereby limiting analytics capability as operations like `JOIN` and `GROUP BY` are unable to be executed (ITaegis, 2024). Therefore, to achieve user anonymity without compromising entity and referential integrity, the SmartBank database has implemented hashing mechanisms on principal identifiers like `AccountID`, `CustomerID`, and `TransactionID` to convert them into pseudonymized outputs. Not only this helps preventing malicious users from performing reverse-engineering techniques and deduce the original plain text data, but the deterministic nature of hashing ensures that identical inputs will always produce the same outputs, making the output hash fully compatible with SQL operations to support analytics tasks while promoting data confidentiality (Buckman, 2025).

Figure 87

Salted Hashing Applied under `Reporting.usp_RunETL`

```
CREATE PROCEDURE Reporting.usp_RunETL AS BEGIN

    DECLARE @Salt NVARCHAR(64) = N'SmartBank$2026$AnalyticsSalt';          -- Secret salt defined --

    INSERT INTO Reporting.DimCustomer (CustomerHash, MaskedName)
    SELECT
        CONVERT(CHAR(64), HASHBYTES('SHA2_256', @Salt + c.CustomerID), 2), -- Hash customer ID --
        LEFT(c.CustomerName, 1) + REPLICATE('*', 7)
    FROM   SmartBankProd.dbo.Customer AS c;

    INSERT INTO Reporting.FactAccount
           (AccountHash, CustomerHash, AccountType, Balance)
    SELECT
        CONVERT(CHAR(64), HASHBYTES('SHA2_256', @Salt + a.AccountID), 2),  -- Hash account ID --
        CONVERT(CHAR(64), HASHBYTES('SHA2_256', @Salt + a.CustomerID), 2), -- Hash customer ID --
        a.AccountType,
        a.Balance
    FROM   SmartBankProd.dbo.Account AS a;

    INSERT INTO Reporting.FactTransaction
           (TransHash, AccountHash, TransDate, Amount, TransactionType)
    SELECT                                                                 -- Hash transaction ID --
        CONVERT(CHAR(64), HASHBYTES('SHA2_256', @Salt + CONVERT(VARCHAR(12), t.TransID)), 2),
        CONVERT(CHAR(64), HASHBYTES('SHA2_256', @Salt + t.AccountID), 2),  -- Hash account ID --
        t.TransDate,
        t.Amount,
        t.TransactionType
    FROM   SmartBankProd.dbo.TransactionRecord AS t;
END;
```

As shown in Figure 87, the salted hashing mechanism is implemented on the `AccountID`, `CustomerID` and `TransactionID` attributes under the procedure `Reporting.usp_RunETL` alongside with static data masking to ensure that key identifiers are directly anonymised when the ETL pipeline is executed. To enhance the robustness of the hashing mechanism, aside from selecting `SHA2_256` as the primary algorithm to avoid hash collision while maximising computational efficiency (Huan & Xie, 2026), a cryptographic salt is also defined and attached as the prefix of each data to introduce structural randomness in the output hash and make pre-computation attacks computationally infeasible to be implemented (Kapadiya, 2025), thereby mitigating the risk of having original values being decoded by unauthorised users or attackers.

To validate that hashing is successfully applied on the identifiers, the ETL pipeline is first executed using the SQL code in Figure 88 before reviewing the contents of the dimension and fact tables, `Reporting.DimCustomer` and `Reporting.FactAccount`. As observed in the output table, the contents in the identifier columns (renamed as `CustomerHash` and `AccountHash`) are formed by 64 hexadecimal characters that are indecipherable to attackers if the underlying cryptographic salt and hashing algorithm remains unknown. The deterministic nature of hashing is further demonstrated via the first and fourth row of the `Reporting.FactAccount` table, where a similar output hash is produced for the same customer who owns two different accounts in SmartBank. This allows the output hash to act as the new primary identifier for these tables, hence enabling the execution of relational SQL operations to support business intelligence and data analysis tasks.

Figure 88

SQL Code to Run ETL Pipeline and Validate the Occurrence of Salted Hashing

```
EXEC Reporting.usp_RunETL;
GO

-- Show that account ID and customer ID is hashed --
SELECT  a.AccountID   AS ProductionAccountID,
        a.CustomerID  AS ProductionCustomerID,
        CONVERT(CHAR(64), HASHBYTES('SHA2_256', N'SmartBank$2026$AnalyticsSalt' + a.AccountID),  2) AS AccountHash,
        CONVERT(CHAR(64), HASHBYTES('SHA2_256', N'SmartBank$2026$AnalyticsSalt' + a.CustomerID), 2) AS CustomerHash
FROM    SmartBankProd.dbo.Account AS a;
SELECT  AccountHash, CustomerHash, AccountType, Balance FROM Reporting.FactAccount;
GO
```

	ProductionAccountID	ProductionCustomerID	AccountHash	CustomerHash
1	ACC1000001	C00001	1B5193BF568576D98F982F689470CFEA0CD0C7AB24584679C5...	F186E19390EE7CEBA10CB16AD7F068A7E87BF559F8AA0E0E...
2	ACC1000002	C00001	C60EFEEDB015C5748B54FF41712BFAFEA71142127AD6B0B66...	F186E19390EE7CEBA10CB16AD7F068A7E87BF559F8AA0E0E...
3	ACC1000003	C00002	EEBECE22B3FEACC2C639E7A8B90D5ECE79329838CE91AA7E...	BAC6B559F1B1149CC49C1B409EE7912700F4B9741E78FD3A4...
4	ACC1000004	C00003	B79705E8647D7786DDAE30653829D7978961A36BA16BF4C2EB...	C5B2DFFA45E38BC0EF05C9938CB774A7ACE562541F548F74...
5	ACC1000005	C00004	8CE82BEA8BDFD0A551405EB2B051EF50E5A0EC4421EDBEB4...	DA9D4554F8D0F1FDE36EC4478F5F4B9A0568E98696F36E8A0...

	AccountHash	CustomerHash	AccountType	Balance
1	1B5193BF568576D98F982F689470CFEA0CD0C7AB24584679C5...	F186E19390EE7CEBA10CB16AD7F068A7E87BF559F8AA0E0E...	Savings	15000.00
2	8CE82BEA8BDFD0A551405EB2B051EF50E5A0EC4421EDBEB4...	DA9D4554F8D0F1FDE36EC4478F5F4B9A0568E98696F36E8A0...	Current	51000.00
3	B79705E8647D7786DDAE30653829D7978961A36BA16BF4C2EB...	C5B2DFFA45E38BC0EF05C9938CB774A7ACE562541F548F74...	Savings	9200.00
4	C60EFEEDB015C5748B54FF41712BFAFEA71142127AD6B0B66...	F186E19390EE7CEBA10CB16AD7F068A7E87BF559F8AA0E0E...	Current	4500.00
5	EEBECE22B3FEACC2C639E7A8B90D5ECE79329838CE91AA7E...	BAC6B559F1B1149CC49C1B409EE7912700F4B9741E78FD3A4...	Savings	28000.00

Backup

When databases are deployed in real world, developers and database administrators often perform multiple operations to maintain its schema and contents. However, statistics from Lawrence (2022) show that approximately two-thirds of database infrastructure and service disruptions are often caused by human errors due to reasons like procedural slips, which leads to system misconfigurations and creates security vulnerabilities. Therefore, it is necessary for organisations, especially SmartBank that is involved in the highly sensitive finance industry, to safeguard their databases with backup and disaster recovery (BDR) strategies, not only to ensure database availability by providing alternatives for restoration when data loss occurs, but also to enhance database integrity by allowing faulty commands or instructions to be reverted.

Backup Strategy

As the SmartBank system implements both production and analytics databases, considering that the data stored in the two databases possesses different criticality, it is more appropriate for each to have its own backup strategy to optimise resource allocation (Netdata, 2025).

Production Database: `SmartBankProd`

Live transactional data are stored in the production database, including account balances and transaction records, where any data loss shall be addressed as soon as possible to minimise any potential financial and reputational loss. Due to the high criticality of such data, the backup strategy of the production database is designed in a robust manner, as shown in Table 5.

Table 5

Backup Strategy for Production Database

Aspect	Configuration	Justification
RPO	15 minutes	Balances data loss against backup overhead
RTO	Less than 1 hour	Bounded by the sequential recovery method (full → differential → log restore)
Recovery model	Full	Required to enable transaction log backups
Backup types and frequency	Full (*weekly*) Differential (*daily*) Transactional log (*every 15 minutes*)	Provides sufficient granularity to support fast and comprehensive recovery
Recovery method	Full → Differential → Log	Restores to the latest point with the fewest files
Retention	28 days	4 weeks is sufficient while preserving disk usage

Analytics Database: `SmartBankAnalytics`

Unlike production, the analytics database stores historical data that are intended for analytical tasks. As these data are loaded from the production database via an ETL pipeline, the analytics database can still rely on the pipeline to fully restore its data even without backups. However, when any pipeline misconfigurations take place, without proper backups, the database will not be able to roll back and recover itself from the faulty pipeline, hence compromising data integrity and availability in the analytics database. Considering that the pipeline may also become resource intensive as the volume of data grows, it is crucial for the analytics database to have a lightweight yet effective backup strategy, as proposed in Table 6.

Table 6

Backup Strategy for Analytics Database

Aspect	Configuration	Justification
RPO	24 hours	As source data can always be retrieved from production, a longer RPO is acceptable to reduce unnecessary backup storage
RTO	One ETL cycle	In the worst case, data is reloaded directly from production, so RTO depends on the ETL pipeline's execution time
Recovery model	Simple	Log backups are not required to rebuild a derived database
Backup types and frequency	Full (*weekly*)	Provides sufficient safeguard without the storage overhead of differential or log backups
Recovery method	Restore full backup, or re-run ETL	Restore recovers the database from a faulty ETL load. Re-run recovers database from infrastructure failures.
Retention	28 days	To align with production's retention window for consistency

In terms of backup encryption, both the production and analytics database are configured with transparent data encryption (TDE). This indicates that all files, including backup files, will be encrypted at rest automatically without requiring further configuration, which plays a crucial role in preserving the confidentiality of the database as unauthorised users, despite being able to bypass access control mechanisms and access the physical backup files, will still be unable to perform data restoration and retrieve database information if they lack the corresponding decryption keys (Ahmed, 2024). Also, enforcing TDE exempts administrators from having to additionally create and manage separate certificates for backups, hence reducing administrative workload to mitigate governance oversight and enhance the overall security of the database.

Implementation

Based on the backup strategy, both the production and analytics database are implemented with their backup mechanisms accordingly.

Scheduled Backup

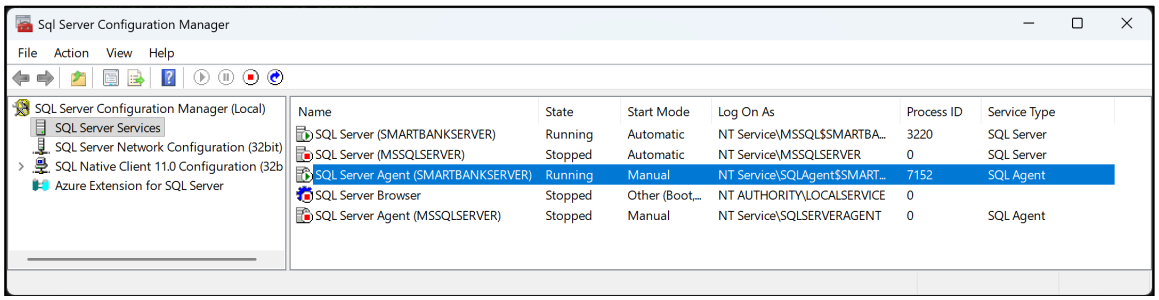
To schedule backups for the production and analytics database, several steps are performed:

- **Ensure that SQL Server Agent is running**

  As the SQL Server Agent is the service that is responsible for handling the automation of scheduled jobs, it is a must to ensure that the service is enabled, as shown by the screenshot of the SQL Server Configuration Manager application in Figure 89, before proceeding to creating scheduled backups. This is to make sure that the service will be able to trigger the backup jobs at the scheduled time automatically.

Figure 89

Status Checking for SQL Server Agent via SQL Server Configuration Manager



It is also possible to verify the status of the SQL Server Agent using the SQL script in Figure 90, where the `status_desc` column should display `Running` if the service is enabled. However, if SQL Server Agent is disabled, the service must be started manually using the SQL Server Configuration Manager application.

Figure 90

SQL Code to Check the Status of SQL Server Agent

```
SELECT servicename, status_desc, startup_type_desc
FROM sys.dm_server_services
WHERE servicename LIKE 'SQL Server Agent%';
```

	servicename	status_desc	startup_type_desc
1	SQL Server Agent (SMARTBANKSERVER)	Running	Manual

- **Establish the backup chain**

  Before configuring the automated scheduled backups, both the production and analytics database are manually backed up first using the code in Figure 91. This is to make sure that differential and transactional log backups are able to locate and reference the initial full backup to ensure subsequent backup operations can be executed successfully. Moreover, performing initial manual backups also helps to verify if both the production and analytics database can have their backup files stored at the designated location correctly, as shown in Figure 92, to reduce the risk of misconfigurations when creating automated backups later.

Figure 91

SQL Code to Execute Manual Full, Differential and Transactional Log Backups

```
-- Manual full backup on production DB --
BACKUP DATABASE SmartBankProd
    TO DISK = 'C:\SmartBankServer\Backup\Production\SmartBankProd_Full.bak'
    WITH INIT, NAME = 'SmartBankProd Full Backup',
               DESCRIPTION = 'Weekly full backup baseline',
               CHECKSUM, STATS = 10;
GO

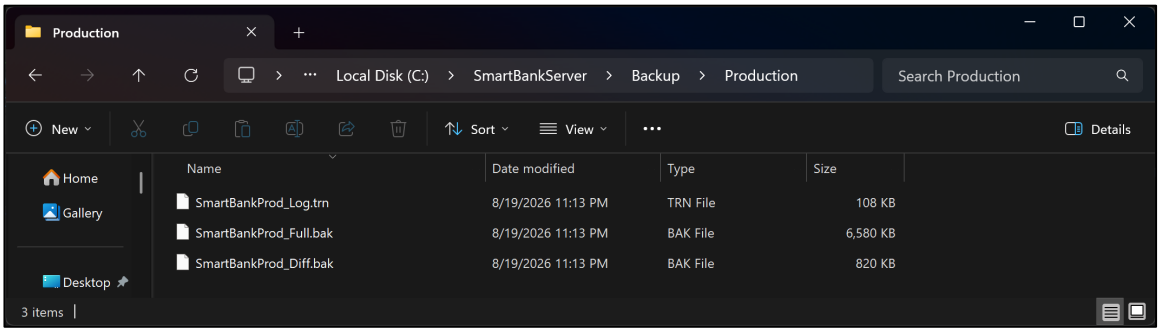
-- Manual differential backup on production DB --
BACKUP DATABASE SmartBankProd
    TO DISK = 'C:\SmartBankServer\Backup\Production\SmartBankProd_Diff.bak'
    WITH DIFFERENTIAL, INIT, NAME = 'SmartBankProd Differential Backup', CHECKSUM, STATS = 10;
GO

-- Manual transactional log backup on production DB --
BACKUP LOG SmartBankProd
    TO DISK = 'C:\SmartBankServer\Backup\Production\SmartBankProd_Log.trn'
    WITH INIT, NAME = 'SmartBankProd Log Backup', CHECKSUM, STATS = 10;
GO

-- Manual full backup on analytics DB --
BACKUP DATABASE SmartBankAnalytics
    TO DISK = 'C:\SmartBankServer\Backup\Analytics\SmartBankAnalytics_Full.bak'
    WITH INIT, NAME = 'Analytics Full Backup', CHECKSUM, STATS = 10;
GO
```

Figure 92

Full, Differential and Transactional Log Backup Files of Production Database in Directory



- **Create the job container**

  After completing the prerequisite configurations, the automated scheduled backup can be defined using the code in Figure 93. Once switching to the `msdb` database that handles all configurations for scheduled jobs, the `sp_add_job` procedure is first executed to initialise an empty job container with details like name and description. The progress of backup is also kept tracked by setting the `notify_level_eventlog` parameter to the value of `2`, where any failed attempts will be recorded and logged onto the server.

Figure 93

SQL Code to Create a Job Container for Production's Full Weekly Backup

```
USE msdb;
GO

EXEC msdb.dbo.sp_add_job
     @job_name = 'SmartBank - Full Backup (Weekly)',
     @description = 'Weekly full backup of SmartBankProd.',
     @notify_level_eventlog = 2;
```

- **Define the backup step**

  Next, the `sp_add_jobstep` procedure is executed to define the actions to be performed by the job container. By specifying parameters like `subsystem` as `TSQL` and `database_name` as `master`, when SQL Server Agent triggers the backup job on the scheduled time, it will automatically connect to the `master` database to execute the commands written in SQL. Taking Figure 94 as an example, when the job is triggered, the `command` script will dynamically generate a timestamped file name, then perform a full backup for the production database and save it to the designated directory.

Figure 94

SQL Code to Define the Backup Step for Production's Full Weekly Backup

```
EXEC msdb.dbo.sp_add_jobstep
     @job_name  = 'SmartBank - Full Backup (Weekly)',
     @step_name = 'Full backup',
     @subsystem = 'TSQL',
     @database_name = 'master',
     @command = N'
     DECLARE @File NVARCHAR(300) = ''C:\SmartBankServer\Backup\Production\SmartBankProd_Full_'' +
                                   FORMAT(SYSDATETIME(), ''yyyyMMdd_HHmmss'') + ''.bak'';
     BACKUP DATABASE SmartBankProd TO DISK = @File
     WITH INIT, NAME = ''SmartBankProd Full'', CHECKSUM, STATS = 10;
     ';
```

- **Create the schedule**

  The `sp_add_schedule` procedure is then executed to create a schedule that specifies when shall the backup job be executed. To ensure that the backup operations will not lead to any interruptions to business users, backups are often scheduled during off-peak hours, where, taking the full production backup as an example, it is intended to be executed weekly on Sunday at 2 a.m. Therefore, as shown in Figure 95, `freq_type` is set to `8` to indicate weekly execution, `freq_interval` is configured to `1` to specify that the job shall be executed on Sundays, and `active_start_time` is set to `020000` to indicate that the job shall begin its execution sharp at 02:00:00 automatically.

Figure 95

SQL Code to Create Job Schedule for Production's Full Weekly Backup

```
EXEC msdb.dbo.sp_add_schedule
     @schedule_name = 'SmartBank_Weekly_Sunday_0200',
     @freq_type = 8,                    -- weekly
     @freq_interval = 1,                -- Sunday
     @freq_recurrence_factor = 1,
     @active_start_time = 020000;       -- 2:00 a.m.
```

- **Attach schedule to job and register to server**

  Once the schedule is created, it shall be linked to the corresponding backup job using the procedure `sp_attach_schedule` before the job is registered to the SQL Server Agent using `sp_add_jobserver`, as shown in Figure 96. This is to ensure that the scheduled backup can be automatically triggered by the SQL Server Agent at the specified time without requiring administrators to manually execute them.

Figure 96

SQL Code to Attach and Register Job Schedule for Production's Full Weekly Backup

```
-- Attach schedule to job --
EXEC msdb.dbo.sp_attach_schedule
     @job_name = 'SmartBank - Full Backup (Weekly)',
     @schedule_name = 'SmartBank_Weekly_Sunday_0200';

-- Register job to server --
EXEC msdb.dbo.sp_add_jobserver @job_name = 'SmartBank - Full Backup (Weekly)';
GO
```

Similar configurations are applied for the differential and transactional log production backups, as well as the weekly full backup for analytics database, as shown in Figures 97, 98, and 99.

Figure 97

SQL Code to Configure Daily Differential Backups for Production Database

```
EXEC msdb.dbo.sp_add_job
     @job_name = 'SmartBank - Differential Backup (Daily)',
     @description = 'Daily differential backup of SmartBankProd.',
     @notify_level_eventlog = 2;

EXEC msdb.dbo.sp_add_jobstep
     @job_name  = 'SmartBank - Differential Backup (Daily)',
     @step_name = 'Differential backup',
     @subsystem = 'TSQL',
     @database_name = 'master',
     @command = N'
     DECLARE @File NVARCHAR(300) = ''C:\SmartBankServer\Backup\Production\SmartBankProd_Diff_'' +
                                   FORMAT(SYSDATETIME(), ''yyyyMMdd_HHmmss'') + ''.bak'';
     BACKUP DATABASE SmartBankProd TO DISK = @File
     WITH DIFFERENTIAL, INIT, NAME = ''SmartBankProd Differential'', CHECKSUM, STATS = 10;';

EXEC msdb.dbo.sp_add_schedule
     @schedule_name = 'SmartBank_Daily_0300',
     @freq_type = 4,                    -- daily
     @freq_interval = 1,
     @active_start_time = 030000;

EXEC msdb.dbo.sp_attach_schedule
     @job_name = 'SmartBank - Differential Backup (Daily)',
     @schedule_name = 'SmartBank_Daily_0300';

EXEC msdb.dbo.sp_add_jobserver @job_name = 'SmartBank - Differential Backup (Daily)';
GO
```

Figure 98

SQL Code to Configure Transactional Log Backups for Production Database

```
EXEC msdb.dbo.sp_add_job
     @job_name = 'SmartBank - Log Backup (15 min)',
     @description = 'Transaction log backup every 15 minutes. Delivers the 15-minute RPO.',
     @notify_level_eventlog = 2;

EXEC msdb.dbo.sp_add_jobstep
     @job_name  = 'SmartBank - Log Backup (15 min)',
     @step_name = 'Log backup',
     @subsystem = 'TSQL',
     @database_name = 'master',
     @command = N'
     DECLARE @File NVARCHAR(300) = ''C:\SmartBankServer\Backup\Production\SmartBankProd_Log_'' +
                                   FORMAT(SYSDATETIME(), ''yyyyMMdd_HHmmss'') + ''.trn'';
     BACKUP LOG SmartBankProd TO DISK = @File
     WITH INIT, NAME = ''SmartBankProd Log'', CHECKSUM, STATS = 10;';

EXEC msdb.dbo.sp_add_schedule
     @schedule_name = 'SmartBank_Every_15_Minutes',
     @freq_type = 4,                    -- daily
     @freq_interval = 1,
     @freq_subday_type = 4,             -- minutes
     @freq_subday_interval = 15,
     @active_start_time = 000000,
     @active_end_time   = 235959;

EXEC msdb.dbo.sp_attach_schedule
     @job_name = 'SmartBank - Log Backup (15 min)',
     @schedule_name = 'SmartBank_Every_15_Minutes';

EXEC msdb.dbo.sp_add_jobserver @job_name = 'SmartBank - Log Backup (15 min)';
GO
```

Figure 99

SQL Code to Configure Weekly Full Backups for Analytics Database

```
EXEC msdb.dbo.sp_add_job
     @job_name = 'SmartBank - Analytics Backup (Weekly)',
     @description = 'Weekly full backup of SmartBankAnalytics. No log backups - SIMPLE recovery.',
     @notify_level_eventlog = 2;

EXEC msdb.dbo.sp_add_jobstep
     @job_name  = 'SmartBank - Analytics Backup (Weekly)',
     @step_name = 'Analytics full backup',
     @subsystem = 'TSQL',
     @database_name = 'master',
     @command = N'
     DECLARE @File NVARCHAR(300) = ''C:\SmartBankServer\Backup\Analytics\SmartBankAnalytics_Full_'' +
                                   FORMAT(SYSDATETIME(), ''yyyyMMdd_HHmmss'') + ''.bak'';
     BACKUP DATABASE SmartBankAnalytics TO DISK = @File
     WITH INIT, NAME = ''Analytics Full'', CHECKSUM, STATS = 10;';

EXEC msdb.dbo.sp_attach_schedule
     @job_name = 'SmartBank - Analytics Backup (Weekly)',
     @schedule_name = 'SmartBank_Weekly_Sunday_0200';        -- Same schedule as production full backup

EXEC msdb.dbo.sp_add_jobserver @job_name = 'SmartBank - Analytics Backup (Weekly)';
GO
```

Due to the periodical execution of scheduled backups, it is also necessary to ensure that expired backup files are removed in time to avoid them from occupying the local storage and causing future backup failures. Therefore, as shown in Figure 100, an additional daily scheduled job is defined to perform cleanup by automatically deleting files that exceed the retention window.

Figure 100

SQL Code to Remove Expired Backup Files for Production and Analytics Database

```
EXEC msdb.dbo.sp_add_job
     @job_name = 'SmartBank - Backup Retention Cleanup (Daily)',
     @description = 'Deletes backup files past their retention window',
     @notify_level_eventlog = 2;

EXEC msdb.dbo.sp_add_jobstep
     @job_name  = 'SmartBank - Backup Retention Cleanup (Daily)',
     @step_name = 'Delete expired backup files',
     @subsystem = 'TSQL',
     @database_name = 'master',
     @command = N'
     EXEC master.dbo.xp_delete_file 0,
         N''C:\SmartBankServer\Backup\Production'', N''bak'', DATEADD(DAY, -28, SYSDATETIME());
     EXEC master.dbo.xp_delete_file 0,
         N''C:\SmartBankServer\Backup\Production'', N''trn'', DATEADD(DAY,  -3, SYSDATETIME());
     EXEC master.dbo.xp_delete_file 0,
         N''C:\SmartBankServer\Backup\Analytics'',  N''bak'', DATEADD(DAY, -28, SYSDATETIME());';

EXEC msdb.dbo.sp_add_schedule
     @schedule_name = 'SmartBank_Daily_0400',
     @freq_type = 4,                    -- daily
     @freq_interval = 1,
     @active_start_time = 040000;       -- at 4:00 am after all backups are performed

EXEC msdb.dbo.sp_attach_schedule
     @job_name = 'SmartBank - Backup Retention Cleanup (Daily)',
     @schedule_name = 'SmartBank_Daily_0400';

EXEC msdb.dbo.sp_add_jobserver @job_name = 'SmartBank - Backup Retention Cleanup (Daily)';
GO
```

To ensure that scheduled backups are properly configured for SmartBank, the two `SELECT` queries in Figure 101 are executed to verify that the configurations for the scheduled backups are aligned with the backup strategy, as well as to evidence the timely execution of different scheduled backups. The former result table shows that all scheduled backups, together with other jobs like backup purging tasks, are enabled with details like schedule frequency and active start time configured correctly, while the first three rows of the latter table indicate that transactional log backups are executed every 15 minutes after configuration, thereby proving that scheduled backup tasks are performed in an automated and timely manner.

Figure 101

SQL Code to Verify the Configuration and Execution of Job Schedules

```
-- Verify that job and schedule configuration is correct --
SELECT  j.name AS JobName,
        j.enabled,
        s.name AS ScheduleName,
        s.freq_type,
        s.freq_subday_interval,
        s.active_start_time
FROM    msdb.dbo.sysjobs AS j
LEFT JOIN msdb.dbo.sysjobschedules AS js ON js.job_id = j.job_id
LEFT JOIN msdb.dbo.sysschedules    AS s  ON s.schedule_id = js.schedule_id
WHERE   j.name LIKE 'SmartBank%'
ORDER BY j.name;
GO

-- Verify that backup was performed --
SELECT  bs.database_name,
        bs.backup_start_date,
        bs.backup_finish_date,
        CASE bs.type WHEN 'D' THEN 'Full'
                     WHEN 'I' THEN 'Differential'
                     WHEN 'L' THEN 'Transaction Log' END AS BackupType,
        CAST(bs.backup_size / 1048576.0 AS DECIMAL(10,2)) AS SizeMB,
        bmf.physical_device_name
FROM    msdb.dbo.backupset AS bs
JOIN    msdb.dbo.backupmediafamily AS bmf ON bmf.media_set_id = bs.media_set_id
WHERE   bs.database_name IN ('SmartBankProd','SmartBankAnalytics')
ORDER BY bs.backup_start_date DESC;
GO
```

	JobName	enabled	ScheduleName	freq_type	freq_subday_interval	active_start_time
1	SmartBank - Analytics Backup (Weekly)	1	SmartBank_Weekly_Sunday_0200	8	0	20000
2	SmartBank - Backup Retention Cleanup (Daily)	1	SmartBank_Daily_0400	4	0	40000
3	SmartBank - Differential Backup (Daily)	1	SmartBank_Daily_0300	4	0	30000
4	SmartBank - Full Backup (Weekly)	1	SmartBank_Weekly_Sunday_0200	8	0	20000
5	SmartBank - Log Backup (15 min)	1	SmartBank_Every_15_Minutes	4	15	0
6	SmartBank - Nightly Analytics ETL	1	SmartBank_Nightly_0100	4	0	10000

	database_name	backup_start_date	backup_finish_date	BackupType	SizeMB	physical_device_name
1	SmartBankProd	2026-08-20 00:00:00.000	2026-08-20 00:00:00.000	Transaction Log	0.08	C:\SmartBankServer\Backup\Production\SmartBankProd_Log_20260820_000000.trn
2	SmartBankProd	2026-08-19 23:45:00.000	2026-08-19 23:45:00.000	Transaction Log	0.08	C:\SmartBankServer\Backup\Production\SmartBankProd_Log_20260819_234500.trn
3	SmartBankProd	2026-08-19 23:30:01.000	2026-08-19 23:30:01.000	Transaction Log	0.08	C:\SmartBankServer\Backup\Production\SmartBankProd_Log_20260819_233000.trn
4	SmartBankProd	2026-08-19 23:13:13.000	2026-08-19 23:13:13.000	Full	6.40	C:\SmartBankServer\Backup\Production\SmartBankProd_Full.bak
5	SmartBankProd	2026-08-19 23:13:13.000	2026-08-19 23:13:13.000	Differential	0.77	C:\SmartBankServer\Backup\Production\SmartBankProd_Diff.bak
6	SmartBankProd	2026-08-19 23:13:13.000	2026-08-19 23:13:13.000	Transaction Log	0.08	C:\SmartBankServer\Backup\Production\SmartBankProd_Log.trn
7	SmartBankAnalytics	2026-08-19 23:13:13.000	2026-08-19 23:13:13.000	Full	5.09	C:\SmartBankServer\Backup\Analytics\SmartBankAnalytics_Full.bak

Restore and Recovery

With backup strategies implemented for the production and analytics databases of SmartBank, this allows any future data restoration and recovery tasks to be performed smoothly when the database experiences unexpected incidents like accidental data modifications, data loss, or database corruptions. Especially for financial institutions like SmartBank, having the capability to execute data restoration actions in a timely manner is critical, not only to mitigate the risk of service outages and provide users with a seamless experience, but also to minimise the negative impacts caused by these incidents towards their reliability and reputational status (Jane, 2025). Therefore, to demonstrate the disaster recovery capabilities of SmartBank's production database to comply with its 15-minute RPO threshold, a scenario involving an accidental data modification is simulated:

> **Scenario:** When using `sysadmin` to perform emergency operations, a bank officer accidentally runs an `UPDATE` operation without the `WHERE` clause, changing all account balance to `0`.

As the scenario leads to incorrect data stored in the production database, performing a simple full restore is unable to resolve the issue, as restoring to the latest transactional log backup or previous full and differential backups might lead to the loss of transactional records that are unrelated to the modification. This makes point-in-time restore more appropriate to be applied as it is able to restore the database up to a specific timestamp to exclude erroneous records while preserving the valid ones. Therefore, point-in-time restore is selected to demonstrate the recovery procedures for the production database:

- **Scenario Setup**

  The scenario of accidental data modification is simulated using the code in Figure 102, where all balances in the `Account` table are updated to `0` using `smartbank_admin`. However, before modification, a safe point-in-time is recorded using `SYSDATETIME()` and displayed in Figure 103 to act as a reference for the subsequent point-in-time restore.

  Figure 102

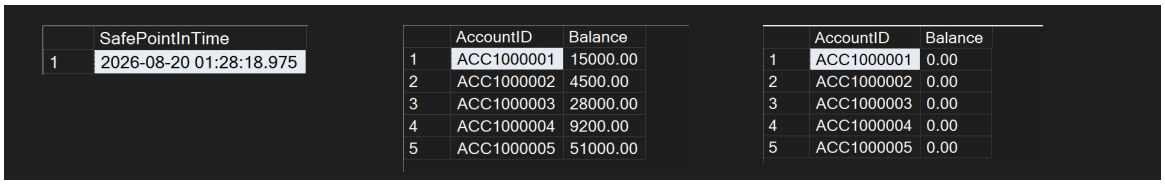
  SQL Code to Simulate Accidental Modification towards Customer Account Balance

```
DECLARE @SafePointInTime DATETIME2(3) = SYSDATETIME();   -- Safe point in time before incident --
SELECT @SafePointInTime AS SafePointInTime;

SELECT AccountID, Balance FROM dbo.Account;              -- Account details before modification --
UPDATE dbo.Account SET Balance = 0;                      -- Simulate incident --
SELECT AccountID, Balance FROM dbo.Account;              -- Account details after modification --
GO
```

Figure 103

Result Tables for Safe Point-in-Time, Account Details Before, and After Modification



	SafePointInTime
1	2026-08-20 01:28:18.975

	AccountID	Balance
1	ACC1000001	15000.00
2	ACC1000002	4500.00
3	ACC1000003	28000.00
4	ACC1000004	9200.00
5	ACC1000005	51000.00

	AccountID	Balance
1	ACC1000001	0.00
2	ACC1000002	0.00
3	ACC1000003	0.00
4	ACC1000004	0.00
5	ACC1000005	0.00

- **Pause existing backup jobs**

  After detecting the existence of anomalous records and identifying that recovery operations shall be performed, the first action is to immediately pause all existing scheduled backups, as shown in Figure 104. This is to prevent any automated backup jobs from overwriting existing files and affecting the restore chain for the subsequent point-in-time restore process.

Figure 104

SQL Code to Pause Existing Scheduled Backups on Production Database

```
EXEC msdb.dbo.sp_update_job @job_name = 'SmartBank - Full Backup (Weekly)',         @enabled = 0;
EXEC msdb.dbo.sp_update_job @job_name = 'SmartBank - Differential Backup (Daily)', @enabled = 0;
EXEC msdb.dbo.sp_update_job @job_name = 'SmartBank - Log Backup (15 min)',          @enabled = 0;
GO
```

- **Perform a tail-log backup**

  To preserve every transactional record taking place since the last transactional log backup, it is crucial to perform a final backup, which is also known as “tail-log backup”, using the code in Figure 105. This helps to ensure that transactions, specifically those that occur right before the modification, will not be lost when performing point-in-time restore.

Figure 105

SQL Code to Perform Tail-Log Backup for Production Database

```
BACKUP LOG SmartBankProd
    TO DISK = 'C:\SmartBankServer\Backup\Production\SmartBankProd_TailLog.trn'
    WITH INIT, NAME = 'Tail log before recovery';
GO
```

- **Restore the backup chain to a recovery database**

  To ensure that the production database remains operational during the data restoration process, all transactional records will first be restored on a separate recovery database with the name of `SmartBankProd_Recovery`. This allows restoration and verifications steps

to be performed separately before changes are migrated to the production database, hence avoiding service downtime and minimising interruptions to business users. As shown in Figure 106, recovery operations are performed following the sequence of full, differential and transactional logs, with `NORECOVERY` attached to ensure that the recovery database remains in a restoring state to facilitate the subsequent roll-forward process of restores.

Figure 106

SQL Code to Perform Full, Differential and Transactional Log Backup Restoration

```
USE master;
GO

-- Restore full backup --
RESTORE DATABASE SmartBankProd_Recovery
    FROM DISK = 'C:\SmartBankServer\Backup\Production\SmartBankProd_Full.bak'
    WITH MOVE 'SmartBankProd'
              TO 'C:\SmartBankServer\Backup\Production\SmartBankProd_Recovery.mdf',
         MOVE 'SmartBankProd_log'
              TO 'C:\SmartBankServer\Backup\Production\SmartBankProd_Recovery.ldf',
         NORECOVERY, REPLACE, STATS = 10;
GO

-- Restore differential backup --
RESTORE DATABASE SmartBankProd_Recovery
    FROM DISK = 'C:\SmartBankServer\Backup\Production\SmartBankProd_Diff.bak'
    WITH NORECOVERY, STATS = 10;
GO

-- Restore transactional log backup --
RESTORE LOG SmartBankProd_Recovery
    FROM DISK = 'C:\SmartBankServer\Backup\Production\SmartBankProd_Log.trn'
    WITH NORECOVERY, STATS = 10;
GO
```

- **Restore the tail-log backup with STOPAT**

  As shown in Figure 107, point-in-time restore is performed by restoring the tail-log backup while specifying the safe point-in-time of the production database using the `STOPAT` parameter. This allows SQL Server to only commit and restore the transactions that occur up till the specified point of time, hence excluding the problematic `UPDATE` statement that caused the accidental modification while preserving valid prior transactional records.

Figure 107

SQL Code to Perform Full, Differential and Transactional Log Backup Restoration

```
RESTORE LOG SmartBankProd_Recovery
    FROM DISK = 'C:\SmartBankServer\Backup\Production\SmartBankProd_TailLog.trn'
    WITH STOPAT = '2026-08-20 01:28:18.975', RECOVERY, STATS = 10;
GO
```

- **Verify data and reconcile records back into production**

  After the point-in-time restore completes, data verification shall be performed using the `SELECT` statement in Figure 108 to ensure that the recovered balances match with the values prior to the accidental modification. Upon confirming that the data values are accurate, the corrected details are then reconciled back to the production database, where in this case, the `Account` and `TransactionRecord` tables affected by the modifications are updated with the correct records using the code in Figure 109. The `UPDATE` commands are coupled under a single transaction to ensure that both tables are updated simultaneously and avoid the reconciliation process from introducing data inconsistency or integrity issues.

Figure 108

SQL Code to Verify if Recovered `Balance` *Values are Accurate*

```
SELECT AccountID, Balance FROM SmartBankProd_Recovery.dbo.Account;
```

	AccountID	Balance
1	ACC1000001	15000.00
2	ACC1000002	4500.00
3	ACC1000003	28000.00
4	ACC1000004	9200.00
5	ACC1000005	51000.00

Figure 109

SQL Code to Update `Account` *and* `TransactionRecord` *Tables with Correct Values*

```
BEGIN TRANSACTION;

UPDATE p                                    -- Account table --
SET    p.Balance = r.Balance
FROM   SmartBankProd.dbo.Account AS p
JOIN   SmartBankProd_Recovery.dbo.Account AS r ON r.AccountID = p.AccountID;

UPDATE p                                    -- Transaction record table --
SET    p.Amount          = r.Amount,
       p.TransactionType = r.TransactionType,
       p.TransDate       = r.TransDate,
       p.ToAccountID     = r.ToAccountID
FROM   SmartBankProd.dbo.TransactionRecord AS p
JOIN   SmartBankProd_Recovery.dbo.TransactionRecord AS r ON r.TransID = p.TransID;

COMMIT TRANSACTION;
GO

SELECT AccountID, Balance FROM SmartBankProd.dbo.Account;
GO
```

	AccountID	Balance
1	ACC1000001	15000.00
2	ACC1000002	4500.00
3	ACC1000003	28000.00
4	ACC1000004	9200.00
5	ACC1000005	51000.00

- **Re-enable existing backup jobs**

  Once all transactions are recovered successfully, scheduled backup jobs must be re-enabled, as shown in Figure 110, to restore the 15-minute RPO of the production database, as well as to ensure that backups can continue to be executed automatically according to their respective schedules to support future recovery processes.

Figure 110

SQL Code to Re-Enable Existing Scheduled Backups in Production

```
EXEC msdb.dbo.sp_update_job @job_name = 'SmartBank - Full Backup (Weekly)',        @enabled = 1;
EXEC msdb.dbo.sp_update_job @job_name = 'SmartBank - Differential Backup (Daily)', @enabled = 1;
EXEC msdb.dbo.sp_update_job @job_name = 'SmartBank - Log Backup (15 min)',         @enabled = 1;
GO

SELECT name, enabled FROM msdb.dbo.sysjobs WHERE name LIKE 'SmartBank%';
GO
```

	name	enabled
1	SmartBank - Nightly Analytics ETL	1
2	SmartBank - Log Backup (15 min)	1
3	SmartBank - Full Backup (Weekly)	1
4	SmartBank - Analytics Backup (Weekly)	1
5	SmartBank - Differential Backup (Daily)	1
6	SmartBank - Backup Retention Cleanup (Daily)	1

- **Verify backups are restorable**

  As a complementary check, it is also crucial to verify that the backup files are valid and can be restored when needed. Therefore, using the code in Figure 111, the integrity of the backup file is verified, where `RESTORE VERIFYONLY` ensures that the backup file is uncorrupted and structurally complete, while `RESTORE HEADERONLY` extracts the metadata of the backup file to ensure that the correct file and database are being referenced, both providing a lightweight check to support the execution of future data restoration operations.

Figure 111

SQL Code to Verify the Status of Backup Files for Future Restoration Process

```
RESTORE VERIFYONLY FROM DISK = 'C:\SmartBankServer\Backup\Production\SmartBankProd_Full.bak';
RESTORE HEADERONLY FROM DISK = 'C:\SmartBankServer\Backup\Production\SmartBankProd_Full.bak';
```

```
The backup set on file 1 is valid.

Completion time: 2026-08-20T01:34:55.4772704+08:00
```

	BackupName	BackupDescription	BackupType	ExpirationDate	Compressed	Position	DeviceType	UserName	ServerName
1	Baseline full	NULL	1	NULL	0	1	2	smartbank_admin	LIMBENGRH

- **Drop the recovery database**

  Once the production database is recovered with its lost data restored, the recovery database shall be removed to prevent it from occupying additional disk space. Hence, using the code in Figure 112, the `SmartBankProd_Recovery` database is dropped, together with its physical `.mdf` and `.ldf` files removed from local directory, as shown in Figure 113. This marks the completion of the point-in-time restore demonstration for production.

Figure 112

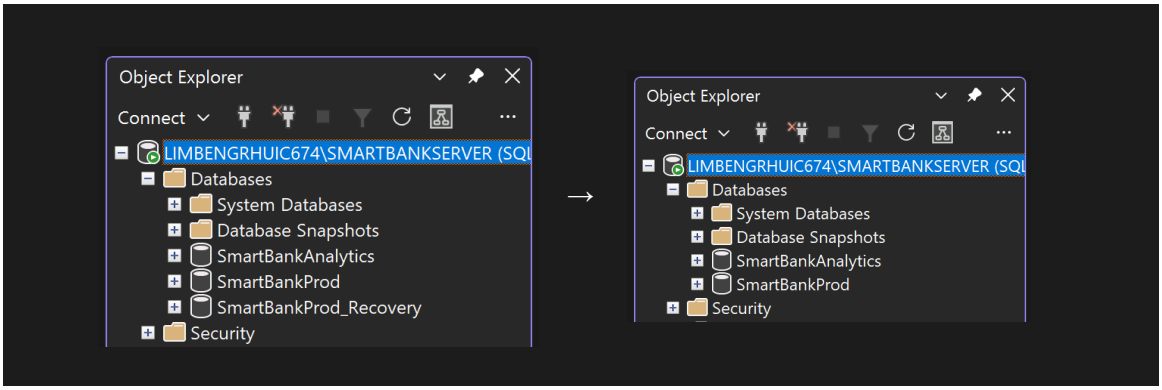
SQL Code to Drop the Recovery Database

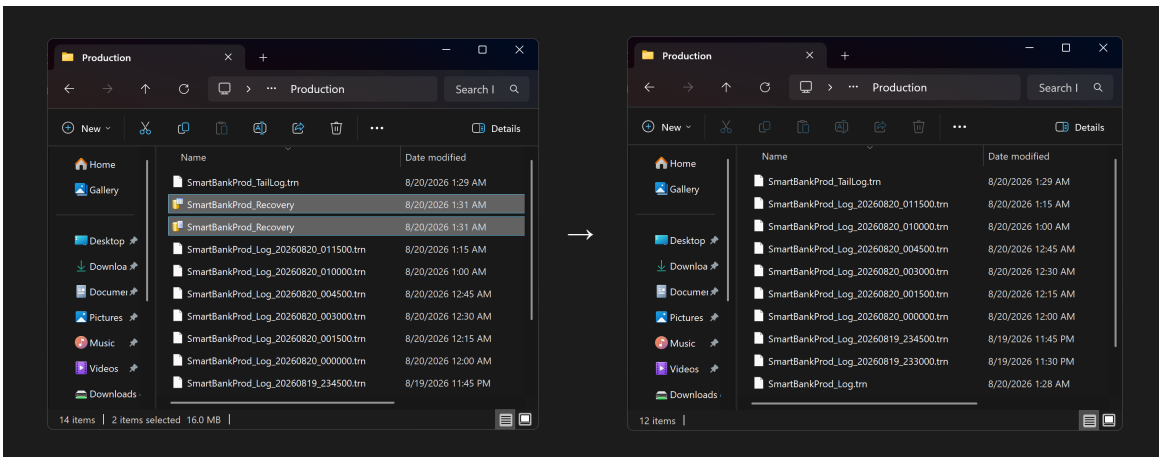
```
USE master;
GO

IF DB_ID('SmartBankProd_Recovery') IS NOT NULL
BEGIN
    ALTER DATABASE SmartBankProd_Recovery SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE SmartBankProd_Recovery;
END
GO
```

Figure 113

Evidence of the Removal of Recovery Database





Audit

Although SmartBank is currently implemented with a comprehensive range of security mechanisms, encompassing areas from access level controls to data layer security, these controls are preventative in nature (Willert, 2022). In other words, if unauthorised users gain the opportunity to bypass these mechanisms and perform illicit read and write operations on databases, there is no option for bank managers or database administrators to detect the execution of these actions and perform back-tracing to identify the culprit. To overcome such a limitation, SmartBank has also implemented database audit mechanisms to complement existing security controls, where suspicious activities taking place in SmartBank's database will be monitored and captured automatically in real time (Gahlot et al., 2018). Aside from supporting the robust implementation of detective and corrective mechanisms, having an audit framework also contributes to operational accountability as each action can be mapped to its corresponding user, hence strengthening the data integrity and transparency of SmartBank's production and analytics databases (SearchInform, 2026).

Audit Strategy

Both the production and analytics database of SmartBank are implemented with the same audit strategy to achieve a consistent audit coverage so that attackers are unable to exploit security disparities and compromise data integrity. Audit is mainly implemented using techniques like:

- **SQL Server Audit**

  Due to its security robustness, the SQL Server Audit is selected as the primary mechanism to capture the audit logs for SmartBank as it is able to capture all actions performed by users as long as the actions are covered by the audit specifications, no matter the client used.

- **System-versioned temporal tables**

  While SQL Server Audit is able to record the actions that occur along with their timestamps, it is unable to directly display the row values of a table at a specific point of time. Hence, system-versioned temporal tables are implemented to overcome such a gap, allowing historical data to be viewed without having to perform complex data restoration operations.

- **Procedure-level logging**

  Since the SQL Server Audit only captures statement that reaches the database engine, it is unable to record the refusals that are triggered by the business-rule logic in user-defined stored procedures. Therefore, the `Security.usp_LogRejection` helper procedure is executed whenever a refusal happens, ensuring that details of any actions are always logged.

Audit Matrix

The audit matrix for SmartBank is shown in Table 7, where each database object is mapped to their corresponding strategy.

Table 7

Audit Matrix for SmartBank

Object Name	Type	What to Capture	Mechanism	Audit Log Location
`SmartBankServer`	Server	• Successful login • Failed login • Logout of every session	SQL Server Audit	`C:\SmartBankServer\Audit`

`SmartBank_ServerAudit`
`SmartBank_ServerAuditSpec`
`SmartBankProd_AuditSpec`
`SmartBankAnalytics_AuditSpec`
	Audit Object	• Create audit
		• Alter audit
		• Disable audit
		• Drop audit
		• Change audit specification
		• Stop or restart the SQL server
		• Run database console commands (DBCC)
		• Refused attempt
			SQL Server Audit	`C:\SmartBankServer\Audit`

`SmartBankProd` `SmartBankAnalytics`	Database	• Create object in database • Alter object in database • Drop object in database • Grant permission • Deny permission • Revoke permission • Change database principal • Alter database settings • Disable TDE • Change recovery model • Back up database • Restore database • Refused attempt	SQL Server Audit	`C:\SmartBankServer\Audit`

Object Name	Type	What to Capture	Mechanism	Audit Log Location
dbo.Staff dbo.Customer dbo.Account dbo.TransactionRecord Security.UserMapping Security.RejectedAttemptLog Reporting.DimCustomer Reporting.FactAccount Reporting.FactTransaction Reporting.ETLRunLog	Table	• Select from table • Insert into table • Update table • Delete from table • Refused attempt	SQL Server Audit	C:\SmartBankServer\Audit
dbo.StaffHistory dbo.CustomerHistory dbo.AccountHistory Security.UserMappingHistory Security.RejectedAttemptLogHistory	Table	• Select from table • Refused attempt	SQL Server Audit	C:\SmartBankServer\Audit
dbo.vw_StaffDirectory dbo.vw_MyStaffRecord dbo.vw_AllStaff dbo.vw_CustomerProfile dbo.vw_CustomerAccounts dbo.vw_CustomerTransactions dbo.vw_CustomerIC Reporting.vw_ETLHealth Reporting.vw_AccountSummary Reporting.vw_TransactionTrend Reporting.vw_CustomerActivity	View	• Select from view • Refused attempt	SQL Server Audit	C:\SmartBankServer\Audit
dbo.usp_Deposit dbo.usp_Withdraw dbo.usp_Transfer dbo.usp_UpdateOwnStaffRecord dbo.usp_UpdateStaffDetails dbo.usp_CreateCustomerAccount dbo.usp_UpdateCustomerAccount dbo.usp_RegisterCustomer dbo.usp_GetCustomerIC Reporting.usp_RunETL	Stored Procedure	• Execute stored procedure • Refused execution	SQL Server Audit	C:\SmartBankServer\Audit

Object Name	Type	What to Capture	Mechanism	Audit Log Location
`Security.usp_LogRejection`	Stored Procedure	• Execute stored procedure • Insert into `Security.RejectedAttemptLog` • Refused direct read of the log • Refused execution	SQL Server Audit	`C:\SmartBankServer\Audit`
`dbo.fn_StaffDirectory`	Function	• Execute function • Alter function • Drop function • Refused attempt	SQL Server Audit	`C:\SmartBankServer\Audit`

`dba_ahmad`
`mgr_aisyah`
`mgr_ravi`
`ofc_lim`
`ofc_nurul`
`cust_tan`
`cust_siti`
`cust_kumar`
`cust_wong`
`analyst_lee`
`svc_ETL`
`smartbank_admin`
	SQL Login	• Create login
		• Alter login
		• Drop login
		• Change server role membership
		• Change server permission
		• Change login password
		• Refused attempt
			SQL Server Audit	`C:\SmartBankServer\Audit`

`dba_ahmad`
`mgr_aisyah`
`mgr_ravi`
`ofc_lim`
`ofc_nurul`
`cust_tan`
`cust_siti`
`cust_kumar`
`cust_wong`
`analyst_lee`
`svc_ETL`
`smartbank_admin`
	Impersonation	• `EXECUTE AS LOGIN`
		• `EXECUTE AS USER`
		• `REVERT`
			SQL Server Audit	`C:\SmartBankServer\Audit`

Object Name	Type	What to Capture	Mechanism	Audit Log Location

`role_DBA`
`role_Manager`
`role_Officer`
`role_Customer`
`role_ETL_Reader`
`role_ETL_Service`
`role_Analyst_ReadOnly`
	Database Role	• Add member to role
		• Remove member from role
		• Refused attempt
			SQL Server Audit	`C:\SmartBankServer\Audit`

`Security.fn_StaffAccessPredicate`
`Security.fn_AccountAccessPredicate`
`Security.fn_TransactionAccessPredicate`
`Security.fn_CustomerAccessPredicate`
`Security.fn_UserMappingPredicate`
	Predicate Function	• Alter predicate function
		• Drop predicate function
		• Refused attempt
			SQL Server Audit	`C:\SmartBankServer\Audit`

`Security.StaffSecurityPolicy`
`Security.AccountSecurityPolicy`
`Security.TransactionSecurityPolicy`
`Security.CustomerSecurityPolicy`
`Security.UserMappingSecurityPolicy`
	Security Policy	• Create security policy
		• Alter security policy
		• Drop security policy
		• Disable row-level security
		• Refused attempt
			SQL Server Audit	`C:\SmartBankServer\Audit`

`dbo.Staff`
`dbo.Customer`
`dbo.Account`
`Security.UserMapping`
`Security.RejectedAttemptLog`
	Table	• Reconstruct prior row version on update
		• Reconstruct prior row version on delete
			System-versioned Temporal Table	`dbo.StaffHistory`
				`dbo.CustomerHistory`
				`dbo.AccountHistory`
				`Security.UserMappingHistory`
				`Security.RejectedAttemptLogHistory`

`dbo.trg_Staff_SalaryGuard`	Trigger	• Reject edit of own salary • Reject salary change by non-manager	Procedure-level Logging	`Security.RejectedAttemptLog`
`dbo.trg_Transaction_NoDelete`	Trigger	• Reject deletion of transaction record	Procedure-level Logging	`Security.RejectedAttemptLog`
`dbo.usp_Deposit`	Stored Procedure	• Reject caller not in `role_Customer` • Reject account not belonging to caller • Reject non-positive amount	Procedure-level Logging	`Security.RejectedAttemptLog`

Object Name	Type	What to Capture	Mechanism	Audit Log Location
`dbo.usp_Withdraw`	Stored Procedure	• Reject caller not in `role_Customer` • Reject account not belonging to caller • Reject non-positive amount • Reject insufficient funds	Procedure-level Logging	`Security.RejectedAttemptLog`
`dbo.usp_Transfer`	Stored Procedure	• Reject caller not in `role_Customer` • Reject source account not belonging to caller • Reject same source and destination account • Reject non-existent destination account • Reject non-positive amount • Reject insufficient funds	Procedure-level Logging	`Security.RejectedAttemptLog`
`dbo.usp_UpdateOwnStaffRecord`	Stored Procedure	• Reject caller not in any staff role • Reject caller with no mapped staff record	Procedure-level Logging	`Security.RejectedAttemptLog`
`dbo.usp_UpdateStaffDetails`	Stored Procedure	• Reject caller not in `role_Manager` • Reject staff record not found • Reject manager updating own staff record	Procedure-level Logging	`Security.RejectedAttemptLog`
`dbo.usp_CreateCustomerAccount`	Stored Procedure	• Reject caller not in `role_Officer` • Reject non-existent customer • Reject negative opening balance	Procedure-level Logging	`Security.RejectedAttemptLog`
`dbo.usp_UpdateCustomerAccount`	Stored Procedure	• Reject caller not in `role_Officer` • Reject account not found	Procedure-level Logging	`Security.RejectedAttemptLog`
`dbo.usp_RegisterCustomer`	Stored Procedure	• Reject caller not in `role_Officer` • Reject customer already registered • Reject IC number failing format validation	Procedure-level Logging	`Security.RejectedAttemptLog`
`dbo.usp_GetCustomerIC`	Stored Procedure	• Reject caller not in `role_Manager`	Procedure-level Logging	`Security.RejectedAttemptLog`

Implementation

Based on the audit matrix defined in Table 7, audit is implemented across different areas for both the production and analytics database of SmartBank.

Initialisation of Audit Objects

Before specifying the actions to be captured, several audit objects must first be initialised, as shown in Figure 114. To define the overall audit behaviour of SmartBank, a server audit named `SmartBank_ServerAudit` is created to allow details like the audit file storage location and failure handling mechanisms to be configured. This is followed by the creation of several audit specifications, including `SmartBank_ServerAuditSpec` to specify the server-level events to be captured, as well as `SmartBankProd_AuditSpec` and `SmartBankAnalytics_AuditSpec` that respectively record database-level events occurring in production and analytics database.

Figure 114

Initialisation of Audit Objects for SmartBank

```
USE master;
GO

CREATE SERVER AUDIT SmartBank_ServerAudit TO FILE (              -- Server audit
    FILEPATH           = 'C:\SmartBankServer\Audit\',
    MAXSIZE            = 100 MB,
    MAX_ROLLOVER_FILES = 10,
    RESERVE_DISK_SPACE = OFF
) WITH (
    QUEUE_DELAY = 0,
    ON_FAILURE  = CONTINUE
);
GO

ALTER SERVER AUDIT SmartBank_ServerAudit WITH (STATE = ON);      -- Enable server audit
GO

CREATE SERVER AUDIT SPECIFICATION SmartBank_ServerAuditSpec      -- Server audit specification
FOR SERVER AUDIT SmartBank_ServerAudit
WITH (STATE = OFF);
GO

USE SmartBankProd;
GO

CREATE DATABASE AUDIT SPECIFICATION SmartBankProd_AuditSpec      -- Audit specification (production)
FOR SERVER AUDIT SmartBank_ServerAudit
WITH (STATE = OFF);
GO

USE SmartBankAnalytics;
GO

CREATE DATABASE AUDIT SPECIFICATION SmartBankAnalytics_AuditSpec -- Audit specification (analytics)
FOR SERVER AUDIT SmartBank_ServerAudit
WITH (STATE = OFF);
GO
```

Login and Logout Audit

As authentication events take place at the server level before users gain access to the database, audits that are associated to login and logout attempts are configured under the server audit specification. Several audit action groups are added to the `SmartBank_ServerAuditSpec` audit specification to capture the full lifecycle of database sessions, as shown in Figure 115. `SUCCESSFUL_LOGIN_GROUP` captures all the successful authentication and login attempts to SmartBank's database, `FAILED_LOGIN_GROUP` handles the authentication operations that are rejected by the server, while `LOGOUT_GROUP` records the end of a connection session.

Figure 115

Configuration of Login and Logout Audit for SmartBank

```
USE master;
GO

ALTER SERVER AUDIT SPECIFICATION SmartBank_ServerAuditSpec
    ADD (SUCCESSFUL_LOGIN_GROUP),
    ADD (FAILED_LOGIN_GROUP),
    ADD (LOGOUT_GROUP);
GO
```

To verify the configuration for the audit of authentication access, the code in Figure 116 is executed to display the details of the three audit action groups. Based on the result, it is shown that all three audit action groups are successfully added to the server audit specification, but the state of the audit specification is currently disabled. This design is intentional as enabling the audit specifications will disrupt further additional configurations, hence all specifications will be disabled currently until the configuration for all audit areas is completed.

Figure 116

SQL Code to Verify the Configuration for Login and Logout Audit

```
SELECT  d.audit_action_name, s.name AS SpecificationName, s.is_state_enabled
FROM    sys.server_audit_specifications AS s
JOIN    sys.server_audit_specification_details AS d
        ON d.server_specification_id = s.server_specification_id
WHERE   d.audit_action_name IN ('SUCCESSFUL_LOGIN_GROUP','FAILED_LOGIN_GROUP','LOGOUT_GROUP')
ORDER BY d.audit_action_name;
GO
```

	audit_action_name	SpecificationName	is_state_enabled
1	FAILED_LOGIN_GROUP	SmartBank_ServerAuditSpec	0
2	LOGOUT_GROUP	SmartBank_ServerAuditSpec	0
3	SUCCESSFUL_LOGIN_GROUP	SmartBank_ServerAuditSpec	0

Data Manipulation Language (DML) Audit

Unlike the login and logout audit that requires different audit action groups to be added to the server audit specification, audits related to data manipulation language (DML) often involve operations like `SELECT`, `INSERT`, `UPDATE` and `DELETE`, which are captured by the database audit specifications at the object and schema level. As shown in Figure 117, individual audit actions are specified for the schemas involved in both the production and analytics database, such as `dbo`, `Security`, and `Reporting`, not only to ensure that operations involving these objects are captured with sufficient granularity to improve audit traceability, but also to allow database objects that are subsequently added to these schemas to be able to automatically inherit any audit configurations set for the schema without requiring manual configuration, hence improving the reliability of audit in SmartBank.

Figure 117

Configuration of Data Manipulation Language (DML) Audit for SmartBank

```
-- DML audit (production) --
USE SmartBankProd;
GO

ALTER DATABASE AUDIT SPECIFICATION SmartBankProd_AuditSpec

    -- dbo schema --
    ADD (SELECT, INSERT, UPDATE, DELETE, EXECUTE ON SCHEMA::dbo BY public),

    -- Security schema --
    ADD (SELECT, INSERT, UPDATE, DELETE, EXECUTE ON SCHEMA::[Security] BY public);
GO

-- DML audit (analytics) --
USE SmartBankAnalytics;
GO

-- Reporting schema --
ALTER DATABASE AUDIT SPECIFICATION SmartBankAnalytics_AuditSpec
    ADD (SELECT, INSERT, UPDATE, DELETE, EXECUTE ON SCHEMA::Reporting BY public);
GO
```

Additionally, system-versioned temporal tables are implemented on tables in the production database, including `Staff`, `Customer`, `Account`, `Security.UserMapping`, and `Security.RejectedAttemptLog`, as shown in Figure 118. By having such a mechanism, in case if unexpected `UPDATE` or `DELETE` occur, the previous values of table rows can be examined without requiring backup restoration to minimise disruption towards live operations. However, to maintain data confidentiality, business users are denied from having direct access to these temporal tables, where only privileged accounts like `smartbank_admin` will have the permission to access these tables when investigations are required to be performed.

Figure 118

Configuration of System-Versioned Temporal Tables for SmartBank

```
USE SmartBankProd;
GO

ALTER TABLE dbo.Staff ADD
    SysStartTime DATETIME2 GENERATED ALWAYS AS ROW START NOT NULL
        CONSTRAINT DF_Staff_SysStart DEFAULT SYSUTCDATETIME(),
    SysEndTime DATETIME2 GENERATED ALWAYS AS ROW END NOT NULL
        CONSTRAINT DF_Staff_SysEnd DEFAULT '9999-12-31 23:59:59.9999999',
    PERIOD FOR SYSTEM_TIME (SysStartTime, SysEndTime);
GO
ALTER TABLE dbo.Staff SET (SYSTEM_VERSIONING = ON (HISTORY_TABLE = dbo.StaffHistory));
GO

ALTER TABLE dbo.Customer ADD
    SysStartTime DATETIME2 GENERATED ALWAYS AS ROW START NOT NULL
        CONSTRAINT DF_Customer_SysStart DEFAULT SYSUTCDATETIME(),
    SysEndTime DATETIME2 GENERATED ALWAYS AS ROW END NOT NULL
        CONSTRAINT DF_Customer_SysEnd DEFAULT '9999-12-31 23:59:59.9999999',
    PERIOD FOR SYSTEM_TIME (SysStartTime, SysEndTime);
GO
ALTER TABLE dbo.Customer SET (SYSTEM_VERSIONING = ON (HISTORY_TABLE = dbo.CustomerHistory));
GO

ALTER TABLE dbo.Account ADD
    SysStartTime DATETIME2 GENERATED ALWAYS AS ROW START NOT NULL
        CONSTRAINT DF_Account_SysStart DEFAULT SYSUTCDATETIME(),
    SysEndTime DATETIME2 GENERATED ALWAYS AS ROW END NOT NULL
        CONSTRAINT DF_Account_SysEnd DEFAULT '9999-12-31 23:59:59.9999999',
    PERIOD FOR SYSTEM_TIME (SysStartTime, SysEndTime);
GO
ALTER TABLE dbo.Account SET (SYSTEM_VERSIONING = ON (HISTORY_TABLE = dbo.AccountHistory));
GO

ALTER TABLE [Security].UserMapping ADD
    SysStartTime DATETIME2 GENERATED ALWAYS AS ROW START NOT NULL
        CONSTRAINT DF_UserMapping_SysStart DEFAULT SYSUTCDATETIME(),
    SysEndTime DATETIME2 GENERATED ALWAYS AS ROW END NOT NULL
        CONSTRAINT DF_UserMapping_SysEnd DEFAULT '9999-12-31 23:59:59.9999999',
    PERIOD FOR SYSTEM_TIME (SysStartTime, SysEndTime);
GO
ALTER TABLE [Security].UserMapping
    SET (SYSTEM_VERSIONING = ON (HISTORY_TABLE = [Security].UserMappingHistory));
GO

ALTER TABLE [Security].RejectedAttemptLog ADD
    SysStartTime DATETIME2 GENERATED ALWAYS AS ROW START NOT NULL
        CONSTRAINT DF_RejectedAttemptLog_SysStart DEFAULT SYSUTCDATETIME(),
    SysEndTime DATETIME2 GENERATED ALWAYS AS ROW END NOT NULL
        CONSTRAINT DF_RejectedAttemptLog_SysEnd DEFAULT '9999-12-31 23:59:59.9999999',
    PERIOD FOR SYSTEM_TIME (SysStartTime, SysEndTime);
GO
ALTER TABLE [Security].RejectedAttemptLog
    SET (SYSTEM_VERSIONING = ON (HISTORY_TABLE = [Security].RejectedAttemptLogHistory));
GO

DENY SELECT, INSERT, UPDATE, DELETE ON dbo.StaffHistory
    TO role_DBA, role_Manager, role_Officer, role_Customer, role_ETL_Reader;
DENY SELECT, INSERT, UPDATE, DELETE ON dbo.CustomerHistory
    TO role_DBA, role_Manager, role_Officer, role_Customer, role_ETL_Reader;
DENY SELECT, INSERT, UPDATE, DELETE ON dbo.AccountHistory
    TO role_DBA, role_Manager, role_Officer, role_Customer, role_ETL_Reader;
DENY SELECT, INSERT, UPDATE, DELETE ON [Security].UserMappingHistory
    TO role_DBA, role_Manager, role_Officer, role_Customer, role_ETL_Reader;
DENY SELECT, INSERT, UPDATE, DELETE ON [Security].RejectedAttemptLogHistory
    TO role_DBA, role_Manager, role_Officer, role_Customer, role_ETL_Reader;
GO
```

To review the configurations for both system-versioned temporal tables and the DML audit, multiple `SELECT` queries are executed. From the result in Figure 119, it is shown that the database objects within the production and analytics database are correctly associated with their audit actions. Also, result from Figure 120 shows that the tables specified by the code in Figure 118 are correctly versioned, with their corresponding history table names displayed under the `HistoryTableName` column. As these history tables contain full row data from their original tables, including sensitive columns like `Salary`, it is crucial to ensure that no business users have direct access to these tables, which is proven by Figure 121 where the permissions to access the raw history tables are assigned `DENY` to all business roles.

Figure 119

SQL Code to Verify the Configuration for DML Audit on Production Database

```
USE SmartBankProd;
GO
SELECT  s.name AS SpecificationName, d.audit_action_name, d.class_desc,
        CASE d.class_desc
            WHEN 'OBJECT_OR_COLUMN' THEN OBJECT_NAME(d.major_id)
            WHEN 'SCHEMA'           THEN SCHEMA_NAME(d.major_id)
        END AS AuditedObject,
        d.audited_principal_id
FROM    sys.database_audit_specifications AS s
JOIN    sys.database_audit_specification_details AS d
        ON d.database_specification_id = s.database_specification_id
WHERE   d.class_desc IN ('OBJECT_OR_COLUMN','SCHEMA')
ORDER BY AuditedObject, audit_action_name;
GO

USE SmartBankAnalytics;
GO
SELECT  s.name AS SpecificationName, d.audit_action_name, d.class_desc,
        CASE d.class_desc
            WHEN 'OBJECT_OR_COLUMN' THEN OBJECT_NAME(d.major_id)
            WHEN 'SCHEMA'           THEN SCHEMA_NAME(d.major_id)
        END AS AuditedObject,
        d.audited_principal_id
FROM    sys.database_audit_specifications AS s
JOIN    sys.database_audit_specification_details AS d
        ON d.database_specification_id = s.database_specification_id
WHERE   d.class_desc IN ('OBJECT_OR_COLUMN','SCHEMA')
ORDER BY AuditedObject, audit_action_name;
GO
```

	SpecificationName	audit_action_name	class_desc	AuditedObject	audited_principal_id
1	SmartBankProd_AuditSpec	DELETE	SCHEMA	dbo	0
2	SmartBankProd_AuditSpec	EXECUTE	SCHEMA	dbo	0
3	SmartBankProd_AuditSpec	INSERT	SCHEMA	dbo	0
4	SmartBankProd_AuditSpec	SELECT	SCHEMA	dbo	0
5	SmartBankProd_AuditSpec	UPDATE	SCHEMA	dbo	0
6	SmartBankProd_AuditSpec	DELETE	SCHEMA	Security	0
7	SmartBankProd_AuditSpec	EXECUTE	SCHEMA	Security	0
8	SmartBankProd_AuditSpec	INSERT	SCHEMA	Security	0
9	SmartBankProd_AuditSpec	SELECT	SCHEMA	Security	0
10	SmartBankProd_AuditSpec	UPDATE	SCHEMA	Security	0

	SpecificationName	audit_action_name	class_desc	AuditedObject	audited_principal_id
1	SmartBankAnalytics_AuditSpec	DELETE	SCHEMA	Reporting	0
2	SmartBankAnalytics_AuditSpec	EXECUTE	SCHEMA	Reporting	0
3	SmartBankAnalytics_AuditSpec	INSERT	SCHEMA	Reporting	0
4	SmartBankAnalytics_AuditSpec	SELECT	SCHEMA	Reporting	0
5	SmartBankAnalytics_AuditSpec	UPDATE	SCHEMA	Reporting	0

Figure 120

SQL Code to Verify that Tables in Production are Versioned

```
USE SmartBankProd;
GO

SELECT  sc.name AS SchemaName, t.name AS TableName, t.temporal_type_desc,
        h.name  AS HistoryTableName
FROM    sys.tables AS t
JOIN    sys.schemas AS sc ON sc.schema_id = t.schema_id
LEFT JOIN sys.tables AS h ON h.object_id = t.history_table_id
WHERE   (sc.name = 'dbo' AND t.name IN ('Staff','Customer','Account'))
   OR   (sc.name = 'Security' AND t.name IN ('UserMapping','RejectedAttemptLog'))
ORDER BY sc.name, t.name;
```

	SchemaName	TableName	temporal_type_desc	HistoryTableName
1	dbo	Account	SYSTEM_VERSIONED_TEMPORAL_TABLE	AccountHistory
2	dbo	Customer	SYSTEM_VERSIONED_TEMPORAL_TABLE	CustomerHistory
3	dbo	Staff	SYSTEM_VERSIONED_TEMPORAL_TABLE	StaffHistory
4	Security	RejectedAttemptLog	SYSTEM_VERSIONED_TEMPORAL_TABLE	RejectedAttemptLogHistory
5	Security	UserMapping	SYSTEM_VERSIONED_TEMPORAL_TABLE	UserMappingHistory

Figure 121

SQL Code to Verify that No One has Access to the History Tables

```
USE SmartBankProd;
GO

SELECT  OBJECT_SCHEMA_NAME(p.major_id) AS SchemaName,
        OBJECT_NAME(p.major_id)        AS HistoryTableName,
        dp.name                        AS GranteeRole,
        p.permission_name, p.state_desc
FROM    sys.database_permissions AS p
JOIN    sys.database_principals  AS dp ON dp.principal_id = p.grantee_principal_id
WHERE   p.major_id IN (
            OBJECT_ID('dbo.StaffHistory'),
            OBJECT_ID('dbo.CustomerHistory'),
            OBJECT_ID('dbo.AccountHistory'),
            OBJECT_ID('Security.UserMappingHistory'),
            OBJECT_ID('Security.RejectedAttemptLogHistory'));
```

	SchemaName	HistoryTableName	GranteeRole	permission_name	state_desc
1	dbo	StaffHistory	role_DBA	DELETE	DENY
2	dbo	StaffHistory	role_DBA	INSERT	DENY
3	dbo	StaffHistory	role_DBA	SELECT	DENY
4	dbo	StaffHistory	role_DBA	UPDATE	DENY
5	dbo	StaffHistory	role_Manager	DELETE	DENY
6	dbo	StaffHistory	role_Manager	INSERT	DENY
7	dbo	StaffHistory	role_Manager	SELECT	DENY
8	dbo	StaffHistory	role_Manager	UPDATE	DENY
9	dbo	StaffHistory	role_Officer	DELETE	DENY
10	dbo	StaffHistory	role_Officer	INSERT	DENY
11	dbo	StaffHistory	role_Officer	SELECT	DENY
12	dbo	StaffHistory	role_Officer	UPDATE	DENY
13	dbo	StaffHistory	role_Customer	DELETE	DENY
14	dbo	StaffHistory	role_Customer	INSERT	DENY
15	dbo	StaffHistory	role_Customer	SELECT	DENY
16	dbo	StaffHistory	role_Customer	UPDATE	DENY
17	dbo	StaffHistory	role_ETL_Reader	DELETE	DENY
18	dbo	StaffHistory	role_ETL_Reader	INSERT	DENY
19	dbo	StaffHistory	role_ETL_Reader	SELECT	DENY
20	dbo	StaffHistory	role_ETL_Reader	UPDATE	DENY
21	dbo	CustomerHistory	role_DBA	DELETE	DENY
22	dbo	CustomerHistory	role_DBA	INSERT	DENY
23	dbo	CustomerHistory	role_DBA	SELECT	DENY
24	dbo	CustomerHistory	role_DBA	UPDATE	DENY

Data Definition Language (DDL) Audit

Data definition language (DDL) audit involves the creation, modification and removal of database objects via operations like `CREATE`, `ALTER` and `DROP`, which, unlike DML audits, can only be captured via predefined audit action groups in SQL Server. Therefore, as shown in Figure 122, different audit action groups are assigned to the server and database audit specifications. `DATABASE_CHANGE_GROUP` is added to the server audit specification as it handles any structural changes related to the database, such as the `ALTER DATABASE` operation, which only requires configuration at the server level to cover both the production and analytics databases at once. Conversely, audit action groups like `DATABASE_OBJECT_CHANGE_GROUP` that audits database-level objects like schemas and users, and `SCHEMA_OBJECT_CHANGE_GROUP` that handles schema-level objects like tables and views, are separately added to the database audit specifications of both the production and analytics database.

Figure 122

Configuration of Data Definition Language (DDL) Audit for SmartBank

```
USE master;
GO

ALTER SERVER AUDIT SPECIFICATION SmartBank_ServerAuditSpec
    ADD (DATABASE_CHANGE_GROUP);        -- Alter database
GO

USE SmartBankProd;
GO

ALTER DATABASE AUDIT SPECIFICATION SmartBankProd_AuditSpec
    ADD (SCHEMA_OBJECT_CHANGE_GROUP),    -- Tables and views
    ADD (DATABASE_OBJECT_CHANGE_GROUP);  -- Stored procedure, functions, policies
GO

USE SmartBankAnalytics;
GO

ALTER DATABASE AUDIT SPECIFICATION SmartBankAnalytics_AuditSpec
    ADD (SCHEMA_OBJECT_CHANGE_GROUP),
    ADD (DATABASE_OBJECT_CHANGE_GROUP);
GO
```

The code in Figure 123 is then executed to review the configuration of DDL audit on all audit specifications. Based on the result, it is evident that all audit specifications are accurately configured with their respective audit action groups, where `DATABASE_CHANGE_GROUP` that only appeared once under `SmartBank_ServerAuditSpec` indicates that its single configuration is able to capture any events related to database changes at a server level, while `DATABASE_OBJECT_CHANGE_GROUP` and `SCHEMA_OBJECT_CHANGE_GROUP` that appears once in each database audit specification confirms that DDL is evenly covered across both databases.

Figure 123

SQL Code to Verify the Configuration for DDL Audit for SmartBank

```
-- Verify for server audit spec --
SELECT  d.audit_action_name, s.name AS SpecificationName, s.is_state_enabled
FROM    sys.server_audit_specifications AS s
JOIN    sys.server_audit_specification_details AS d
        ON d.server_specification_id = s.server_specification_id
WHERE   d.audit_action_name IN ('DATABASE_CHANGE_GROUP')
ORDER BY d.audit_action_name;
GO

-- Verify for database audit spec (production & analytics) --
SELECT  'SmartBankProd' AS DatabaseName, s.name AS SpecificationName,
        d.audit_action_name, d.class_desc, d.audited_principal_id
FROM    SmartBankProd.sys.database_audit_specifications AS s
JOIN    SmartBankProd.sys.database_audit_specification_details AS d
        ON d.database_specification_id = s.database_specification_id
WHERE   d.audit_action_name IN ('SCHEMA_OBJECT_CHANGE_GROUP','DATABASE_OBJECT_CHANGE_GROUP')
UNION ALL
SELECT  'SmartBankAnalytics', s.name,
        d.audit_action_name, d.class_desc, d.audited_principal_id
FROM    SmartBankAnalytics.sys.database_audit_specifications AS s
JOIN    SmartBankAnalytics.sys.database_audit_specification_details AS d
        ON d.database_specification_id = s.database_specification_id
WHERE   d.audit_action_name IN ('SCHEMA_OBJECT_CHANGE_GROUP','DATABASE_OBJECT_CHANGE_GROUP')
ORDER BY DatabaseName, audit_action_name;
GO
```

	audit_action_name	SpecificationName	is_state_enabled
1	DATABASE_CHANGE_GROUP	SmartBank_ServerAuditSpec	0

	DatabaseName	SpecificationName	audit_action_name	class_desc	audited_principal_id
1	SmartBankAnalytics	SmartBankAnalytics_AuditSpec	DATABASE_OBJECT_CHANGE_GROUP	DATABASE	0
2	SmartBankAnalytics	SmartBankAnalytics_AuditSpec	SCHEMA_OBJECT_CHANGE_GROUP	DATABASE	0
3	SmartBankProd	SmartBankProd_AuditSpec	DATABASE_OBJECT_CHANGE_GROUP	DATABASE	0
4	SmartBankProd	SmartBankProd_AuditSpec	SCHEMA_OBJECT_CHANGE_GROUP	DATABASE	0

Permission Change Audit

Comprising of operations like permission granting, denial and revocation, actions related to permission changes shall be constantly monitored via audit to ensure that any unauthorised permission change to database objects can be traced and investigated. Similar to DDL audit, permission change operations can only be captured through predefined audit action groups, hence the audit specifications are configured correspondingly, as shown in Figure 124. The audit action groups configured for the audit specifications are categorised as shown in Table 8.

Table 8

Different Audit Action Groups for Permission Change Audit with Description

Suffix	Description
`PRINCIPAL_CHANGE_GROUP`	Changes to logins, users, and roles.
`ROLE_MEMBER_CHANGE_GROUP`	Changes to role membership
`PERMISSION_CHANGE_GROUP`	`GRANT`, `DENY` and `REVOKE` operations
`LOGIN_CHANGE_PASSSWORD_GROUP`	Changes to login passwords
`PRINCIPAL_IMPERSONATION_GROUP`	Principal impersonation activities
`OWNERSHIP_CHANGE_GROUP`	Changes to object ownership

Figure 124

Configuration of Permission Change Audit for SmartBank

```
USE master;
GO
ALTER SERVER AUDIT SPECIFICATION SmartBank_ServerAuditSpec
    ADD (SERVER_PRINCIPAL_CHANGE_GROUP),
    ADD (SERVER_ROLE_MEMBER_CHANGE_GROUP),
    ADD (SERVER_PERMISSION_CHANGE_GROUP),
    ADD (LOGIN_CHANGE_PASSWORD_GROUP),
    ADD (SERVER_PRINCIPAL_IMPERSONATION_GROUP),
    ADD (DATABASE_OWNERSHIP_CHANGE_GROUP);
GO

USE SmartBankProd;
GO
ALTER DATABASE AUDIT SPECIFICATION SmartBankProd_AuditSpec
    ADD (DATABASE_PERMISSION_CHANGE_GROUP),
    ADD (SCHEMA_OBJECT_PERMISSION_CHANGE_GROUP),
    ADD (DATABASE_ROLE_MEMBER_CHANGE_GROUP),
    ADD (DATABASE_PRINCIPAL_CHANGE_GROUP),
    ADD (DATABASE_PRINCIPAL_IMPERSONATION_GROUP),
    ADD (SCHEMA_OBJECT_OWNERSHIP_CHANGE_GROUP);
GO

USE SmartBankAnalytics;
GO
ALTER DATABASE AUDIT SPECIFICATION SmartBankAnalytics_AuditSpec
    ADD (DATABASE_PERMISSION_CHANGE_GROUP),
    ADD (SCHEMA_OBJECT_PERMISSION_CHANGE_GROUP),
    ADD (DATABASE_ROLE_MEMBER_CHANGE_GROUP),
    ADD (DATABASE_PRINCIPAL_CHANGE_GROUP),
    ADD (DATABASE_PRINCIPAL_IMPERSONATION_GROUP),
    ADD (SCHEMA_OBJECT_OWNERSHIP_CHANGE_GROUP);
GO
```

The code in Figure 125 is used to verify the configurations for permission change audit across different audit specifications. Based on the result, server-level audit action groups are only added to `SmartBank_ServerAuditSpec`, while the remaining database and schema-object level audit action groups appear once in each of the database audit specifications. This indicates that permission change audit is captured evenly across the production and analytics databases.

Figure 125

SQL Code to Verify the Configuration for Permission Change Audit for SmartBank

```
SELECT  d.audit_action_name, s.name AS SpecificationName, s.is_state_enabled
FROM    sys.server_audit_specifications AS s
JOIN    sys.server_audit_specification_details AS d
        ON d.server_specification_id = s.server_specification_id
WHERE   d.audit_action_name IN ('SERVER_PRINCIPAL_CHANGE_GROUP',
                                'SERVER_ROLE_MEMBER_CHANGE_GROUP',
                                'SERVER_PERMISSION_CHANGE_GROUP',
                                'LOGIN_CHANGE_PASSWORD_GROUP',
                                'SERVER_PRINCIPAL_IMPERSONATION_GROUP',
                                'DATABASE_OWNERSHIP_CHANGE_GROUP')
ORDER BY d.audit_action_name;
GO

SELECT  'SmartBankProd' AS DatabaseName, s.name AS SpecificationName,
        d.audit_action_name, d.class_desc, d.audited_principal_id
FROM    SmartBankProd.sys.database_audit_specifications AS s
JOIN    SmartBankProd.sys.database_audit_specification_details AS d
        ON d.database_specification_id = s.database_specification_id
WHERE   d.audit_action_name LIKE '%PERMISSION_CHANGE_GROUP'
     OR d.audit_action_name LIKE '%PRINCIPAL_CHANGE_GROUP'
     OR d.audit_action_name LIKE '%ROLE_MEMBER_CHANGE_GROUP'
     OR d.audit_action_name LIKE '%IMPERSONATION_GROUP'
     OR d.audit_action_name LIKE '%OWNERSHIP_CHANGE_GROUP'
UNION ALL
SELECT  'SmartBankAnalytics', s.name,
        d.audit_action_name, d.class_desc, d.audited_principal_id
FROM    SmartBankAnalytics.sys.database_audit_specifications AS s
JOIN    SmartBankAnalytics.sys.database_audit_specification_details AS d
        ON d.database_specification_id = s.database_specification_id
WHERE   d.audit_action_name LIKE '%PERMISSION_CHANGE_GROUP'
     OR d.audit_action_name LIKE '%PRINCIPAL_CHANGE_GROUP'
     OR d.audit_action_name LIKE '%ROLE_MEMBER_CHANGE_GROUP'
     OR d.audit_action_name LIKE '%IMPERSONATION_GROUP'
     OR d.audit_action_name LIKE '%OWNERSHIP_CHANGE_GROUP'
ORDER BY DatabaseName, audit_action_name;
GO
```

	audit_action_name	SpecificationName	is_state_enabled
1	DATABASE_OWNERSHIP_CHANGE_GROUP	SmartBank_ServerAuditSpec	0
2	LOGIN_CHANGE_PASSWORD_GROUP	SmartBank_ServerAuditSpec	0
3	SERVER_PERMISSION_CHANGE_GROUP	SmartBank_ServerAuditSpec	0
4	SERVER_PRINCIPAL_CHANGE_GROUP	SmartBank_ServerAuditSpec	0
5	SERVER_PRINCIPAL_IMPERSONATION_GROUP	SmartBank_ServerAuditSpec	0
6	SERVER_ROLE_MEMBER_CHANGE_GROUP	SmartBank_ServerAuditSpec	0

	DatabaseName	SpecificationName	audit_action_name	class_desc	audited_principal_id
1	SmartBankAnalytics	SmartBankAnalytics_AuditSpec	DATABASE_PERMISSION_CHANGE_GROUP	DATABASE	0
2	SmartBankAnalytics	SmartBankAnalytics_AuditSpec	DATABASE_PRINCIPAL_CHANGE_GROUP	DATABASE	0
3	SmartBankAnalytics	SmartBankAnalytics_AuditSpec	DATABASE_PRINCIPAL_IMPERSONATION_GROUP	DATABASE	0
4	SmartBankAnalytics	SmartBankAnalytics_AuditSpec	DATABASE_ROLE_MEMBER_CHANGE_GROUP	DATABASE	0
5	SmartBankAnalytics	SmartBankAnalytics_AuditSpec	SCHEMA_OBJECT_OWNERSHIP_CHANGE_GROUP	DATABASE	0
6	SmartBankAnalytics	SmartBankAnalytics_AuditSpec	SCHEMA_OBJECT_PERMISSION_CHANGE_GROUP	DATABASE	0
7	SmartBankProd	SmartBankProd_AuditSpec	DATABASE_PERMISSION_CHANGE_GROUP	DATABASE	0
8	SmartBankProd	SmartBankProd_AuditSpec	DATABASE_PRINCIPAL_CHANGE_GROUP	DATABASE	0
9	SmartBankProd	SmartBankProd_AuditSpec	DATABASE_PRINCIPAL_IMPERSONATION_GROUP	DATABASE	0
10	SmartBankProd	SmartBankProd_AuditSpec	DATABASE_ROLE_MEMBER_CHANGE_GROUP	DATABASE	0
11	SmartBankProd	SmartBankProd_AuditSpec	SCHEMA_OBJECT_OWNERSHIP_CHANGE_GROUP	DATABASE	0
12	SmartBankProd	SmartBankProd_AuditSpec	SCHEMA_OBJECT_PERMISSION_CHANGE_GROUP	DATABASE	0

Audit, Backup and Restore Logging

To further protect the audit trail, four additional audit action groups are added to the server audit specification to complement the other audit areas configured, as shown in Figure 126. `AUDIT_CHANGE_GROUP` is added to capture any changes that are related to audit configurations, ensuring that operations like altering or disabling the audit, including unauthorised attempts, will be recorded. Similarly, operations that are relevant to backup and restore are also captured by `BACKUP_RESTORE_GROUP` so that anyone who attempts to retrieve the existing contents of SmartBank's database by using backups to bypass access controls can be identified and traced. Additionally, `SERVER_STATE_CHANGE_GROUP` and `DBCC_GROUP` are added to respectively capture unexpected changes to the server instance, as well as the execution of database console commands that might expose database information when performing maintenance tasks.

Figure 126

Configuration of Audit, Backup and Restore Logging for SmartBank

```
USE master;
GO
ALTER SERVER AUDIT SPECIFICATION SmartBank_ServerAuditSpec
    ADD (AUDIT_CHANGE_GROUP), ADD (BACKUP_RESTORE_GROUP),
    ADD (SERVER_STATE_CHANGE_GROUP), ADD (DBCC_GROUP);
GO
```

Similar to other audit areas, a `SELECT` query is executed to verify the configurations for audit, backup and restore logging on the server audit specification. The result in Figure 127 shows that all the specified audit action groups are correctly added to the server audit specification, thereby protecting audit trails across both databases without requiring duplicated configuration.

Figure 127

SQL Code to Verify the Configuration for Audit, Backup and Restore Logging for SmartBank

```
SELECT  d.audit_action_name, s.name AS SpecificationName, s.is_state_enabled
FROM    sys.server_audit_specifications AS s
JOIN    sys.server_audit_specification_details AS d
        ON d.server_specification_id = s.server_specification_id
WHERE   d.audit_action_name IN ('AUDIT_CHANGE_GROUP',
                                'BACKUP_RESTORE_GROUP',
                                'SERVER_STATE_CHANGE_GROUP',
                                'DBCC_GROUP')
ORDER BY d.audit_action_name;
GO
```

	audit_action_name	SpecificationName	is_state_enabled
1	AUDIT_CHANGE_GROUP	SmartBank_ServerAuditSpec	0
2	BACKUP_RESTORE_GROUP	SmartBank_ServerAuditSpec	0
3	DBCC_GROUP	SmartBank_ServerAuditSpec	0
4	SERVER_STATE_CHANGE_GROUP	SmartBank_ServerAuditSpec	0

Enabling the Audit Specifications

With all the audit areas configured, to ensure that the audit mechanism is able to perform its tasks to capture and record activities taking place in SmartBank, the last step of implementation is to enable all the audit specifications with the code in Figure 128. This is then verified using the code in Figure 129, where, based on the result table, `SmartBank_ServerAuditSpec`, `SmartBankProd_AuditSpec`, and `SmartBankAnalytics_AuditSpec`, has a value of `1` for the `is_state_enabled` column, indicating that these specifications are successfully enabled and will automatically perform audit capturing and logging for subsequent operations taking place at server, database or schema-object level.

Figure 128

SQL Code to Enable All Server and Database Audit Specifications for SmartBank

```
USE master;
GO
ALTER SERVER AUDIT SPECIFICATION SmartBank_ServerAuditSpec WITH (STATE = ON);
GO

USE SmartBankProd;
GO
ALTER DATABASE AUDIT SPECIFICATION SmartBankProd_AuditSpec WITH (STATE = ON);
GO

USE SmartBankAnalytics;
GO
ALTER DATABASE AUDIT SPECIFICATION SmartBankAnalytics_AuditSpec WITH (STATE = ON);
GO
```

Figure 129

SQL Code to Verify the Activation of Audit Specifications of SmartBank

```
SELECT 'SERVER' AS Scope, name, is_state_enabled
FROM   master.sys.server_audit_specifications
UNION ALL
SELECT 'SmartBankProd', name, is_state_enabled
FROM   SmartBankProd.sys.database_audit_specifications
UNION ALL
SELECT 'SmartBankAnalytics', name, is_state_enabled
FROM   SmartBankAnalytics.sys.database_audit_specifications;
GO
```

	Scope	name	is_state_enabled
1	SERVER	SmartBank_ServerAuditSpec	1
2	SmartBankProd	SmartBankProd_AuditSpec	1
3	SmartBankAnalytics	SmartBankAnalytics_AuditSpec	1

Audit Verification

To ensure that all the configured audit mechanisms are able to function as intended, different operations are performed for each audit area to demonstrate that attempts, no matter successful or rejected, are correctly captured and recorded in the audit.

Login and Logout Audit

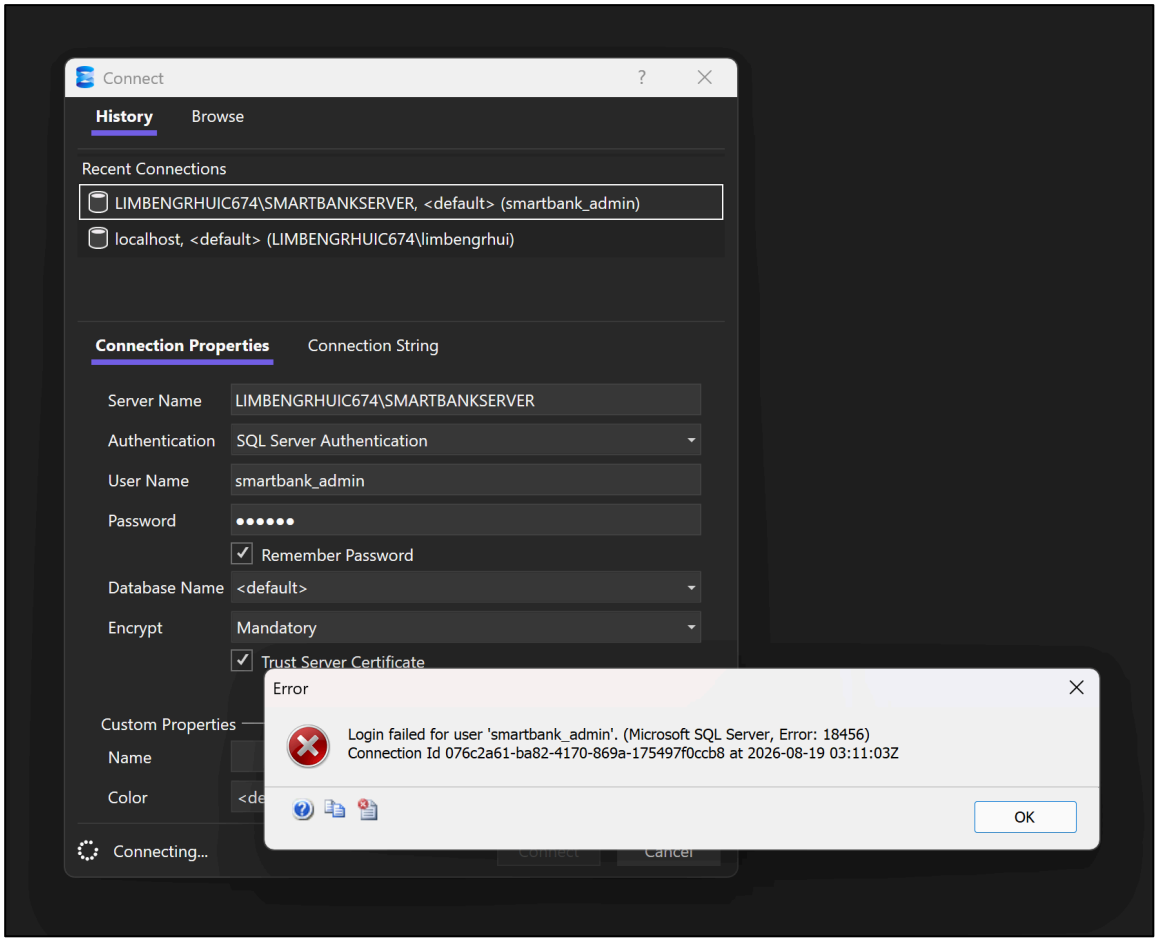
The mechanism for login and logout auditing is tested by performing different authentication activities using the `smartbank_admin` account, as outlined below:

- **Login with an incorrect password**

  The `smartbank_admin` account is firstly used to perform a login with an incorrect password of `123456`. As shown in Figure 130, since `smartbank_admin` is not authenticated, the SmartBank server refused to establish a connection with the user, making the account unable to be used to perform any actions towards the server and the database.

Figure 130

Screenshot of Failed Login Attempt to SmartBank

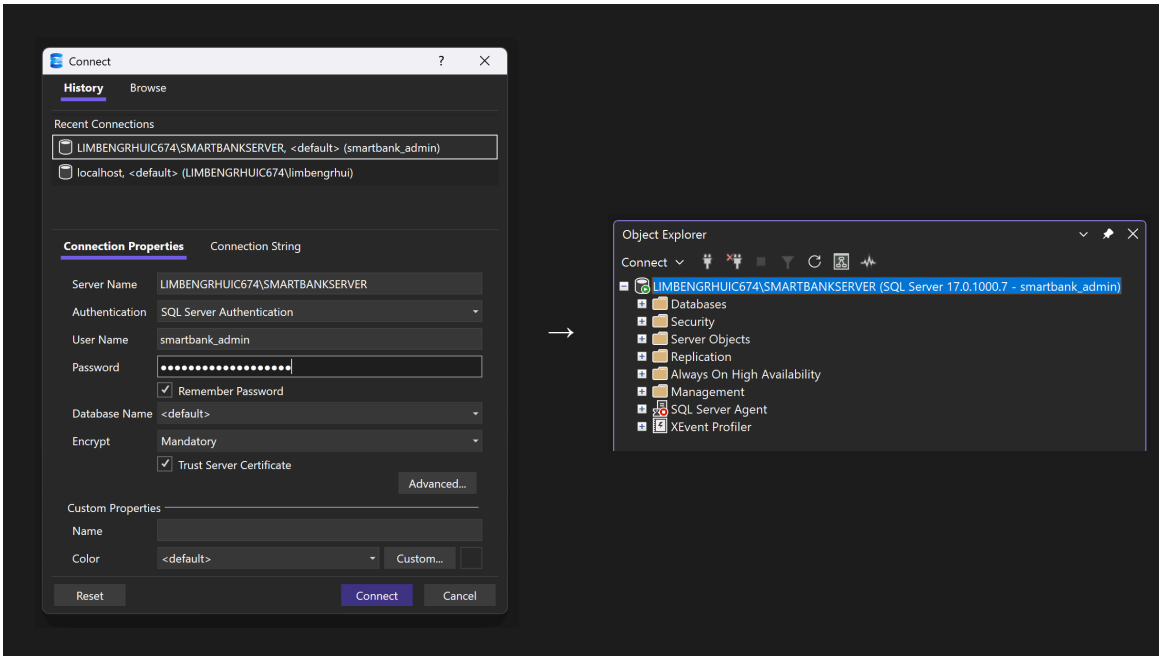


- **Login with the correct password**

  Next, the same `smartbank_admin` account is used to perform login but with the correct password. As shown in Figure 131, entering the valid password allows `smartbank_admin` to successful establish a connection with the server, allowing it to perform different database operations and queries.

Figure 131

Screenshot of Success Login Attempt to SmartBank

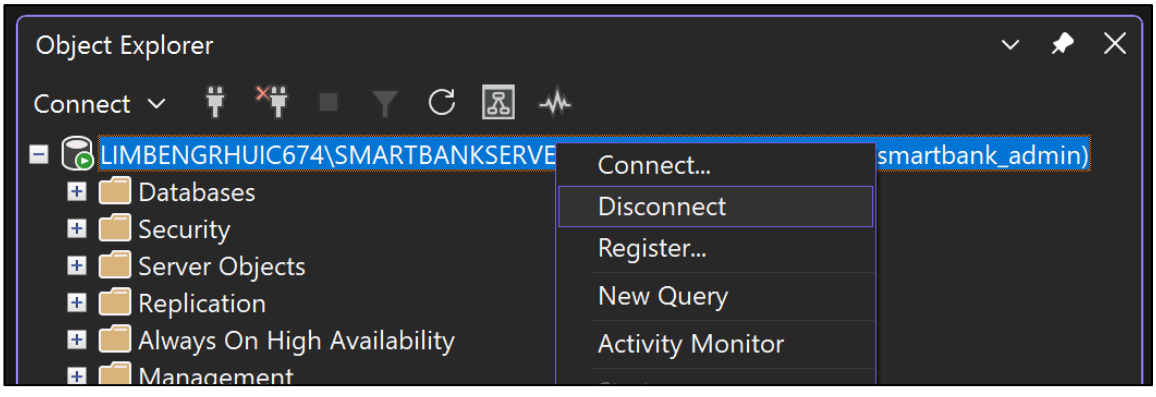


- **Disconnection from the server**

  Lastly, the `smartbank_admin` account is disconnected from SmartBank, as shown in Figure 132. This terminates the session between `smartbank_admin` and the database.

Figure 132

Screenshot of Terminating Database Connection



To verify that these actions are recorded in the audit log, the `SELECT` query in Figure 133 is used to extract the corresponding records from the audit file. Based on the different rows of the result table, it is observed that:

- **Row 20**

  This row records the failed login attempt of `smartbank_admin`, with action ID `LGIF`.

- **Row 3 to 19**

  These rows consist of repeated `LGIS` / `LGO` pairs that represent the multiple SSMS internal connections established when `smartbank_admin` connects to the server, which involves the creation of new connections and the closing of outdated connections.

- **Row 1 and 2**

  These row, with action ID `LGO`, record the operation of `smartbank_admin` terminating its connection with the SmartBank server.

These records confirm that the login and logout operations performed by `smartbank_admin` are correctly captured by the authentication access audit.

Figure 133

SQL Code for Verifying Login and Logout Audit through Audit Actions

```
SELECT  event_time, action_id, succeeded, server_principal_name, statement
FROM    sys.fn_get_audit_file('C:\SmartBankServer\Audit\*.sqlaudit', DEFAULT, DEFAULT)
WHERE   action_id IN ('LGIS','LGIF','LGO')
ORDER BY event_time DESC;
GO
```

	event_time	action_id	succeeded	server_principal_name	statement
1	2026-08-19 18:09:18.3553000	LGO	1	smartbank_admin	
2	2026-08-19 18:09:18.3553000	LGO	1	smartbank_admin	
3	2026-08-19 18:09:08.6770537	LGO	1	smartbank_admin	
4	2026-08-19 18:09:08.6770537	LGO	1	smartbank_admin	
5	2026-08-19 18:09:08.6770537	LGIS	1	smartbank_admin	-- network protocol: LPC set quoted_identifier on set arithabort off set nu...
6	2026-08-19 18:09:08.6759994	LGIS	1	smartbank_admin	-- network protocol: LPC set quoted_identifier on set arithabort off set nu...
7	2026-08-19 18:09:08.6242184	LGO	1	smartbank_admin	
8	2026-08-19 18:09:08.6071769	LGIS	1	smartbank_admin	-- network protocol: LPC set quoted_identifier on set arithabort off set nu...
9	2026-08-19 18:09:08.5761107	LGIS	1	smartbank_admin	-- network protocol: LPC set quoted_identifier on set arithabort off set nu...
10	2026-08-19 18:09:08.4729738	LGIS	1	smartbank_admin	-- network protocol: LPC set quoted_identifier on set arithabort off set nu...
11	2026-08-19 18:09:08.4719692	LGO	1	smartbank_admin	
12	2026-08-19 18:09:08.4679686	LGO	1	smartbank_admin	
13	2026-08-19 18:09:08.4631928	LGIS	1	smartbank_admin	-- network protocol: LPC set quoted_identifier on set arithabort off set nu...
14	2026-08-19 18:09:08.3691616	LGO	1	smartbank_admin	
15	2026-08-19 18:09:08.3664701	LGIS	1	smartbank_admin	-- network protocol: LPC set quoted_identifier on set arithabort off set nu...
16	2026-08-19 18:09:08.3662478	LGO	1	smartbank_admin	
17	2026-08-19 18:09:08.3642825	LGIS	1	smartbank_admin	-- network protocol: LPC set quoted_identifier on set arithabort off set nu...
18	2026-08-19 18:09:08.2585059	LGIS	1	smartbank_admin	-- network protocol: LPC set quoted_identifier on set arithabort off set nu...
19	2026-08-19 18:09:08.1424602	LGIS	1	smartbank_admin	-- network protocol: LPC set quoted_identifier on set arithabort off set nu...
20	2026-08-19 18:08:57.1392772	LGIF	0	smartbank_admin	Login failed for user 'smartbank_admin'. Reason: Password did not match...

Data Manipulation Language (DML) Audit

Several operations that involve data manipulation language (DML) are also performed using the bank officer account `ofc_lim`, as outlined below:

- **Select from a permitted view**

  The `ofc_lim` account is used to perform `SELECT` on `dbo.vw_StaffDirectory`, as shown in Figure 134. As all authorised users in the production database are granted with permission to perform `SELECT` on the view, the `ofc_lim` account is able to view staff details like `StaffName`, `Phone` and `Branch`.

Figure 134

SQL Code to Perform `SELECT` *Operation from a Permitted View*

```
EXECUTE AS USER = 'ofc_lim';
SELECT * FROM dbo.vw_StaffDirectory;
REVERT;
GO
```

	StaffID	StaffName	Phone	Branch
1	S00001	Ahmad bin Hassan	012-3450001	HQ Kuala Lumpur
2	S00002	Aisyah binti Rahman	012-3450002	Petaling Jaya
3	S00003	Ravi Subramaniam	012-3450003	Shah Alam
4	S00004	Lim Wei Sheng	012-3450004	Petaling Jaya
5	S00005	Nurul Izzah	012-3450005	Shah Alam

- **Insert directly into a restricted base table**

  The `ofc_lim` account is then used to perform a direct `INSERT` operation on the `Account` table. As direct write access is explicitly denied for all business users, `ofc_lim` is unable to bypass access control mechanisms, leading to rejected operation as shown in Figure 135.

Figure 135

SQL Code to Perform an Unauthorised `INSERT` *Operation*

```
EXECUTE AS USER = 'ofc_lim';
INSERT INTO dbo.Account (AccountID, CustomerID, AccountType, Balance)
VALUES ('ACC9999999', 'C00001', 'Savings', 0);
REVERT;
GO
```

```
Msg 229, Level 14, State 5, Line 455
The INSERT permission was denied on the object 'Account', database 'SmartBankProd', schema 'dbo'.

Completion time: 2026-08-20T02:16:45.3708071+08:00
```

- **Update directly on a restricted base table**

  Next, the `ofc_lim` account is used to perform a direct `UPDATE` operation towards the `Customer` table, which, as shown in Figure 136, is also rejected as bank officers are not permitted to perform direct retrieval and modification towards all the base tables in the production database.

Figure 136

SQL Code to Perform an Unauthorised `UPDATE` Operation

```
EXECUTE AS USER = 'ofc_lim';
UPDATE dbo.Customer SET Address = '123 Test Street' WHERE CustomerID = 'C00001';
REVERT;
GO
```

```
Msg 229, Level 14, State 5, Line 463
The SELECT permission was denied on the object 'Customer', database 'SmartBankProd', schema 'dbo'.
Msg 229, Level 14, State 5, Line 463
The UPDATE permission was denied on the object 'Customer', database 'SmartBankProd', schema 'dbo'.

Completion time: 2026-08-20T02:17:51.5737558+08:00
```

- **Execute a permitted stored procedure**

  Lastly, the stored procedure `usp_UpdateOwnStaffRecord` is executed using `ofc_lim` to update its staff details. Since all staffs, including `ofc_lim` that belongs to the `role_Officer` role, are granted with the permission to execute the stored procedure, the operation is performed successfully, as shown in Figure 137, indicating that the new staff detail is updated into the corresponding `Staff` table.

Figure 137

SQL Code to Execute an Authorised Stored Procedure

```
EXECUTE AS USER = 'ofc_lim';
EXEC dbo.usp_UpdateOwnStaffRecord @Phone = '012-3450099';
REVERT;
GO
```

```
Your staff record has been updated.

Completion time: 2026-08-20T02:18:47.0767261+08:00
```

To verify that all the performed DML operations are captured by the audit, the SQL audit file is queried using the code in Figure 138 to extract the relevant records. Based on the result table, it is observed that:

- **Row 8 and 9**

  Two rows with action ID `SL` are displayed with `succeeded` having a value of `1`. This indicates that the successful execution of `SELECT` on the view is captured by the audit.

- **Row 7**

  This row captures the rejected `INSERT` (`IN`) operation performed on `Account` table.

- **Row 5 to 6**

  When the server tries to perform write operations like `UPDATE` (`UP`), it implicitly performs a `SELECT` (`SL`) to retrieve relevant rows, leading to two audit records.

- **Row 1 to 4**

  Three rows are captured for the execution of stored procedures, which aligns with its content where `StaffID` is checked first, and an implicit `SELECT` (`SL`) operation is executed before the `UPDATE` (`UP`) and `INSERT` (`IN`) operations are performed.

Additionally, the system-versioned temporal table in Figure 139 is also able to record the values of rows before and after `Phone` is modified in the `Staff` table. Hence, these results are able to show that the DML audit is configured correctly and is able to function as intended.

Figure 138

SQL Code for Verifying DML Audit through Audit Actions

```
SELECT  event_time, action_id, succeeded, server_principal_name,
        database_principal_name, object_name, statement
FROM    sys.fn_get_audit_file('C:\SmartBankServer\Audit\*.sqlaudit', DEFAULT, DEFAULT)
WHERE   action_id IN ('IN','UP','DL','SL') AND server_principal_name = 'ofc_lim'
ORDER BY event_time DESC;
```

	event_time	action_id	succeeded	server_principal_name	database_principal_name	object_name	statement
1	2026-08-19 18:18:47.0512510	IN	1	ofc_lim	ofc_lim	StaffHistory	UPDATE dbo.Staff SET Phone = @Phone, Branch = ISNULL(@Branch, Branch) ...
2	2026-08-19 18:18:47.0507512	UP	1	ofc_lim	ofc_lim	Staff	UPDATE dbo.Staff SET Phone = @Phone, Branch = ISNULL(@Branch, Branch) ...
3	2026-08-19 18:18:47.0496116	SL	1	ofc_lim	ofc_lim	Staff	UPDATE dbo.Staff SET Phone = @Phone, Branch = ISNULL(@Branch, Branch) ...
4	2026-08-19 18:18:47.0487980	SL	1	ofc_lim	ofc_lim	UserMapping	SELECT @MyStaffID = m.StaffID FROM Security.UserMapping AS m WHERE m.Data...
5	2026-08-19 18:17:51.5702285	UP	0	ofc_lim	ofc_lim	Customer	UPDATE dbo.Customer SET Address = '123 Test Street' WHERE CustomerID = 'C00001'
6	2026-08-19 18:17:51.5698511	SL	0	ofc_lim	ofc_lim	Customer	UPDATE dbo.Customer SET Address = '123 Test Street' WHERE CustomerID = 'C00001'
7	2026-08-19 18:16:45.3620419	IN	0	ofc_lim	ofc_lim	Account	INSERT INTO dbo.Account (AccountID, CustomerID, AccountType, Balance) VALUES ('ACC...
8	2026-08-19 18:16:22.2373025	SL	1	ofc_lim	ofc_lim	fn_StaffDirectory	SELECT * FROM dbo.vw_StaffDirectory
9	2026-08-19 18:16:22.2363703	SL	1	ofc_lim	ofc_lim	vw_StaffDirectory	SELECT * FROM dbo.vw_StaffDirectory

Figure 139

SQL Code for Verifying DML Audit through System-Versioned Temporal Tables

```
SELECT StaffID, StaffName, Phone, SysStartTime, SysEndTime FROM dbo.Staff
FOR SYSTEM_TIME ALL WHERE StaffID = 'S00004'
ORDER BY SysStartTime DESC;
```

	StaffID	StaffName	Phone	SysStartTime	SysEndTime
1	S00004	Lim Wei Sheng	012-3450099	2026-08-19 18:18:47.0496116	9999-12-31 23:59:59.9999999
2	S00004	Lim Wei Sheng	012-3450004	2026-08-19 17:42:33.2579077	2026-08-19 18:18:47.0496116

Data Definition Language (DDL) Audit

To validate that actions associated to data definition language (DDL) are captured by the audit, several operations are performed, including:

- **Create a table**

  As shown in Figure 140, a temporary table is created using the `dbo` account.

  Figure 140

  SQL Code to Create a Table

```
CREATE TABLE dbo.AuditProbe_TempTable (ProbeID INT);
```

```
Commands completed successfully.

Completion time: 2026-08-20T02:28:50.2098286+08:00
```

- **Drop a table**

  The table is then removed in Figure 141 to avoid it from affecting the existing `dbo` schema.

  Figure 141

  SQL Code to Drop a Table

```
DROP TABLE IF EXISTS dbo.AuditProbe_TempTable;
```

```
 Commands completed successfully.

 Completion time: 2026-08-20T02:29:41.3044730+08:00
```

- **Attempt to create a table as a customer**

  Using the `cust_tan` account, a new table is attempted to be created, which, as shown in Figure 142, is rejected as customers does not have the access to perform `CREATE TABLE`.

  Figure 142

  SQL Code to Create a Table using a Customer Account

```
EXECUTE AS USER = 'cust_tan';
CREATE TABLE dbo.AuditProbe_Unauthorised (ProbeID INT);
REVERT;
```

```
Msg 262, Level 14, State 1, Line 506
CREATE TABLE permission denied in database 'SmartBankProd'.

Completion time: 2026-08-20T02:30:25.2437778+08:00
```

- **Alter the database recovery model**

  Lastly, using the `smartbank_admin` account, the recovery model is set to `FULL`. Since the database is initially configured as `FULL` recovery, although no actual changes will take place, the statement will still be captured to ensure that similar commands will be recorded by the audit mechanism.

Figure 143

SQL Code to Alter the Database Recovery Model

```
USE master;
GO
ALTER DATABASE SmartBankProd SET RECOVERY FULL;
GO
```

```
Commands completed successfully.

Completion time: 2026-08-20T02:31:19.0185507+08:00
```

The audit records associated to DDL are then extracted using the code in Figure 144, where based on the result, it is observed that all the actions performed are captured by the audit. Rows 4 and 5 record the table creation process, where the `dbo` schema is altered (`AL`) before the `AuditProbe_TempTable` is created (`CR`). This is followed by row 3 that captures the deletion of the table using the `DROP` (`DR`) operation, before the `CREATE TABLE` operation (`CR`) of `cust_tan` is rejected in row 2. Lastly, the database altering operation (`AL`) that modifies the recovery model of the production database is recorded in row 1. All these records demonstrate that the audit mechanism is able to capture operations related to DDL accurately.

Figure 144

SQL Code for Verifying DDL Audit through Audit Actions

```
SELECT  a.event_time, a.action_id, a.succeeded, a.server_principal_name,
        a.database_principal_name, a.object_name, a.statement
FROM    sys.fn_get_audit_file('C:\SmartBankServer\Audit\*.sqlaudit', DEFAULT, DEFAULT) AS a
CROSS APPLY (VALUES (LTRIM(REPLACE(REPLACE(a.statement, CHAR(13), ' '), CHAR(10), ' ')))) AS
n(NormalisedStatement)
WHERE   a.action_id IN ('CR','AL','DR')        -- create, alter, drop
   OR   n.NormalisedStatement LIKE 'CREATE %'
   OR   n.NormalisedStatement LIKE 'ALTER %'
   OR   n.NormalisedStatement LIKE 'DROP %'
ORDER BY a.event_time DESC;
GO
```

	event_time	action_id	succeeded	server_principal_name	database_principal_name	object_name	statement
1	2026-08-19 18:31:19.0155494	AL	1	smartbank_admin	dbo	SmartBankProd	ALTER DATABASE SmartBankProd
2	2026-08-19 18:30:25.2407868	CR	0	cust_tan	cust_tan		CREATE TABLE dbo.AuditProbe_Ur
3	2026-08-19 18:29:41.2934345	DR	1	smartbank_admin	dbo	AuditProbe_TempTable	DROP TABLE IF EXISTS dbo.AuditF
4	2026-08-19 18:28:50.2006280	CR	1	smartbank_admin	dbo	AuditProbe_TempTable	CREATE TABLE dbo.AuditProbe_Te
5	2026-08-19 18:28:50.1986520	AL	1	smartbank_admin	dbo	dbo	CREATE TABLE dbo.AuditProbe_Te

Permission Change Audit

Several operations related to permission change are also performed, as outlined below:

- **Grant a permission**

  As shown in Figure 145, the permission to perform the `SELECT` operation on the view `dbo.vw_StaffDirectory` is re-granted to the `role_Customer` role. By selecting a permission that was initially possessed by customers, this allows the audit mechanism to be verified without introducing any actual change to the current access control mechanisms.

  Figure 145

  SQL Code to Grant Existing Permission to Customer Role

```
GRANT SELECT ON dbo.vw_StaffDirectory TO role_Customer;
```

```
Commands completed successfully.

Completion time: 2026-08-20T02:34:34.1414327+08:00
```

- **Attempt to revoke a permission as a customer**

  Additionally, the `cust_tan` account is used to perform a `REVOKE` operation. As shown in Figure 146, the operation is denied by the SQL server as the `role_Customer` role does not have the authorisation to modify existing permission change configurations.

  Figure 146

  SQL Code to Attempt Granting Permission as a Customer

```
EXECUTE AS USER = 'cust_tan';
REVOKE SELECT ON dbo.vw_StaffDirectory TO role_Customer;
GO
REVERT;
GO
```

```
Msg 15151, Level 16, State 1, Line 539
Cannot find the object 'vw_StaffDirectory', because it does not exist or you do not have permission.

Completion time: 2026-08-20T02:35:40.8679061+08:00
```

After performing the operations, the code in Figure 147 is used to review the audit records related to permission changes. Based on the result, it is demonstrated that the two actions are correctly captured by the audit mechanism, where row 3 displays the successful `GRANT` (`G`) operation performed, while rows 1 and 2 reflect the impersonation activity (`IMP`) as `cust_tan` to demonstrate the invalid operation performed to revoke (`R`) the `SELECT` permission.

Figure 147

SQL Code for Verifying Permission Change Audit through Audit Actions

```
SELECT  a.event_time, a.action_id, a.succeeded, a.server_principal_name,
        a.database_principal_name, a.object_name, a.statement
FROM    sys.fn_get_audit_file('C:\SmartBankServer\Audit\*.sqlaudit', DEFAULT, DEFAULT) AS a
CROSS APPLY (VALUES (LTRIM(REPLACE(REPLACE(a.statement, CHAR(13), ' '), CHAR(10), ' ')))) AS
n(NormalisedStatement)
WHERE   a.action_id IN ('G','D','R','GWG','RWG','PWCG','IMP','TO')
   OR   n.NormalisedStatement LIKE 'GRANT %'
   OR   n.NormalisedStatement LIKE 'DENY %'
   OR   n.NormalisedStatement LIKE 'REVOKE %'
   OR   n.NormalisedStatement LIKE 'ALTER ROLE%'
   OR   n.NormalisedStatement LIKE 'ALTER LOGIN%'
   OR   n.NormalisedStatement LIKE 'EXECUTE AS%'
   OR   n.NormalisedStatement LIKE 'ALTER AUTHORIZATION%'
ORDER BY a.event_time DESC;
GO
```

	event_time	action_id	succeeded	server_principal_name	database_principal_name	object_name	statement
1	2026-08-19 18:35:40.8638960	R	0	cust_tan	cust_tan	vw_StaffDirectory	REVOKE SELECT ON dbo.vw_StaffDirectory
2	2026-08-19 18:35:40.8538940	IMP	1	smartbank_admin	dbo	cust_tan	EXECUTE AS USER = 'cust_tan';
3	2026-08-19 18:34:34.1332540	G	1	smartbank_admin	dbo	vw_StaffDirectory	GRANT SELECT ON dbo.vw_StaffDirectory T

Audit, Backup and Restore Logging

Similar to other audit area, several operations are also performed to verify if any audit, backup or restore activities are captured by audit, as detailed below:

- **Perform a database backup**

  As shown in Figure 148, a full backup is performed on the production database, with `COPY_ONLY` specified to ensure that the current full backup does not disrupt subsequent differential and transactional log backups.

Figure 148

SQL Code to Perform a Full Database Backup

```
BACKUP DATABASE SmartBankProd
    TO DISK = 'C:\SmartBankServer\Backup\Production\AuditProbe_CopyOnly.bak'
    WITH COPY_ONLY, INIT, COMPRESSION,
         NAME = 'SmartBankProd audit probe (copy-only)';
GO
```

```
Processed 832 pages for database 'SmartBankProd', file 'SmartBankProd' on file 1.
Processed 2 pages for database 'SmartBankProd', file 'SmartBankProd_log' on file 1.
BACKUP DATABASE successfully processed 834 pages in 0.020 seconds (325.585 MB/sec).

Completion time: 2026-08-20T02:38:37.0169219+08:00
```

- **Disable and re-enable the audit**

  The server audit is also first disabled, as shown in Figure 149, then immediately enabled to ensure that the audit mechanism remains active after performing verification.

Figure 149

SQL Code to Disable and Re-enable the Audit

```
ALTER SERVER AUDIT SmartBank_ServerAudit WITH (STATE = OFF);
GO

ALTER SERVER AUDIT SmartBank_ServerAudit WITH (STATE = ON);
GO
```

```
Commands completed successfully.

Completion time: 2026-08-20T02:39:55.1404455+08:00
```

```
Commands completed successfully.

Completion time: 2026-08-20T02:40:04.8451073+08:00
```

The code in Figure 150 is then executed to verify the audit records related to audit, backup and restore. Based on the results, it is evident that the logging mechanism performs as intended, where row 2 captures the full backup (`BA`) operation performed on `SmartBankProd` , while row 1 records the alteration (`AL`) operation that disables the audit mechanism. However, due to the audit's disabled state, this makes the operation of re-enabling the audit to not be recorded, which reflects the tamper-evident nature of auditing mechanisms where any attempt to disable the audit mechanism is only logged. Therefore, to overcome such a limitation, additional security controls, such as role-based access control (RBAC), play a critical role by ensuring that only privileged accounts like `smartbank_admin` are granted with access to configure audit-related settings, hence reducing the risk of unauthorised users modifying audit configurations.

Figure 150

SQL Code for Verifying Audit, Backup and Restore Logging through Audit Actions

```
SELECT  a.event_time, a.action_id, a.succeeded, a.server_principal_name,
        a.database_name, a.object_name, a.statement
FROM    sys.fn_get_audit_file('C:\SmartBankServer\Audit\*.sqlaudit', DEFAULT, DEFAULT) AS a
CROSS APPLY (VALUES (LTRIM(REPLACE(REPLACE(a.statement, CHAR(13), ' '), CHAR(10), ' ')))) AS
n(NormalisedStatement)
WHERE   a.action_id IN ('SVCN','SVPD','SVSD','SVSR','DBCC')
   OR   n.NormalisedStatement LIKE 'ALTER SERVER AUDIT%'
   OR   n.NormalisedStatement LIKE 'CREATE SERVER AUDIT%'
   OR   n.NormalisedStatement LIKE 'DROP SERVER AUDIT%'
   OR   n.NormalisedStatement LIKE 'ALTER DATABASE%'
   OR   n.NormalisedStatement LIKE 'BACKUP %'
   OR   n.NormalisedStatement LIKE 'RESTORE %'
   OR   n.NormalisedStatement LIKE 'DBCC %'
ORDER BY a.event_time DESC;
GO
```

	event_time	action_id	succeeded	server_principal_name	database_name	object_name	statement
1	2026-08-19 18:39:55.1261958	AL	1	smartbank_admin	master	SmartBank_ServerAudit	ALTER SERVER AUDIT Sm
2	2026-08-19 18:38:36.9549744	BA	1	smartbank_admin	SmartBankProd	SmartBankProd	BACKUP DATABASE Smart

Procedure-Level Logging

To ensure that business-rule rejections that occur inside stored procedures are also captured, the following action is performed:

- **Create an account with negative balance**

  The `ofc_lim` account is used to demonstrate the scenario where a customer account with a negative opening balance of `-500.00` is created, as shown in Figure 151. Although `ofc_lim` possesses the permission to execute the `dbo.usp_CreateCustomerAccount` procedure, the operation shall not be permitted as it violates business rules, which shall be captured by the procedure-level logging to ensure data integrity and accuracy in database.

Figure 151

SQL Code to Create an Account with Negative Balance

```
USE SmartBankProd;
GO

EXECUTE AS USER = 'ofc_lim';
EXEC dbo.usp_CreateCustomerAccount @AccountID = 'ACC9999998',
                                   @CustomerID = 'C00001',
                                   @AccountType = 'Savings',
                                   @OpeningBalance = -500.00;
GO
REVERT;
GO
```

```
Msg 51015, Level 16, State 1, Procedure dbo.usp_CreateCustomerAccount, Line 35 [Batch Start Line 595]
Opening balance cannot be negative.

Completion time: 2026-08-20T02:44:02.9282171+08:00
```

The `Security.RejectedAttemptLog` table is then queried, as shown in Figure 152. Result shows that the business-rule rejection is captured correctly with details like `ProcedureName` and `SysStartTime` recorded accurately, hence verifying that procedure-level logging occurs.

Figure 152

SQL Code to Verify that Procedure-Level Logging is Performed

```
SELECT TOP (100) * FROM Security.RejectedAttemptLog
ORDER BY AttemptID DESC;
GO
```

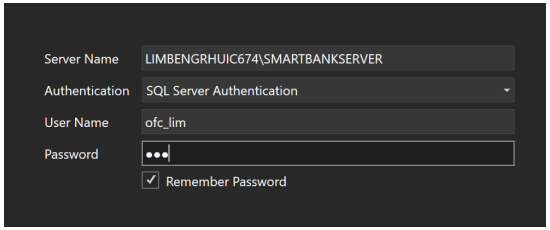
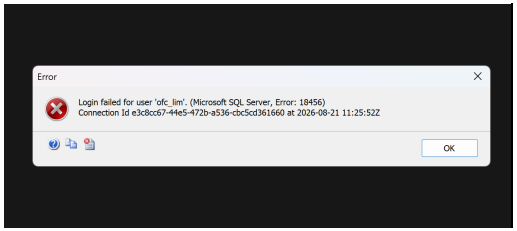
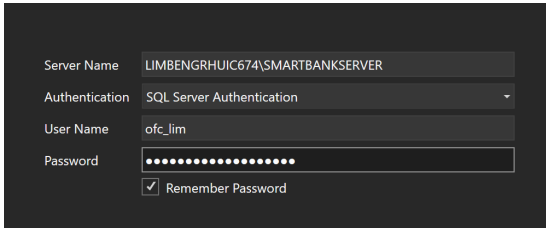
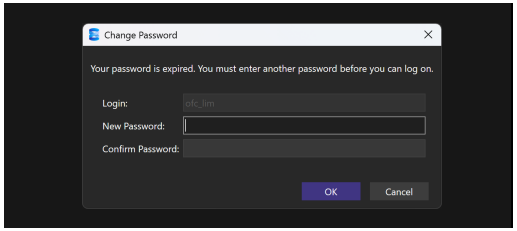
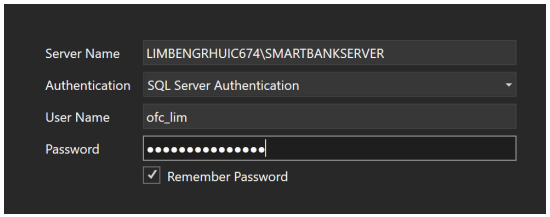
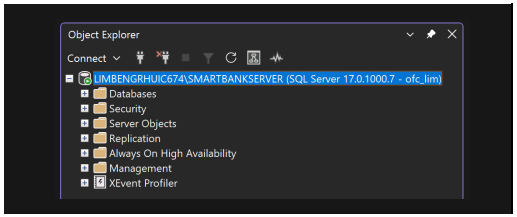
	AttemptID	AttemptedOn	LoginName	ProcedureName	Reason	Detail	SysStartTime	SysEndTime
1	1	2026-08-20 02:44:03	smartbank_admin	usp_CreateCustomerAccount	Negative opening balance	ACC9999998	2026-08-19 18:44:02.9193708	9999-12-31 23:5

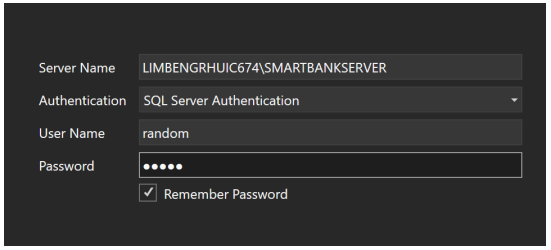
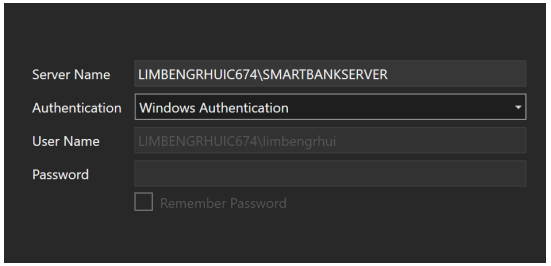
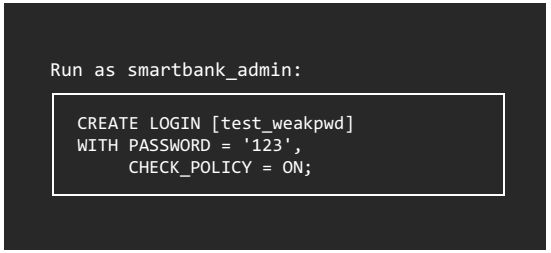
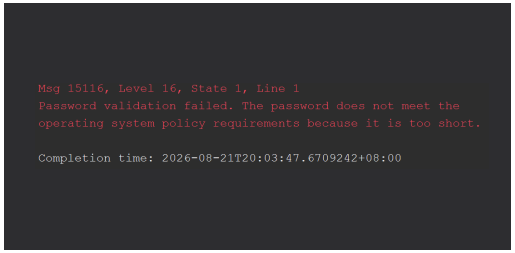
Test Case

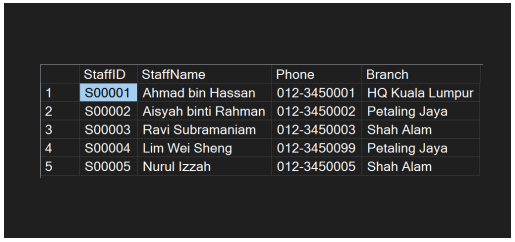
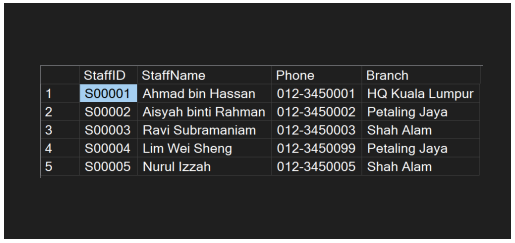
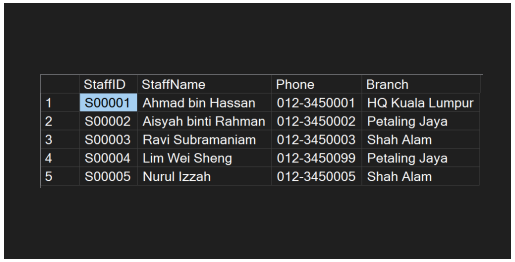
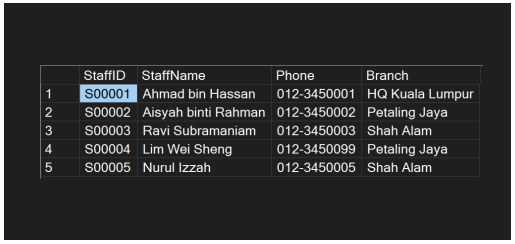
To ensure that all security mechanisms are implemented and aligned with the business requirements of SmartBank, multiple test cases are performed and documented in Table 9, encompassing areas in the CIA triad as well as mechanisms like backup and audit.

Table 9

Test Cases Performed to Verify the Implementation of Security Controls in SmartBank

No.	Description	Execution	Results	Passed / Failed
Confidentiality: All users must be authenticated.				
1.	Connect as `ofc_lim` with a wrong password. Expected Login failed.			Passed
2.	Connect as `ofc_lim` using the correct initial password for the first time. Expected Password must be changed before login.			Passed
3.	Connect as `ofc_lim` again, but using valid username and password. Expected Connection is established.			Passed

No.	Description	Execution	Results	Passed / Failed
4.	Connect with a login that does not exist. Expected Login failed.			Passed
5.	Connect with the disabled Windows `sysadmin` account. Expected Login failed.			Passed
6.	Submit ten consecutive wrong passwords when login as `ofc_lim`, then submit the correct one. Expected Account is locked out.			Passed
7.	Create a login with a weak password while `CHECK_POLICY` is on. Expected Password validation failed.			Passed

No.	Description	Execution	Results	Passed / Failed
Confidentiality: All authenticated users may view staff name and contact number.				
8.	View public staff directory as `mgr_aisyah`. Expected Five rows are returned, showing `StaffID`, `StaffName`, `Phone` and `Branch`.	Run as mgr_aisyah: `SELECT * FROM dbo.vw_StaffDirectory;`		Passed
9.	View public staff directory as `ofc_nurul`. Expected Five rows are returned, showing `StaffID`, `StaffName`, `Phone` and `Branch`.	Run as ofc_nurul: `SELECT * FROM dbo.vw_StaffDirectory;`		Passed
10.	View public staff directory as `dba_ahmad`. Expected Five rows are returned, showing `StaffID`, `StaffName`, `Phone` and `Branch`.	Run as dba_ahmad: `SELECT * FROM dbo.vw_StaffDirectory;`		Passed
11.	View public staff directory as `cust_tan`. Expected Five rows are returned, showing `StaffID`, `StaffName`, `Phone` and `Branch`.	Run as cust_tan: `SELECT * FROM dbo.vw_StaffDirectory;`		Passed

No.	Description	Execution	Results	Passed / Failed

12.	Attempt to select `Salary` from the public staff directory.
	
	Expected
	Error returned. `Salary` is not a valid column.
		Run as smartbank_admin:
		`SELECT Salary`
		`FROM dbo.vw_StaffDirectory;`
			Msg 207, Level 16, State 1, Line 4
			Invalid column name 'Salary'.
			
			Completion time: 2026-08-23T15:22:21.3477976+08:00
				Passed

13.	Attempt to perform direct `SELECT` on the `dbo.Staff` base table as `mgr_aisyah`.
	
	Expected
	Error returned. Permission denied on the `Staff` table.
		Run as mgr_aisyah:
		`SELECT *`
		`FROM dbo.Staff;`
			Msg 229, Level 14, State 5, Line 4
			The SELECT permission was denied on the object 'Staff', database 'SmartBankProd', schema 'dbo'.
			
			Completion time: 2026-08-23T15:24:24.5190700+08:00
				Passed

14.	Attempt to perform direct `SELECT` on the `dbo.Staff` base table as `ofc_nurul`.
	
	Expected
	Error returned. Permission denied on the `Staff` table.
		Run as ofc_nurul:
		`SELECT *`
		`FROM dbo.Staff;`
			Msg 229, Level 14, State 5, Line 4
			The SELECT permission was denied on the object 'Staff', database 'SmartBankProd', schema 'dbo'.
			
			Completion time: 2026-08-23T15:27:10.5055526+08:00
				Passed

15.	Attempt to perform direct `SELECT` on the `dbo.Staff` base table as `dba_ahmad`.
	
	Expected
	Error returned. Permission denied on the `Staff` table.
		Run as dba_ahmad:
		`SELECT *`
		`FROM dbo.Staff;`
			Msg 229, Level 14, State 5, Line 4
			The SELECT permission was denied on the object 'Staff', database 'SmartBankProd', schema 'dbo'.
			
			Completion time: 2026-08-23T15:29:12.0119932+08:00
				Passed

No.	Description	Execution	Results	Passed / Failed

16.	Attempt to perform direct `SELECT` on the `dbo.Staff` base table as `cust_tan`.
	
	Expected
	Error returned. Permission denied on the `Staff` table.
		Run as cust_tan:
		`SELECT *`
		`FROM dbo.Staff;`
			Msg 229, Level 14, State 5, Line 4
			The SELECT permission was denied on the object 'Staff', database 'SmartBankProd', schema 'dbo'.
			
			Completion time: 2026-08-23T15:30:23.3002402+08:00
				Passed

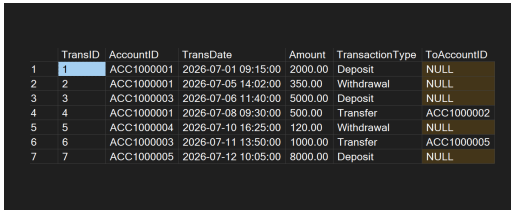
Confidentiality: Bank managers may view own record, all staff records, all customer accounts and all transactions.				

17.	View own staff record as `mgr_aisyah`.
	
	Expected
	One row shown:
	`StaffID = S00002`
	`StaffName = Aisyah binti Rahman`
	`Salary = 12000.00`
		Run as mgr_aisyah:
		`SELECT *`
		`FROM dbo.vw_MyStaffRecord;`
			StaffID | StaffName | Position | Branch | Phone | Salary
			1 | S00002 | Aisyah binti Rahman | Manager | Petaling Jaya | 012-3450002 | 12000.00
				Passed

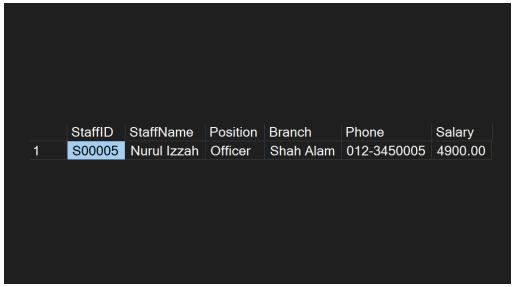
18.	View all staff records as `mgr_aisyah`.
	
	Expected
	Five rows shown, covering staff member from `S00001` to `S00005`.
		Run as mgr_aisyah:
		`SELECT *`
		`FROM dbo.vw_AllStaff`
		`ORDER BY StaffID;`
			StaffID | StaffName | Position | Branch | Phone | Salary
			1 | S00001 | Ahmad bin Hassan | DBA | HQ Kuala Lumpur | 012-3450001 | 9500.00
			2 | S00002 | Aisyah binti Rahman | Manager | Petaling Jaya | 012-3450002 | 12000.00
			3 | S00003 | Ravi Subramaniam | Manager | Shah Alam | 012-3450003 | 11500.00
			4 | S00004 | Lim Wei Sheng | Officer | Petaling Jaya | 012-3450099 | 5200.00
			5 | S00005 | Nurul Izzah | Officer | Shah Alam | 012-3450005 | 4900.00
				Passed

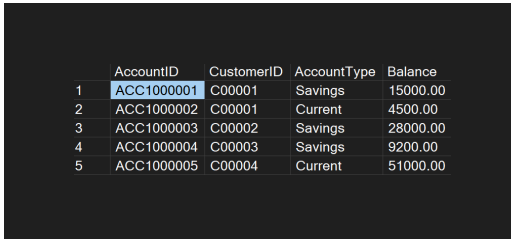
19.	View all customer accounts as `mgr_aisyah`.
	
	Expected
	Five rows shown, covering accounts from `ACC1000001` to `ACC1000005`.
		Run as mgr_aisyah:
		`SELECT *`
		`FROM dbo.vw_CustomerAccounts`
		`ORDER BY AccountID;`
			AccountID | CustomerID | AccountType | Balance
			1 | ACC1000001 | C00001 | Savings | 15000.00
			2 | ACC1000002 | C00001 | Current | 4500.00
			3 | ACC1000003 | C00002 | Savings | 28000.00
			4 | ACC1000004 | C00003 | Savings | 9200.00
			5 | ACC1000005 | C00004 | Current | 51000.00
				Passed

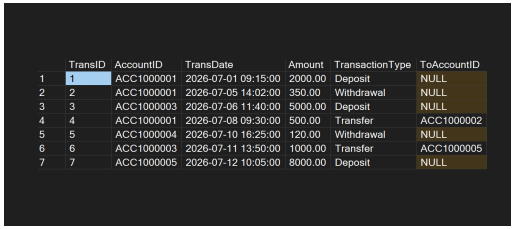
No.	Description	Execution	Results	Passed / Failed

20.	View all transactions as `mgr_aisyah`.
	Expected
	All transaction rows are displayed.
		Run as mgr_aisyah:
		`SELECT *`
		`FROM dbo.vw_CustomerTransactions`
		`ORDER BY TransID;`
				Passed

Confidentiality: Bank officers may view own record, all customer accounts and all transactions.				

21.	View own staff record as `ofc_nurul`.
	Expected
	One row shown:
	`StaffID = S00005`
	`StaffName = Nurul Izzah`
	`Salary = 4900.00`
		Run as ofc_nurul:
		`SELECT *`
		`FROM dbo.vw_MyStaffRecord;`
				Passed

22.	View all customer accounts as `ofc_nurul`.
	Expected
	Five rows shown, covering accounts from `ACC1000001` to `ACC1000005`.
		Run as ofc_nurul:
		`SELECT *`
		`FROM dbo.vw_CustomerAccounts`
		`ORDER BY AccountID;`
				Passed

23.	View all transactions as `ofc_nurul`.
	Expected
	All transaction rows are displayed.
		Run as ofc_nurul:
		`SELECT *`
		`FROM dbo.vw_CustomerTransactions`
		`ORDER BY TransID;`
				Passed

No.	Description	Execution	Results	Passed / Failed

24.	Attempt to view all staff record as `ofc_nurul`.
	
	Expected
	Error returned. Permission denied on `vw_AllStaff`.
		Run as ofc_nurul:
		`SELECT *`
		`FROM dbo.vw_AllStaff;`
			Msg 229, Level 14, State 5, Line 4
			The SELECT permission was denied on the object 'vw_AllStaff', database 'SmartBankProd', schema 'dbo'.
			
			Completion time: 2026-08-23T15:37:31.9233968+08:00
				Passed

Confidentiality: DBA may view own record only.				

25.	View own staff record as `dba_ahmad`.
	
	Expected
	One row shown:
	StaffID = S00001
	StaffName = Ahmad bin Hassan
	Salary = 9500.00
		Run as dba_ahmad:
		`SELECT *`
		`FROM dbo.vw_MyStaffRecord;`
			StaffID | StaffName | Position | Branch | Phone | Salary
			1 | S00001 | Ahmad bin Hassan | DBA | HQ Kuala Lumpur | 012-3450001 | 9500.00
				Passed

26.	Attempt to view all staff record as `dba_ahmad`.
	
	Expected
	Error returned. Permission denied on `vw_AllStaff`.
		Run as dba_ahmad:
		`SELECT *`
		`FROM dbo.vw_AllStaff;`
			Msg 229, Level 14, State 5, Line 4
			The SELECT permission was denied on the object 'vw_AllStaff', database 'SmartBankProd', schema 'dbo'.
			
			Completion time: 2026-08-23T15:38:40.5035440+08:00
				Passed

No.	Description	Execution	Results	Passed / Failed

27.	Attempt to view customer profile as `dba_ahmad`.
	
	Expected
	Error returned. Permission denied on `vw_CustomerProfile`.
		Run as dba_ahmad:
		`SELECT *`
		`FROM dbo.vw_CustomerProfile;`
			Msg 229, Level 14, State 5, Line 4
			The SELECT permission was denied on the object 'vw_CustomerProfile', database 'SmartBankProd', schema 'dbo'.
			
			Completion time: 2026-08-23T15:39:24.7865327+08:00
				Passed

28.	Attempt to view customer accounts as `dba_ahmad`.
	
	Expected
	Error returned. Permission denied on `vw_CustomerAccounts`.
		Run as dba_ahmad:
		`SELECT *`
		`FROM dbo.vw_CustomerAccounts`
		`ORDER BY AccountID;`
			Msg 229, Level 14, State 5, Line 4
			The SELECT permission was denied on the object 'vw_CustomerAccounts', database 'SmartBankProd', schema 'dbo'.
			
			Completion time: 2026-08-23T15:39:51.1860461+08:00
				Passed

29.	Attempt to view customer transactions as `dba_ahmad`.
	
	Expected
	Error returned. Permission denied on `vw_CustomerTransactions`.
		Run as dba_ahmad:
		`SELECT *`
		`FROM dbo.vw_CustomerTransactions;`
			Msg 229, Level 14, State 5, Line 4
			The SELECT permission was denied on the object 'vw_CustomerTransactions', database 'SmartBankProd', schema 'dbo'.
			
			Completion time: 2026-08-23T15:40:16.4112784+08:00
				Passed

30.	Attempt to perform direct `SELECT` on `dbo.Customer` base table as `dba_ahmad`.
	
	Expected
	Error returned. Permission denied on the `Customer` table.
		Run as dba_ahmad:
		`SELECT *`
		`FROM dbo.Customer;`
			Msg 229, Level 14, State 5, Line 4
			The SELECT permission was denied on the object 'Customer', database 'SmartBankProd', schema 'dbo'.
			
			Completion time: 2026-08-23T15:40:45.2564303+08:00
				Passed

No.	Description	Execution	Results	Passed / Failed

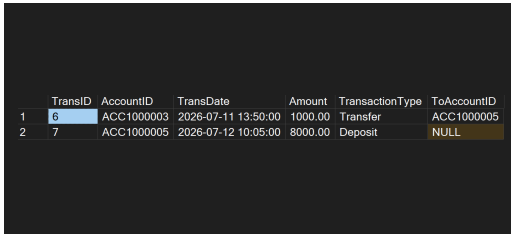
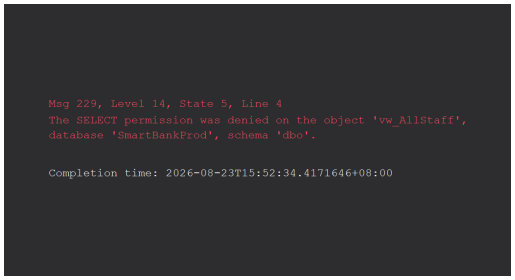
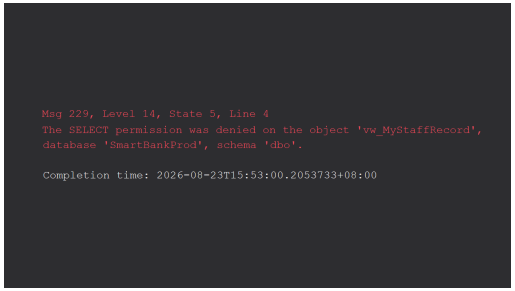
31.	Attempt to perform direct `SELECT` on `dbo.Account` base table as `dba_ahmad`.
	
	Expected
	Error returned. Permission denied on the `Account` table.
		Run as dba_ahmad:
		`SELECT *`
		`FROM dbo.Account;`
			Msg 229, Level 14, State 5, Line 4
			The SELECT permission was denied on the object 'Account', database 'SmartBankProd', schema 'dbo'.
			
			Completion time: 2026-08-23T15:41:06.5482759+08:00
				Passed

Confidentiality: Customer may view own accounts and own transactions.				

32.	View own customer account as `cust_tan`.
	
	Expected
	Two rows shown, covering accounts from `ACC1000001` and `ACC1000002`.
		Run as cust_tan:
		`SELECT *`
		`FROM dbo.vw_CustomerAccounts;`
			AccountID / CustomerID / AccountType / Balance
			1 ACC1000001 C00001 Savings 15000.00
			2 ACC1000002 C00001 Current 4500.00
				Passed

33.	View own customer account as `cust_siti`.
	
	Expected
	One row shown, covering account from `ACC1000003`.
		Run as cust_siti:
		`SELECT *`
		`FROM dbo.vw_CustomerAccounts;`
			AccountID / CustomerID / AccountType / Balance
			1 ACC1000003 C00002 Savings 28000.00
				Passed

34.	View own transactions as `cust_tan`.
	
	Expected
	Only transactions of `ACC1000001` and `ACC1000002` are covered.
		Run as cust_tan:
		`SELECT *`
		`FROM dbo.vw_CustomerTransactions`
		`ORDER BY TransID;`
			TransID / AccountID / TransDate / Amount / TransactionType / ToAccountID
			1 1 ACC1000001 2026-07-01 09:15:00 2000.00 Deposit NULL
			2 2 ACC1000001 2026-07-05 14:02:00 350.00 Withdrawal NULL
			3 4 ACC1000001 2026-07-08 09:30:00 500.00 Transfer ACC1000002
				Passed

No.	Description	Execution	Results	Passed / Failed
35.	View own transactions as `cust_wong`. Expected Only transactions on `ACC1000005` are covered.	Run as cust_wong: `SELECT * FROM dbo.vw_CustomerTransactions ORDER BY TransID;`		Passed
36.	Attempt to read customer profile as `cust_tan`. Expected Error returned. Permission denied on `vw_CustomerProfile`.	Run as cust_tan: `SELECT * FROM dbo.vw_CustomerProfile;`		Passed
37.	Attempt to read all staff profile as `cust_tan`. Expected Error returned. Permission denied on `vw_AllStaff`.	Run as cust_tan: `SELECT * FROM dbo.vw_AllStaff;`		Passed
38.	Attempt to read own-staff-record view as `cust_tan`. Expected Error returned. Permission denied on `vw_MyStaffRecord`.	Run as cust_tan: `SELECT * FROM dbo.vw_MyStaffRecord;`		Passed

No.	Description	Execution	Results	Passed / Failed

39.	Attempt to perform direct `SELECT` on `dbo.Account` base table as `cust_tan`.
	
	Expected
	Error returned. Permission denied on the `Account` table.
		Run as cust_tan:
		`SELECT *`
		`FROM dbo.Account;`
			Msg 229, Level 14, State 5, Line 4
			The SELECT permission was denied on the object 'Account', database 'SmartBankProd', schema 'dbo'.
			
			Completion time: 2026-08-23T15:53:37.5498249+08:00
				Passed

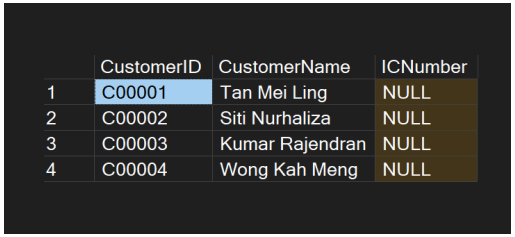
40.	Attempt to perform direct `SELECT` on `dbo.TransactionRecord` base table as `cust_tan`.
	
	Expected
	Error returned. Permission denied on the `TransactionRecord` table.
		Run as cust_tan:
		`SELECT *`
		`FROM dbo.TransactionRecord;`
			Msg 229, Level 14, State 5, Line 4
			The SELECT permission was denied on the object 'TransactionRecord', database 'SmartBankProd', schema 'dbo'.
			
			Completion time: 2026-08-23T15:54:02.8644882+08:00
				Passed

Confidentiality: No authenticated user may view another user's identity link.				

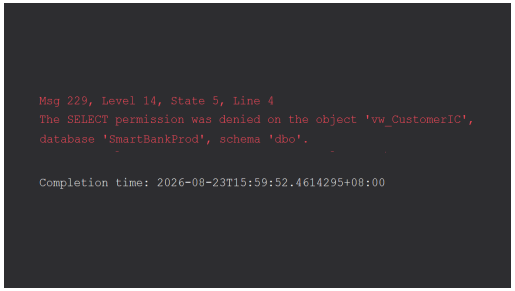
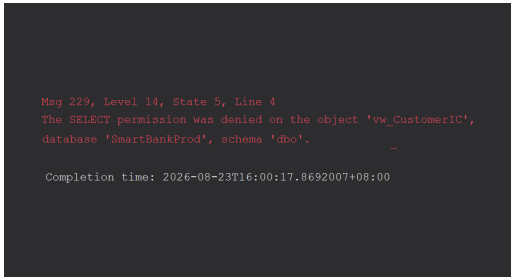
41.	Read the identity mapping table as `cust_tan`.
	
	Expected
	Only the `cust_tan` row is returned.
		Run as cust_tan:
		`SELECT *`
		`FROM Security.UserMapping;`
			DatabaseUser LoginName StaffID CustomerID SysStartTime SysEndTime
			1 cust_tan cust_tan NULL C00001 2026-08-22 13:33:28.0916600 9999-12-31 23:59:59.9999999
				Passed

42.	Read the identity mapping table as `ofc_nurul`.
	
	Expected
	Only the `ofc_nurul` row is returned.
		Run as ofc_nurul:
		`SELECT *`
		`FROM Security.UserMapping;`
			DatabaseUser LoginName StaffID CustomerID SysStartTime SysEndTime
			1 ofc_nurul ofc_nurul S00005 NULL 2026-08-22 13:33:28.0916600 9999-12-31 23:59:59.9999999
				Passed

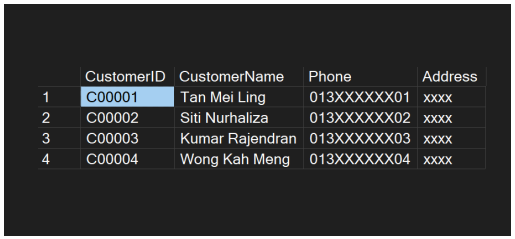
No.	Description	Execution	Results	Passed / Failed
Confidentiality: The database and its encryption keys cannot be exported or backed up without authorisation.				
43.	Attempt to back up the production database as `ofc_nurul`. Expected Error returned. Permission for `BACKUP DATABASE` is denied.	Run as ofc_nurul: `BACKUP DATABASE SmartBankProd TO DISK = 'C:\SmartBankServer\Backup\Production\Unauthorised.bak';`	Msg 262, Level 14, State 1, Line 4 BACKUP DATABASE permission denied in database 'SmartBankProd'. Msg 3013, Level 16, State 1, Line 4 BACKUP DATABASE is terminating abnormally. Completion time: 2026-08-23T15:55:59.2476154+08:00	Passed
44.	Attempt to back up the customer PII certificate as `ofc_nurul`. Expected Error returned. Permission for `BACKUP CERTIFICATE` is denied.	Run as ofc_nurul: `BACKUP CERTIFICATE Cert_CustomerPII TO FILE = 'C:\SmartBankServer\Encryption\Stolen.cer';`	Msg 15247, Level 16, State 1, Line 4 User does not have permission to perform this action. Completion time: 2026-08-23T15:56:27.2360801+08:00	Passed
Confidentiality: Customer IC numbers can only be viewed by bank managers.				
45.	Retrieve the customer IC number as `mgr_aisyah`. Expected Plain-text IC number is returned.	Run as mgr_aisyah: `EXEC dbo.usp_GetCustomerIC @CustomerID = 'C00001';`	CustomerID: C00001 CustomerName: Tan Mei Ling ICNumber: 900314-14-5521	Passed

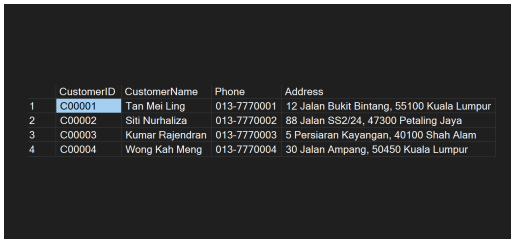
No.	Description	Execution	Results	Passed / Failed
46.	Retrieve the customer IC number as `ofc_nurul`. Expected IC column returns NULL.	Run as ofc_nurul: `SELECT * FROM dbo.vw_CustomerIC;`	 CustomerID / CustomerName / ICNumber 1 C00001 Tan Mei Ling NULL 2 C00002 Siti Nurhaliza NULL 3 C00003 Kumar Rajendran NULL 4 C00004 Wong Kah Meng NULL	Passed

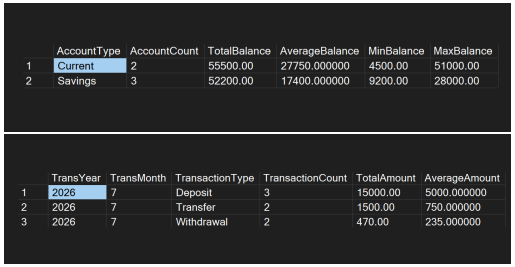
47.	Attempt to call the IC decryption procedure as `ofc_nurul`.
	
	Expected
	Error returned. Permission denied on `usp_GetCustomerIC`.
		Run as ofc_nurul:
		`EXEC dbo.usp_GetCustomerIC`
		`@CustomerID = 'C00001';`
			
			Msg 229, Level 14, State 5, Procedure dbo.usp_GetCustomerIC, Line 1 [Batch Start Line 3]
			The EXECUTE permission was denied on the object 'usp_GetCustomerIC', database 'SmartBankProd', schema 'dbo'.
			Completion time: 2026-08-23T15:58:20.2273423+08:00
				Passed

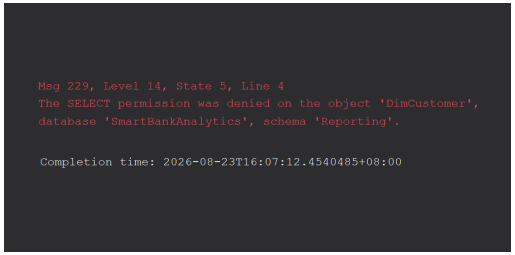
48.	Attempt to read the customer IC view as `dba_ahmad`. Expected Error returned. Permission denied on `vw_CustomerIC`.	Run as dba_ahmad: `SELECT * FROM dbo.vw_CustomerIC;`	 Msg 229, Level 14, State 5, Line 4 The SELECT permission was denied on the object 'vw_CustomerIC', database 'SmartBankProd', schema 'dbo'. Completion time: 2026-08-23T15:59:52.4614295+08:00	Passed
49.	Attempt to read the customer IC view as `cust_tan`. Expected Error returned. Permission denied on `vw_CustomerIC`.	Run as cust_tan: `SELECT * FROM dbo.vw_CustomerIC;`	 Msg 229, Level 14, State 5, Line 4 The SELECT permission was denied on the object 'vw_CustomerIC', database 'SmartBankProd', schema 'dbo'. Completion time: 2026-08-23T16:00:17.8692007+08:00	Passed

No.	Description	Execution	Results	Passed / Failed
Confidentiality: Customer phone number and address are visible only to bank managers. Analysts may see aggregated figures only.				

50.	Retrieve the customer profile as `ofc_nurul`.
	Expected
	`Phone` and `Address` are masked.
		Run as ofc_nurul:
		`SELECT *`
		`FROM dbo.vw_CustomerProfile;`
			
			CustomerID | CustomerName | Phone | Address
			1 | C00001 | Tan Mei Ling | 013XXXXXX01 | xxxx
			2 | C00002 | Siti Nurhaliza | 013XXXXXX02 | xxxx
			3 | C00003 | Kumar Rajendran | 013XXXXXX03 | xxxx
			4 | C00004 | Wong Kah Meng | 013XXXXXX04 | xxxx
				Passed

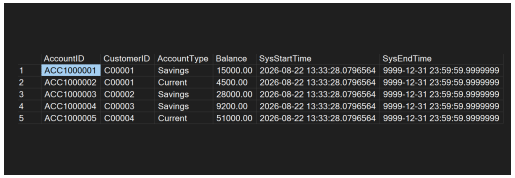
51.	Retrieve the customer profile as `mgr_aisyah`.
	Expected
	`Phone` and `Address` show the real values.
		Run as mgr_aisyah:
		`SELECT *`
		`FROM dbo.vw_CustomerProfile;`
			
			CustomerID | CustomerName | Phone | Address
			1 | C00001 | Tan Mei Ling | 013-7770001 | 12 Jalan Bukit Bintang, 55100 Kuala Lumpur
			2 | C00002 | Siti Nurhaliza | 013-7770002 | 88 Jalan SS2/24, 47300 Petaling Jaya
			3 | C00003 | Kumar Rajendran | 013-7770003 | 5 Persiaran Kayangan, 40100 Shah Alam
			4 | C00004 | Wong Kah Meng | 013-7770004 | 30 Jalan Ampang, 50450 Kuala Lumpur
				Passed

52.	Retrieve the aggregated reporting view as `analyst_lee`.
	Expected
	The account summary and transaction trend views return aggregated rows.
		Run as analyst_lee:
		`SELECT *`
		`FROM Reporting.vw_AccountSummary;`
		`SELECT *`
		`FROM Reporting.vw_TransactionTrend;`
			
			AccountType | AccountCount | TotalBalance | AverageBalance | MinBalance | MaxBalance
			1 | Current | 2 | 55500.00 | 27750.000000 | 4500.00 | 51000.00
			2 | Savings | 3 | 52200.00 | 17400.000000 | 9200.00 | 28000.00
			TransYear | TransMonth | TransactionType | TransactionCount | TotalAmount | AverageAmount
			1 | 2026 | 7 | Deposit | 3 | 15000.00 | 5000.000000
			2 | 2026 | 7 | Transfer | 2 | 1500.00 | 750.000000
			3 | 2026 | 7 | Withdrawal | 2 | 470.00 | 235.000000
				Passed

53.	Attempt to read the customer dimension table as `analyst_lee`.
	Expected
	Error returned. Permission denied on the `Reporting.DimCustomer` table.
		Run as analyst_lee:
		`SELECT *`
		`FROM Reporting.DimCustomer;`
			
			Msg 229, Level 14, State 5, Line 4
			The SELECT permission was denied on the object 'DimCustomer', database 'SmartBankAnalytics', schema 'Reporting'.
			Completion time: 2026-08-23T16:07:12.4540485+08:00
				Passed

No.	Description	Execution	Results	Passed / Failed
54.	Attempt to read the account fact table as `analyst_lee`. Expected Error returned. Permission denied on the `Reporting.FactAccount` table.	Run as analyst_lee: `SELECT * FROM Reporting.FactAccount;`	Msg 229, Level 14, State 5, Line 4 The SELECT permission was denied on the object 'FactAccount', database 'SmartBankAnalytics', schema 'Reporting'. Completion time: 2026-08-23T16:07:49.3797682+08:00	Passed
Confidentiality: ETL service account may only read production data for extraction.				
55.	Run the ETL pipeline as `svc_ETL`. Expected ETL completed successfully. The health view shows a `Success` entry.	Run as svc_ETL: `EXEC Reporting.usp_RunETL;`	ETL completed successfully. Completion time: 2026-08-23T16:09:53.4611574+08:00 RunID 3 StartedOn 2026-08-23 16:09:53 CompletedOn 2026-08-23 16:09:53 RunStatus Success CustomerRows 4 AccountRows 5 TransactionRows 7 Message NULL	Passed
56.	Compare the analytics row counts with records in production. Expected `ProductionRows` equals `AnalyticsRows` for customers, accounts and transactions.	Run as svc_ETL: `SELECT 'Customer' AS Entity, (SELECT COUNT(*) FROM SmartBankProd.dbo.Customer) AS ProductionRows, (SELECT COUNT(*) FROM Reporting.DimCustomer) AS AnalyticsRows UNION ALL SELECT 'Account', (SELECT COUNT(*) FROM SmartBankProd.dbo.Account), (SELECT COUNT(*) FROM Reporting.FactAccount) UNION ALL SELECT 'Transaction', (SELECT COUNT(*) FROM SmartBankProd.dbo.TransactionRecord), (SELECT COUNT(*) FROM Reporting.FactTransaction);`	1 Customer 4 4 2 Account 5 5 3 Transaction 7 7	Passed

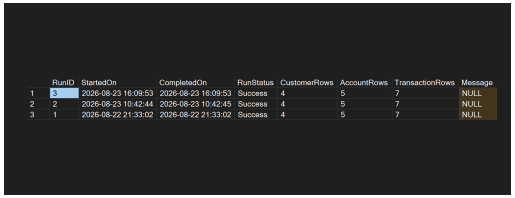
No.	Description	Execution	Results	Passed / Failed

57.	Read production data as `svc_ETL`.
	Expected
	`SELECT` query executes successfully.
		Run as svc_ETL:
		`SELECT *`
		`FROM dbo.Account;`
				Passed

58.	Attempt to perform direct `UPDATE` on production data as `svc_ETL`.
	Expected
	Error returned. Permission denied on the `Account` table.
		Run as svc_ETL:
		`UPDATE dbo.Account`
		`SET Balance = 0`
		`WHERE AccountID = 'ACC1000001';`
			Msg 229, Level 14, State 5, Line 7
			The UPDATE permission was denied on the object 'Account', database 'SmartBankProd', schema 'dbo'.
			Completion time: 2026-08-23T16:13:31.4898075+08:00
				Passed

59.	Attempt to read `Staff` table as `svc_ETL`.
	Expected
	Error returned. Permission denied on the `Staff` table.
		Run as svc_ETL:
		`SELECT *`
		`FROM dbo.Staff;`
			Msg 229, Level 14, State 5, Line 7
			The SELECT permission was denied on the object 'Staff', database 'SmartBankProd', schema 'dbo'.
			Completion time: 2026-08-23T16:13:55.2900677+08:00
				Passed

60.	Attempt to execute ETL procedure as `analyst_lee`. Expected Error returned. Permission denied on `usp_RunETL`.	Run as analyst_lee: `EXEC Reporting.usp_RunETL;`	Msg 229, Level 14, State 5, Procedure Reporting.usp_RunETL, Line 1 [Batch Start Line 3] The EXECUTE permission was denied on the object 'usp_RunETL', database 'SmartBankAnalytics', schema 'Reporting'. Completion time: 2026-08-23T16:14:19.4135787+08:00	Passed

61.	Read the ETL run log as `smartbank_admin`.
	Expected
	`RunStatus` show `Success`, with data no more than one day old.
		Run as smartbank_admin:
		`SELECT *`
		`FROM Reporting.vw_ETLHealth`
		`ORDER BY RunID DESC;`
				Passed

No.	Description	Execution	Results	Passed / Failed
Integrity: DBA may only create and manage tables but not data.				
62.	Create a table as `dba_ahmad`. Expected The table is created successfully.	Run as dba_ahmad: `CREATE TABLE dbo.TestCase_Table (TestID INT NOT NULL);`	Commands completed successfully. Completion time: 2026-08-23T16:15:33.2716120+08:00	Passed
63.	Add a new column to the test table as `dba_ahmad`. Expected The new column is added successfully.	Run as dba_ahmad: `ALTER TABLE dbo.TestCase_Table ADD TestDesc VARCHAR(50) NULL;`	Commands completed successfully. Completion time: 2026-08-23T16:16:01.5952186+08:00 TestID | TestDesc	Passed
64.	Drop the test table as `dba_ahmad`. Expected The table is dropped successfully.	Run as dba_ahmad: `DROP TABLE dbo.TestCase_Table;`	Commands completed successfully. Completion time: 2026-08-23T16:18:30.7619890+08:00	Passed
65.	Attempt to create a table as `mgr_aisyah`. Expected Error returned. Permission for `CREATE TABLE` is denied.	Run as mgr_aisyah: `CREATE TABLE dbo.TestCase_Table (TestID INT NOT NULL);`	Msg 262, Level 14, State 1, Line 4 CREATE TABLE permission denied in database 'SmartBankProd'. Completion time: 2026-08-23T16:19:24.1136575+08:00	Passed

No.	Description	Execution	Results	Passed / Failed
66.	Attempt to create a table as `ofc_nurul`. Expected Error returned. Permission for `CREATE TABLE` is denied.	Run as ofc_nurul: `CREATE TABLE dbo.TestCase_Table (TestID INT NOT NULL);`	Msg 262, Level 14, State 1, Line 4 CREATE TABLE permission denied in database 'SmartBankProd'. Completion time: 2026-08-23T16:20:21.5743878+08:00	Passed
67.	Attempt to create a table as `cust_tan`. Expected Error returned. Permission for `CREATE TABLE` is denied.	Run as cust_tan: `CREATE TABLE dbo.TestCase_Table (TestID INT NOT NULL);`	Msg 262, Level 14, State 1, Line 4 CREATE TABLE permission denied in database 'SmartBankProd'. Completion time: 2026-08-23T16:21:47.9441466+08:00	Passed
68.	Attempt to perform direct `SELECT` on `dbo.Account` as `dba_ahmad`. Expected Error returned. Permission denied on the `Account` table.	Run as dba_ahmad: `SELECT * FROM dbo.Account;`	Msg 229, Level 14, State 5, Line 4 The SELECT permission was denied on the object 'Account', database 'SmartBankProd', schema 'dbo'. Completion time: 2026-08-23T16:22:18.2507673+08:00	Passed
69.	Attempt to perform direct `INSERT` into `dbo.Account` as `dba_ahmad`. Expected Error returned. Permission denied on the `Account` table.	Run as dba_ahmad: `INSERT INTO dbo.Account (AccountID, CustomerID, AccountType, Balance) VALUES ('ACC3000001', 'C00001', 'Savings', 100);`	Msg 229, Level 14, State 5, Line 4 The INSERT permission was denied on the object 'Account', database 'SmartBankProd', schema 'dbo'. Completion time: 2026-08-23T16:22:41.5280200+08:00	Passed

No.	Description	Execution	Results	Passed / Failed

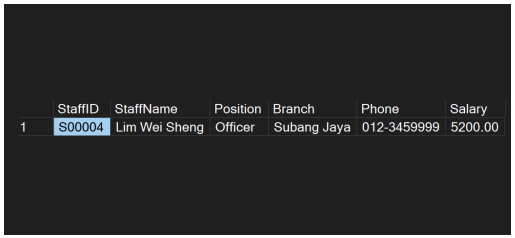
70.	Attempt to perform direct `UPDATE` on `dbo.Account` as `dba_ahmad`.
	
	Expected
	Error returned. Permission denied on the `Account` table.
		Run as dba_ahmad:
		`UPDATE dbo.Account`
		`SET Balance = 0`
		`WHERE AccountID = 'ACC1000001';`
			Msg 229, Level 14, State 5, Line 4
			The SELECT permission was denied on the object 'Account', database 'SmartBankProd', schema 'dbo'.
			Msg 229, Level 14, State 5, Line 4
			The UPDATE permission was denied on the object 'Account', database 'SmartBankProd', schema 'dbo'.
			
			Completion time: 2026-08-23T16:23:09.9448384+08:00
				Passed

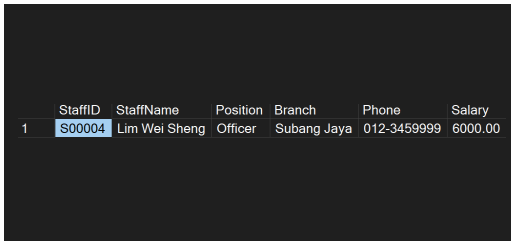
71.	Attempt to perform direct `DELETE` on `dbo.Account` as `dba_ahmad`.
	
	Expected
	Error returned. Permission denied on the `Account` table.
		Run as dba_ahmad:
		`DELETE FROM dbo.Account`
		`WHERE AccountID = 'ACC1000001';`
			Msg 229, Level 14, State 5, Line 4
			The SELECT permission was denied on the object 'Account', database 'SmartBankProd', schema 'dbo'.
			Msg 229, Level 14, State 5, Line 4
			The DELETE permission was denied on the object 'Account', database 'SmartBankProd', schema 'dbo'.
			
			Completion time: 2026-08-23T16:23:41.0738087+08:00
				Passed

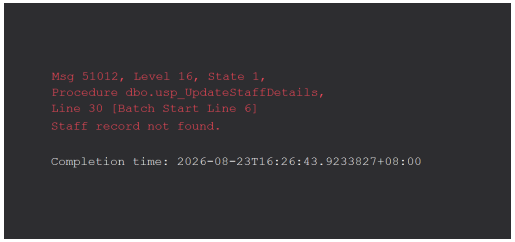
72.	Attempt to add `dba_ahmad` to `role_Manager` as `dba_ahmad`.
	
	Expected
	Error returned. Permission for `ALTER ROLE` is denied.
		Run as dba_ahmad:
		`ALTER ROLE role_Manager`
		`ADD MEMBER [dba_ahmad];`
			Msg 15151, Level 16, State 2, Line 4
			Cannot alter the role 'role_Manager', because it does not exist or you do not have permission.
			
			Completion time: 2026-08-23T16:24:09.0649530+08:00
				Passed

73.	Attempt to create a new database user as `dba_ahmad`.
	
	Expected
	Error returned. Permission for `CREATE USER` is denied.
		Run as dba_ahmad:
		`CREATE USER [test_user]`
		`WITHOUT LOGIN;`
			Msg 15247, Level 16, State 1, Line 4
			User does not have permission to perform this action.
			
			Completion time: 2026-08-23T16:24:45.5391504+08:00
				Passed

No.	Description	Execution	Results	Passed / Failed
Integrity: Only bank manager may manage staff details.				

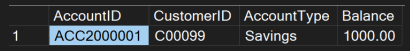
74.	Update staff member's phone and branch as `mgr_aisyah`.
	
	Expected
	Staff record is updated successfully.
		Run as mgr_aisyah:
		`EXEC dbo.usp_UpdateStaffDetails`
		`@StaffID = 'S00004',`
		`@Phone = '012-3459999',`
		`@Branch = 'Subang Jaya';`
				Passed

75.	Update staff member's salary as `mgr_aisyah`.
	
	Expected
	Staff record is updated successfully.
		Run as mgr_aisyah:
		`EXEC dbo.usp_UpdateStaffDetails`
		`@StaffID = 'S00004',`
		`@Salary = 6000.00;`
				Passed

76.	Update a `StaffID` that does not exist as `mgr_aisyah`.
	
	Expected
	Error returned. Staff record not found.
		Run as mgr_aisyah:
		`EXEC dbo.usp_UpdateStaffDetails`
		`@StaffID = 'S99999',`
		`@Phone = '012-0000000';`
				Passed

77.	Attempt to edit own details as `mgr_aisyah`.
	
	Expected
	Error returned. Managers may not update their own details.
	
	[*Note: Own details are updated using the procedure* `usp_UpdateOwnStaffRecord`.]
		Run as mgr_aisyah:
		`EXEC dbo.usp_UpdateStaffDetails`
		`@StaffID = 'S00002',`
		`@Phone = 012-2348765;`
				Passed

No.	Description	Execution	Results	Passed / Failed
78.	Attempt to update details of other staff as `ofc_nurul`. Expected Error returned. Permission denied for `usp_UpdateStaffDetails`.	Run as ofc_nurul: `EXEC dbo.usp_UpdateStaffDetails @StaffID = 'S00005', @Salary = 20000.00;`	Msg 229, Level 14, State 5, Procedure dbo.usp_UpdateStaffDetails, Line 1 [Batch Start Line 3] The EXECUTE permission was denied on the object 'usp_UpdateStaffDetails', database 'SmartBankProd', schema 'dbo'. Completion time: 2026-08-23T16:30:58.2324976+08:00	Passed
79.	Attempt to update details of other staff as `dba_ahmad`. Expected Error returned. Permission denied for `usp_UpdateStaffDetails`.	Run as dba_ahmad: `EXEC dbo.usp_UpdateStaffDetails @StaffID = 'S00005', @Salary = 20000.00;`	Msg 229, Level 14, State 5, Procedure dbo.usp_UpdateStaffDetails, Line 1 [Batch Start Line 3] The EXECUTE permission was denied on the object 'usp_UpdateStaffDetails', database 'SmartBankProd', schema 'dbo'. Completion time: 2026-08-23T16:31:24.0625191+08:00	Passed
80.	Attempt to update details of other staff as `cust_tan`. Expected Error returned. Permission denied for `usp_UpdateStaffDetails`.	Run as cust_tan: `EXEC dbo.usp_UpdateStaffDetails @StaffID = 'S00005', @Salary = 20000.00;`	Msg 229, Level 14, State 5, Procedure dbo.usp_UpdateStaffDetails, Line 1 [Batch Start Line 3] The EXECUTE permission was denied on the object 'usp_UpdateStaffDetails', database 'SmartBankProd', schema 'dbo'. Completion time: 2026-08-23T16:31:48.9614762+08:00	Passed
Integrity: Only bank officers may manage customer accounts.				
81.	Register a new customer with valid IC number as `ofc_nurul`. Expected Customer is registered successfully.	Run as ofc_nurul: `EXEC dbo.usp_RegisterCustomer @CustomerID = 'C00099', @CustomerName = 'Test Cust', @ICNumber = '990101-14-9999', @Phone = '012-3456789', @Address = 'Test Address';`	CustomerID, CustomerName, Phone, Address 1, C00099, Test Cust, 012XXXXXX89, xxxx	Passed

No.	Description	Execution	Results	Passed / Failed
82.	Register a new customer with invalid IC number as `ofc_nurul`. Expected Error returned. Invalid IC number format.	Run as ofc_nurul: `EXEC dbo.usp_RegisterCustomer @CustomerID = 'C00098', @CustomerName = 'Test Cust', @ICNumber = '99010114-9999';`	Msg 51042, Level 16, State 1, Procedure dbo.usp_RegisterCustomer, Line 42 [Batch Start Line 3] IC number must be in the format YYMMDD-PB-###G. Completion time: 2026-08-23T16:33:09.0017110+08:00	Passed
83.	Register a new customer with a `CustomerID` that already exist, as `ofc_nurul`. Expected Error returned. `CustomerID` is registered.	Run as ofc_nurul: `EXEC dbo.usp_RegisterCustomer @CustomerID = 'C00001', @CustomerName = 'Test Cust', @ICNumber = '990101-14-9999';`	Msg 51041, Level 16, State 1, Procedure dbo.usp_RegisterCustomer, Line 33 [Batch Start Line 3] That CustomerID is already registered. Completion time: 2026-08-23T16:34:12.9371086+08:00	Passed
84.	Attempt to register a new customer as `cust_tan`. Expected Error returned. Permission denied for `usp_RegisterCustomer`.	Run as cust_tan: `EXEC dbo.usp_RegisterCustomer @CustomerID = 'C00097', @CustomerName = 'Test Cust', @ICNumber = '900101-14-1234';`	Msg 229, Level 14, State 5, Procedure dbo.usp_RegisterCustomer, Line 1 [Batch Start Line 3] The EXECUTE permission was denied on the object 'usp_RegisterCustomer', database 'SmartBankProd', schema 'dbo'. Completion time: 2026-08-23T16:35:03.7254844+08:00	Passed
85.	Create a savings account for the new customer as `ofc_nurul`. Expected Customer account created successfully.	Run as ofc_nurul: `EXEC dbo.usp_CreateCustomerAccount @AccountID = 'ACC2000001', @CustomerID = 'C00099', @AccountType = 'Savings', @OpeningBalance = 1000.00;`	 AccountID: ACC2000001 | CustomerID: C00099 | AccountType: Savings | Balance: 1000.00	Passed

No.	Description	Execution	Results	Passed / Failed
86.	Create an account with a negative opening balance as `ofc_nurul`. Expected Error returned. `OpeningBalance` cannot be negative.	Run as ofc_nurul: `EXEC dbo.usp_CreateCustomerAccount @AccountID = 'ACC2000002', @CustomerID = 'C00099', @AccountType = 'Savings', @OpeningBalance = -500.00;`	Msg 51015, Level 16, State 1, Procedure dbo.usp_CreateCustomerAccount, Line 36 [Batch Start Line 3] Opening balance cannot be negative. Completion time: 2026-08-23T16:36:11.6228180+08:00	Passed
87.	Create an account for a customer that does not exist as `ofc_nurul`. Expected Error returned. Customer does not exist.	Run as ofc_nurul: `EXEC dbo.usp_CreateCustomerAccount @AccountID = 'ACC2000003', @CustomerID = 'C99999', @AccountType = 'Savings', @OpeningBalance = 100.00;`	Msg 51014, Level 16, State 1, Procedure dbo.usp_CreateCustomerAccount, Line 27 [Batch Start Line 3] Line 27 [Batch Start Line 3] Customer does not exist. Completion time: 2026-08-23T16:36:40.2496201+08:00	Passed
88.	Attempt to create a customer account as `mgr_aisyah`. Expected Error returned. Permission denied for `usp_CreateCustomerAccount`.	Run as mgr_aisyah: `EXEC dbo.usp_CreateCustomerAccount @AccountID = 'ACC2000004', @CustomerID = 'C00001', @AccountType = 'Savings', @OpeningBalance = 100.00;`	Msg 229, Level 14, State 5, Procedure dbo.usp_CreateCustomerAccount, Line 1 [Batch Start Line 6] The EXECUTE permission was denied on the object 'usp_CreateCustomerAccount', database 'SmartBankProd', schema 'dbo'. Completion time: 2026-08-23T16:36:59.6582035+08:00	Passed
89.	Attempt to create a customer account as `dba_ahmad`. Expected Error returned. Permission denied for `usp_CreateCustomerAccount`.	Run as dba_ahmad: `EXEC dbo.usp_CreateCustomerAccount @AccountID = 'ACC2000005', @CustomerID = 'C00001', @AccountType = 'Savings', @OpeningBalance = 100.00;`	Msg 229, Level 14, State 5, Procedure dbo.usp_CreateCustomerAccount, Line 1 [Batch Start Line 3] The EXECUTE permission was denied on the object 'usp_CreateCustomerAccount', database 'SmartBankProd', schema 'dbo'. Completion time: 2026-08-23T16:37:27.7414916+08:00	Passed

No.	Description	Execution	Results	Passed / Failed
90.	Attempt to create a customer account as `cust_tan`. Expected Error returned. Permission denied for `usp_CreateCustomerAccount`.	Run as cust_tan: `EXEC dbo.usp_CreateCustomerAccount @AccountID = 'ACC2000006', @CustomerID = 'C00001', @AccountType = 'Savings', @OpeningBalance = 100.00;`	Msg 229, Level 14, State 5, Procedure dbo.usp_CreateCustomerAccount, Line 1 [Batch Start Line 3] The EXECUTE permission was denied on the object 'usp_CreateCustomerAccount', database 'SmartBankProd', schema 'dbo'. Completion time: 2026-08-23T16:37:53.6857020+08:00	Passed
91.	Change the account type of an existing account as `ofc_nurul`. Expected Account successfully converted to `Current` account.	Run as ofc_nurul: `EXEC dbo.usp_UpdateCustomerAccount @AccountID = 'ACC2000001', @AccountType = 'Current';`	AccountID | CustomerID | AccountType | Balance 1 | ACC2000001 | C00099 | Current | 1000.00	Passed
92.	Change the account type of an account that does not exist as `ofc_nurul`. Expected Error returned. Account does not exist.	Run as ofc_nurul: `EXEC dbo.usp_UpdateCustomerAccount @AccountID = 'ACC9999999', @AccountType = 'Savings';`	Msg 51016, Level 16, State 1, Procedure dbo.usp_UpdateCustomerAccount, Line 30 [Batch Start Line 3] Account not found. Completion time: 2026-08-23T16:38:41.5810795+08:00	Passed
93.	Attempt to perform direct `INSERT` into `dbo.Account` as `ofc_nurul`. Expected Error returned. Permission denied on the `Account` table.	Run as ofc_nurul: `INSERT INTO dbo.Account (AccountID, CustomerID, AccountType, Balance) VALUES ('ACC2000007', 'C00001', 'Savings', 999999.00);`	Msg 229, Level 14, State 5, Line 4 The INSERT permission was denied on the object 'Account', database 'SmartBankProd', schema 'dbo'. Completion time: 2026-08-23T16:39:02.7945742+08:00	Passed

No.	Description	Execution	Results	Passed / Failed

94.	Attempt to perform direct `UPDATE` on `dbo.Account` as `ofc_nurul`.
	Expected
	Error returned. Permission denied on the `Account` table.
		Run as ofc_nurul:
		`UPDATE dbo.Account`
		`SET Balance = 999999.00`
		`WHERE AccountID = 'ACC1000001';`
			Msg 229, Level 14, State 5, Line 4
			The SELECT permission was denied on the object 'Account', database 'SmartBankProd', schema 'dbo'.
			Msg 229, Level 14, State 5, Line 4
			The UPDATE permission was denied on the object 'Account', database 'SmartBankProd', schema 'dbo'.
			Completion time: 2026-08-23T16:39:31.9536367+08:00
				Passed

Integrity: Each staff can update their own record except salary.				

95.	Update own phone number and branch as `ofc_nurul`.
	Expected
	Record updated successfully.
		Run as ofc_nurul:
		`EXEC dbo.usp_UpdateOwnStaffRecord`
		`@Phone = '012-3450044',`
		`@Branch = 'Cheras';`
			StaffID | StaffName | Position | Branch | Phone | Salary
			1 | S00005 | Nurul Izzah | Officer | Cheras | 012-3450044 | 4900.00
				Passed

96.	Attempt to perform a direct `UPDATE` on own salary as `ofc_nurul`.
	Expected
	Error returned. Permission denied on the `Staff` table.
		Run as ofc_nurul:
		`UPDATE dbo.Staff`
		`SET Salary = 50000.00`
		`WHERE StaffID = 'S00005';`
			Msg 229, Level 14, State 5, Line 5
			The SELECT permission was denied on the object 'Staff', database 'SmartBankProd', schema 'dbo'.
			Msg 229, Level 14, State 5, Line 5
			The UPDATE permission was denied on the object 'Staff', database 'SmartBankProd', schema 'dbo'.
			Completion time: 2026-08-23T16:40:31.0395519+08:00
				Passed

97.	Attempt to raise own salary as `mgr_aisyah`.
	Expected
	Error returned. Managers cannot raise their own salary.
		Run as mgr_aisyah:
		`EXEC dbo.usp_UpdateStaffDetails`
		`@StaffID = 'S00002',`
		`@Salary = 50000.00;`
			Msg 51013, Level 16, State 1,
			Procedure dbo.usp_UpdateStaffDetails,
			Line 44 [Batch Start Line 6]
			Managers may not update their own staff record.
			Use usp_UpdateOwnStaffRecord instead.
			Completion time: 2026-08-23T16:40:54.7659503+08:00
				Passed

No.	Description	Execution	Results	Passed / Failed

98.	Attempt to perform direct `UPDATE` on salary as `smartbank_admin`.
	
	Expected
	Error returned. Update is rolled back by the `trg_Staff_SalaryGuard` trigger.
		Run as smartbank_admin:
		`UPDATE dbo.Staff`
		`SET Salary = 1.00`
		`WHERE StaffID = 'S00005';`
			Msg 51021, Level 16, State 1,
			Procedure trg_Staff_SalaryGuard,
			Line 44 [Batch Start Line 62]
			Refused: only Bank Managers may modify salary.
			
			Completion time: 2026-08-23T16:58:27.2839490+08:00
				Passed

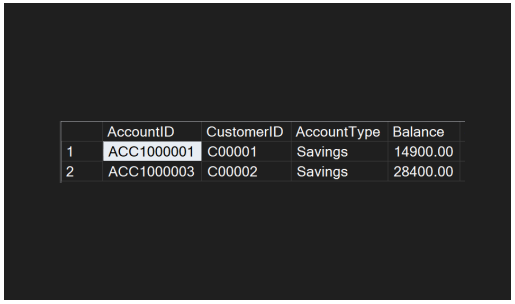
99.	Attempt to execute staff self-update procedure as `cust_tan`.
	
	Expected
	Error returned. Permission denied for `usp_UpdateOwnStaffRecord`.
		Run as cust_tan:
		`EXEC dbo.usp_UpdateOwnStaffRecord`
		`@Phone = '013-0000000;`
			Msg 229, Level 14, State 5,
			Procedure dbo.usp_UpdateOwnStaffRecord, Line 1 [Batch Start Line 3]
			The EXECUTE permission was denied on the object 'usp_UpdateOwnStaffRecord',
			database 'SmartBankProd', schema 'dbo'.
			
			Completion time: 2026-08-23T16:47:13.5743360+08:00
				Passed

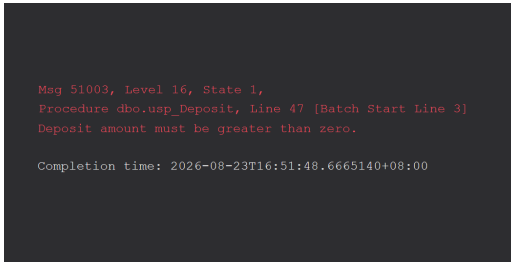
Integrity: Only customers may perform transactions.				

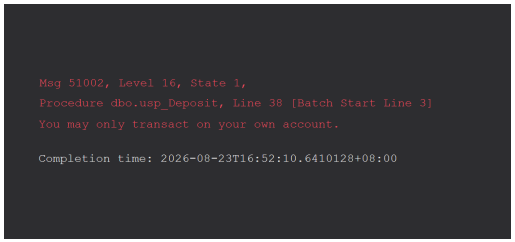
100.	Perform deposit of RM500 into own account as `cust_tan`.
	
	Expected
	Deposit performed successfully with balance of `ACC1000001` updated.
		Run as cust_tan:
		`EXEC dbo.usp_Deposit`
		`@AccountID = 'ACC1000001',`
		`@Amount = 500.00;`
			AccountID / CustomerID / AccountType / Balance
			1 ACC1000001 C00001 Savings 15500.00
				Passed

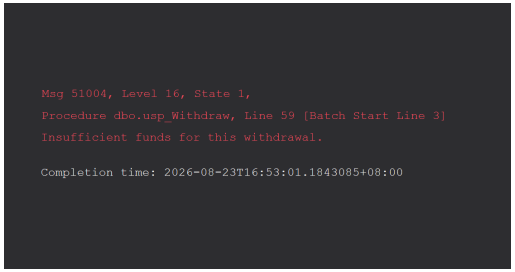
101.	Perform withdrawal of RM200 from own account as `cust_tan`.
	
	Expected
	Withdrawal performed successfully with balance of `ACC1000001` updated.
		Run as cust_tan:
		`EXEC dbo.usp_Withdraw`
		`@AccountID = 'ACC1000001',`
		`@Amount = 200.00;`
			AccountID / CustomerID / AccountType / Balance
			1 ACC1000001 C00001 Savings 15300.00
				Passed

No.	Description	Execution	Results	Passed / Failed

102.	Perform transfer of RM400 to another customer account as `cust_tan`.
	
	Expected
	Transfer performed successfully with balance of `ACC1000001` and `ACC1000003` updated.
		Run as cust_tan:
		`EXEC dbo.usp_Transfer`
		`@FromAccountID = 'ACC1000001',`
		`@ToAccountID = 'ACC1000003',`
		`@Amount = 400.00;`
			
			AccountID / CustomerID / AccountType / Balance
			1 ACC1000001 C00001 Savings 14900.00
			2 ACC1000003 C00002 Savings 28400.00
				Passed

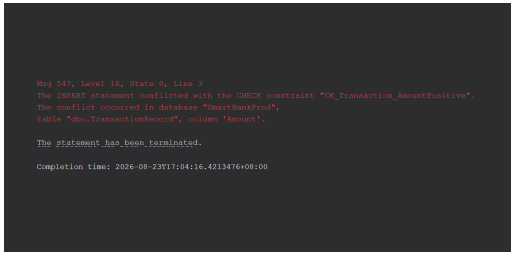
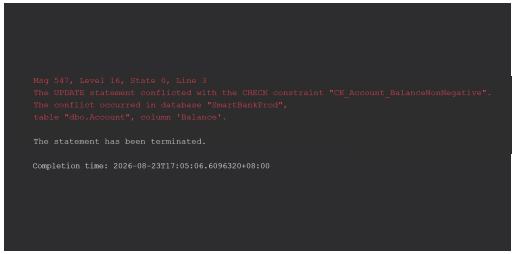
103.	Perform deposit with a negative amount as `cust_tan`.
	
	Expected
	Error returned. Deposit amount must be greater than zero.
		Run as cust_tan:
		`EXEC dbo.usp_Deposit`
		`@AccountID = 'ACC1000001',`
		`@Amount = -100.00;`
			
			Msg 51003, Level 16, State 1,
			Procedure dbo.usp_Deposit, Line 47 [Batch Start Line 3]
			Deposit amount must be greater than zero.
			Completion time: 2026-08-23T16:51:48.6665140+08:00
				Passed

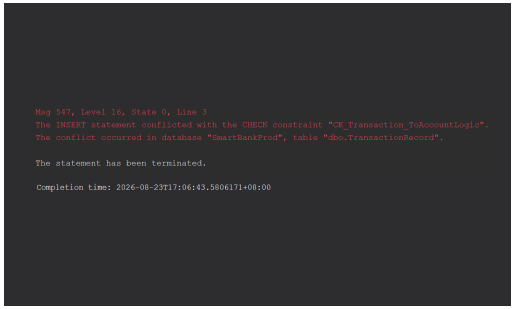
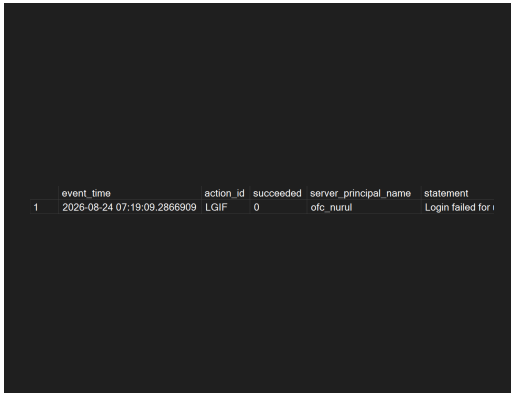
104.	Perform deposit into an account belonging to a different customer as `cust_tan`.
	
	Expected
	Error returned. Deposit can only be performed on own account.
		Run as cust_tan:
		`EXEC dbo.usp_Deposit`
		`@AccountID = 'ACC1000003',`
		`@Amount = 100.00;`
			
			Msg 51002, Level 16, State 1,
			Procedure dbo.usp_Deposit, Line 38 [Batch Start Line 3]
			You may only transact on your own account.
			Completion time: 2026-08-23T16:52:10.6410128+08:00
				Passed

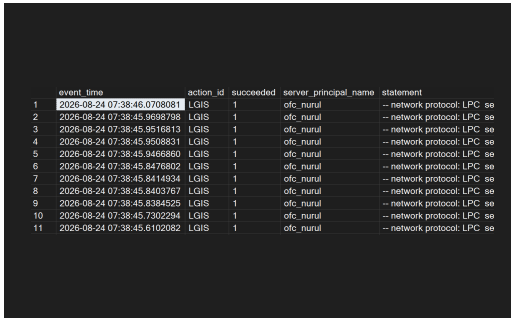
105.	Perform withdrawal of amount more than available balance as `cust_tan`.
	
	Expected
	Error returned. Insufficient funds available for withdrawal.
		Run as cust_tan:
		`EXEC dbo.usp_Withdraw`
		`@AccountID = 'ACC1000001',`
		`@Amount = 99999999.00;`
			
			Msg 51004, Level 16, State 1,
			Procedure dbo.usp_Withdraw, Line 59 [Batch Start Line 3]
			Insufficient funds for this withdrawal.
			Completion time: 2026-08-23T16:53:01.1843085+08:00
				Passed

No.	Description	Execution	Results	Passed / Failed
106.	Perform transfer to a destination account that does not exist, as `cust_tan`. Expected Error returned. Destination account does not exist.	Run as cust_tan: `EXEC dbo.usp_Transfer @FromAccountID = 'ACC1000001', @ToAccountID = 'ACC0000000', @Amount = 50.00;`	Msg 51006, Level 16, State 1, Procedure dbo.usp_Transfer, Line 60 [Batch Start Line 3] Destination account does not exist. Completion time: 2026-08-23T16:53:21.3142898+08:00	Passed
107.	Perform transfer from an account to itself as `cust_tan`. Expected Error returned. Source and destination accounts must differ.	Run as cust_tan: `EXEC dbo.usp_Transfer @FromAccountID = 'ACC1000001', @ToAccountID = 'ACC1000001', @Amount = 50.00;`	Msg 51005, Level 16, State 1, Procedure dbo.usp_Transfer, Line 51 [Batch Start Line 3] Source and destination accounts must differ. Completion time: 2026-08-23T16:53:52.5858335+08:00	Passed
108.	Attempt to perform deposit as `ofc_nurul`. Expected Error returned. Permission denied for `usp_Deposit`.	Run as ofc_nurul: `EXEC dbo.usp_Deposit @AccountID = 'ACC1000001', @Amount = 100.00;`	Msg 229, Level 14, State 5, Procedure dbo.usp_Deposit, Line 1 [Batch Start Line 4] The EXECUTE permission was denied on the object 'usp_Deposit', database 'SmartBankProd', schema 'dbo'. Completion time: 2026-08-23T16:54:37.4584060+08:00	Passed
109.	Attempt to perform transfer as `mgr_aisyah`. Expected Error returned. Permission denied for `usp_Transfer`.	Run as mgr_aisyah: `EXEC dbo.usp_Transfer @FromAccountID = 'ACC1000001', @ToAccountID = 'ACC1000003', @Amount = 100.00;`	Msg 229, Level 14, State 5, Procedure dbo.usp_Transfer, Line 1 [Batch Start Line 6] The EXECUTE permission was denied on the object 'usp_Transfer', database 'SmartBankProd', schema 'dbo'. Completion time: 2026-08-23T16:55:01.6147819+08:00	Passed

No.	Description	Execution	Results	Passed / Failed
110.	Attempt to perform withdrawal as `dba_ahmad`. Expected Error returned. Permission denied for `usp_Withdraw`.	Run as dba_ahmad: `EXEC dbo.usp_Withdraw @AccountID = 'ACC1000001', @Amount = 100.00;`	Msg 229, Level 14, State 5, Procedure dbo.usp_Withdraw, Line 1 [Batch Start Line 3] The EXECUTE permission was denied on the object 'usp_Withdraw', database 'SmartBankProd', schema 'dbo'. Completion time: 2026-08-23T16:55:29.2783397+08:00	Passed
111.	Attempt to perform a direct `INSERT` into `dbo.TransactionRecord` as `cust_tan`. Expected Error returned. Permission denied on the `TransactionRecord` table.	Run as cust_tan: `INSERT INTO dbo.TransactionRecord (AccountID, Amount, TransactionType) VALUES ('ACC1000001', 1000000.00, 'Deposit');`	Msg 229, Level 14, State 5, Line 4 The INSERT permission was denied on the object 'TransactionRecord', database 'SmartBankProd', schema 'dbo'. Completion time: 2026-08-23T16:56:23.4786894+08:00	Passed
112.	Attempt to perform a direct `UPDATE` on own account balance as `cust_tan`. Expected Error returned. Permission denied on the `Account` table.	Run as cust_tan: `UPDATE dbo.Account SET Balance = 1000000.00 WHERE AccountID = 'ACC1000001';`	Msg 229, Level 14, State 5, Line 4 The SELECT permission was denied on the object 'Account', database 'SmartBankProd', schema 'dbo'. Msg 229, Level 14, State 5, Line 4 The UPDATE permission was denied on the object 'Account', database 'SmartBankProd', schema 'dbo'. Completion time: 2026-08-23T17:00:21.7633319+08:00	Passed
Integrity: Data must remain accurate, consistent and tamper-proof.				
113.	Insert an account that references to an invalid customer as `smartbank_admin`. Expected Error returned. Foreign key conflict raised by `FK_Account_Customer`.	Run as smartbank_admin: `INSERT INTO dbo.Account (AccountID, CustomerID, AccountType, Balance) VALUES ('ACC9999999', 'C99999', 'Savings', 100.00);`	Msg 547, Level 16, State 0, Line 4 The INSERT statement conflicted with the FOREIGN KEY constraint "FK_Account_Customer". The conflict occurred in database "SmartBankProd", table "dbo.Customer", column 'CustomerID'. The statement has been terminated. Completion time: 2026-08-23T17:02:00.4804051+08:00	Passed

No.	Description	Execution	Results	Passed / Failed
114.	Insert a transaction record with type other than deposit, withdrawal, or transfer, as `smartbank_admin`. Expected Error returned. Insertion is rejected by `CK_Transaction_Type`.	Run as smartbank_admin: `INSERT INTO dbo.TransactionRecord (AccountID, Amount, TransactionType) VALUES ('ACC1000001', 100.00, 'Loan Disbursement');`		Passed
115.	Insert a transaction record with a negative amount as `smartbank_admin`. Expected Error returned. Insertion is rejected by `CK_Transaction_AmountPositive`.	Run as smartbank_admin: `INSERT INTO dbo.TransactionRecord (AccountID, Amount, TransactionType) VALUES ('ACC1000001', -50.00, 'Deposit');`		Passed
116.	Update an account balance to a negative value as `smartbank_admin`. Expected Error returned. Update is rejected by `CK_Account_BalanceNonNegative`.	Run as smartbank_admin: `UPDATE dbo.Account SET Balance = -1.00 WHERE AccountID = 'ACC1000001';`		Passed
117.	Insert a transaction record with transfer type but without a destination account, as `smartbank_admin`. Expected Error returned. Insertion is rejected by `CK_Transaction_ToAccountLogic`.	Run as smartbank_admin: `INSERT INTO dbo.TransactionRecord (AccountID, Amount, TransactionType, ToAccountID) VALUES ('ACC1000001', 100.00, 'Transfer', NULL);`		Passed

No.	Description	Execution	Results	Passed / Failed
118.	Insert a transaction record with deposit type but with a destination account, as `smartbank_admin`. Expected Error returned. Insertion is rejected by `CK_Transaction_ToAccountLogic`.	Run as smartbank_admin: `INSERT INTO dbo.TransactionRecord (AccountID, Amount, TransactionType, ToAccountID) VALUES ('ACC1000001', 100.00, 'Deposit', 'ACC1000003');`		Passed
119.	Attempt to delete a transaction record as `smartbank_admin`. Expected Error returned. Deletion is rejected by `trg_Transaction_NoDelete`.	Run as smartbank_admin: `DELETE FROM dbo.TransactionRecord WHERE TransID = 1;`		Passed
Integrity: All database and data manipulations, valid or invalid, must be logged.				
120.	Login as `ofc_nurul` and provide an invalid password. Then, read audit trail for failed login attempts as `smartbank_admin`. Expected The topmost LGIF row shows `server_principal_name = ofc_nurul` and `succeeded = 0`.	Run as smartbank_admin: `SELECT TOP (50) event_time, action_id, succeeded, server_principal_name, statement FROM sys.fn_get_audit_file( 'C:\SmartBankServer\Audit\*.sqlaudit', DEFAULT, DEFAULT) WHERE action_id = 'LGIF' ORDER BY event_time DESC;`		Passed

No.	Description	Execution	Results	Passed / Failed
121.	Login as `ofc_nurul` with the correct password. Then, read audit trail for successful logins as `smartbank_admin`. Expected The topmost LGIS row shows `server_principal_name = ofc_nurul` and `succeeded = 1`.	Run as smartbank_admin: `SELECT TOP (50) event_time, action_id, succeeded, server_principal_name, statement FROM sys.fn_get_audit_file( 'C:\SmartBankServer\Audit\*.sqlaudit', DEFAULT, DEFAULT) WHERE action_id = 'LGIS' ORDER BY event_time DESC;`		Passed
122.	Logout as `ofc_nurul`. Then, read audit trail for logouts as `smartbank_admin`. Expected The topmost LGO row shows `server_principal_name = ofc_nurul`.	Run as smartbank_admin: `SELECT TOP (50) event_time, action_id, succeeded, server_principal_name, statement FROM sys.fn_get_audit_file( 'C:\SmartBankServer\Audit\*.sqlaudit', DEFAULT, DEFAULT) WHERE action_id = 'LGO' ORDER BY event_time DESC;`		Passed

123.	Perform a permitted `SELECT` as `ofc_nurul`. Then, read audit trail as `smartbank_admin`.
	
	Expected
	The topmost SL row shows `database_principal_name = ofc_nurul` with `succeeded = 1`.
		Run as ofc_nurul:
		`SELECT * FROM dbo.vw_StaffDirectory;`
		
		Run as smartbank_admin:
		`SELECT TOP (20) event_time action_id, succeeded, database_principal_name, object_name FROM sys.fn_get_audit_file( 'C:\SmartBankServer\Audit\*.sqlaudit', DEFAULT, DEFAULT) WHERE action_id = 'SL' ORDER BY event_time DESC;`
				Passed

No.	Description	Execution	Results	Passed / Failed

124.	Execute a permitted procedure as `ofc_nurul`. Then, read audit trail as `smartbank_admin`.
	
	Expected
	The two most recent `EX` and `UP` rows show `database_principal_name = ofc_nurul` with `succeeded = 1`.
		Run as ofc_nurul:
		`EXEC dbo.usp_UpdateOwnStaffRecord @Phone = '012-3450045';`
		
		Run as smartbank_admin:
		`SELECT TOP (20) event_time, action_id, succeeded, database_principal_name, object_name FROM sys.fn_get_audit_file( 'C:\SmartBankServer\Audit\*.sqlaudit', DEFAULT, DEFAULT) WHERE action_id IN ('EX', 'UP') ORDER BY event_time DESC;`
			
			event_time / action_id / succeeded / database_principal_name / object_name
			1 2026-08-24 07:43:55.6356090 / UP / 1 / ofc_nurul / Staff
			2 2026-08-24 07:43:55.6081011 / EX / 1 / ofc_nurul / usp_UpdateOwnStaffRecord
				Passed

125.	Perform a denied `INSERT` as `cust_tan`. Then, read audit trail as `smartbank_admin`.
	
	Expected
	The topmost IN row shows `database_principal_name = cust_tan` with `succeeded = 0`.
		Run as cust_tan:
		`INSERT INTO dbo.Account (AccountID, CustomerID, AccountType, Balance) VALUES ('ACC4000001', 'C00001', 'Savings', 5000.00);`
		
		Run as smartbank_admin:
		`SELECT TOP (20) event_time, action_id, succeeded, database_principal_name, object_name FROM sys.fn_get_audit_file( 'C:\SmartBankServer\Audit\*.sqlaudit', DEFAULT, DEFAULT) WHERE action_id = 'IN' ORDER BY event_time DESC;`
			
			event_time / action_id / succeeded / database_principal_name / object_name
			1 2026-08-24 07:45:49.2133418 / IN / 0 / cust_tan / Account
				Passed

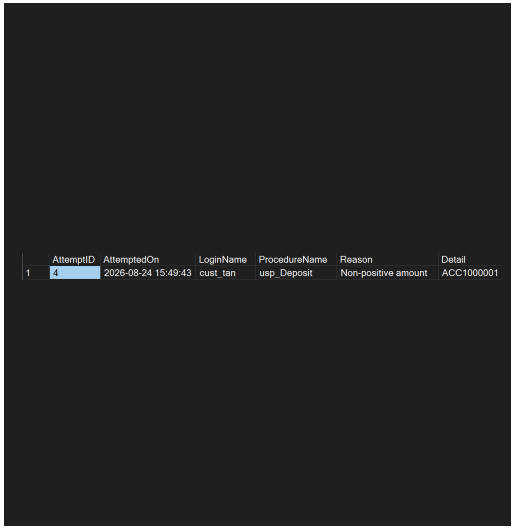
No.	Description	Execution	Results	Passed / Failed

126.	Perform a denied `UPDATE` as `cust_tan`. Then, read audit trail as `smartbank_admin`.
	
	Expected
	The topmost UP row shows `database_principal_name = cust_tan` with `succeeded = 0`.
		Run as cust_tan:
		`UPDATE dbo.Account`
		`SET Balance = 1000000`
		`WHERE AccountID = 'Acc1000001';`
		
		Run as smartbank_admin:
		`SELECT TOP (20)`
		`event_time, action_id, succeeded,`
		`database_principal_name, object_name`
		`FROM sys.fn_get_audit_file(`
		`'C:\SmartBankServer\Audit\*.sqlaudit',`
		`DEFAULT,`
		`DEFAULT)`
		`WHERE action_id = 'UP'`
		`ORDER BY event_time DESC;`
				Passed

127.	Attempt to create a table as `cust_tan`. Then, read audit trail as `smartbank_admin`.
	
	Expected
	The topmost row shows `statement` containing `CREATE TABLE`, with `database_principal_name = cust_tan` and `succeeded = 0`.
		Run as cust_tan:
		`CREATE TABLE dbo.Test_Table (ID INT);`
		
		Run as smartbank_admin:
		`SELECT TOP (20)`
		`event_time, action_id, succeeded,`
		`database_principal_name, statement`
		`FROM sys.fn_get_audit_file(`
		`'C:\SmartBankServer\Audit\*.sqlaudit',`
		`DEFAULT,`
		`DEFAULT)`
		`WHERE statement LIKE '%CREATE TABLE%'`
		`ORDER BY event_time DESC;`
				Passed

No.	Description	Execution	Results	Passed / Failed

128.	Attempt to revoke permission as `cust_tan`. Then, read audit trail as `smartbank_admin`.
	
	Expected
	The topmost row shows `statement` containing `REVOKE`, with `database_principal_name = cust_tan` and `succeeded = 0`.
		Run as cust_tan:
		`REVOKE SELECT ON dbo.vw_StaffDirectory TO role_Customer;`
		
		Run as smartbank_admin:
		`SELECT TOP (20) event_time, action_id, succeeded, database_principal_name, statement FROM sys.fn_get_audit_file( 'C:\SmartBankServer\Audit\*.sqlaudit', DEFAULT, DEFAULT) WHERE statement LIKE '%REVOKE%' ORDER BY event_time DESC;`
				Passed

129.	Deposit a negative amount as `cust_tan`. Then, read `Security.RejectedAttemptLog` as `smartbank_admin`.
	
	Expected
	The topmost row shows
	`LoginName = cust_tan`
	`ProcedureName = usp_Deposit`
	`Reason` describing the invalid amount
		Run as cust_tan:
		`EXEC dbo.usp_Deposit @AccountID = 'ACC1000001', @Amount = -100.00;`
		
		Run as smartbank_admin:
		`SELECT TOP (20) AttemptID, AttemptedOn, LoginName, ProcedureName, Reason, Detail FROM Security.RejectedAttemptLog ORDER BY AttemptID DESC;`
				Passed

No.	Description	Execution	Results	Passed / Failed

130.	Update the branch of `S00004` as `mgr_aisyah`. Then, query the temporal history of `S00004` as `smartbank_admin`.
	
	Expected
	Several rows are returned. The topmost row shows `Branch = Klang`, while the earlier rows show the prior values.
		Run as mgr_aisyah:
		`EXEC dbo.usp_UpdateStaffDetails @StaffID = 'S00004', @Branch = 'Klang';`
		
		Run as smartbank_admin:
		`SELECT StaffID, StaffName, Branch, Phone, Salary, SysStartTime, SysEndTime FROM dbo.Staff FOR SYSTEM_TIME ALL WHERE StaffID = 'S00004' ORDER BY SysStartTime DESC;`
			
			StaffID StaffName Branch Phone Salary SysStartTime SysEndTime
			1 S00004 Lim Wei Sheng Klang 012-3450099 5200.00 2026-08-24 07:51:52.5565595 9999-12-31 23:59:59.9999999
			2 S00004 Lim Wei Sheng Petaling Jaya 012-3450099 5200.00 2026-08-19 18:18:47.0496116 2026-08-24 07:51:52.5565595
				Passed

131.	Attempt to read the staff history table as `mgr_aisyah`. Then, read audit trail as `smartbank_admin`.
	
	Expected
	Permission denied on the `StaffHistory` table. The topmost `SL` row shows `database_principal_name = mgr_aisyah`, `object_name = StaffHistory`, and `succeeded = 0`.
		Run as mgr_aisyah:
		`SELECT * FROM dbo.StaffHistory;`
		
		Run as smartbank_admin:
		`SELECT TOP (20) event_time, action_id, succeeded, database_principal_name, object_name FROM sys.fn_get_audit_file( 'C:\SmartBankServer\Audit\*.sqlaudit', DEFAULT, DEFAULT) WHERE action_id = 'SL' ORDER BY event_time DESC;`
			
			event_time action_id succeeded database_principal_name object_name
			1 2026-08-24 07:52:54.6127740 SL 0 mgr_aisyah StaffHistory
				Passed

No.	Description	Execution	Results	Passed / Failed

132.	Attempt to read the rejection log as `dba_ahmad`. Then, read audit trail as `smartbank_admin`.
	
	Expected
	Permission denied on the `Security.RejectedAttemptLog` table. The topmost `SL` row shows `database_principal_name = dba_ahmad`, `object_name = RejectedAttemptLog`, and `succeeded = 0`.
		Run as dba_ahmad:
		`SELECT * FROM Security.RejectedAttemptLog;`
		
		Run as smartbank_admin:
		`SELECT TOP (20) event_time, action_id, succeeded, database_principal_name, object_name FROM sys.fn_get_audit_file( 'C:\SmartBankServer\Audit\*.sqlaudit', DEFAULT, DEFAULT) WHERE action_id = 'SL' ORDER BY event_time DESC;`
			
			event_time: 2026-08-24 07:54:42.1532342, action_id: SL, succeeded: 0, database_principal_name: dba_ahmad, object_name: RejectedAttemptLog
				Passed

133.	Attempt to remove the rejection log as `dba_ahmad`. Then, read audit trail as `smartbank_admin`.
	
	Expected
	Permission denied on the `Security.RejectedAttemptLog` table. The topmost `DL` row shows `database_principal_name = dba_ahmad`, `object_name = RejectedAttemptLog`, and `succeeded = 0`.
		Run as dba_ahmad:
		`DELETE FROM Security.RejectedAttemptLog;`
		
		Run as smartbank_admin:
		`SELECT TOP (20) event_time, action_id, succeeded, database_principal_name, object_name FROM sys.fn_get_audit_file( 'C:\SmartBankServer\Audit\*.sqlaudit', DEFAULT, DEFAULT) WHERE action_id = 'DL' ORDER BY event_time DESC;`
			
			event_time: 2026-08-24 07:55:33.5037133, action_id: DL, succeeded: 0, database_principal_name: dba_ahmad, object_name: RejectedAttemptLog
				Passed

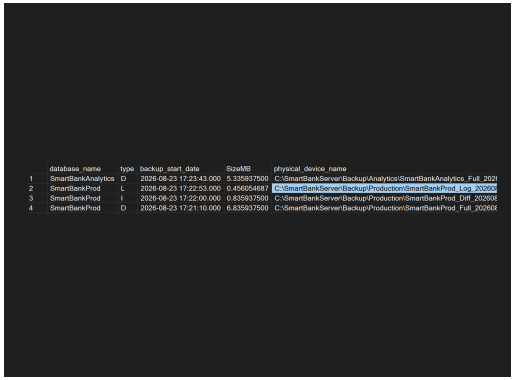
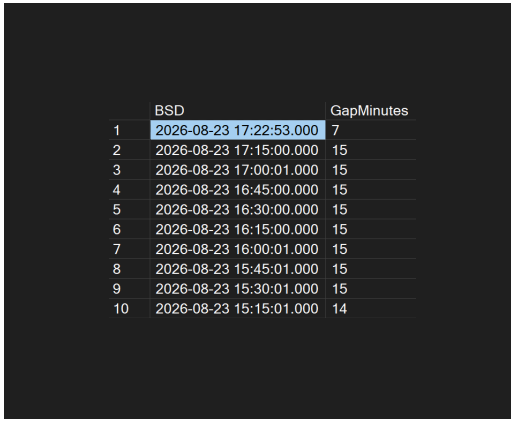
No.	Description	Execution	Results	Passed / Failed
Availability: Full, differential, and transaction log backups are automated.				

134.	Start the weekly full backup job and view the job history as `smartbank_admin`.
	
	Expected
	Backup runs successfully. Job history shows `RunStatus = Succeeded.`
		Run as smartbank_admin:
		`EXEC msdb.dbo.sp_start_job @job_name = 'SmartBank - Full Backup (Weekly)';`
		`WAITFOR DELAY '00:00:20';`
			Job 'SmartBank - Full Backup (Weekly)' started successfully.
			Completion time: 2026-08-23T17:21:29.4983657+08:00
				Passed

135.	Start the daily differential backup job and view the job history as `smartbank_admin`.
	
	Expected
	Backup runs successfully. Job history shows `RunStatus = Succeeded.`
		Run as smartbank_admin:
		`EXEC msdb.dbo.sp_start_job @job_name = 'SmartBank - Differential Backup (Daily)';`
		`WAITFOR DELAY '00:00:20';`
			Job 'SmartBank - Differential Backup (Daily)' started successfully.
			Completion time: 2026-08-23T17:22:20.2200232+08:00
				Passed

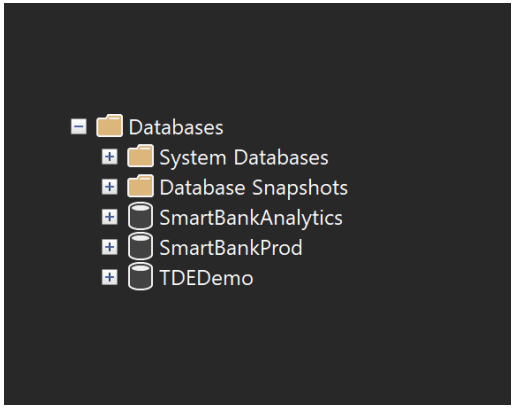
136.	Start the 15-minute transactional log backup job and view the job history as `smartbank_admin`.
	
	Expected
	Backup runs successfully. Job history shows `RunStatus = Succeeded.`
		Run as smartbank_admin:
		`EXEC msdb.dbo.sp_start_job @job_name = 'SmartBank - Log Backup (15 min)';`
		`WAITFOR DELAY '00:00:20';`
			Job 'SmartBank - Log Backup (15 min)' started successfully.
			Completion time: 2026-08-23T17:23:13.0148033+08:00
				Passed

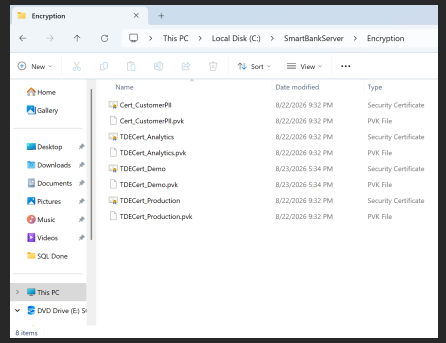
137.	Start the weekly analytics backup job and view the job history as `smartbank_admin`.
	
	Expected
	Backup runs successfully. Job history shows `RunStatus = Succeeded.`
		Run as smartbank_admin:
		`EXEC msdb.dbo.sp_start_job @job_name = 'SmartBank - Analytics Backup (Weekly)';`
		`WAITFOR DELAY '00:00:20';`
			Job 'SmartBank - Analytics Backup (Weekly)' started successfully.
			Completion time: 2026-08-23T17:24:02.8948422+08:00
				Passed

No.	Description	Execution	Results	Passed / Failed
138.	Review the backup history for both databases as `smartbank_admin`. Expected Rows are present for full, differential and transaction log backups on `SmartBankProd`, and a full backup on `SmartBankAnalytics`, each with a valid `backup_start_date`, size and file path.	Run as smartbank_admin: `SELECT bs.database_name, bs.type, bs.backup_start_date, bs.backup_size / 1048576.0 AS SizeMB, bmf.physical_device_name FROM msdb.dbo.backupset AS bs JOIN msdb.dbo.backupmediafamily AS bmf ON bmf.media_set_id = bs.media_set_id WHERE bs.database_name IN ('SmartBankProd', 'SmartBankAnalytics') ORDER BY bs.backup_start_date DESC;`		Passed
Availability: Data loss in production is bounded to a maximum of 15 minutes by the defined RPO.				
139.	Measure the interval between consecutive transactional log backups. Expected `GapMinutes` never exceeds 15.	Run as smartbank_admin: `WITH L AS ( SELECT backup_start_date AS BSD FROM msdb.dbo.backupset WHERE database_name = 'SmartBankProd' AND type = 'L' ) SELECT BSD, DATEDIFF(MINUTE, LAG(BSD) OVER (ORDER BY BSD), BSD) AS GapMinutes FROM L ORDER BY BSD DESC;`		Passed
Availability: Accidental or unauthorised changes can be recovered.				

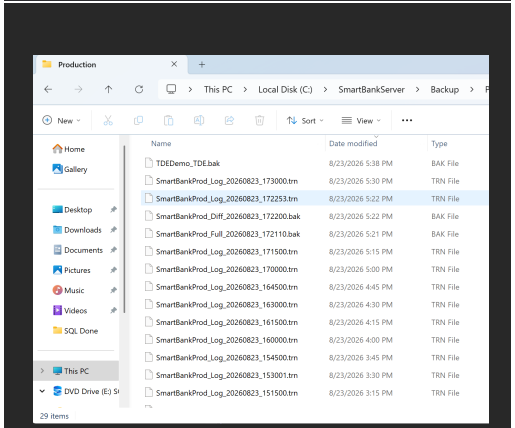
This is demonstrated in Backup: Restore and Recovery. Hence, it is not included in the current test case. See page 95.

No.	Description	Execution	Results	Passed / Failed
Availability: Recovery of an encrypted backup depends on custody of its certificate.				

140.	Prepare a demonstration database as `smartbank_admin`.
	
	Expected
	`TDEDemo` is created.
		Run as smartbank_admin:
		`CREATE DATABASE TDEDemo;`
		`GO`
		
		`USE TDEDemo;`
		`GO`
		
		`CREATE TABLE dbo.SecretData (`
		`ID INT,`
		`SecretText VARCHAR(100)`
		`);`
		`INSERT INTO dbo.SecretData`
		`VALUES (1, 'SECRET_MARKER');`
		`GO`
				Passed

141.	Prepare the certificate for the demonstration database as `smartbank_admin`.
	
	Expected
	The certificates `TDECert_Demo.cer` / `.pvk` are written to disk.
		Run as smartbank_admin:
		`CREATE CERTIFICATE TDECert_Demo`
		`WITH SUBJECT = 'TDE Demo Cert';`
		`GO`
		
		`BACKUP CERTIFICATE TDECert_Demo`
		`TO FILE =`
		`'C:\SmartBankServer\'`
		`+ 'Encryption\TDECert_Demo.cer'`
		`WITH PRIVATE KEY (`
		`FILE =`
		`'C:\SmartBankServer\'`
		`+ 'Encryption\TDECert_Demo.pvk',`
		`ENCRYPTION BY PASSWORD =`
		`'C3rtBackup#Demo2026'`
		`);`
		`GO`
			`Commands completed successfully.`
			
			`Completion time: 2026-08-23T17:34:33.7047967+08:00`
			
			 
				Passed

No.	Description	Execution	Results	Passed / Failed

142.	Prepare the encrypted keys for backup of the demonstration database as `smartbank_admin`.
	
	Expected
	The encrypted backup `TDEDemo_TDE.bak` is written to disk.
		Run as smartbank_admin:
		`CREATE DATABASE ENCRYPTION KEY`
		`  WITH ALGORITHM = AES_256`
		`  ENCRYPTION BY SERVER CERTIFICATE`
		`  TDECert_Demo;`
		`GO`
		
		`ALTER DATABASE TDEDemo SET ENCRYPTION ON;`
		`GO`
		
		`USE master;`
		`GO`
		
		`BACKUP DATABASE TDEDemo`
		`  TO DISK =`
		`  'C:\SmartBankServer\'`
		`  + 'Backup\Production\'`
		`  + 'TDEDemo_TDE.bak'`
		`  WITH INIT,`
		`  NAME = 'TDEDemo encrypted backup';`
		`GO`
			Processed 552 pages for database 'TDEDemo', file 'TDEDemo' on file 1.
			Processed 2 pages for database 'TDEDemo', file 'TDEDemo_log' on file 1.
			BACKUP DATABASE successfully processed 554 pages in 0.031 seconds (139.490 MB/sec).
			
			Completion time: 2026-08-23T17:30:59.1499687+08:00
			
			 
				Passed

143.	Delete the certificate, then attempt to restore the backup as `smartbank_admin`.
	
	Expected
	Error returned. Restore is rejected.
		Run as smartbank_admin:
		`USE master;`
		`GO`
		
		`ALTER DATABASE TDEDemo`
		`  SET SINGLE_USER`
		`  WITH ROLLBACK IMMEDIATE;`
		`DROP DATABASE TDEDemo;`
		`DROP CERTIFICATE TDECert_Demo;`
		`GO`
		
		`RESTORE DATABASE TDEDemo`
		`  FROM DISK =`
		`  'C:\SmartBankServer\'`
		`  + 'Backup\Production\'`
		`  + 'TDEDemo_TDE.bak'`
		`  WITH REPLACE, RECOVERY, STATS = 10;`
		`GO`
			Msg 33111, Level 16, State 3, Line 34
			Cannot find server certificate with thumbprint '0xCC5687AF27ECEE7666EA58C04859B89CD3670F30'.
			Msg 3013, Level 16, State 1, Line 34
			RESTORE DATABASE is terminating abnormally.
			
			Completion time: 2026-08-23T17:40:38.3868216+08:00
				Passed

No.	Description	Execution	Results	Passed / Failed

144.	Recreate the certificate from its backup files, then reperform the same restore as `smartbank_admin`.
	
	Expected
	Restore performed successfully with table contents readable.
		Run as smartbank_admin:
		
		`USE master;`
		`GO`
		`CREATE CERTIFICATE TDECert_Demo`
		`  FROM FILE =`
		`  'C:\SmartBankServer\'`
		`  + 'Encryption\TDECert_Demo.cer'`
		`  WITH PRIVATE KEY (`
		`  FILE =`
		`  'C:\SmartBankServer\'`
		`  + 'Encryption\TDECert_Demo.pvk',`
		`  DECRYPTION BY PASSWORD =`
		`  'C3rtBackup#Demo2026'`
		`  );`
		`GO`
		
		`RESTORE DATABASE TDEDemo`
		`  FROM DISK =`
		`  'C:\SmartBankServer\'`
		`  + 'Backup\Production\'`
		`  + 'TDEDemo_TDE.bak'`
		`  WITH REPLACE, RECOVERY, STATS = 10;`
		`GO`
			10 percent processed.
			20 percent processed.
			31 percent processed.
			40 percent processed.
			50 percent processed.
			60 percent processed.
			71 percent processed.
			80 percent processed.
			90 percent processed.
			100 percent processed.
			Processed 552 pages for database 'TDEDemo', file 'TDEDemo' on file 1.
			Processed 2 pages for database 'TDEDemo', file 'TDEDemo_log' on file 1.
			RESTORE DATABASE successfully processed 554 pages in 0.035 seconds (123.549 MB/sec).
			
			Completion time: 2026-08-23T17:41:34.2398579+08:00
			
			ID SecretText
			1 1 SECRET_MARKER
				Passed

Conclusion

Overall, this report demonstrates the design and implementation of a comprehensive database security architecture for SmartBank. As the firm handles a large volume of sensitive information, including customer details, transaction histories, and employee information, it is essential for SmartBank to prioritise on database security and conform with the components of the CIA triad throughout its daily business operations. Hence, a variety of security mechanisms, including role-based access control (RBAC), integrity controls, data protection techniques, backup and restore, as well as audit techniques, are applied on the production and analytics database of SmartBank to align with best practices like the principle of least privilege (PoLP) and defence in depth (DiD). By having a combination of detective, preventive, and recovery controls across different levels of the SmartBank system, not only this helps to protect the information assets of the company to minimise any potential financial loss or privacy violations, but also contribute to the reliability of the database environment in supporting business needs and daily operational tasks.

References

Ahmed, I. (2024, February 2). *What is Transparent Data Encryption (TDE)? The Ultimate Guide*. Percona. https://www.percona.com/blog/transparent-data-encryption-tde/

Alkhaldi, N. (2023, February 28). *What is data masking, and how to implement it the right way?* ITRex. https://itrexgroup.com/blog/what-is-data-masking-and-how-to-implement-it-the-right-way/

Almaiah, M. A., Saqr, L. M., Al-Rawwash, L. A., Altellawi, L. A., Al-Ali, R., & Almomani, O. (2024). Classification of Cybersecurity Threats, Vulnerabilities and Countermeasures in Database Systems. *Computers, Materials & Continua*, *81*(2), 3189–3220. https://doi.org/10.32604/cmc.2024.057673

Amoresano, K., & Yankson, B. (2023). Human Error - A Critical Contributing Factor to the Rise in Data Breaches: A Case Study of Higher Education. *Holistica*, *14*(1), 110–132. https://doi.org/10.2478/hjbpa-2023-0007

AWS. (2026, August 7). *What is Data Masking?* AWS. https://aws.amazon.com/what-is/data-masking/

AxialSQL. (2021, October 7). *Understanding SQL Check Constraints*. AxialSQL. https://axial-sql.com/info/understanding-sql-check-constraints/

Azad, M. A., Abdullah, S., Arshad, J., Lallie, H., & Ahmed, Y. H. (2024). Verify and trust: A multidimensional survey of zero-trust security in the age of IoT. *Internet of Things*, *27*, 101227. https://doi.org/10.1016/j.iot.2024.101227

Banerjee, S. (2023, November 18). *Enhancing Data Security with Column-Level Encryption: Best Practices*. DEV Community. https://dev.to/supratipb/enhancing-data-security-with-column-level-encryption-best-practices-2hol

Beek, M. (2021, July 22). *How to configure Row Level Security - A Basic Example.* ThoughtSpot. https://community.thoughtspot.com/customers/s/article/Row-Level-Security-A-Basic-Example

Bowman, K. (2024, June 20). *5 Data Masking Techniques and Why You Need Them.* Pathlock. https://pathlock.com/learn/5-data-masking-techniques-and-why-you-need-them/

Buckman, B. (2025). *What is Hashing in Cybersecurity? A Guide for Professionals.* Huntress. https://www.huntress.com/cybersecurity-101/topic/what-is-hashing-cybersecurity

Butt, A. U. R., Mahmood, T., Saba, T., Bahaj, S. A. O., Alamri, F. S., Iqbal, M. W., & Khan, A. R. (2023). An Optimized Role-Based Access Control Using Trust Mechanism in E-Health Cloud Environment. *IEEE Access*, *11*, 138813–138826. https://doi.org/10.1109/access.2023.3335984

Dharani. (2025). *What are Constraints in SQL? Types, Examples & Advantages.* Acte. https://www.acte.in/overview-of-constraints-in-sql

Douglas, R. (2023, May 9). *Choosing a SQL Authentication Mode.* It Depends. https://gumption.co.nz/2023/05/choosing-a-sql-authentication-mode/

Erik. (2026, June 4). *Encryption at Rest: Secure Stored Data Everywhere*. Secure Systems. https://www.securesystems.com/data-at-rest-encryption/

Farhadighalati, N., Estrada-Jimenez, L. A., Nikghadam-Hojjati, S., & Barata, J. (2025). A Systematic Review of Access Control Models: Background, Existing Research, and Challenges. *IEEE Access*, *13*, 17777–17806. https://doi.org/10.1109/access.2025.3533145

Fernandez, R., Cheng, P. C.-H., Nhlabatsi, A., Khan, K. Md., & Fetais, N. (2023). Effective Collaboration in the Management of Access Control Policies: A Survey of Tools. *IEEE Access*, *11*, 13929–13947. https://doi.org/10.1109/access.2023.3242863

Gahlot, S., Verma, B., Khandelwal, A. Dayanand. (2018). Database Security: Attacks, Threats and Control Methods. *International Journal of Engineering Research & Technology*, *5*(10). https://doi.org/10.17577/IJERTCONV5IS10011

Gomes, A. F. D., Santos, C., Wanzeller, C., & Martins, P. (2021). Database Encryption for Balance Between Performance and Security. *Journal of Information Assurance & Cybersecurity*, 1–9. https://doi.org/10.5171/2021.614511

Gundapaneni, M. N. (2025). *Security Practices in Database Access: A Technical Review.* Journal of Computer Science and Technology Studies, *7*(8), 771-778. https://doi.org/10.32996/jcsts.2025.7.8.90

Huan, B., & Xie, H. (2026). Scalable and energy efficient hybrid cryptographic framework for IoT security using advanced symmetric and asymmetric techniques. *Scientific Reports*, *16*(1). https://doi.org/10.1038/s41598-025-22225-6

Imperva. (2024, January 8). *Static vs Dynamic Data Masking.* Imperva. https://www.imperva.com/learn/data-security/static-dynamic-data-masking/

ITaegis. (2024, December 14). *Unpacking Data Protection: Encryption, Encoding, Hashing, Tokenization, and Masking Explained*. Medium. https://medium.com/@ITaegis/unpacking-data-protection-encryption-encoding-hashing-tokenization-and-masking-explained-3d21dfb6485f

Jane, K. (2025). Backup and Recovery Solutions: Disaster recovery planning for critical databases.

Johnston, B. (2023, September 3). *SQL Server Row-Level Security: Setup, access predicates, and examples (complete guide, part two)*. Simple Talk. https://www.red-gate.com/simple-talk/databases/sql-server/sql-server-rls-setup/

Kapadiya, P. (2025). A Research Paper on Salting Hashed Passwords for Securing Web Applications. *International Journal For Multidisciplinary Research*, *7*(6). https://doi.org/10.36948/ijfmr.2025.v07i06.57752

Karthik. (2025, January 24). *Row and Column Level Security in Synapse/SQL Server— Day 45 of 100 Days of Data Engineering, AI and Azure Challenge*. Medium. https://medium.com/@krthiak/row-and-column-level-security-in-synapse-sql-server-day-45-of-100-days-of-data-engineering-ai-and-15b08678cc47

Lawrence, A. (2022, June 8). *Outages: understanding the human factor*. Uptime Institute Blog. https://journal.uptimeinstitute.com/outages-understanding-the-human-factor/

Madyatmadja, E. D., Hakim, A. N., & Sembiring, D. J. M. (2021). Performance testing on Transparent Data Encryption for SQL Server's reliability and efficiency. *Journal of Big Data*, *8*(1). https://doi.org/10.1186/s40537-021-00520-z

McCallister, E., Grance, T., & Scarfone, K. (2010, April). *Guide to Protecting the Confidentiality of Personally Identifiable Information (PII) Recommendations of the National Institute of Standards and Technology.* NIST. https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-122.pdf

Microsoft. (2026a, July 20). *Primary and foreign key constraint*. MicrosoftLearn. https://learn.microsoft.com/en-us/sql/relational-databases/tables/primary-and-foreign-key-constraints?view=sql-server-ver17

Microsoft. (2026b, March 6). *Connect to SQL Server with strict encryption*. MicrosoftLearn. https://learn.microsoft.com/en-us/sql/relational-databases/security/networking/connect-with-strict-encryption?view=sql-server-ver17&tabs=new-ag

Mohamed, A., Auer, D., Hofer, D., & Küng, J. (2023). A systematic literature review of authorization and access control requirements and current state of the art for different database models. *International Journal of Web Information Systems*, *20*(1), 1-23. https://doi.org/10.1108/ijwis-04-2023-0072

Mollashaik, A. S. (2023). Dynamic data masking: Enhancing data security in real-time. *International Journal of Science and Research Archive*, *8*(1), 1053–1070. https://doi.org/10.30574/ijsra.2023.8.1.0145

Netdata. (2025, April 28). *Database Backup - Types, Process and Benefits.* Netdata. https://www.netdata.cloud/academy/what-is-database-backup/

Panja, M., Chakraborty, T., & Kumar, U. (2022). Data Dictionary. *Encyclopedia of Earth Sciences Series*, 1–5. https://doi.org/10.1007/978-3-030-26050-7_75-1

PentaSecurity. (2020, June 10). *Database Encryption.* Penta Security Inc. https://www.pentasecurity.com/product/encryption/

Rajeshkumarin. (2026, February 21). *What is encryption in transit? Meaning, Examples, Use Cases & Complete Guide*. DevSecOps. https://www.devsecopsnow.com/encryption-in-transit/

Richman, J. (2026, June 2). *What Are Database Triggers? Types, Examples, Syntax To Use*. Estuary. https://estuary.dev/blog/database-triggers/

Ridge Security (2025, August 17). *Compliance with Malaysia Risk Management in Technology (RMiT) Using RidgeBot™*. Ridge Security. https://ridgesecurity.ai/ridgebot/malaysian-rmit-compliance/

Robidoux, G. (2025, April 29). *SQL Server Recovery Models*. MSSQLTips. https://www.mssqltips.com/tutorial/sql-server-recovery-models/

SearchInform. (2026). *Data Audit: What Is a Data Auditing and Why Is It Important?* Searchinform. https://searchinform.com/articles/cybersecurity/measures/audit/data-audit/

Singh, K. (2026b, February 2). *RBAC Security in Data: Field-Level to Row-Level Data Control*. LoginRadius. https://www.loginradius.com/blog/identity/rbac-data-security-access-control

Singh, S. (2026a, January 6). *Data Breach Statistics (2026) – Trends, Costs & Impact*. DemandSage. https://www.demandsage.com/data-breach-statistics/

Slawinska, M. (2021, April 27). *What Is the Benefit of Foreign Keys in SQL?* LearnSQL. https://learnsql.com/blog/why-use-foreign-key-in-sql/

Sun, B., Zhao, S., Tian, G. (2024). SQL queries over encrypted databases: a survey. *Connection Science*, *36*(1). https://doi.org/10.1080/09540091.2024.2323059

Tenable (2024). *DISA STIG SQL Server 2014 Instance DB Audit v2r4.* Tenable. https://www.tenable.com/audits/DISA_STIG_MSSQL_2014_Instance_Database_v2r4

Willert, R. (2022, February 22). *Preventive Controls & Their Importance To the Security Control Environment*. Linford & Company LLP. https://linfordco.com/blog/importance-of-preventive-controls/

Wongaphai, C., & Ongtang, M. (2025). Enhancing Financial Service Systems Through OLAP and OLTP Integration. *2025 10Th International Conference on Business and Industrial Research (ICBIR)*, 264–269. https://doi.org/10.1109/icbir65229.2025.11163347

Appendix A

Initial Database Design

```
CREATE DATABASE SmartBankDB;
GO

USE SmartBankDB;
GO

CREATE TABLE Staff(
       StaffID            VARCHAR(6)         PRIMARY KEY,
       StaffName          VARCHAR(100),
       Position           VARCHAR(20),
       Branch             VARCHAR(50),
       Phone              VARCHAR(20),
       Salary             DECIMAL(10,2)
);

CREATE TABLE Customer(
       CustomerID         VARCHAR(6)         PRIMARY KEY,
       CustomerName       VARCHAR(100),
       ICNumber           VARCHAR(20),
       Phone              VARCHAR(20),
       [Address]          VARCHAR(200)
);

CREATE TABLE Account(
       AccountID          VARCHAR(10)        PRIMARY KEY,
       CustomerID         VARCHAR(6),
       AccountType        VARCHAR(20),
       Balance            DECIMAL(12,2)
);

CREATE TABLE TransactionRecord(
       TransID            INT                IDENTITY          PRIMARY KEY,
       AccountID          VARCHAR(10),
       TransDate          DATETIME,
       Amount             DECIMAL(12,2),
       TransactionType    VARCHAR(20)
);
```